

Numerical Control (CNC) Communication Software

FCSB1224W000

Reference Manual

INTRODUCTION

Thank you for purchasing the Numerical Control (CNC) communication software FCSB1224W000. This user's reference manual describes how to use the OLE/COM interface of FCSB1224W000.

Read this manual before use to get familiar with and correctly use the functions of FCSB1224W000.

Precautions for Safety

(Read carefully before use.)

Before using the product, read the user's reference manual and other related manuals. Pay careful attention to safety when using the product.

The safety instructions in this manual are intended for this product. Do not use this product until you have a full knowledge of general and safety information and precautions about the computerized numerical controller.

In this manual, the safety instruction levels are classified into "WARNING" and "CAUTION".



When there is a great risk that the user could be subject to fatalities or serious injuries if handling is mistaken.



When the user could be subject to medium or slight injuries or when physical damage could occur if handling is mistaken.

Note that even items ranked as , may lead to major results depending on the situation. In any case, important information that must always be observed is described.

Keep this manual in a safe place for future reference.

[Mechanical precautions]



- When connecting the product with the computerized numerical controller, consider the risk of external power supply failure and computer malfunction, and install the external safety circuit as fail-safe of the entire system.
- There is a risk of accident due to output error or malfunction.
- Writing to the computerized numerical controller will directly be reflected in machine control.
- Input error of setup or other parameter may cause accidental operation.
- Check all things before execution.

[Startup and maintenance precautions]



- Operation error may cause machine damage or accident.
- Some functions may be different or unavailable depending on the version of the computerized numerical controller.

Trademarks

MELDAS, MELSEC, EZSocket, EZMotion, iQ Platform, MELSOFT, GOT, CC-Link, CC-Link/LT and CC-Link IE are either trademarks or registered trademarks of Mitsubishi Electric Corporation in Japan and/or other countries.

Ethernet is a registered trademark of Xerox Corporation in the United States and/or other countries.

Microsoft®, Windows®, Visual C++®, Visual C#® and Visual Basic® are either trademarks or registered trademarks of Microsoft Corporation in the United States and/or other countries.

CompactFlash and CF are either trademarks or registered trademarks of SanDisk Corporation in the United States and/or other countries.

SD Logos are trademarks or registered trademarks of SD-3C, LLC in the United States, other countries or both.

Other company and product names that appear in this manual are trademarks or registered trademarks of the respective companies.

CONTENTS

1. OUTLINE	1
1.1 Features	1
1.2 Software Types	1
1.3 Development Environment	2
1.4 Applicable Models	2
1.5 Connection Configuration	3
1.5.1 Connection with M700 series	3
1.5.2 Connection with M800 series	3
1.5.3 Connection with C70/C80	4
1.6 Installation	7
1.6.1 Directory structure after installation	7
1.7 Redistributable Installer for Run-time Library	8
1.8 Header Files for Development	8
1.9 Interface	9
1.9.1 Custom interface	9
1.9.2 Automation interface	10
1.10 Program Flow	11
1.10.1 VC++ program flow	11
1.10.2 VB program flow	12
1.10.3 VC# Program flow	13
2. I/F DETAILED SPECIFICATIONS	15
2.1 Common Items	15
2.2 Method List	18
2.3 Precautions for Using Methods	23
2.3.1 When an unknown error code responses or memory access error occurs	23
2.3.2 Precautions for opening/closing of NC control unit system line	23
2.4 IEZNCCommunication3 Interface	24
2.5 IEZNCSystem2 Interface	44
2.6 IEZNCPosition Interface	52
2.7 IEZNCCommand2 Interface	72
2.8 IEZNCProgram2 Interface	78
2.9 IEZNCTime Interface	85
2.10 IEZNCAxisMonitor Interface	95
2.11 IEZNCRunStatus Interface	123
2.12 IEZNCFile6 Interface	131
2.13 IEZNCCommonVariable2 Interface	172
2.14 IEZNCLocalVariable2 Interface	178
2.15 IEZNCTool3 Interface	181
2.16 IEZNCATC3 Interface	216
2.17 IEZNCParameter3 Interface	226
2.18 IEZNCOperation Interface	230
2.19 IEZNCDevice Interface	233
2.20 IEZNCSubFunction3 Interface	252
3. ERROR CODE LIST	258
4. OPERATING PROCEDURE	269
4.1 Initialization for Enabling OLE/COM Interface	270
4.2 Object Creation	270
4.3 VB Programming of Automation Interface	271
4.3.1 Using OLE automation interface with VB	271
4.3.2 VB program flow (early binding)	273

4.3.3 VB program flow (late binding).....	274
4.4 VC# Programming of EZSocket Automation Interface	275
4.4.1 Using EZSocketNc OLE automation interface with VC#	275
4.4.2 VC# program flow (early binding)	276
4.4.3 VC# program flow (late binding).....	277

5. SAMPLE APPLICATION278

5.1 Overview.....	278
5.2 Position Data Display Application.....	278
5.2.1 Operating requirements	278
5.2.2 Installation and uninstallation	278
5.2.3 Executing the sample application	279
5.2.4 Function list	279
5.2.5 Screen structure and functions	280
5.2.6 Setting project workspaces	283
5.2.7 IEZNcCommunication object.....	283
5.2.8 IEZNcPosition object.....	283
5.3 Monitoring Application	284
5.3.1 Operating requirements	284
5.3.2 Installation and uninstallation	284
5.3.3 Executing the sample application	284
5.3.4 Function list	285
5.3.5 Screen structure and functions	286
5.3.6 Setting project workspaces	287
5.4 CONSOLE PROGRAM SAMPLE.....	288
5.4.1 Console Program to Connect C70 (via Ethernet)	288

1. OUTLINE

The Mitsubishi Electric CNC communication software FCSB1224W000 is designed to help development of an application with Windows interface for Mitsubishi Electric CNC M700/M800 series and C70/C80 series which operates on the EZSocket environment. (Hereinafter referred to as the product.)

The product can accelerate development by eliminating necessity to know about internal processing of the computerized numerical controller and enabling use of the common OLE interface on the Mitsubishi computerized numerical controller.

Note:

- EZSocket is a common platform for each software.
When installing other applications causes a problem, reinstall the product or update it to the latest version.

1.1 Features

- Functions of Mitsubishi Electric CNC M700/M800 series, Mitsubishi Electric CNC C70 series and Mitsubishi Electric CNC C80 series can be used on the Windows application with VC++, VC#, VB or VBA macro.
- Communication and other complex processing with Mitsubishi Electric CNC M700/M800 series, C70 series and C80 series will be conducted by the product so that the user can focus on development of the value-added Windows application.
- As the product will be upgraded in accordance with new models in the future, upgrading of the user-created application will also be easy.

1.2 Software Types

There are two types of softwares for the product:

- FCSB1224W000

The software development kit (SDK) for engineers

Header files and sample programs for each I/F are included.

- FCSB1224W100

The run-time library

Header files and sample programs for each I/F are not included.

When the software version of them are the same, the same version of the run-time library is installed.

1.3 Development Environment

Environmental requirements for application development with this product are as below.

Item	Specifications
Personal computer	Personal Computer AT compatible machine (x86 processor) Personal Computer AT compatible machine (x64 processor)
CPU (*2)	-
OS (*3)	Microsoft Windows 10 Japanese/English
Required memory (*2)	-
Disc space (*2)	-
Peripheral equipment (*2)	-
Development language (*1) (*2)	Microsoft Visual C# 2015, 2017 Microsoft Visual C++.NET 2003, 2005, 2008, 2010, 2015, 2017 Microsoft Visual C++ Ver.6.0 Microsoft Visual Basic Ver.6.0 Microsoft Visual Basic for Applications Ver.6.0 (Excel 2000 VBA equivalent)
Note	(*1) When the language is C++, the development uses the native code (VC++). Development with managed code is not supported. Development with VC# uses the automation interface. (*2) See the Windows operating environment recommended by Microsoft Corporation. (*3) As the product is 32 bit module, it will run on the Windows 32-bit On Windows 64-bit (WOW64) subsystem if executed on x64 platform. It does not support 64 bit native operation.

1.4 Applicable Models

The product is applicable to the following models. Check compatibility before use.

- Mitsubishi Electric CNC M700 series (M700V/M70V series, M700/M70 series and E70) (hereinafter referred to as M700 series)
- Mitsubishi Electric CNC M800 series (M800V/M80V series, M800/M80 series and E80) (hereinafter referred to as M800 series)
- Mitsubishi Electric CNC C70 (hereinafter referred to as C70)
- Mitsubishi Electric CNC C80 (hereinafter referred to as C80)

1.5 Connection Configuration

This section explains about connection configuration between the Mitsubishi Electric CNC and personal computer. Prepare the computerized numerical controller, personal computer, cable and other necessary equipment for communication using this product. For connection of equipment, refer to the instruction manual for each Mitsubishi Electric CNC.

1.5.1 Connection with M700 series

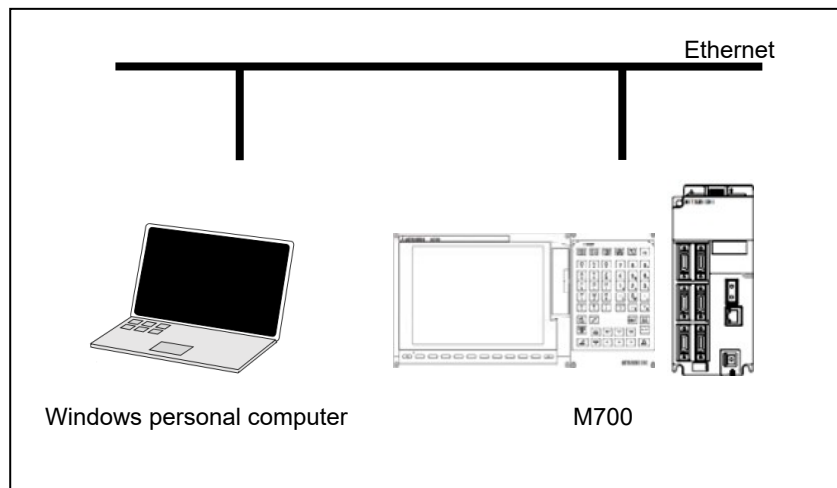


Figure1-1 Connection with M700 series

1.5.2 Connection with M800 series

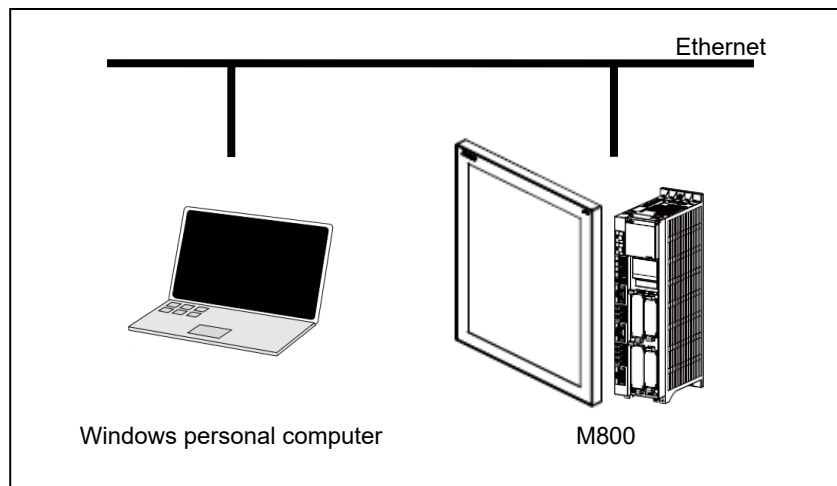


Figure1-2 Connection with M800 series

1.5.3 Connection with C70/C80

See the main connection pathes between C70/C80 and personal computer.

Model	Connection path	Connection module	Figure
C70	Ethernet connection	MELSEC QnUDE CPU C70 Ethernet I/F Module (QJ71E71)	Figure 1-3
	USB connection * Refer to the manual of the connection destination module for the USB specification.	MELSEC QnUD Series CPU MELSEC QnUDE Series CPU	Figure 1-4
	RS-232C connection	MELSEC QnUD Series CPU	Figure 1-5
	GOT (bus connection) transparent	Base unit	Figure 1-6
	GOT (Ethernet) transparent	MELSEC QnUDE Series CPU C70 Ethernet I/F Module (QJ71E71)	Figure 1-7
C80	Ethernet connection	MELSEC iQ-R Series CPU C80 Ethernet I/F Module (RJ71EN71)	Figure 1-3
	USB connection * Refer to the manual of the connection destination module for the USB specification.	MELSEC iQ-R Series CPU	Figure 1-4
	GOT (Ethernet) transparent	MELSEC iQ-R Series CPU C80 Ethernet I/F Module (RJ71EN71)	Figure 1-7

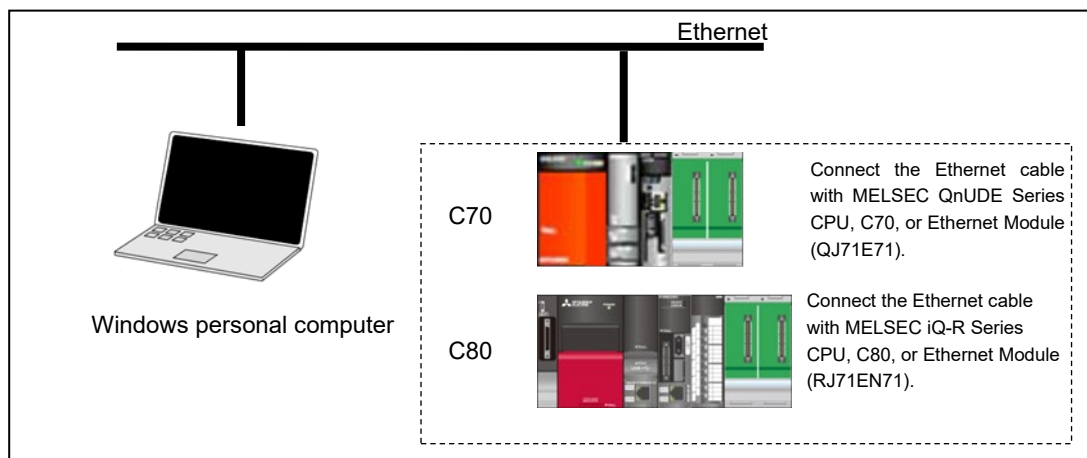


Figure1-3 Ethernet connection with C70/C80

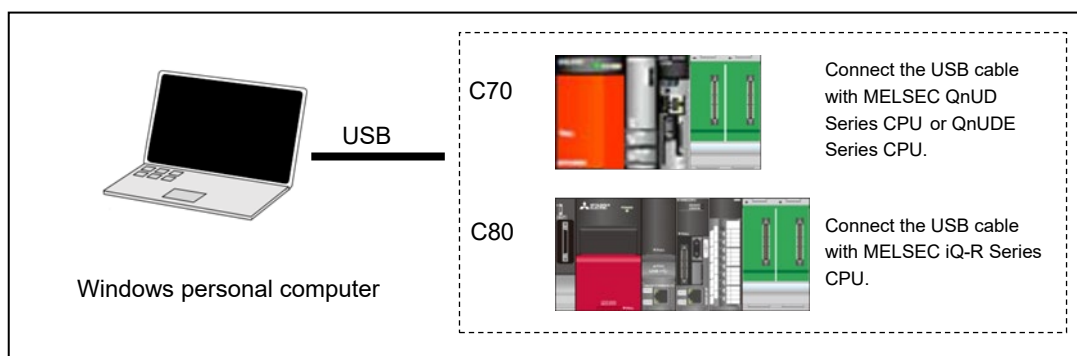


Figure1-4 USB connection with C70/C80

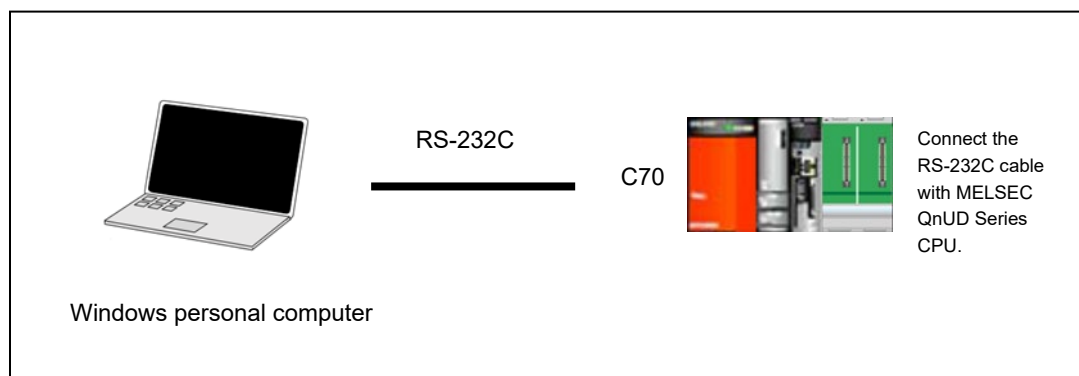


Figure1-5 RS-232C connection with C70

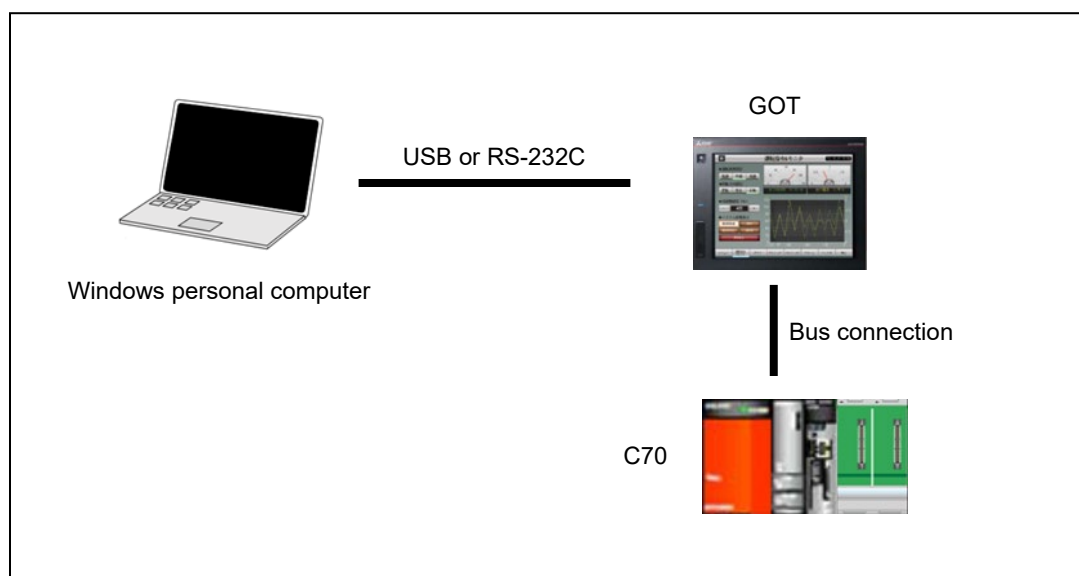


Figure1-6 C70 and GOT (bus connection) transparent

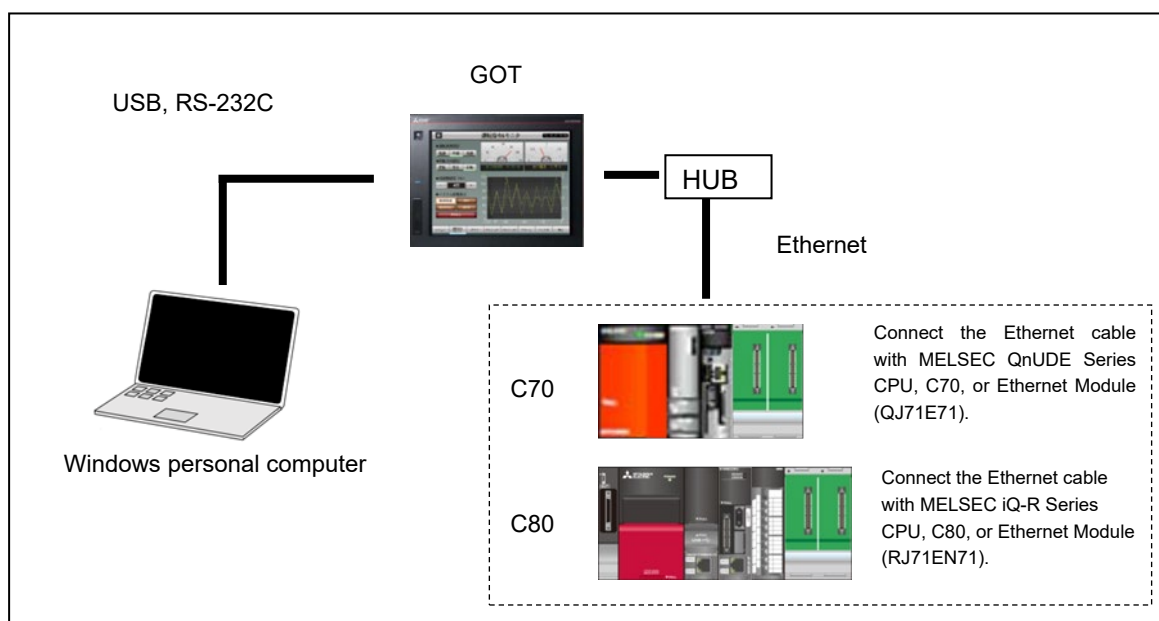


Figure1-7 C70/C80 and GOT (Ethernet) transparent

1.6 Installation

Execute the installer which is supplied with the product.

Note:

When using the product with the Windows 10 PC, enable ".NET Framework3.5 (includes .NET 2.0 and 3.0)". When ".NET Framework3.5 (includes .NET 2.0 and 3.0)" is disabled, an error dialog is displayed when the product is installed, and some incorrect operation may occur after the installation.

When an error occurs, reinstall the product after enabling ".NET Framework3.5 (includes .NET 2.0 and 3.0)" with the following procedures.

(1) Insert the Windows 10 installation medium to a PC.

Contact with the PC manufacturer when there is no Windows 10 installation media.

(2) Execute a command prompt as an administrator from the start menu.

(3) Execute one of the following commands from the command prompt.

Dism/online/enable-feature/featurename:NetFx3/All/Source:x:\sources\sxs\LimitAccess

Dism/online/enable-feature/featurename:NetFx3/All/Source:x:\sources\sxs\LimitAccess

For "x:", specify the drive where a Windows 10 installation medium is inserted.

1.6.1 Directory structure after installation

Directory structure after installation is as follows.

When the EZSocket has already been installed by any arbitrary software, the EZSocket may be stored in the different directory.

"Program Files" or "Program Files" (x86)

└ EZSocket (Default installation directory)

├ EasySocket

├ EZSCommon

├ EZSocketGOT

├ EZSocketNc

| └ CommServer For system use

| └ Include Header files for development are stored.

| └ Paramter For system use

| └ Samples Sample programs are stored.

| └ tools For system use

└ RuntimeInstaller Redistributable installer for run-time library is stored.

└ redistributable-setup.exe (Redistributable installer for run-time library)

• Windows

└ melcfg.ini Communication setting file (for system use)

1.7 Redistributable Installer for Run-time Library

When you install the product, a redistributable installer for run-time library "redistributable-setup.exe" is stored in the "RuntimeInstaller" folder in the installation directory. The said installer is to be redistributed or is to be added to the application installer of the product, which allows the run-time library to be copied to any other person's environment. For details of the license by redistributing the installer, refer to the license agreement of the product.

1.8 Header Files for Development

To create an application using this product with VC++, VB or VBA, the following appropriate include files or modules are required. The table below shows the default folders of when the installation is made in the C drive from DVD-ROM.

	VC++	VB or VBA
Installation destination folder	%ProgramFiles%\EZSocket\EZSocketNc\include\Vc	%ProgramFiles%\EZSocket\EZSocketNc\include\Vb
File	EZSocketNc.h EZSocketNcStr.h EZSocketNc_i.c EZSocketNcDef.h EZSocketNcErr.h EZSocketCommonErr.h	EZNcDef.bas EZNcErr.bas EZComErr.bas

To use the product with VC++, include necessary header file in the table above in the project.

```
#include "EZSocketNc.h"           .....Header file for method definitions
                                   * "EZSocketNc_i.c" is also included by this header file, so you
                                   do not need to include "EZSocketNc_i.c" separately using
                                   any user application.

#include "EZSocketNcStr.h"        .....Header file for structure definitions
#include "EZSocketNcDef.h"        .....Header file for miscellaneous definitions
#include "EZSocketNcErr.h"        .....Header file for error definitions
#include "EZSocketCommonErr.h"    .....Header file for EZSocket common error
```

To use the product with VB or VBA, include "bas" file in the table above.

Dedicated definition files for VC# are not included in the software. Define the value, which is used in each method, in the program as necessary.

1.9 Interface

The product provides two types of interface as DLL inprocess server: custom interface and automation interface. The two types of interface have similar data access functions.

The custom interface works well with VC++ and the automation interface works well with VC#, VB and VBA. The interface can be selected according to the development language.

As the interfaces of the product are based on Microsoft Component Object Model (**COM**), general knowledge about **COM** will be necessary to use the interfaces from the application. Note that general explanation about **COM** is not described in this manual.

1.9.1 Custom interface

See the custom interface list below.

Component	Interface	Category
EZNcCommunication	IEZNcCommunication3	Communication
	IEZNcSystem2	NC System
	IEZNcPosition	Position
	IEZNcCommand2	Command
	IEZNcProgram2	Program
	IEZNcTime	Time
	IEZNcAxisMonitor	Axis monitor
	IEZNcRunStatus	Operation status
	IEZNcFile6	File
	IEZNcCommonVariable2	Common variable
	IEZNcLocalVariable2	Local variable
	IEZNcTool3	Tool
	IEZNcATC3	ATC
	IEZNcParameter3	Parameter
	IEZNcOperation	Operation
	IEZNcDevice	Programmable controller device
EZNcSubFunction		
	IEZNcSubFunction3	Subfunction

(Note 1) Though the interface name may change due to version upgrade, the former interface can still be used since the new interface inherits its former version.

Example) IEZNcFile5 → IEZNcFile6

Note that the former interface is for backward compatibility and the new interface is required to use the latest version of product.

1.9.2 Automation interface

See the automation interface list below.

They are handy for use with VB since all functions are contained in a single automation interface.

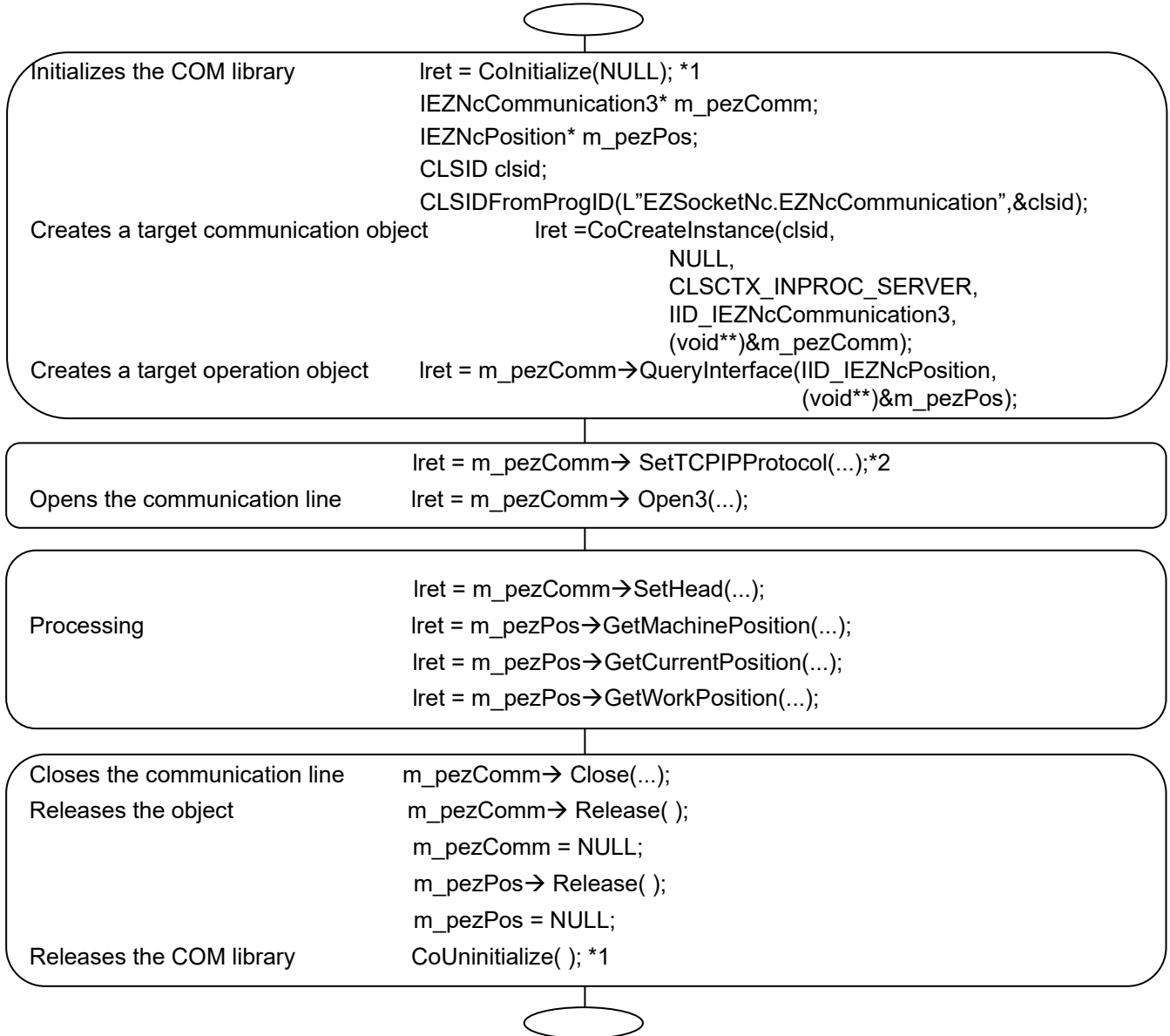
Component	Interface	Category
DispEZNcCommunication	IDispEZNcCommunication	Communication
		NC System
		Position
		Command
		Program
		Time
		Axis monitor
		Operation status
		File
		Common variable
		Local variable
		Tool
		ATC
		Parameter
		Operation
		Programmable controller device
DispEZNcSubFunction	IDispEZNcSubFunction	Subfunction

(Note 1) C70/C80 does not support automation interface.

1.10 Program Flow

1.10.1 VC++ program flow

This section explains the program flow outline to create an application for M700/M800 series or C70/C80 using the custom interface with VC++.



*1 In the thread using this product, call the COM library function `CoInitialize()` before using this product and `CoUninitialize( )` after using this product. When finishing the use of the objects of this product, call `Release()` to release the objects (decrement the reference count).

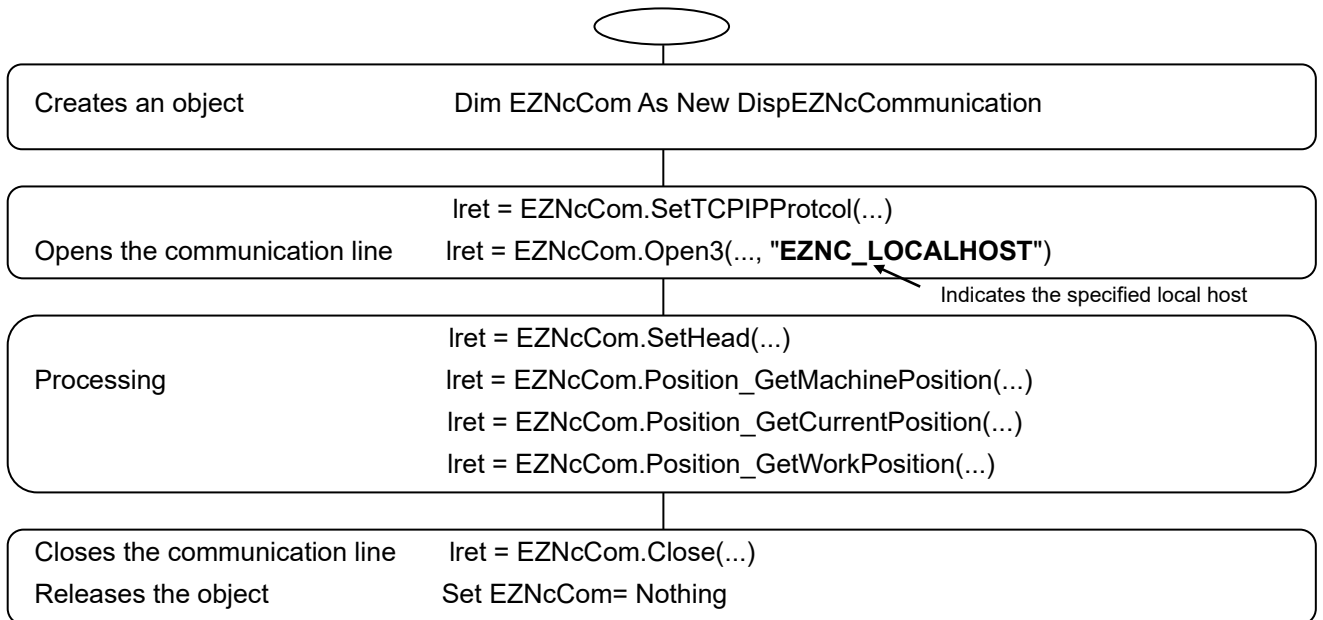
*2 When creating an M700/M800 series application, call `SetTCPIPProtocol` before `Open`. When creating a C70/C80 application, call `SetMelsecProtocol` instead of `SetTCPIPProtocol`.

1.10.2 VB program flow

This section explains the program flow outline using the automation interface with VB.

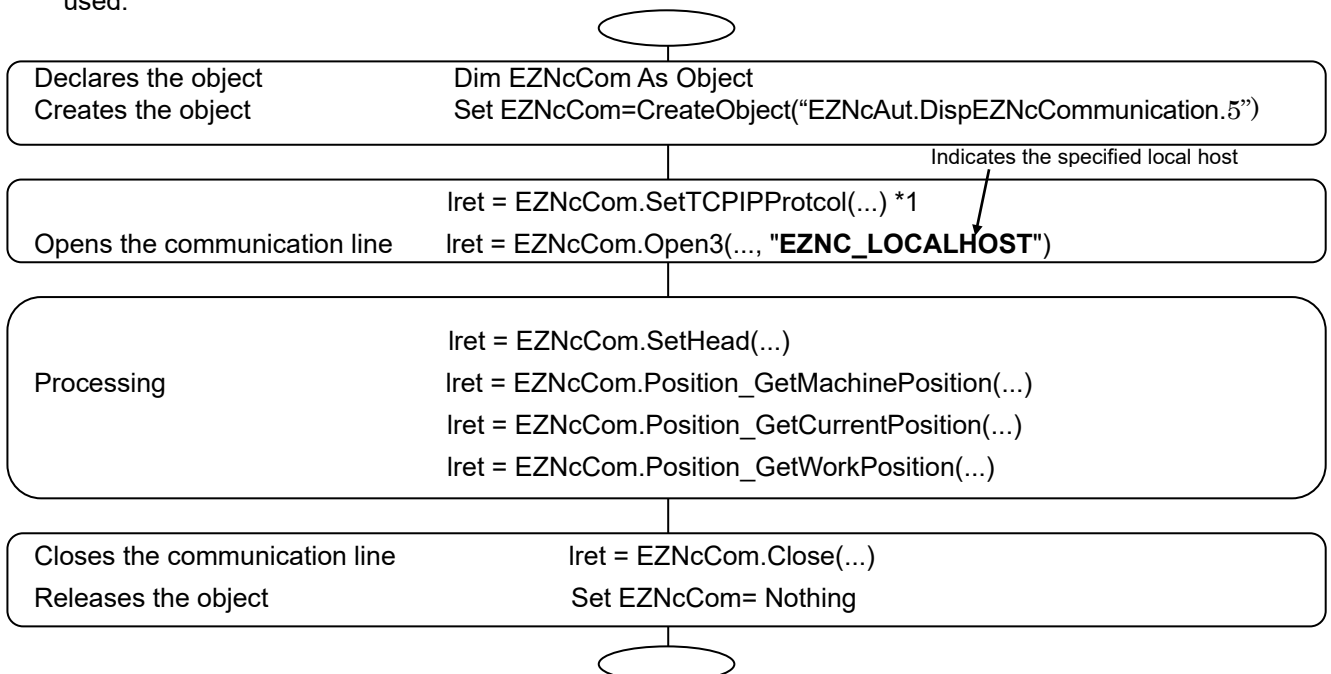
(1) Early binding call

Early binding requires reference of type library of the automation interface to be set in advance.



(2) Late binding call

Late binding does not require reference setting. Note that the object browser function with VB cannot be used.



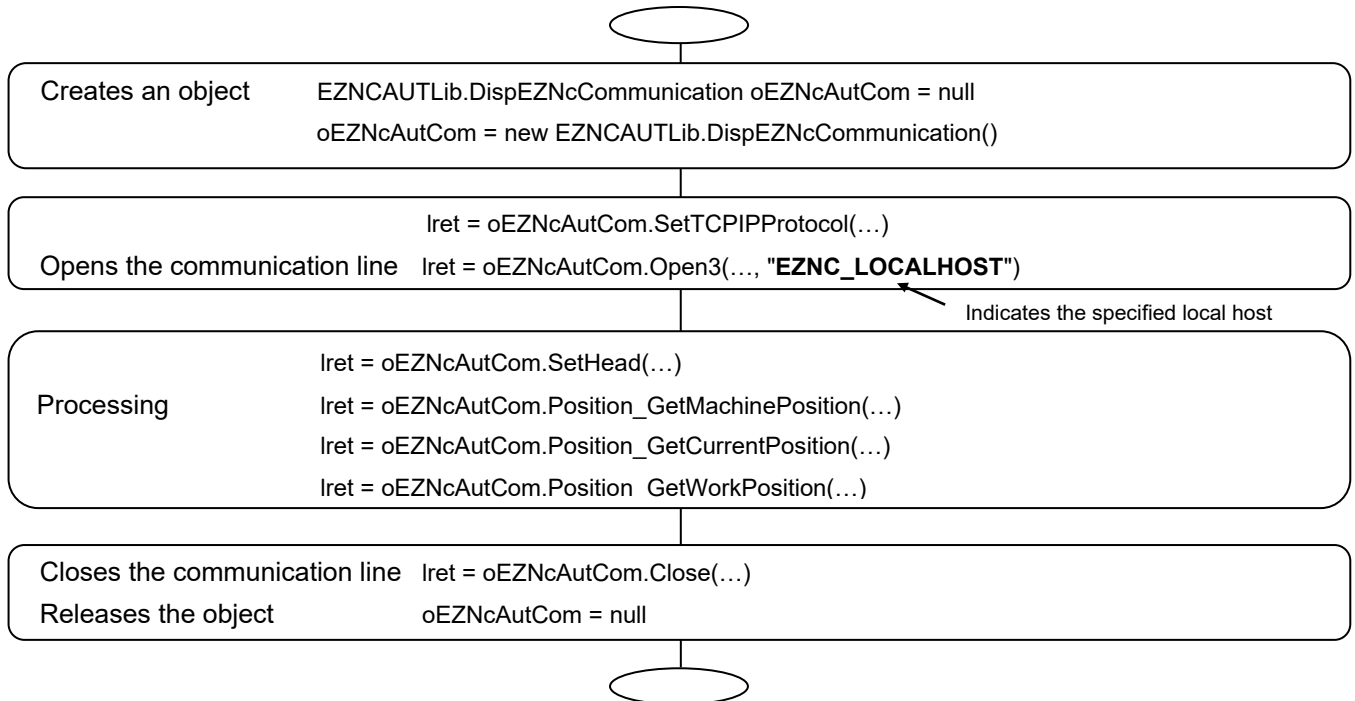
*1 When creating an M700/M800 series application, call SetTCPIPProtocol before Open. When creating a C70 application, call SetMelsecProtocol instead of SetTCPIPProtocol.

1.10.3 VC# Program flow

This section explains the program flow outline to create an application for M700/M800 series using the automation interface with VC#.

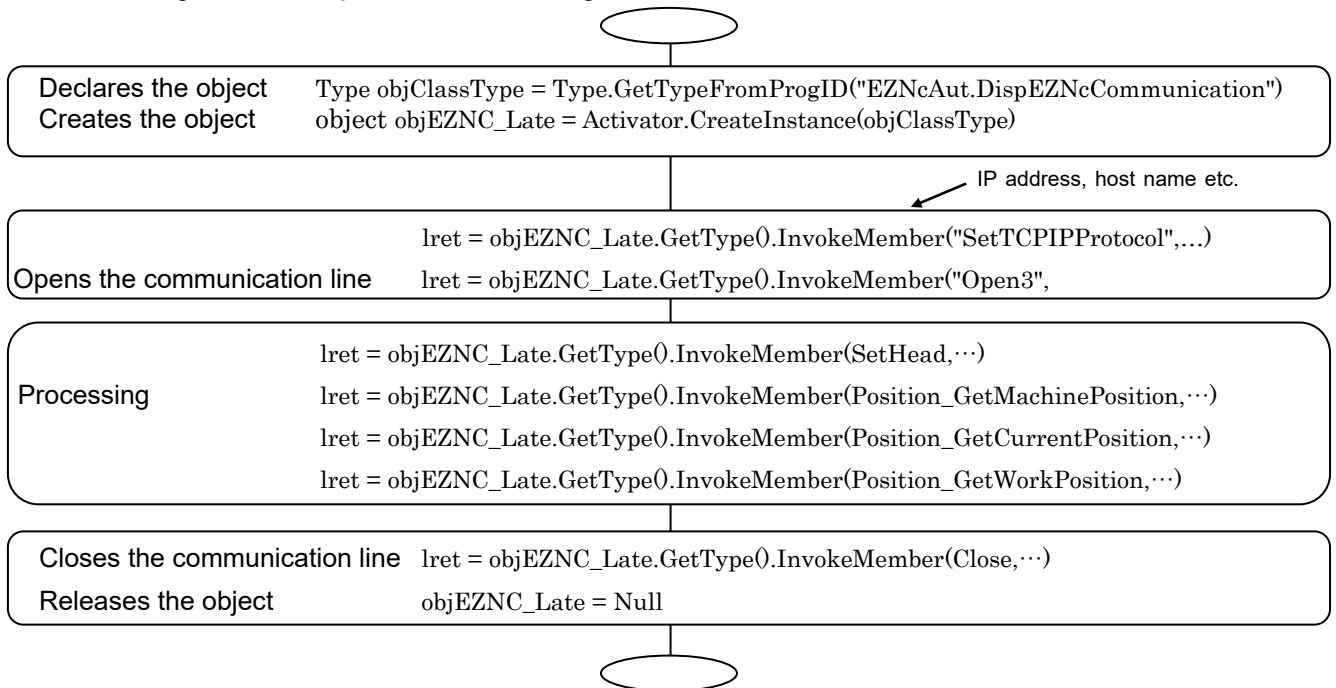
(1) Early binding call

Early binding requires reference of type library of the automation interface to be set in advance.



(2) Late binding call

Late binding does not require reference settings.



The method which uses output argument is required the following settings.

- When VC#7.0 or later version is used for the development environment, set "out" to the output argument.

(Example) Gets workpiece coordinate system offset amount

```
object[] getwork_arg = { 1, 54, out pdoffset};
lret = objEZNC_Late.GetType().InvokeMember("Tool_GetToolWorkOffSet",
BindingFlags.InvokeMethod, null, objEZNC_Late, getwork_arg);
```

- When the earlier version than VC#7.0 is used for the development environment, the ParameterModifier argument of InvokeMember() is used to output the data for reference.

(Example) Gets workpiece coordinate system offset amount

```
object[] getwork_arg = { 1, 54, pdoffset};
ParameterModifier getwork_mod = new ParameterModifier(3);
getwork_mod[0] = false; // In
getwork_mod[1] = false; // In
getwork_mod[2] = true; // Out
ParameterModifier[] getwork_mods = { getwork_mod };
lret = objEZNC_Late.GetType().InvokeMember("Tool_GetToolWorkOffSet",
BindingFlags.InvokeMethod, null, objEZNC_Late, getwork_arg, getwork_mods, null, null );
```

2. I/F DETAILED SPECIFICATIONS

2.1 Common Items

(1) Character code

All character string parameters in the interface of this product use UNICODE.

(2) Character string handling

With the method for handling character string data, if returning the character string data to the application, memory is allocated on the product side. Character string data memory that is no longer needed is freed up on the application side. If it is not freed up, a memory leak may occur.

To develop applications with custom interfaces for use with VC++, use **CoTaskMemFree()** to explicitly free up character string area memory.

To develop applications with automation interfaces for use with VC++, use **SysFreeString()** to explicitly free up character string area memory.

(3) Handling of common error codes

The method return value will be either for successful completion (**S_OK**) or failure (**S_FALSE**) of the method. A detailed method error is returned as an argument.

Commonly used error code include the following.

EZ_ERR_NOT_OPEN: Communication lines are not open.

EZ_ERR_DOUBLE_OPEN: Double open error

EZ_ERR_DATA_TYPE: Invalid argument data type

EZ_ERR_DATA_RANGE: Invalid argument data range

EZ_ERR_NOT_SUPPORT: Not supported

EZ_ERR_NULLPTR: Argument is **NULL** pointer

(4) Error handling when calling an I/F incompatible with the model

If the I/F of a function not supported by the model is called, **EZ_ERR_NOT_SUPPORT** is output as an unsupported error code.

(5) Notations used in Chapter 2.4 and later in this document

The meaning of notations used on pages describing individual method detailed specifications in Chapter 2.4 and later are as follows.

"□ Argument" section: Describes the argument specifications of the method.

"□ Return value" section: Describes the return values of the method.

"□ Function" section: Describes the general function of the method.

"□ Reference" section: Describes the methods related to the method.

"□ Specification" section: Indicates that specification of the system number (including PLC axis systems) and axis numbers is necessary when executing the method. If used without specifying, note that operation will not be guaranteed.

Required specification details are abbreviated with the following marks.

System: Set the part system number. Use SetHead() for the part system number setting.

PLC axis: Set the PLC axis system number. Use SetHead() for PLC axis system number setting.

Axis number: Set the axis number.

In addition, even if the **System** mark is listed, sometimes system specification is not required depending on the argument value. For more information, see the supplementary information described in the "□ Specification" section on an individual method's page.

(6) Restrictions on methods that require part system setting

For disabled systems, if a method that requires part system setting is executed, the method's execution result will be null. Check beforehand whether the set system is an enabled system.

In addition, system 1 will be set after a line opens.

(7) File name specification

In this product, the Mitsubishi CNC (*1) is regarded as a single drive, and the various data on the NC control unit (machining program, tool offset, etc.) are treated as a file. To access an NC control unit file, set the file name according to the following form unless otherwise noted.

Drive name + ":" + \directory name + \file name

Make sure to set the file name with an absolute path.*2

Also, the drive name should correspond to an NC control unit number as follows.

NC control unit number	Drive name (NC memory)
01	M01
02	M02
03	M03
:	:
:	:
FF	MFF

*1: Described in the following sections of this manual as "NC control unit".

*2: Set the file name with upper-case characters.

In English-version operating systems, set by changing "\" to a backslash.

(8) Restrictions on calls from multiple threads of the C70

With the C70, if the number of threads the product uses exceeds 1000, error codes may no longer be output correctly.

Keep in mind the number of threads the product uses when creating a C70 application.

(9) Reading/writing methods of device

The following methods can be used to read/write to the NC devices.

M800 series supports multi-project function and can read/write devices to the multiple projects.

Operation	Method name
Set device	SetDevice
Read/write device	ReadDevice2 WriteDevice2
Batch read/write devices	ReadBlockDevice2 WriteBlockDevice2
Random read/write device	ReadRandomDevice2 WriteRandomDevice2

Read/write the devices by performing the following steps.

When multi-project function is enabled

- Reading/writing device method using ReadDevice2() or WriteDevice2()
 - (a) Defines the device to be used in the user PLC from SetDevice().
 - (b) Read or write the device of the project number set in ReadDevice2() or WriteDevice2() with an argument.
- Reading/writing device method using ReadBlockDevice2() or WriteBlockDevice2()
 - (a) Read or write in batch the device of the project number set in ReadBlockDevice2() or WriteBlockDevice2() with an argument.
- Reading/writing device method using ReadRandomDevice2() or WriteRandomDevice2()
 - (a) Randomly read or write the device of the project number set in ReadRandomDevice2() or WriteRandomDevice2() with an argument.

When multi-project function is disabled

- Reading/writing device method using ReadDevice2() or WriteDevice2()
 - (a) Defines the part system of the NC axis and PLC axis from SetHead().
 - (b) Defines the part system of the NC axis and PLC axis from SetDevice().
 - (c) Read or write the device of the project number set in ReadDevice2() or WriteDevice2() with an argument.
- Reading/writing device method using ReadBlockDevice2() or WriteBlockDevice2()
 - (a) Defines the part system of the NC axis and PLC axis from SetHead().
 - (b) Read or write in batch the device of the project number set in ReadBlockDevice2() or WriteBlockDevice2() with an argument.
- Reading/writing device method using ReadRandomDevice2() or WriteRandomDevice2()
 - (a) Defines the part system of the NC axis and PLC axis from SetHead().
 - (b) Randomly read or write the device of the project number set in ReadRandomDevice2() or WriteRandomDevice2() with an argument.

2.2 Method List

○: Supported, -: Not supported

Chapter number	Class (Interface)	Method	Function	Target			
				M700 series	M800 series	C70 ¹	C80 ¹
2.4	IEZNCCommunication3 (Communication)	Open2	Open line	○	○	○	-
		Open3	Open line	○	○	○	○
		Close	Close line	○	○	○	○
		Close2	Close line	○	○	-	-
		SetHead	Set part system number	○	○	○	○
		GetHead	Get part system number	○	○	○	○
		SetTCP/IPProtocol	Set TCP/IP communication protocol	○	○	-	-
		SetMelsecProtocol	Set MELSEC communication protocol	-	-	○	○
		SetNCMode	Set NC mode	○	○	-	-
		GetNCMode	Get NC mode	○	○	-	-
		SetPathLoadModule	Set NCAPI load module path	○	○	-	-
		GetPathLoadModule	Get NCAPI load module path	○	○	-	-
2.5	IEZNCSystem2 (NC System)	GetVersion	Get NC system S/W number, name	○	○	○	○
		GetSystemInformation	Get NC system information	○	○	○	○
		GetSerialNo	Get NC serial No. (product No.)	○	○	-	○
		GetAlarm	Get alarm information	○	-	○	-
		GetAlarm2	Get alarm information	○	○	○	○
		GetAlarm3	Get alarm information	○	○	-	○
2.6	IEZNCPosition (Position)	GetWorkPosition	Get workpiece coordinate position (skip ON-compliant)	○	○	○	-
		GetWorkPosition2	Get workpiece coordinate position (skip ON-compliant)	○	○	○	-
		GetMachinePosition	Get machine position (skip ON-compliant)	○	○	○	○
		GetMachinePosition2	Get machine position (skip ON-compliant)	○	○	○	○
		GetCurrentPosition	Get current position	○	○	○	-
		GetDistance	Get remaining command (skip ON-compliant)	○	○	○	○
		GetDistance2	Get remaining command (skip ON-compliant)	○	○	○	○
		GetNextDistance	Get next command	○	○	○	-
		GetFeedSpeed	Get feed speed	○	○	○	○
		GetTCPSpeed	Get tip speed	○	○	-	-
		GetManualOverlap	Get manual interrupt amount	○	○	○	-
		GetManualOverlap2	Get manual interrupt amount	○	○	○	-
		GetProgramPosition	Get program position	○	○	○	-
		GetProgramPosition3	Get program position	○	○	○	-
		GetTCPMachinePosition	Get tip machine position	○	○	-	-
		GetTCPWorkPosition	Get tip workpiece position	○	○	-	-
		GetFeedbackPosition	Get feedback position	○	○	-	-
		GetTableCoordinationPosition	Get table coordinate system counter	○	○	-	-
		GetWorkInstallationPosition	Get workpiece installation coordinate system counter	○	○	-	-
		GetInclinedSurfacePosition	Get inclined surface coordinate system counter	○	○	-	-
2.7	IEZNCCommand2 (Command)	GetGCodeCommand	Get G code modal command value	○	○	○	○
		GetToolCommand	Get tool compensation number	○	○	○	-

Chapter number	Class (Interface)	Method	Function	Target			
				M700 series	M800 series	C70 ¹	C80 ¹
		GetFeedCommand	Get feed speed command value	○	○	○	-
		GetCommand2	Get M/S/T/B function command modal value	○	○	○	-
		SetCommand2	Set manual numerical value command for M/S/T/B functions	○	○	○	-
2.8	IEZNCProgram2 (Program end)	CurrentBlockRead	Read current program block	○	○	○	-
		GetProgramNumber2	Get program number (main, sub)	○	○	○	-
		GetSequenceNumber	Get sequence number (main, sub)	○	○	○	-
		GetBlockNumber	Get block number (main, sub)	○	○	○	○
		GetSubProLevel	Get subprogram call level	○	○	○	○
		GetInformation	Get program information	○	○	○	-
		GetCurrentBlockByByte	Get number of bytes from start of program	○	○	-	-
2.9	IEZNCTime (Time)	GetClockData	Get date and time	○	○	○	-
		SetClockData	Set date and time	○	○	○	-
		GetAliveTime	Get power-on time	○	○	○	-
		SetAliveTime	Set power-on time	○	○	○	-
		GetRunTime	Get automatic operation time	○	○	○	-
		SetRunTime	Set automatic operation time	○	○	○	-
		GetStartTime	Get automatic start time	○	○	○	-
		SetStartTime	Set automatic start time	○	○	○	-
		GetEstimateTime	Get external integration time (1, 2)	○	○	○	-
		SetEstimateTime	Set external integration time (1, 2)	○	○	○	-
2.10	IEZNCAxisMonitor (Axis monitor)	GetServoMonitor	Get servo monitor	○	○	○	○
		GetServoMonitorEx	Get servo monitor	○	○	-	○
		GetServoVersion	Get servo axis unit information	○	○	○	-
		GetServoDiagnosis	Get servo diagnostics information	○	○	○	-
		GetServoMotorTemperature	Get servo motor temperature	-	○	-	○
		GetPowerVersion	Get power supply version information	○	○	○	-
		GetPowerDiagnosis	Get power supply diagnostics information	○	○	○	-
		GetSpindleMonitor	Monitor spindle	○	○	○	○
		GetSpindleVersion	Get spindle unit version information	○	○	○	-
		GetSpindleDiagnosis	Get spindle diagnostics information	○	○	○	-
		GetSpindleMotorTemperature	Get spindle motor temperature	-	○	-	○
		GetAbsPositionMonitor	Get absolute position monitor information	○	○	○	-
		GetAuxAxisMonitor	Get auxiliary axis monitor information	○	○	-	-
		GetAuxAxisVersion	Get auxiliary axis version information	○	○	-	-
		GetDwellTime	Get remaining dwell time	○	○	○	○
		GetPowerConsumption	Get current power consumption	-	○	-	-
		GetIntegralPower	Get integral power	-	○	-	-
		GetAccumulatedTravelDistance	Get accumulated travel distance	-	○	-	○
2.11	IEZNCRunStatus (Operation status)	GetInvalidStatus	Get disabled status	○	○	○	-
		GetCommandStatus	Get operation command status	○	○	○	-

Chapter number	Class (Interface)	Method	Function	Target			
				M700 series	M800 series	C70 ¹	C80 ¹
		GetCuttingMode	Get cutting mode status	○	○	○	-
		GetAxisStatus	Get servo axis status	○	○	○	-
		GetRunStatus	Get operation status	○	○	○	○
		GetCycleTime	Get cycle time	-	○	-	○
2.12	IEZNCFile6 (File)	FindDir2	Search directory	○	○	○	○
		FindNextDir2	Search next directory	○	○	○	○
		ResetDir	Terminate directory search	○	○	○	○
		CreateDir	Create directory	○	○	-	-
		DeleteDir	Delete directory	○	○	-	-
		Copy2	Copy file	○	○	○	○
		Delete2	Delete file	○	○	○	○
		Rename2	Change file name	○	○	○	○
		GetDriveInformation	Get drive information	○	○	○	○
		GetDriveSize	Get free drive space	○	○	○	-
		GetDriveSize2	Get free drive space	-	○	-	-
		OpenFile3	Open file	○	○	○	○
		OpenFile4	Open file	○	○	○	○
		CloseFile2	Close file	○	○	○	○
		CloseFile3	Close file	○	○	○	○
		AbortFile2	Force close file	○	○	○	-
		AbortFile3	Force close file	○	○	○	-
		ReadFile2	Read file	○	○	○	○
		ReadFile3	Read file	○	○	○	○
		WriteFile	Write file	○	○	○	○
		ForceReadFile	Force read file	-	-	○	-
		SpecialReadFile	Special read file	○	○	-	○
		OpenNCFile2	Open machining program dedicated file	○	○	-	-
		OpenNCFile3	Open machining program dedicated file	○	○	-	-
		CloseNCFile2	Close machining program dedicated file	○	○	-	-
		CloseNCFile3	Close machining program dedicated file	○	○	-	-
		AbortNCFile2	Force close machining program dedicated file	○	○	-	-
		AbortNCFile3	Force close machining program dedicated file	○	○	-	-
		ReadNCFile2	Read machining program dedicated file	○	○	-	-
		ReadNCFile3	Read machining program dedicated file	○	○	-	-
		WriteNCFile	Write machining program dedicated file	○	○	-	-
2.13	IEZNCCommonVariable2 (Common variable)	CommonVRead	Read common variables (#100, #500)	○	○	○	○
		CommonVWrite	Write common variables (#100, #500)	○	○	○	-
		GetSize	Get number of sets for common variables (#100, #500)	○	○	○	-
		GetName	Get names of common variables (#500 to 519)	○	○	○	-
		SetName	Set names of common variables (#500 to 519)	○	○	○	-
		GetCVNullData	Get value when no numerical value is set	○	○	○	○
2.14	IEZNCLocalVariable2 (Local variable)	LocalVRead	Read local variable	○	○	○	-
		GetMacroLevel	Get macro subprogram call level (level 0 to 4)	○	○	○	-
		GetLVNullData	Get value when no numerical value is set	○	○	○	-
2.15	IEZNCTool3 (Tool)	GetToolSetSize	Get number of sets for tool offset	○	○	○	-

Chapter number	Class (Interface)	Method	Function	Target			
				M700 series	M800 series	C70 ¹	C80 ¹
		GetType	Get tool offset type	○	○	○	-
		GetOffset3	Get tool offset amount	○	○	○	-
		SetOffset2	Set tool offset amount	○	○	○	-
		GetToolWorkOffset	Get workpiece coordinate system offset (#54 to 60)	○	○	○	○
		GetToolWorkOffset2	Get workpiece coordinate system offset (#54 to 60)	○	○	○	-
		SetToolWorkOffset	Set workpiece coordinate system offset (#54 to 60)	○	○	○	-
		SetToolWorkOffset2	Set workpiece coordinate system offset	○	○	○	-
		GetToolWorkOffsetEX	Get extended workpiece coordinate system offset	○	○	-	-
		SetToolWorkOffsetEX	Set extended workpiece coordinate system offset	○	○	-	-
		GetSurface	Get reference surface height	○	○	○	-
		GetSurface2	Get reference surface height	○	○	○	-
		SetSurface	Set reference surface height	○	○	○	-
		GetToolLifeType2	Get tool life management method	○	○	○	-
		SetToolLifeType2	Select tool life management method	○	○	○	-
		GetToolLifeGroupList	Get Tool Life management group No. list	○	○	○	-
		ChangeToolLifeGroup	Change tool life management group No.	○	○	○	-
		DeleteToolLifeGroup	Delete tool life management group No.	○	○	○	-
		GetToolLifeToolNoList	Get tool No. list within tool life management group	○	○	○	-
		AddToolLifeToolNo	Add tool No. within tool life management group	○	○	○	-
		ChangeToolLifeToolNo	Change tool life management tool No.	○	○	○	-
		DeleteToolLifeToolNo	Delete tool life management tool No.	○	○	○	-
		GetToolLifeValue	Get tool life management data	○	○	○	-
		SetToolLifeValue	Set individual tool life management data	○	○	○	-
		SetToolLifeValue2	Set tool life management data	○	○	○	-
2.16	IEZNCATC3 (ATC)	GetMGNControl	Get ATC tool registration control parameter	○	○	○	-
		GetMGNSize	Get total number of sets of magazine pots for ATC tool registration	○	○	○	-
		GetMGNSize2	Get number of sets of pots for each magazine for ATC tool registration	○	○	○	-
		GetMGNReady2	Get tool number for ATC tool registration	○	○	○	-
		GetMGNPot	Get tool number for magazine pot for ATC tool registration	○	○	○	-
		GetMGNPot3	Get tool number for each magazine pot for ATC tool registration	○	○	○	-
		SetMGNPot	Set tool number for magazine pots for ATC tool registration	○	○	○	-

Chapter number	Class (Interface)	Method	Function	Target			
				M700 series	M800 series	C70*1	C80*1
		SetMGNPot3	Set tool number for each magazine pot for ATC tool registration	○	○	○	-
		GetMGNAux	Get user PLC interface for ATC tool registration	○	○	○	-
		SetMGNAux	Set user PLC interface for ATC tool registration	○	○	○	-
2.17	IEZNCParameter3 (Parameter)	GetParameterData2	Get parameters	○	-	○	-
		GetParameterData3	Get parameters	○	○	○	-
		SetParameterData2	Set parameters	○	-	○	-
		SetParameterData3	Set parameters	○	○	○	-
2.18	IEZNCOperation (Operation)	Search	Operation search	○	○	○	-
		Run	Start PLC program	○	○	○	-
		Stop	Stop PLC program	○	○	○	-
2.19	IEZNCDevice (Device)	SetDevice	Set device	○	○	-	-
		DeleteDeviceAll	Delete all devices	○	○	-	-
		ReadDevice2	Read device	○	○	-	-
		WriteDevice2	Write device	○	○	-	-
		ReadBlockDevice2	Batch read devices	○	○	-	-
		WriteBlockDevice2	Batch write devices	○ *2	○	-	-
		ReadRandomDevice2	Random read device	○	○	-	-
		WriteRandomDevice2	Random write device	○ *2	○	-	-
2.20	IEZNCSubFunction3 (Subfunction)	ChangeInit2	Initialize subfunction	○	○	○	-
		GetToolWorkOffsetOfFile	Get data from workpiece offset file	○	○	○	-
		SetToolWorkOffsetOfFile	Set data to workpiece offset file	○	○	○	-

*1) C70/C80 does not support automation interface.

*2) Only M700V/M70V is supported.

2.3 Precautions for Using Methods

2.3.1 When an unknown error code responses or memory access error occurs

Check that the product itself or run-time library is installed. And also check the version of the product is appropriate.

2.3.2 Precautions for opening/closing of NC control unit system line

- (1) When the product is to be connected to multiple NC control units, designate the different unit No. (setting range of 1 to 255) to each control unit.
- (2) For M700/M800 series, "NC control unit No." whose system line is opened becomes empty No. by designating "EZNC_RESET_SIMPLE" or "EZNC_RESET_SIMPLE" with "Close2()".

2.4 IEZNcCommunication3 Interface

C70

M700

M800

IEZNcCommunication3::Open2

Open line

□ Custom call procedure

```
HRESULT      Open2 (
                LONG ISystemType,           // (I) NC system type
                LONG IMachine,              // (I) NC control unit
                LONG ITimeOut,              // (I) Communication time-out value
                LONG* pIRet                  // (O) Error code
            )
```

□ Automation call procedure

```
Open2 (
    ISystemType As LONG,           // (I) NC system type
    IMachine As LONG,              // (I) NC control unit
    ITimeOut As LONG,              // (I) Communication time-out value
    IpcwszHostName As STRING      // (I) Host name
) As LONG                         // (O) Error code
```

□ Argument	<i>ISystemType</i> : Sets the NC system type.	
	Value	Meaning
	EZNC_SYS_MELDASC70	With the C70, performs a line connection.
	EZNC_SYS_MELDAS700M	With M700 M systems, performs a line connection.
	EZNC_SYS_MELDAS700L	With M700 L system, performs a line connection.
	EZNC_SYS_MELDAS800M	With M800 M systems, performs a line connection.
	EZNC_SYS_MELDAS800L	With M800 L system, performs a line connection.

For the M700/M800 series, specify the NC system type by combining the following value. Close the line by designating EZNC_RESET_SIMPLE with Close2(). Regardless of the model, the settings of the all lines should be the same.

Value	Meaning
EZNC_SYS_MULTI	Enables to communicate with multi-thread.

(Example for use)

```
ISystemType = EZNC_SYS_MELDAS800M | EZNC_SYS_MULTI ;
```

IMachine: Sets the NC control unit number. When connecting to multiple Mitsubishi CNCs, specify the different NC control unit numbers for each Mitsubishi CNC. Value: 1 to 255

ITimeOut: Sets the communication time-out value. However, with C70, this value is disabled. The C70 communication time-out is set by *ITimeOut* in **SetMelsecProtocol()**.

Value	Meaning
1 to 3000	Time-out value (unit: 100 ms) (M700/M800 series is 10 or more, and if a time-out error occurs, increase the value.)

IpcwszHostName: Sets the NC system host name to connect. The IP address can also be specified.

When connecting to a local host, set **"EZNC_LOCALHOST"** as the character string.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_SYSFUNC_IOCTL_ADDR: Invalid NC control unit number

EZNC_SYSFUNC_IOCTL_NOTOPEN: Device is not open

EZNC_SYSFUNC_IOCTL_DATA: Invalid communication parameter data range

EZNC_COMM_NOTSETUP_PROTOCOL: TCP/IP communication has not been set (M700/M800 series only)

EZNC_COMM_NOTMODULE: No submodule

EZNC_COMM_CREATEPC: EZSocketPc objects cannot be created (C70 only)

EZNC_COMM_CANNOT_OPEN: When connecting to local host with automation call, host name **EZNC_LOCALHOST** has not been set.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Executes connection of the communication line for an **IEZNCCommunication3** object. For the M700/M800 series, before open, make sure to execute **SetTCPIPProtocol()** to configure the communication settings. Also, for the C70, before open, make sure to execute **SetMelsecProtocol()** to configure the communication settings. If it is not executed, an error will occur when **Open2()** is executed.
- Set **EZNC_SYS_MULTI** for **ISystemType** with the M700/M800 series. When **EZNC_SYS_MULTI** is set, multi-thread communication is enabled, and threads which have been made in the same process can communicate with different NCs simultaneously.
- For details, refer to the explanations below.

◆Multi-thread operation

When two threads in the same process are to be connected to the different NCs, each thread can communicate with NC simultaneously without wait time.

Thread (1) (a) Connect to the 1st NC with **Open3()**.

(b) Communication with the 1st NC is executed using any other method.

-> In this case, even if the thread (2) (b) is communicating, this thread can also communicate with the 1st NC without wait time.

Thread (2) (a) Connect to the 2nd NC with **Open3()**.

(b) Communication with the 2nd NC is executed using any other method.

-> In this case, even if the thread (1) (b) is communicating, this thread can also communicate with the 2nd NC without wait time.

The NC types which can use this function by combining with **EZNC_SYS_MULTI** are **EZNC_SYS_MELDAS700M**, **EZNC_SYS_MELDAS700L**, **EZNC_SYS_MELDAS800M** and **EZNC_SYS_MELDAS800L**. If you set **EZNC_SYS_MULTI** to any other models, the setting is ignored.

This function is not supported with C80. (**EZ_ERR_NOT_SUPPORT** is returned to *plRet*.)

(Note 1) When **EZNC_SYS_MULTI** is not set, closing the line may fail.

- **Reference** **Close()**, **SetTCPIPProtocol()**, **SetMelsecProtocol()**

- **Specifica-
tion**

IEZNCCommunication3::Open3**Open line**

□ Custom call procedure

```

HRESULT      Open3 (
                LONG ISystemType,           // (I) NC system type
                LONG IMachine,              // (I) NC control unit
                LONG ITimeOut,              // (I) Communication time-out value
                LONG* pIRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Open3 (
    ISystemType As LONG,           // (I) NC system type
    IMachine As LONG,             // (I) NC control unit
    ITimeOut As LONG,             // (I) Communication time-out value
    lpcwszHostName As STRING      // (I) Host name
) As LONG                        // (O) Error code

```

□ *ISystemType*: Sets the NC system type.

Argument	Value	Meaning
	EZNC_SYS_MELDASC70	With the C70, performs a line connection.
	EZNC_SYS_MELDAS700M	With M700 M systems, performs a line connection.
	EZNC_SYS_MELDAS700L	With M700 L system, performs a line connection.
	EZNC_SYS_MELDAS800M	With M800 M systems, performs a line connection.
	EZNC_SYS_MELDAS800L	With M800 L system, performs a line connection.
	EZNC_SYS_CNCC80	With the C80, performs a line connection.

For the M700/M800 series, specify the NC system type by combining the following value. Close the line by designating EZNC_RESET_SIMPLE with Close2(). Regardless of the model, the settings of the all lines should be the same.

Value	Meaning
EZNC_SYS_MULTI	Enables to communicate with multi-thread.

(Example for use)

```
ISytemType = EZNC_SYS_MELDAS800M | EZNC_SYS_MULTI ;
```

IMachine: Sets the NC control unit number. When connecting to multiple Mitsubishi CNCs, specify the different NC control unit numbers for each Mitsubishi CNC. Value: 1 to 255

ITimeOut: Sets the communication time-out value. However, with C70/C80, this value is disabled. The C70/C80 communication time-out is set by *ITimeOut* in **SetMelsecProtocol()**.

Value	Meaning
5 to 250	Time-out value (unit: 100 ms) (M700/M800 series is 10 or more, and if a time-out error occurs, increase the value.)

lpcwszHostName: Sets the NC system host name to connect. The IP address can also be specified.

When connecting to a local host, set **"EZNC_LOCALHOST"** as the character string.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_SYSFUNC_IOCTL_ADDR: Invalid NC control unit number

EZNC_SYSFUNC_IOCTL_NOTOPEN: Device is not open

EZNC_SYSFUNC_IOCTL_DATA: Invalid communication parameter data range

EZNC_COMM_NOTSETUP_PROTOCOL: TCP/IP communication has not been set (M700/M800 series only)

EZNC_COMM_NOTMODULE: No submodule

EZNC_COMM_CREATEPC: EZSocketPc objects cannot be created (C70 only)

EZNC_COMM_CANNOT_OPEN: When connecting to local host with automation call, host name EZNC_LOCALHOST has not been set.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Function Executes connection of the communication line for an IEZNCCommunication3 object. For the M700/M800 series, before open, make sure to execute SetTCPIPProtocol() to configure the communication settings. Also, for the C70/C80, before open, make sure to execute SetMelsecProtocol() to configure the communication settings. If it is not executed, an error will occur when Open3() is executed. Set EZNC_SYS_MULTI for ISystemType with the M700/M800 series. When EZNC_SYS_MULTI is set, multi-thread communication is enabled, and threads which have been made in the same process can communicate with different NCs simultaneously. For details, refer to the explanations below. ◆Multi-thread operation When two threads in the same process are to be connected to the different NCs, each thread can communicate with NC simultaneously without wait time. Thread (1) (a) Connect to the 1st NC with Open3(). (b) Communication with the 1st NC is executed using any other method. -> In this case, even if the thread (2) (b) is communicating, this thread can also communicate with the 1st NC without wait time. Thread (2) (a) Connect to the 2nd NC with Open3(). (b) Communication with the 2nd NC is executed using any other method. -> In this case, even if the thread (1) (b) is communicating, this thread can also communicate with the 2nd NC without wait time. The NC types which can use this function by combining with EZNC_SYS_MULTI are EZNC_SYS_MELDAS700M, EZNC_SYS_MELDAS700L, EZNC_SYS_MELDAS800M and EZNC_SYS_MELDAS800L. If you set EZNC_SYS_MULTI to any other models, the setting is ignored. (Note 1) The existing interface IEZNCCommunication3::Open2() can be used as a backward compatible model. Use this method when newly starting use of EZSocket. (Note 2) When EZNC_SYS_MULTI is not set, closing the line may fail.	
□	Close(), Close2(), SetTCPIPProtocol(), SetMelsecProtocol()	
□	Reference	
□	Specifica- tion	

IEZNCCommunication3::Close**Close line**□ **Custom call procedure**

```

HRESULT      Close (
                LONG* pIRet
            )
// (O) Error code

```

□ **Automation call procedure**

```

Close ( ) As LONG
// (O) Error code

```

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_SYSDFUNC_IOCTL_NOTOPEN: Device is not open

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Failure

□ **Function** The communication lines of **IEZNCCommunication3** objects connected by **Open2()** or **Open3()** will be disconnected.

For the M700/M800 series, even if the line is closed, the content set by **SetTCPIPProtocol()** is maintained, so **Open2()** or **Open3()** can be performed again.

For the C70/C80, even if the line is closed, the content set by **SetMelsecProtocol()** is maintained, so **Open2()** or **Open3()** can be performed again.

□ **Reference** **Open2()**, **Open3()**, **SetTCPIPProtocol()**, **SetMelsecProtocol()**

□ **Specifica-
tion**

IEZNCCommunication3::Close2**Close line**□ **Custom call procedure**

```

HRESULT      Close2 (
                LONG IReset,                // (I) NC reset method
                LONG* pIRet                // (O) Error code
            )

```

□ **Automation call procedure**

```

                Close2 (
                    IReset As LONG          // (I) NC reset method
                ) As LONG                  // (O) Error code

```

- **Argument** *IReset*: Sets how to reset the NC system. Regardless of which method is selected, communication line close process is performed.

Value	Meaning
EZNC_RESET_NONE	Does not reset the NC system.
EZNC_RESET_SIMPLE	Resets the NC system which is currently opened.
EZNC_RESET_ALL	Resets all the NC systems.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_SYSFUNC_IOCTL_NOTOPEN: Device is not open

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Failure

- **Function** The communication lines of **IEZNCCommunication3** objects connected by **Open3()** will be disconnected.

This is valid for the M700/M800 series. This function is not supported with C70/C80.

You can set how to reset the NC system which is being connected with *IReset*. Regardless of which method is selected, communication line close process is performed. This method is valid only when the argument of **Open3()** "ISystemType" is set to **EZNC_SYS_MULTIPLE**. Even though **EZNC_SYS_MULTIPLE** is not set, this method terminates properly. However the NC system is not reset. Resetting the NC system makes the NC card No. empty, which enables you to set the other IP address for the empty No. When the NC system is not reset, the NC card No. maintains as it is, thus the NC can be connected with the same IP address.

Information of resetting the NC system is to be shared with all the threads in the same process. When **EZNC_RESET_SIMPLE** is set, the reset process is performed to the NC system which is currently being connected. However, if the other thread is connected to the same NC system, the thread cannot communicate with the NC. When **EZNC_RESET_ALL** is set, all the communications by the other threads stop. Note that monitor the connecting status of each thread, and perform the NC reset only when it is needed.

For the M700/M800 series, even if the line is closed, the content set by **SetTCPIPProtocol()** is maintained, so **Open3()** can be performed again.

(Note 1) The existing interface **IEZNCCommunication3::Close()** can be used as a backward compatible model. Use this method when newly starting use of **EZSocket**.

□ **Open3(), SetTCPIPProtocol()**□ **Reference**□ **Specification**

IEZNCCommunication3::SetHead**Set part system number**□ **Custom call procedure**

```

HRESULT      SetHead(
                LONG IHead,           // (I) Part system number
                LONG* pIRet           // (O) Error code
            )

```

□ **Automation call procedure**

```

SetHead(
    IHead As LONG           // (I) Part system number.
) As LONG                  // (O) Error code

```

- **Argument** *IHead*: Sets the part system number. or the PLC axis system. Value: The range is determined by Mitsubishi CNC specifications (optional) and the machine manufacturer setting values (parameter). A value of 0 means "Not setting". When setting the PLC axis system, sets **EZNC_PLCAxis**.

pIRet: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Failure

- **Function** Set the NC axis / PLC axis system number.
 Set the part system number before executing a method that requires system number. setting.
 The set part system remains enabled until it is changed.
 System 1 is set after a line opens.

□ **Reference** **GetHead()**□ **Specifica-
tion**

IEZNCCommunication3::GetHead**Get part system number**□ **Custom call procedure**

```

HRESULT      GetHead(
                LONG* pIHead,           // (O) Part system number.
                LONG* pIRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

GetHead(
    pIHead As LONG*           // (O) Part system number.
) As LONG                    // (O) Error code

```

□ **Argument** *pIHead*: Returns the part system number. or the PLC axis system. Value: The range is determined by Mitsubishi CNC specifications (optional) and the machine manufacturer setting values (parameter). A value of 0 means "Not setting".
The PLC axis system returns **EZNC_PLCAxis**.

pIRet: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Failure

□ **Function** Gets the NC axis / PLC axis system number. The PLC axis system number gets **EZNC_PLCAxis**.

□ **Reference** **SetHead()**□ **Specification**

IEZNCCommunication3::SetTCPIPProtocol**Set TCP/IP communication protocol**

□ Custom call procedure

```

HRESULT      SetTCPIPProtocol (
                LPCOLESTR IpcwszIPAddress,           // (I) IP address
                LONG IPort,                          // (I) Port number
                LONG* pIRet                          // (O) Error code
            )

```

□ Automation call procedure

```

                SetTCPIPProtocol (
                    IpcwszIPAddress As STRING,        // (I) IP address
                    IPort As LONG                    // (I) Port number
                ) As LONG                             // (O) Error code

```

□ **Argument** *IpcwszIPAddress*: Sets the IP address of the M700/M800 series connection destination.
(Example: "192.168.1.1")

IPort: Sets the M700/M800 series connection destination port number.
For the port number, check the settings on the Mitsubishi CNC side. For the M700/M800 series, the port number becomes 683.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_COMM_ALREADYOPENED: Cannot be set because communication is already in progress

EZ_ERR_DATA_RANGE: Invalid IP address or port number

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Sets TCP/IP communication protocol.
For the M700/M800 series, call before performing **Open3()**. If it is not called, an error will occur when **Open3()** is executed.
The setting details are retained until the object is released by **Release()**.
Temporarily, until **Close()** is performed if **Open3()** is performed, re-setting with **SetTCPIPProtocol()** cannot be done. An error will occur.
This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pIRet.)

□ **Reference** **Open3(), Close()**

□ **Specifica-
tion**

IEZNCCommunication3::SetMelsecProtocol

Set MELSEC communication
protocol

□ Custom call procedure

```

HRESULT      SetMelsecProtocol (
                EZNCST_OPEN* pstOpen,           // (I) Line
                LONG* plRet                      // (O) Error code
            )

```

□ **Argument** *pstOpen*: A pointer referring to the **EZNCST_OPEN** structure which sets the parameters for the open. Refer to the following for members of the structure of **EZNCST_OPEN**.

(Note) The structure is created with the assumption of connection types other than Ethernet communication and serial communication. According to the following structure member descriptions, set 0 for unneeded parameters.

INetworkNumber: Sets the network number for the MELSEC communication.

Value	Meaning
0x00	Sets a host station or a host network of multi-drop destination
0x01	Sets another network of multi-drop destination
0x01 to 0xEF	Sets another station

IStationNumber: Sets the station number for the MELSEC communication

Value	Meaning
0xFF	Sets a host station
0x01 to 0x78	Sets another station
0x00	Sets a host network of multi-drop destination
0x01	Set another network of multi-drop destination

IUnitNumber: Sets the module number of the computer link (serial communication) module or the station number for a Qn communication system intelligent special module. However, sets "0x00" with the setting of a QnA series host station (module mounted to the host station CPU). This is disabled with computer link (serial communication) communication and a Qn intelligent special target. With a multi-drop link, the module number of the target destination computer link (serial communication) module station is set.

IConnectUnitNumber: Sets the module number of the computer link (serial communication) module or the Ethernet module for QnA/Qn. With a multi-drop link, the module number of the request source computer link (serial communication) module station is set. However, with a multi-drop via a CPU direct connection, the module number of the request source station is not required. ("0x00".) Sets "0x00" when not a multi-drop link. Sets a relay destination station number when using a QnA or Qn Ethernet module. ("0x00" is fixed when accessing within a host network.) If accessing other networks via MNET/10, sets the station number set in the parameter of the connected Ethernet module.

IIONumber: Sets the module I/O number.

Value	Target
0x3E0 to 0x3E3	Individual CPU module with multiple CPUs (Module 1 to 4)

ICpuType: Sets the target CPU (NC module) that performs communication.

Value	Target
0x0901	Q173NCCPU (C70)
0x1018	R16NCCPU (C80)

IUnitType: Sets the module connected to a physical port on the computer.

Value	Meaning
0x01	Direct to ACPU-RS422 port
0x02	Direct to QnACPU-RS422 port
0x03	Direct to C24 module for A
0x04	Direct to UC24 module for A
0x05	Direct to QC24 module for QnA
0x06	Connection to Ethernet LAN for A
0x07	Connection to Ethernet LAN for QnA
0x0D	Direct to AJ65BT-G4 (-S3) module (ACPU access)
0x0E	Direct to AJ65BT-G4 (-S3) module (QnA access)
0x0F	Direct to FXCPU-RS422 port
0x13	Direct to QnCPU (Q mode) RS232C port
0x15	Direct to QnCPU (A mode) RS232C
0x16	Direct to QnCPU (Q mode) USB port
0x18	Direct to QnCPU (A mode) USB port
0x19	Direct to C24 module for Q
0x1A	Connection to Ethernet LAN for Q / Connection to Ethernet LAN for C70
0x1B	Direct to AJ65BT-G4-S3 module (Qn access)
0x1C	Direct to Q motion -RS232C port
0x1D	Direct to Q motion USB port
0x21	Connection to GOT900 series
0x23	Connection to FXCPU computer link (RS232C)
0x24	Connection to FXCPU computer link (RS485)
0x27	Connection to Ethernet adapter for FX
0x2C	Connection to QnCPU (Q mode) Ethernet port
0x2D	Direct connection to QnCPU (Q mode) Ethernet port
0x40	Connection to QJ71E71 module through GOT1000 series
0x41	Connection to QnCPU Ethernet port through GOT1000 series
0x1001	Connection through R series compatible E71
0x1002	Connection to RCPU Ethernet port (IP is set)
0x1003	Direct to RCPU Ethernet port
0x1004	Direct to RCPU USB port
0x1005	Connection through R series compatible E71 (direct)
0x1051	Connection to R series compatible E71 through GOT2000/1000 series
0x1052	Connection to RCPU Ethernet port through GOT2000/1000 series

IPacketType: Sets the computer link or Ethernet packet transmission format. The following format is set for this parameter.

Value	Meaning
0x01	CPU protocol format (except the following formats or when AJ71E7/AJ71QE71 is set)
0x02	Dedicated protocol format (when AJ71(U)C24, AJ71E71/AJ71QE71 is set)
0x03	Dedicated protocol format (when AJ71E71/AJ71QE71 is set)

IProtocolType: Sets the communication protocol type of the module (board) to connect. Select connection through a serial port + modem with communication via AJAJ71QC24N/QJ71C24/LJ71C24 + modem. (If directly connecting to AJ71QC24N/QJ71C24/LJ71C24, select connection through a serial port.)

Value	Meaning
0x04	Through serial port
0x05	Through TCP/IP
0x08	Through UDP/IP
0x0A	Through Q6TEL, A6TEL
0x0D	Through USB port
0x0E	Through serial port + modem
0x13	Through GOT USB port

IPortNumber: Sets the port number for connection between the physical port on the computer and the module set by *IUnitType*. However, an arbitrary value is set as the request source (personal computer) port number with an Ethernet connection. If "0" is set as the port number, the MNET/10 routing method will be an automatic response method. In addition, set a fixed value of "5001" when not selecting an automatic response method via QE71 or when not setting TCP/IP for E71/QE71.

Value	Meaning
1	Communication port 1
2	Communication port 2
3	Communication port 3
4	Communication port 4
5	Communication port 5
6	Communication port 6
7	Communication port 7
8	Communication port 8
9	Communication port 9
10	Communication port 10

However, when an Ethernet connection is made, the following is applicable.

Model	Protocol	Port number
QJ71E71 AJ71QE71 (UDP)	UDP	Method other than automatic response
		Automatic response method
	TCP	-
AJ71QE71 (TCP) AJ71E71	UDP	Sets by matching the port number set in the sequence
	TCP	If sets by the sequence: Set by matching the set port number
		If not set by the sequence: Arbitrarily set an open port in the personal computer

IBaudRate: Sets the baud rate with serial communication. This parameter can be one of the following values.

Value	Meaning
2400	2400 bps
4800	4800 bps
9600	9600 bps
14400	14400 bps
19200	19200 bps
38400	38400 bps
56000	56000 bps
57600	57600 bps
115200	115200 bps
128000	128000 bps
256000	256000 bps

IDataBits: Sets the number of sent and received byte data bits (6 to 8).

IParity: Sets the parity bit. This parameter can be one of the following values.
It becomes effective only during serial communication.

Value	Meaning
0	No parity
1	Odd number
2	Even number
3	Mark

IStopBits: Sets the number of stop bits used. This parameter can be one of the following values.
It becomes effective only during serial communication.

Value	Meaning
0	1 stop bit
1	1.5 stop bits
2	2 stop bits

IControl: Sets the control signal. This parameter can be one of the following values.
It becomes effective only during serial communication.

Value	Meaning
0x01	DTR control
0x02	RTS control
0x06	No flow control
0x08	DTR or RTS control
0x8001	DTR control (with CD control)
0x8002	RTS control (with CD control)
0x8008	DTR or RTS control (with CD control)

IpcwszHostAddress: Sets the connected host name (IP address) as a UNICODE character string with Ethernet communication. Set NULL when Ethernet is not set.

ICpuTimeOut: Sets the CPU monitoring timer with Ethernet communication. The unit is *250 ms. (The default is 4.)

ITimeOut: Sets the communication time-out value. The unit is ms. (The default is 1000 ms.)

* The time-out starts counting from when data communication ends.

ISumCheck: Sets whether a sum check is enabled or disabled. One of the following values is set for this parameter. It is enabled only when connecting through a computer link module or an A-series Ethernet module (TCP/IP).

Value	Meaning
0	There is no sum check
1	There is a sum check

ISourceNetworkNumber: Sets the request source network number when Ethernet for QnA and Qn (via AJ71QE71 and QJ71E71) is set
Set the same network No. (network No. specified by the network parameter) as the Ethernet for a QnA or Qn connection.

ISourceStationNumber: Sets the request source station number (station number on the personal computer side) when an Ethernet for QnA and Qn (via AJ71QE71 and QJ71E71) is set
Set the station number so as not to overlap with the QE71 station numbers set in the same Ethernet loop.

IDestinationPortNumber: Sets the port number of the target destination module when Ethernet is set. Sets the relay destination port number when accessing other networks. The following applies when other than an automatic response method, E71, or QE71 (TCP/IP).

* Depending on the module, port numbers other than the following can be set. Refer to the manual of each module for details.

- Ethernet interface module (AJ71QE71)
 - UDP/IP : "5001" is fixed
- Ethernet interface module (QJ71E71)
 - UDP/IP : "5001" is fixed
 - TCP/IP : "5002" is fixed
- Ethernet interface module (RJ71EN71)
 - UDP/IP : "5001" is fixed
 - TCP/IP : "5002" is fixed
 - Ethernet port direct communication : "5003" is fixed
- MELSEC QnUD, QnUDE, and iQ-R series CPU (Ethernet port)
 - UDP/IP : "5006" is fixed
 - TCP/IP : "5007" is fixed
 - Ethernet port direct communication : "5008" is fixed
- C70 (Display I/F)
 - UDP/IP : "5001" is fixed
 - TCP/IP : "5002" is fixed
- C80 (Display I/F)
 - UDP/IP : "5006" is fixed
 - TCP/IP : "5007" is fixed

IDestinationIONumber: Sets the unit input/output No. (0x3E0 to 0x3E3) of the last access target station with a Qn multi-drop connection (via CC-Link, serial communication).

IConnectChannelNumber: Sets the connected channel No. (Ch1/Ch2) when connection of the serial communication module for Qn is set. Because this is used for system reservation, do not set anything. (Specify 0x00.)

IMultiDropChannelNumber: Sets the multi-drop connection channel No. (Ch1/Ch2) with a Qn multidrop connection. This will be invalid if any other connection is set.

Value	Meaning
0x01	Connection to channel 1
0x02	Connection to channel 2

IThroughNetworkType: Sets whether to include MNET/10 mode in the networks to pass through when accessing other stations via MELSECNET/10H.

Value	Meaning
0x00	Does not include MNET/10 mode
0x01	Includes MNET/10 mode

IIntelligentPreferenceBit: Sets whether to connect through the multi-drop link destination network with a Qn multi-drop connection (via CC-Link, serial communication). (This is to distinguish the host network module.)

Value	Meaning
0x00	Multi-drop does not access other destination networks
0x01	Multi-drop accesses other destination networks

IDidPropertyBit: With Q series host station intelligent special access (intelligent special module mounted to the CPU of the host station), by disabling the following settings, it is not necessary to set "IUnitNumber". (Set only by the module I/O number of "IIONumber".)

Value	Meaning
0x00	Enables module number
0x01	Disables module number

IDsidPropertyBit: With a Qn multi-drop connection, as the following are disabled, there is no need to set "*IDestinationIONumber*". However, if the following settings are disabled, make sure to enable "*IDidPropertyBit*". (Set by "*IUnitNumber*".)

Value	Meaning
0x00	Enables the I/O number of the last access target station
0x01	Disables the I/O number of the last access target station

plRet: Returns an error code.

S_OK: Normal termination

EZNC_COMM_ALREADYOPENED: Cannot be set because communication is already in progress

EZ_ERR_DATA_TYPE: Invalid argument data type

EZ_ERR_DATA_RANGE: Invalid argument data range

EZ_ERR_NOT_SUPPORT: Not supported

Value	Meaning
S_OK	Normal termination
S_FALSE	Communication failure

☐ **Return value**

☐ **Function**

Configures MELSEC communication settings. This is valid only for the C70/C80. This function is not supported with the M700/M800 Series. (EZ_ERR_NOT_SUPPORT is returned to *plRet*.)

Call before executing **Open3()**. If it is not called, an error will occur when **Open3()** is executed. The setting details are retained until the object is released by **Release()**. Temporarily, until **Close()** is performed if **Open3()** is performed, re-setting with **SetMelsecProctol()** cannot be done. An error will occur. If an error occurs in **SetMelsecProctol()**, the setting before the error occurred is maintained. The setting contents that resulted in the error are disabled. Sets NULL to the pointer setting argument such as "*lpcwszHostAddress*" when not used.

(Note) This method does not support automation interfaces. Use is limited to a custom interface.

☐ Structure	<pre> typedef struct EZNcStOpen{ LONG INetworkNumber; // Network number LONG IStationNumber; // Station number LONG IUnitNumber; // Module number LONG IConnectUnitNumber; // Module number LONG IIONumber; // Module I/O number LONG ICpuType; // Target CPU LONG IUnitType; // Connected module LONG IPacketType; // Packet transmission type LONG IProtocolType; // Communication protocol type LONG IPortNumber; // Connection port number LONG IBaudRate; // Baud rate LONG IDataBits; // Number of byte data bits LONG IParity; // Parity bit LONG IStopBits; // Stop bit LONG IControl; // Control signal WCHAR * lpcwszHostAddress; // Connection host name (IP address) LONG ICpuTimeOut; // CPU monitoring timer LONG ITimeOut; // Time-out value LONG ISumCheck; // Sum check LONG ISourceNetworkNumber; // Request source network number LONG ISourceStationNumber; // Request source station number (personal computer side station number) LONG IDestinationPortNumber; // Port number LONG IDestinationIONumber; // Actual input/output No. LONG IConnectChannelNumber; // Channel No. LONG IMultiDropChannelNumber; // Multi-drop connection channel No. LONG IThroughNetworkType; // MNET/10 mode LONG IIntelligentPreferenceBit; // Via multi-drop link destination LONG IDidPropertyBit; // Intelligent special module setting LONG IDsidPropertyBit; // Multi-drop connection setting } EZNCST_OPEN; </pre>
---	---

☐	Open3(), Close()
☐	Reference

☐	Specifica- tion
--------------------------	----------------------------

IEZNcCommunication3::SetNcMode**Set NC mode**□ **Custom call procedure**

```

HRESULT      SetNcMode (
                LONG IMode,                // (I) NC mode
                LONG* pIRet                // (O) Error code
            )

```

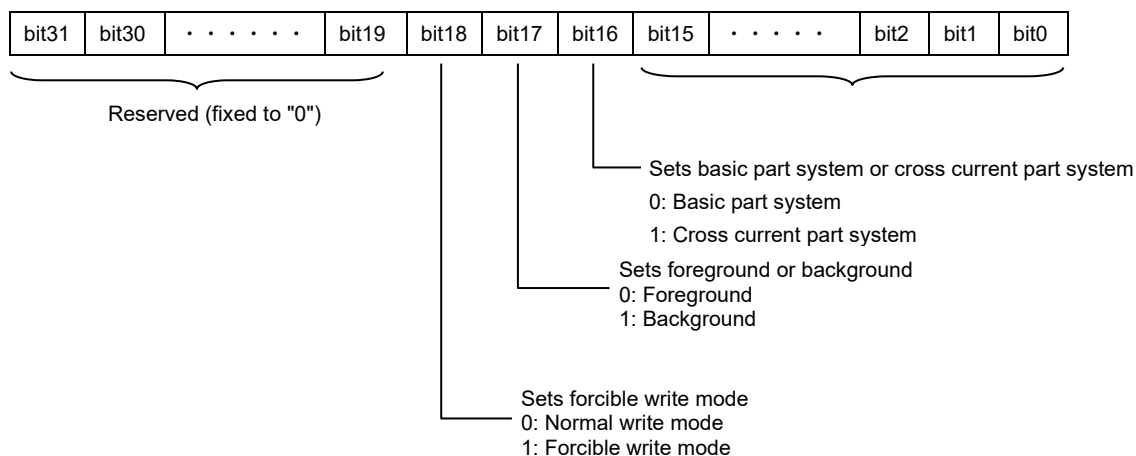
□ **Automation call procedure**

```

SetNcMode (
    IMode As LONG                // (I) NC mode
) As LONG                      // (O) Error code

```

□ *IMode*: Sets the mode of NC system which is being connected.

Argument

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Failure

□ **Function** Sets the mode of the NC system which is being connected. The set NC mode is used to communicate with an NC system in each method. The NC mode which currently being used can be get with GetNcMode(). The set NC mode is valid until communication line is closes. After the communication line of EZSocket is opened, all the setting are "0".
 This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pIRet.)

□ **GetNcMode()**

Reference

□ **Specifica-
tion**

IEZNcCommunication3::GetNcMode

Get NC mode

□ Custom call procedure

```

HRESULT      GetNcMode (
                LONG* pMode,           // (O) NC mode
                LONG* pRet             // (O) Error code
            )

```

□ Automation call procedure

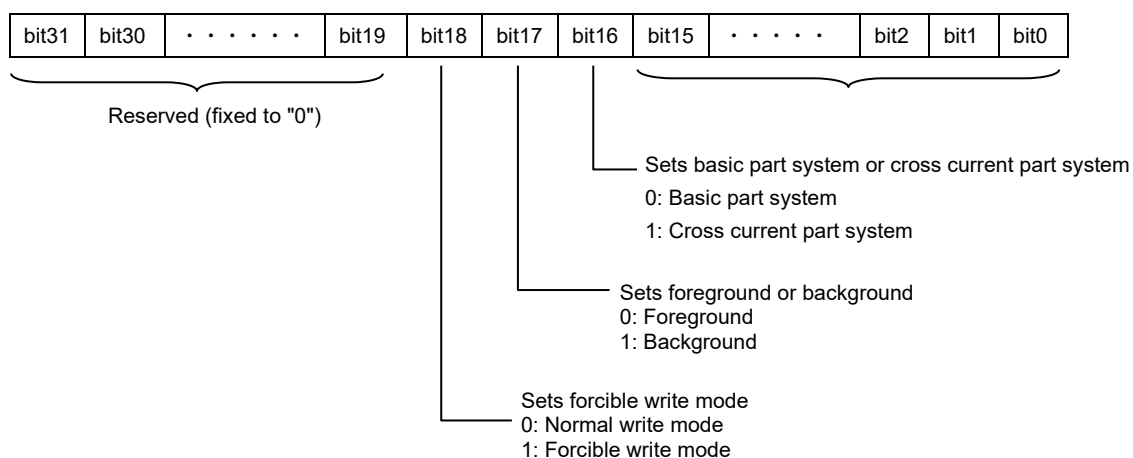
```

GetNcMode (
    pMode As LONG           // (O) NC mode
) As LONG                  // (O) Error code

```

□ *pMode*: Gets the mode of NC system which is being connected.

Argument



pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Failure
□	Gets the mode of the NC system which is being connected. Use SetNcMode() to change the setting of NC mode. This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pRet.)	
□	SetNcMode()	
Reference		
□		
Specifica- tion		

IEZNCCommunication3::SetPathLoadModule**Set NCAPI load module path**□ **Custom call procedure**

```

HRESULT      SetPathLoadModule (
                    LPCOLESTR lpcwszPath,      // (I) NCAPI load module path
                    LONG* pRet                // (O) Error code
                )

```

□ **Automation call procedure**

```

SetPathLoadModule (
    lpcwszPath As STRING                // (I) NCAPI load module path
) As LONG                               // (O) Error code

```

- **Argument** *lpcwszPath*: Sets the path of the NCAPI module (ncapi32.dll) which is to be loaded onto M700/M800 series. It must be an absolute path.
(Example) C:\NCAPI\Module\ncapi32.dll
The path must consist of 260 characters or less.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_COMM_ALREADYOPENED: Cannot be set because communication is already in progress

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Failure

- **Function** When the communication line opens, sets the path of the NCAPI module (ncapi32.dll) which is to be loaded onto M700/M800 series. The set value is invalid if the line opens with the model other than M700/M800 series. The correctness of the path is not checked. If no NCAPI module exists in the directory specified by the path, an error occurs when Open3() is executed.

When this method is not used, the path is set to the default one. The default path means the one which is based on the installation path of the product. The installation path of the product is referred to the contents which is stored in the registry indicated the table below.

Table Reference for registry

	OS environment	
	x86 platform	x64 platform
Reference for registry	HKEY_LOCAL_MACHINE └ SOFTWARE └ MITSUBISHI └ EZSocketNc └ CurrentVersion └ InstallPath	HKEY_LOCAL_MACHINE └ SOFTWARE └ Wow6432Node └ MITSUBISHI └ EZSocketNc └ CurrentVersion └ InstallPath

(Example) When the installation path stored in the registry "InstallPath" is "C:\Program Files (x86)\EZSocket\EZSocketNc\", the path of the NCAPI module which is to be loaded onto M700/M800 series is as follows:

C:\Program Files (x86)\EZSocket\EZSocketNc\CommServer\M700\ncapi32.dll

(Note 1) When the path of the NCAPI module is changed, the path of the system environment variable needs to be added with the same discretion.

(Note 2) If the path of the NCAPI module is changed, the machine may operate unpredictably. Change the path after you verify on the application that the change does not affect the machine operation.

- **Open3(), GetPathLoadModule()**

□ **Reference**□ **Specification**

IEZNCCommunication3::GetPathLoadModule**Get NCAPI load module path**□ **Custom call procedure**

```

HRESULT      GetPathLoadModule (
                    LPCOLESTR* lppwszPath,           // (O) NCAPI load module path
                    LONG* pRet                       // (O) Error code
                )

```

□ **Automation call procedure**

```

GetPathLoadModule (
    lppwszPath As STRING*           // (O) NCAPI load module path
) As LONG                          // (O) Error code

```

□ **Argument** *lppwszPath*: Gets the path of the NCAPI module which is to be loaded onto M700/M800 series.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Failure

□ **Function** Gets current path of the NCAPI module which is to be loaded onto M700/M800 series. When the communication line opens with M700/M800 series, the NCAPI module specified by the path is loaded onto the machine.

The product has dedicated character string area memory. When the development with VC++ client, use `CoTaskMemFree()` to explicitly free up character string area memory.

□ **Reference** **Open3()**, **SetPathLoadModule()**

□ **Specifica-
tion**

2.5 IEZNCSystem2 Interface

C70

C80

M700

M800

IEZNCSystem2::GetVersion

Get NC system S/W number, name

□ Custom call procedure

```
HRESULT      GetVersion(
                LONG IAxisNo,           // (I) Axis number
                LONG IIndex,           // (I) Data number
                LPOLESTR* lppwszBuffer, // (O) NC system S/W number, name
                LONG* plRet             // (O) Error code
            )
```

□ Automation call procedure

```
System_GetVersion(
    IAxisNo As LONG,           // (I) Axis number
    IIndex As LONG,           // (I) Data number
    lppwszBuffer As STRING*    // (O) NC system S/W number, name
) As LONG                    // (O) Error code
```

□ **Argument** *IAxisNo*: Sets the axis number (From Axis 1 = from 1)

IIndex: Sets the desired data number. Refer to the table below.

lppwszBuffer: Returns the system S/W number, name, and control S/W version as a **UNICODE** character string.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_DATASIZE: Application does not fit into prepared buffer

EZNC_DATA_READ_READ: Data is not readable

<i>IIndex</i>	Description	Data range
0	NC system S/W number, name, and PLC version	Depends on the system specifications.
1	Control unit, extension unit	Depends on the system specifications.
2	RIO unit, terminal RIO unit Axis setting is necessary only for C70.	Depends on the system specifications.

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Failure

□ Function	Gets the various NC system S/W version information as a UNICODE character string. 0: Gets the system S/W number, name, and PLC version of the NC system. The format of the character string data becomes as follows. NC system S/W number\tNC system name\tprogrammable controller system number\0 A TAB code is inserted between the NC system number and the NC system S/W name. The end of the data becomes a NULL code. Output example: "BND-2005W000-A0 MITSUBISHI CNC 830WM" If there is no item, a TAB code will follow. If a termination item does not exist, a NULL code will follow the TAB code. Up to three characters are got from the version information portion of NC system S/W number. When the version information portion has four or more characters, the fourth and subsequent characters are not got. For example, when the system S/W number is "BND-2005W000-A0AA", the character string to be got is "BND-2005W000-A0A". 1: Gets the control unit and extension unit versions. The format of the character string data becomes as follows. Control unit number\tExtension unit number\0 A TAB code is inserted between the control unit number and the extension unit number. The end of the data becomes a NULL code. 2: Gets the RIO unit and terminal RIO unit versions. The format of the character string data becomes as follows. RIO unit number\tTerminal RIO unit number\0 M700 has 24 items with RIO unit 1\t RIO unit 2\t...\0. M700/M800 series have up to 32 items (*) with RIO unit 1\t RIO unit 2\t5 ... \0. * Confirm the number of RIO unit with MTB. A TAB code is inserted between the RIO unit number and the terminal RIO unit number. The end of the data becomes a NULL code. As the character string area memory is allocated in this product, clients using VC++ need to release the character string area memory explicitly with CoTaskMemFree().
------------------------------	--

□

Reference

□

**Specifica-
tion**

IEZNCSystem2::GetSystemInformation**Get NC system information**□ **Custom call procedure**

```

HRESULT      GetSystemInformation(
                    LONG IType,           // (I) Information type
                    LONG* pISystem,       // (O) System information
                    LONG* pIRet          // (O) Error code
                )

```

□ **Automation call procedure**

```

System_GetSystemInformation(
    IType As LONG,           // (I) Information type
    pISystem As LONG*       // (O) System information
) As LONG                   // (O) Error code

```

□ **Argument** *IType*: Sets the NC system information type. Refer to the table below.

pISystem: Returns the NC system information.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_READ: Data is not readable

<i>IType</i>	Description	Data range
0	Part system enabled/disabled	0: Part system disabled 1: Part system enabled
1	Number of axes each part system	This will be different for NC systems with 1 [Axis] or more.

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets information regarding the NC system.

□ **Reference**

□ **Specifica-
tion** System

IEZNCSystem2::GetSerialNo**Get NC serial No. (product No.)**□ **Custom call procedure**

```

HRESULT      GetSerialNo (
                    LPOLESTR* lppwszSerialNo,           // (O) NC serial No. (product No.)
                    LONG* plRet                        // (O) Error code
                )

```

□ **Automation call procedure**

```

System_GetSerialNo (
    lppwszSerialNo As STRING           // (O) NC serial No. (product No.)
) As LONG                             // (O) Error code

```

□ *lppwszSerialNo*: Returns NC serial No. (product No.) as a **UNICODE** string.

□ **Argument**

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Failure

□ **Function** Gets the NC serial No. (product No.).
 The character string area memory is allocated in this product. When the development with VC++ client, use CoTaskMemFree() to explicitly free up character string area memory.

This is valid for the M700/M800 series and C80. This function is not supported with the other models.

□ **Reference**

□ **Specifica-
tion**

IEZNCSystem2::GetAlarm**Get alarm information**□ **Custom call procedure**

```

HRESULT      GetAlarm(
                LONG IMessageNumber,           // (I) Number of messages to get
                LONG IAlarmType,               // (I) Alarm type to get
                LPOLESTR* lppwszBuffer,        // (O) Message character string
                LONG* plRet                     // (O) Error code
            )

```

□ **Automation call procedure**

```

System_GetAlarm(
    IMessageNumber As LONG,           // (I) Number of messages to get
    IAlarmType As LONG,               // (I) Alarm type to get
    lppwszBuffer As STRING*          // (O) Message character string
) As LONG                             // (O) Error code

```

□ **Argument** *IMessageNumber*: Sets the number of messages to get. Value: **1 to 10 (maximum)**

IAlarmType: Sets the alarm type to get.

Value	Meaning
M_ALM_NC_ALARM	NC alarm
M_ALM_STOP_CODE	Stop code
M_ALM_PLC_ALARM	PLC alarm message
M_ALM_OPE_MSG	Operator message
M_ALM_ALL_ALARM	No alarm type distinction

lppwszBuffer: Gets the alarm message as a **UNICODE** character string.

The message format includes **CR**, **LF** codes to distinguish between messages. In addition, **NULL** is inserted at the end of the message.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_OPE_CURRALM_ADDR: Invalid part system, axis No. setting

EZNC_OPE_CURRALM_ALMTYPE: Invalid alarm type

EZNC_OPE_CURRALM_DATAERR: Error in communication data between NC and personal computer

EZNC_OPE_CURRALM_DATASIZE: Application does not fit into prepared buffer

EZNC_OPE_CURRALM_NOS: The number of obtained messages is invalid.

EZ_ERR_NOT_SUPPORT: Not supported

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the alarm message of the alarm currently generated in the setting NC control unit. The language of the alarm message adheres to an NC parameter (#1043 lang). Messages are retrieved in descending order of importance. As the character string area memory is allocated in this product, clients using VC++ need to release the character string area memory explicitly with **CoTaskMemFree()**. This function is not supported with the M800 Series and C80. (EZ_ERR_NOT_SUPPORT is returned to plRet.)

□ **Reference**

□ **Specifica-
tion** System (All systems when "0" is set)

IEZNCSystem2::GetAlarm2**Get alarm information**

□ Custom call procedure

```

HRESULT      GetAlarm2(
                LONG IMessageNumber,           // (I) Number of messages to get
                LONG IAlarmType,               // (I) Alarm type to get
                LPOLESTR* lppwszBuffer,        // (O) Message character string
                LONG* plRet                     // (O) Error code
            )

```

□ Automation call procedure

```

System_GetAlarm2(
    IMessageNumber As LONG,           // (I) Number of messages to get
    IAlarmType As LONG,               // (I) Alarm type to get
    lppwszBuffer As STRING*          // (O) Message character string
) As LONG                             // (O) Error code

```

□ **Argument** *IMessageNumber*: Sets the number of messages to get. Value: **1 to 10 (maximum)**

IAlarmType: Sets the alarm type to get.

Value	Meaning
M_ALM_NC_ALARM	NC alarm
M_ALM_STOP_CODE	Stop code
M_ALM_PLC_ALARM	PLC alarm message
M_ALM_OPE_MSG	Operator message
M_ALM_ALL_ALARM	No alarm type distinction

lppwszBuffer: Gets the alarm message as a **UNICODE** character string.

The message format includes **CR**, **LF** codes to distinguish between messages. In addition, **NULL** is inserted at the end of the message.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_OPE_CURRALM_ADDR: Invalid part system, axis No. setting

EZNC_OPE_CURRALM_ALMTYPE: Invalid alarm type

EZNC_OPE_CURRALM_DATAERR: Error in communication data between NC and personal computer

EZNC_OPE_CURRALM_DATASIZE: Application does not fit into prepared buffer

EZNC_OPE_CURRALM_NOS: The number of obtained messages is invalid.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the alarm message of the alarm currently generated in the setting NC control unit. The language of the alarm message adheres to an NC parameter (#1043 lang). Messages are retrieved in descending order of importance. As the character string area memory is allocated in this product, clients using VC++ need to release the character string area memory explicitly with **CoTaskMemFree()**.

□ **Reference**

□ **Specifica-
tion** System (All systems when "0" is set)

IEZNCSystem2::GetAlarm3**Get alarm information**

□ Custom call procedure

```

HRESULT      GetAlarm3(
                LONG IMessageNumber,           // (I) Number of messages to get
                LONG IAlarmType,               // (I) Alarm type to get
                LONG* pISystem,                // (O) Array of part system of got alarm
                BOOL* pbWarning,               // (O) Array of warning of got alarm
                LONG* pIAlarmType,             // (O) Array of detail type of got alarm
                LPOLESTR* lppwszBuffer,        // (O) Message character string
                LONG* pIRet                    // (O) Error code
            )

```

□ Automation call procedure

```

System_GetAlarm3(
    IMessageNumber As LONG           // (I) Number of messages to get
    IAlarmType As LONG               // (I) Alarm type to get
    pISystem As LONG*                // (O) Array of part system of got alarm
    pbWarning As BOOL*               // (O) Array of warning of got alarm
    pIAlarmType As LONG*             // (O) Array of detail type of got alarm
    lppwszBuffer As STRING*          // (O) Message character string
) As LONG                           // (O) Error code

```

□ **Argument** *IMessageNumber*: Sets the number of messages to get. Value: **1 to 10 (maximum)**

IAlarmType: Sets the alarm type to get.

Value	Meaning
M_ALM_NC_ALARM	NC alarm
M_ALM_STOP_CODE	Stop code
M_ALM_PLC_ALARM	PLC alarm message
M_ALM_OPE_MSG	Operator message
M_ALM_ALL_ALARM	No alarm type distinction

* When "**M_ALM_ALL_ALARM** No alarm type distinction" is specified for *IAlarmType*, the type of the got alarm can be checked by referring to the *pIAlarmType* value.

pISystem: Gets the part system where the got alarm occurs, as an array. Allocate the array for *IMessageNumber* on the application side, and specify the pointer.

pbWarning: Gets whether the got alarm is treated as a warning, as an array. Allocate the array for *IMessageNumber* on the application side, and specify the pointer.

TRUE: NC warning message

FALSE: Not NC warning message

pIAlarmType: Gets the type of the got alarm as an array. Allocate the array for *IMessageNumber* on the application side, and specify the pointer.

Value	Meaning
0x100: M_ALM_NC_ALARM	NC alarm
0x200: M_ALM_STOP_CODE	Stop code
0x300: M_ALM_PLC_ALARM	PLC alarm message
0x400: M_ALM_OPE_MSG	Operator message

lppwszBuffer: Gets the alarm message as a **UNICODE** character string.

The message format includes **CR**, **LF** codes to distinguish between messages. In addition, **NULL** is inserted at the end of the message.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_OPE_CURRALM_ADDR: Invalid system, spindle specification

EZNC_OPE_CURRALM_ALMTYPE: Invalid alarm type

EZNC_OPE_CURRALM_DATAERR: Error in communication data between NC and personal computer

EZNC_OPE_CURRALM_DATASIZE: Application does not fit into prepared buffer

EZNC_OPE_CURRALM_NOS: The number of obtained messages is invalid.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Function Gets the alarm message and alarm type of the alarm currently generated in the setting NC control unit. The language of the alarm message adheres to an NC parameter (#1043 lang). Messages are retrieved in descending order of importance. Judge the number of the got alarm information (actual alarm information) by the number of CR , LF codes to distinguish between messages in <i>lppwszBuffer</i> . As the character string area memory is allocated in EZSocket, clients using VC++ need to release the character string area memory explicitly with CoTaskMemFree() .	
□	Reference	
□	Specifica- tion System (All systems when "0" is set)	

2.6 IEZNcPosition Interface

C70

M700

M800

IEZNcPosition::GetWorkPosition

Get workpiece coordinate position

□ Custom call procedure

```
HRESULT      GetWorkPosition(
                LONG IAxisNo,           // (I) Axis number setting
                DOUBLE* pdPosition,     // (O) Workpiece coordinate position
                LONG ISkipOn,           // (I) Skip on flag
                LONG* plRet              // (O) Error code
            )
```

□ Automation call procedure

```
Position_GetWorkPosition(
    IAxisNo As LONG,           // (I) Axis number setting
    pdPosition As DOUBLE*,     // (O) Workpiece coordinate position
    ISkipOn As LONG            // (I) Skip on flag
) As LONG                     // (O) Error code
```

□ **Argument** *IAxisNo*: Sets the axis number (From Axis 1 = from 1)

pdPosition: Returns the workpiece coordinate position of the set axis number of the set part system.

Data range: -99999.999 to 99999.999 [mm]

ISkipOn : Sets the skip on flag.

Value	Meaning
1	Skip is on
0	Normal

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the workpiece coordinate position of the set system/axis number
If 1 is set for the skip on flag, the workpiece coordinate position at the time the skip on signal is input will be got.
This function is not supported with C80.

□ Reference

□ **Specifica-
tion** System Axis number

IEZNCPosition::GetWorkPosition2**Get workpiece coordinate position**□ **Custom call procedure**

```

HRESULT      GetWorkPosition2(
                LONG IAxisNo,                // (I) Axis number
                DOUBLE* pdPosition,          // (O) Workpiece coordinate position
                LONG ISkipOn,                // (I) Skip on flag
                LONG* plRet                  // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_GetWorkPosition2(
    IAxisNo As LONG,                // (I) Axis number
    pdPosition As DOUBLE*,          // (O) Workpiece coordinate position
    ISkipOn As LONG                 // (I) Skip on flag
) As LONG                          // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis number (From Axis 1 = from 1)

pdPosition: Returns the workpiece coordinate position of the set axis number of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

ISkipOn : Sets the skip on flag.

Value	Meaning
1	Skip is on
0	Normal

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the workpiece coordinate position of the set system/axis number.
 If 1 is set for the skip on flag, the workpiece coordinate position at the time the skip on signal is input will be got.
 This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetMachinePosition**Get machine position**□ **Custom call procedure**

```

HRESULT      GetMachinePosition(
                LONG IAxisNo,           // (I) Axis number
                DOUBLE* pdPosition,    // (O) Machine position
                LONG ISkipOn,          // (I) Skip on flag
                LONG* plRet           // (O) Error code
                )

```

□ **Automation call procedure**

```

Position_GetMachinePosition(
    IAxisNo As LONG,           // (I) Axis number
    pdPosition As DOUBLE*,    // (O) Machine position
    ISkipOn As LONG          // (I) Skip on flag
    ) As LONG                 // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis. (From Axis 1 = from 1)

pdPosition: Returns the machine position of the set axis number of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

ISkipOn : Sets the skip on flag.

Value	Meaning
1	Skip is on
0	Normal

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis number setting

EZNC_DATA_READ_READ: Data is not readable

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the machine coordinate position for the set system/axis number (coordinate position in a basic machine coordinate system).
If **1** is set for the skip on flag, the machine coordinate position at the time the skip on signal is input will be got.

□ **Reference**

□ **Specifica-
tion** System Axis number

IEZNCPosition::GetMachinePosition2**Get machine position**□ **Custom call procedure**

```

HRESULT      GetMachinePosition2(
                LONG IAxisNo,           // (I) Axis number
                DOUBLE* pdPosition,    // (O) Machine position
                LONG ISkipOn,          // (I) Skip on flag
                LONG* plRet           // (O) Error code
                )

```

□ **Automation call procedure**

```

Position_GetMachinePosition2(
    IAxisNo As LONG,           // (I) Axis number
    pdPosition As DOUBLE*,    // (O) Machine position
    ISkipOn As LONG          // (I) Skip on flag
    ) As LONG                 // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis. (From Axis 1 = from 1)

pdPosition: Returns the machine position of the set axis number of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

ISkipOn : Sets the skip on flag.

Value	Meaning
1	Skip is on
0	Normal

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis number setting

EZNC_DATA_READ_READ: Data is not readable

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the machine coordinate position for the set system/axis number (coordinate position in a basic machine coordinate system).
If **1** is set for the skip on flag, the machine coordinate position at the time the skip on signal is input will be got.

□ **Reference**

□ **Specifica-
tion** System Axis number

IEZNCPosition::GetCurrentPosition**Get current position**□ **Custom call procedure**

```

HRESULT      GetCurrentPosition(
                LONG IAxisNo,                // (I) Axis number
                DOUBLE* pdPosition,          // (O) Relative position
                LONG* pIRet                   // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_GetCurrentPosition(
    IAxisNo As LONG,                // (I) Axis number
    pdPosition As DOUBLE*          // (O) Relative position
) As LONG                          // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

pdPosition: Returns the relative position from the position at a completion of the dog type zero point return or from the preset position configured by G92/origin set/counter set.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis number setting

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the relative position to the position at a completion of the dog type zero point return or to the preset position configured by G92/origin set/counter set of the set system/axis number. This function is not supported with C80.	
Function		

□

Reference

□

**Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetDistance**Get remaining command**

□ Custom call procedure

```

HRESULT      GetDistance(
                LONG IAxisNo,                // (I) Axis number
                DOUBLE* pdDistance,          // (O) Remaining command
                LONG ISkipOn,                // (I) Skip on flag
                LONG* pRet                   // (O) Error code
            )

```

□ Automation call procedure

```

Position_GetDistance(
    IAxisNo As LONG,                // (I) Axis number
    pdDistance As DOUBLE*,          // (O) Remaining command
    ISkipOn As LONG                 // (I) Skip on flag
) As LONG                          // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis number (From Axis 1 = from **1**)

pdDistance: Returns the remaining command of the travel command being executed in the set axis No. of the set system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

ISkipOn : Sets the skip on flag.

Value	Meaning
1	Skip is on
0	Normal

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis number setting

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the remaining command of the travel command being executed for the set part system/axis number.
If **1** is set for the skip on flag, the remaining command of the travel command at the time the skip on signal is input will be got.

□ **Reference**

□ **Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetDistance2**Get remaining command**□ **Custom call procedure**

```

HRESULT      GetDistance2(
                LONG IAxisNo,                // (I) Axis number
                DOUBLE* pdDistance,          // (O) Remaining command
                LONG ISkipOn,                // (I) Skip on flag
                LONG* pIRet                   // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_GetDistance2(
    IAxisNo As LONG,                // (I) Axis number
    pdDistance As DOUBLE*,          // (O) Remaining Command
    ISkipOn As LONG                 // (I) Skip on flag
) As LONG                          // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis number. (From Axis 1 = from 1)

pdDistance: Returns the remaining command of the travel command being executed in the set axis number. of the set system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

ISkipOn : Sets the skip on flag.

Value	Meaning
1	Skip is on
0	Normal

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis number setting

EZNC_DATA_READ_READ: Data is not readable

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the remaining command of the travel command being executed for the set system/axis number.
If 1 is set for the skip on flag, the remaining command of the travel command at the time the skip on signal is input will be got.

□ **Reference**

□ **Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetNextDistance**Get next command**

□ Custom call procedure

```

HRESULT      GetNextDistance(
                LONG IAxisNo,                // (I) Axis No. setting
                DOUBLE* pdDistance,          // (O) Next command
                LONG* pIRet                   // (O) Error code
            )

```

□ Automation call procedure

```

Position_GetNextDistance(
    IAxisNo As LONG,                // (I) Axis No. setting
    pdDistance As DOUBLE*          // (O) Next command
) As LONG                          // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

pdDistance: Returns the travel command of the next block for the set axis No. of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the travel command of the next block for the set system/axis No.	
Function	This function is not supported with C80.	

□

Reference

□

**Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetFeedSpeed**Get feed speed**

□ Custom call procedure

```

HRESULT      GetFeedSpeed(
                LONG IFeedType,           // (I) Feed speed type
                DOUBLE* pdSpeed,         // (O) Feed speed
                LONG* pIRet               // (O) Error code
            )

```

□ Automation call procedure

```

Position_GetFeedSpeed(
    IFeedType As LONG,           // (I) Feed speed type
    pdSpeed As DOUBLE*         // (O) Feed speed
) As LONG                       // (O) Error code

```

□ *IFeedType*: Sets the type of feed speed to get.

Argument	Value	Meaning
	0	F command feed speed (FA)
	1	Manual effective feed speed (FM)
	2	Synchronization feed speed (FS)
	3	Automatic effective feed speed (Fc)
	4	Screw lead (FE)

pdSpeed: Returns the feed speed of the specified system.

Feed speed type	Data range
FA	0.000 to 1,000,000.000 [mm/min]
FM	0.000 to 1,000,000.000 [mm/min]
FS	0.000 to 1,000.0000000 [mm/rev]
Fc	0.000 to 1,000,000.000 [mm/min]
FE	0.000 to 1,000.0000000 [mm] (Unit will vary)

The number of digits is determined by the NC type, options, and MTB setting values (parameters).

pIRet: Returns an error code. (Upon automation, the return value is used.)**S_OK**: Normal termination**EZ_ERR_DATA_TYPE**: Invalid argument data type**EZNC_DATA_READ_ADDR**: Invalid part system, axis No. setting**EZNC_DATA_READ_READ**: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ Gets the feed speed of the set system.

Function□ **Reference**

□ **Specifica-
tion**

System

IEZNCPosition::GetTCPSpeed**Get tip speed**□ **Custom call procedure**

```

HRESULT      GetTCPSpeed (
                DOUBLE* pdPosition,           // (O) Tip speed
                LONG* plRet                    // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_ GetTCPSpeed (
    pdPosition As DOUBLE*           // (O) Tip speed
) As LONG                          // (O) Error code

```

□ *pdPosition*: Returns the tip speed of the set system.

□ **Argument**

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid system specification

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the tip speed of the set part system.
 If 1 is set for the skip on flag, the workpiece coordinate position at the time the skip on signal is input will be got.
 This function is not supported with C70/C80.

□ **Reference**

□ **Specifica-
tion** System

IEZNCPosition::GetManualOverlap**Get manual interrupt amount**

□ Custom call procedure

```

HRESULT      GetManualOverlap(
                LONG IAxisNo,           // (I) Axis No. setting
                LONG IType,             // (I) Type
                DOUBLE* pdLength,       // (O) Manual interrupt amount
                LONG* pIRet              // (O) Error code
            )

```

□ Automation call procedure

```

Position_GetManualOverlap(
    IAxisNo As LONG,           // (I) Axis No. setting
    IType As LONG,            // (I) Type
    pdLength As DOUBLE*,      // (O) Manual interrupt amount
    ) As LONG                  // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

IType: Sets the manual interrupt amount type to get.

Value	Meaning
0	If getting the manual interrupt amount while the manual ABS switch is off
1	If getting the manual interrupt amount while the manual ABS switch is on

pdLength: Returns the manual interrupt amount for the set axis No. of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the manual interrupt amount of the set part system/axis.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** System Axis number

IEZNCPosition::GetManualOverlap2**Get manual interrupt amount**

□ Custom call procedure

```

HRESULT      GetManualOverlap2(
                LONG IAxisNo,           // (I) Axis No.
                LONG IType,             // (I) Type
                DOUBLE* pdLength,       // (O) Manual interrupt amount
                LONG* pIRet              // (O) Error code
            )

```

□ Automation call procedure

```

Position_GetManualOverlap2(
    IAxisNo As LONG,           // (I) Axis No.
    IType As LONG,             // (I) Type
    pdLength As DOUBLE*       // (O) Manual interrupt amount
) As LONG                     // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

IType: Sets the manual interrupt amount type to get.

Value	Meaning
0	If getting the manual interrupt amount while the manual ABS switch is off
1	If getting the manual interrupt amount while the manual ABS switch is on

pdLength: Returns the manual interrupt amount for the set axis No. of the set part system.
Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis specification

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the manual interrupt amount of the set part axis of the set part system.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** System Axis number

IEZNCPosition::GetProgramPosition**Get program position**

□ Custom call procedure

```

HRESULT      GetProgramPosition(
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdPosition,     // (O) Program position
                LONG* pIRet              // (O) Error code
            )

```

□ Automation call procedure

```

Position_GetProgramPosition(
    IAxisNo As LONG,           // (I) Axis No.
    pdPosition As DOUBLE*     // (O) Program position
) As LONG                     // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

pdPosition: Returns the program position.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets program position.	
Function	This function is not supported with C80.	
□		
Reference		
□		
Specifica- tion	System	Axis number

IEZNCPosition::GetProgramPosition3**Get program position**□ **Custom call procedure**

```

HRESULT      GetProgramPosition3(
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdPosition,     // (O) Program position
                LONG* pRet               // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_GetProgramPosition3(
    IAxisNo As LONG,           // (I) Axis No.
    pdPosition As DOUBLE*     // (O) Program position
) As LONG                     // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

pdPosition: Returns the program position.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No.setting

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets program position.	
Function	This function is not supported with C80.	
□		
Reference		
□		
Specifica- tion	System	Axis number

IEZNCPosition::GetTCPMachinePosition**Get tip machine position**□ **Custom call procedure**

```

HRESULT      GetTCPMachinePosition (
                LONG IAxisNo,                // (I) Axis No.
                DOUBLE* pdPosition,          // (O) Tip machine position
                LONG* pIRet                   // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_ GetTCPMachinePosition (
    IAxisNo As LONG,                // (I) Axis No.
    pdPosition As DOUBLE*          // (O) Tip machine position
) As LONG                          // (O) Error code

```

□ *IAxisNo*: Set the axis No. (From Axis 1 = from 1)

Argument

pdPosition: Returns the tip machine position of the set axis No. of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the tip machine position.	
Function	This function is not supported with C70/C80.	

□

Reference

□ **Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetTCPWorkPosition**Get tip workpiece position**□ **Custom call procedure**

```

HRESULT      GetTCPWorkPosition (
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdPosition,     // (O) Tip workpiece position
                LONG* pIRet              // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_ GetTCPWorkPosition (
    IAxisNo As LONG,           // (I) Axis No.
    pdPosition As DOUBLE*     // (O) Tip workpiece position
) As LONG                    // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

pdPosition: Returns the tip workpiece position of the set axis No. of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Gets the tip workpiece position.	
	This function is not supported with C70/C80.	

□
Reference

□ **Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetFeedbackPosition**Get feedback position**□ **Custom call procedure**

```

HRESULT      GetFeedbackPosition (
                LONG IAxisNo,                // (I) Axis No.
                DOUBLE* pdPosition,          // (O) Feedback position
                LONG* pRet                    // (O) Error code
            )

```

□ **Automation call procedure**

```

Position_ GetFeedbackPosition (
    IAxisNo As LONG,                // (I) Axis No.
    pdPosition As DOUBLE*          // (O) Feedback position
) As LONG                          // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

□ **Argument**

pdPosition: Returns the feedback position of the set axis of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□ Function	Gets the feedback position.	
	This function is not supported with C70/C80.	

□ **Reference**

□ **Specifica-
tion**

System	Axis number
--------	-------------

IEZNCPosition::GetTableCoordinationPosition**Get table coordinate system counter**

□ Custom call procedure

```

HRESULT      GetTableCoordinationPosition (
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdPosition,     // (O) Table coordinate system counter
                LONG* plRet              // (O) Error code
            )

```

□ Automation call procedure

```

Position_ GetTableCoordinationPosition (
            IAxisNo As LONG,           // (I) Axis No.
            pdPosition As DOUBLE*     // (O) Table coordinate system counter
        ) As LONG                     // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

pdPosition: Returns the table coordinate system counter of the set axis of the set part system.
 Data range: -99999.999 to 99999.999 [mm]
 This will vary according to the NC system specifications and parameters.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the table coordinate system counter.
 This function is not supported with C70/C80.

□ **Reference**

□ **Specifica-
tion** System Axis number

IEZNCPosition::GetWorkInstallationPosition**Get workpiece installation
coordinate system counter**□ **Custom call procedure**

```

HRESULT      GetWorkInstallationPosition (
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdPosition,    // (O) Workpiece installation coordinate system counter
                LONG* plRet           // (O) Error code
                )

```

□ **Automation call procedure**

```

Position_ GetWorkInstallationPosition (
    IAxisNo As LONG,           // (I) Axis No.
    pdPosition As DOUBLE*    // (O) Workpiece installation coordinate system counter
    ) As LONG                 // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)□ **Argument**

pdPosition: Returns the workpiece installation coordinate system counter of the set axis of the set system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the workpiece installation coordinate system counter.
This function is not supported with C70/C80.

□ **Reference**

□ **Specifica-
tion** System Axis number

IEZNCPosition::GetInclinedSurfacePosition**Get inclined surface
coordinate system counter**□ **Custom call procedure**

```

HRESULT      GetInclinedSurfacePosition (
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdPosition,    // (O) Inclined surface coordinate system counter
                LONG* plRet             // (O) Error code
                )

```

□ **Automation call procedure**

```

Position_ GetInclinedSurfacePosition (
    IAxisNo As LONG,           // (I) Axis No.
    pdPosition As DOUBLE*    // (O) Inclined surface coordinate system counter
    ) As LONG                 // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)□ **Argument**

pdPosition: Returns the inclined surface coordinate system counter of the set axis No. of the set part system.

Data range: -99999.999 to 99999.999 [mm]

This will vary according to the NC system specifications and parameters.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□ Function	Gets the inclined surface coordinate system counter. This function is not supported with C70/C80.	

□ **Reference**

□ **Specifica-
tion**

System	Axis number
--------	-------------

2.7 IEZNcCommand2 Interface

C70

C80

M700

M800

IEZNcCommand2::GetGCodeCommand

Get G code modal command value

□ Custom call procedure

```
HRESULT      GetGCodeCommand(
                LONG IType,                // (I) Type
                DOUBLE* pdValue,           // (O) Command value
                LONG* plRet                 // (O) Error code
            )
```

□ Automation call procedure

```
Command_GetGCodeCommand(
    IType As LONG,                // (I) Type
    pdValue As DOUBLE*           // (O) Command value
) As LONG                        // (O) Error code
```

□ *IType*: Sets the G code modal command value type to get.

Argument The following describes examples for the M700/M800 series M system. The content will differ depending on the type and the settings.

Value	Meaning
1	Group 1 (Interpolation mode) G00, G01, G02, G03, G33, G02.1, G03.1, G02.3, G03.3, G02.4, G03.4, G062 command modal
2	Group 2 (Plane selection) G17, G18, G19 command modal
3	Group 3 (Absolute) G90, (incremental) G91 command modal
4	Group 4 (Chuck barrier) G22, G23 command modal
5	Group 5 (Feed mode) G93, G94, G95 command modal
6	Group 6 (Inch) G20, (millimeter) G21 command modal
7	Group 7 (Radial compensation mode) G40, G41, G42, G41.2, G42.2 command modal
8	Group 8 (Length compensation mode) G43, G44, G43.1, G43.4, G43.5, G49 command modal
9	Group 9 (Fixed cycle mode) G70, G71, G72, G73, G74, G75, G76, G77, G78, G79, G80, G81, G82, G83, G84, G85, G86, G87, G88, G89 command modal
10	Group 10 (Initial point return) G98, (R point return) G99 command modal
11	Group 11 G50, G51 command modal (only for M700/M800/C80 series M system)
12	Group 12 (Workpiece coordinate system modal) G54, G54.1, G55, G56, G57, G58, G59 command modal
13	Group 13 (Cutting mode) G61, G61.1, G61.2, G62, G63, G63.1, G63.2, G64 command modal
14	Group 14 (Modal call) G66, G66.1, G67 command modal
15	Group 15 (Normal control) G40.1, G41.1, G42.1 command modal (only for M700/M800/C80 series M system)
16	Group 16 (Coordinate rotation) G68, G68.2, G68.3, G69 (only for M700/M800/C80 series M system)
17	Group 17 (Constant surface speed control) G96, G97 command modal
18	Group 18 (Polar coordinate command) G15, G16 command modal
19	Group 19 (G command mirror image) G50.1, G51.1 command modal
20	Group 20 (Spindle selection) G43.1, G44.1, G47.1 command modal
21	Group 21 (Cylindrical interpolation / polar coordinate interpolation) G07.1, G107, G12.1, G112, G13.1, G113 (only for M700/M800/C80 series M system)

pdValue : Returns the current G code modal command value of the set part system.

Value (Example) Meaning

2	G02
17	G17
50.2	G50.2

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid system

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the G code command modal value of the set part system. Refer to the "Programming Manual" of each model for the individual model's G command and group list. Also, depending on the model, different operation than with the original G code command may be incorporated by using the G code macro call. Check after referring to the instructions issued by MTB.	
□	Reference	
□	Specification	
	System	

IEZNCCommand2::GetToolCommand**Get tool compensation number**□ **Custom call procedure**

```

HRESULT      GetToolCommand(
                LONG IAxisNo,           // (I) Axis No.
                LONG IType,             // (I) Type
                LONG* pIValue,          // (O) Tool compensation number
                LONG* pIRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Command_GetToolCommand(
    IAxisNo As LONG,           // (I) Axis No.
    IType As LONG,            // (I) Type
    pIValue As LONG*          // (O) Tool compensation number
) As LONG                    // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis when getting the length compensation number.
(From Axis 1 = from 1)

IType: Sets the tool compensation type to get.

Value	Meaning
0	D command value of the shape compensation number
1	D command value of the wear compensation number
2	H command value of the length compensation number (axis specification necessary)

pIValue : Returns the shape/wear compensation number of the tool for the set part system and the tool length compensation number of the set axis No. in the set part system.
Data range: 1 to 200 (range depends on the number of tool offset sets)
Value meaning: 1 = D1, 1 = H1

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the shape/wear compensation number of the tool for the set part system and the tool length compensation number of the set axis No. in the set part system.
This function is not supported with C80.

□ **Reference**

□ Specifica- tion	System Axis number
------------------------------	--

IEZNCCommand2::GetFeedCommand**Get feed speed command value**

□ Custom call procedure

```

HRESULT      GetFeedCommand(
                LONG IType,                // (I) Type
                DOUBLE* pdValue,          // (O) Command value
                LONG* pRet                 // (O) Error code
            )

```

□ Automation call procedure

```

Command_GetFeedCommand(
    IType As LONG,                // (I) Type
    pdValue As DOUBLE*           // (O) Command value
) As LONG                        // (O) Error code

```

□ *IType*: Sets the command value type to get.

Argument	Value	Meaning
	0	F command feed speed (FA)
	1	Manual effective feed speed (FM)
	2	Synchronization feed speed (FS)
	3	Automatic effective feed speed (FC)
	4	Screw lead (FE)
	5	Tip speed (TCP) (M700/M800 series only)

pdValue: Returns the current feed speed command value of the set part system.

Feed speed type	Data range	
	M700/M800 series	C70
FA	0.000 to 10,000,000.000 [mm/min]	0.000 to 1,000,000.000 [mm/min]
FM	0.000 to 1,000,000.000 [mm/min]	0.000 to 1,000,000.000 [mm/min]
FS	0.000 to 1,000,000.000000 [mm/rev]	0.000 to 1,000,000.000 [mm/rev]
FC	0.000 to 100,000.000 [mm/min]	0.000 to 1,000,000.000 [mm/min]
FE	0.000 to 100,000.000 [mm/rev] (Unit will vary)	0.000 to 999.9999 [mm/rev]

The total number of digits is determined by the NC model, options, and MTB setting values (parameters).

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid system

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Gets the current feed speed command value of the set part system. This function is not supported with C80.	

□ Reference

□ Specification

System

IEZNCCommand2::GetCommand2**Get M/S/T/B function command modal value**

□ Custom call procedure

```

HRESULT      GetCommand2(
                LONG IType,                // (I) Command type
                LONG IIndex                // (I) Command number
                LONG* pIValue,            // (O) Command value
                LONG* pIRet                // (O) Error code
            )

```

□ Automation call procedure

```

Command_GetCommand2(
    IType As LONG,                // (I) Command type
    IIndex As LONG,              // (I) Command number
    pIValue As LONG*             // (O) Command value
) As LONG                        // (O) Error code

```

□ *IType*: Sets the command value type to get.

Argument	Value	Meaning
	EZNC_M	M command (sub function M command value)
	EZNC_S	S command (spindle rotation speed S command value)
	EZNC_T	T command (tool change T command value)
	EZNC_B	B command (second sub function command value (specification of index table position, etc.))

IIndex: Set the command number.Example) When *IType* = EZNC_M and *IIndex* = 1, the M command will be 1.

Model Command	M700/M800 series	C70
M	1 to 4	1 to 4
S	1 to maximum number of spindles*	1 to 7
T	1	1 to 4
B	1 to 4	1 to 4

*For the maximum number of spindles, refer to the product catalog for each Mitsubishi CNC.

pIValue : Returns the current command value of the set part system.

Data range: 0 to 99999.999 (maximum)

pIRet: Returns an error code. (Upon automation, the return value is used.)**S_OK**: Normal termination**EZ_ERR_DATA_TYPE**: Invalid argument data type**EZNC_DATA_READ_READ**: Data is not readable**EZNC_DATA_READ_ADDR**: Invalid part system setting**EZNC_DATA_READ_SUBJECT**: Invalid subsection number**EZ_ERR_NOT_SUPPORT**: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ Gets the current command modal value for the M/S/T/B function of the set part system.

Function This function is not supported with C80.

□ **SetCommad2()**

Reference

□ **Specifica-
tion** System (PLC axis system cannot set)

IEZNCCommand2::SetCommand2**Set manual numerical value command
for M/S/T/B functions**

□ Custom call procedure

```

HRESULT      SetCommand2(
                LONG IType,                // (I) Type
                LONG IIndex                // (I) Command number
                LONG IValue,                // (I) Command value
                LONG* pIRet                // (O) Error code
            )

```

□ Automation call procedure

```

Command_SetCommand2(
    IType As LONG,                // (I) Type
    IIndex As LONG,                // (I) Command number
    IValue As LONG                // (I) Command value
) As LONG                        // (O) Error code

```

□ *IType*: Sets the command value type to get.

Argument	Value	Meaning
	EZNC_M	M command (sub function M command value)
	EZNC_S	S command (spindle rotation speed S command value)
	EZNC_T	T command (tool change T command value)
	EZNC_B	B command (second sub function command value (specification of index table position, etc.))

IIndex: Sets the set number.Example) When *IType* = EZNC_M and *IIndex*= 1, the M command will be 1.

Model	M700/M800 series	C70
Command		
M	1 to 4	1
S	1 to maximum number of spindles*	1 to 7
T	1	1
B	1 to 4	1

*For the maximum number of spindles, refer to the product catalog for each Mitsubishi CNC.

IValue: Sets the command value of the set part system or the axis No.

Data range: 0 to 99999999 (maximum)

pIRet: Returns an error code. (Upon automation, the return value is used.)**S_OK**: Normal termination**EZ_ERR_DATA_TYPE**: Invalid argument data type**EZNC_DATA_WRITE_WRITE**: Data is not writable**EZNC_DATA_WRITE_ADDR**: Invalid part system setting**EZNC_DATA_WRITE_SUBSECT**: Invalid subsection number**EZ_ERR_NOT_SUPPORT**: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Sets the manual numerical value command value of the M/S/T/B function of the axis No. or the set part system. This function is not supported with C80.	
□	GetCommand2()	
Reference		
□	System (PLC axis system cannot set)	
Specification		

2.8 IEZNcProgram2 Interface

C70

M700

M800

IEZNCProgram2::CurrentBlockRead

Read current program block

Custom call procedure

HRESULTCurrentBlockRead(

LONGIBlockNumber, // (I) Number of blocks

LPOLESTR* lppwszProgramData, // (O) Program storage

LONG* plCurrentBlockNo, // (O) Block number being executed

LONG* plRet // (O) Error code

)

Automation call procedure

Program_CurrentBlockRead(

IBlockNumber As LONG, // (I) Number of blocks

lppwszProgramData As STRING*, // (O) Program storage

plCurrentBlockNo As LONG* // (O) Block number being executed

) As LONG // (O) Error code

Argument

IBlockNumber: Sets the number of blocks to get. Value: 1 to 10

lppwszProgramData: Gets the program blocks as a **UNICODE** character string. To separate program blocks, **CR**, **LF** codes are inserted between them. In addition, **NULL** is inserted at the end.

plCurrentBlockNo: Returns the block number being executed in the got blocks.

Value	Meaning
0	Not in operation
1	1st block
2	2nd block

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK : Normal termination

EZNC_OPE_GETPRGBLK_ADDR: Invalid part system setting

EZNC_OPE_GETPRGBLK_DATAERR: Error in communication data between NC and personal computer

EZNC_OPE_GETPRGBLK_DATASIZE: Application does not fit into prepared buffer

EZNC_OPE_GETPRGBLK_NOS: The number of blocks setting is invalid

Return value

Value	Meaning
S_OK	Normal termination
S_FALSE	Communication failure

Function

Gets the program for which the operation search has been completed or the program currently being executed. Reads the program for which the operation search has been completed for the set part system or the program block in operation.

If no operation search has been completed, the following applies.

lppwszProgramData = "\0"

plCurrentBlockNo = 0

As the character string area memory is allocated in this product, clients using VC++ need to release the character string area memory explicitly with **CoTaskMemFree()**.

Even if no operation search has been completed, the character string area memory must be released.

This function is not supported with C80.

Reference

IEZNCOperation::Search()

Specification

System

IEZNCProgram2::GetProgramNumber2**Get program number**

□ Custom call procedure

```

HRESULT      GetProgramNumber2(
                LONG IProgramType,           // (I) Program type
                LPOLESTR* IppwszProgramNo,    // (O) Program No.
                LONG* pIRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Program_GetProgramNumber2(
    IProgramType As LONG,           // (I) Program type
    IppwszProgramNo As STRING*    // (O) Program No.
) As LONG                          // (O) Error code

```

Argument	<i>IProgramType</i> : Sets the program type.
	Value Meaning
	EZNC_MAINPRG Main program
	EZNC_SUBPRG Subprogram

IppwszProgramNo: Returns the number of the program for which search has been completed or currently in automatic operation as a **UNICODE** character string. The program number is got as the program file name with the M700/M800 series.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Returns the number of the program for which search has been completed or currently in automatic operation.
 As the character string area memory is allocated in this product, clients using VC++ need to release the character string area memory explicitly with **CoTaskMemFree()**.
 This function is not supported with C80.

Reference	GetSequenceNumber(), GetBlockNumber(), GetSubProLevel()
-----------	--

Specification	System
---------------	--------

IEZNCProgram2::GetSequenceNumber**Get sequence number**□ **Custom call procedure**

```

HRESULT      GetSequenceNumber(
                LONG IProgramType,           // (I) Program type
                LONG* pISequenceNo,         // (O) Sequence number
                LONG* pIRet                  // (O) Error code
            )

```

□ **Automation call procedure**

```

Program_GetSequenceNumber(
    IProgramType As LONG,           // (I) Program type
    pISequenceNo As LONG*,         // (O) Sequence number
) As LONG                          // (O) Error code

```

Argument	Value	Meaning
	EZNC_MAINPRG	Main program
	EZNC_SUBPRG	Subprogram

IProgramType: Sets the program type.

pISequenceNo: Returns the sequence number of the program for which search has been completed or currently in automatic operation.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK	: Normal termination
EZ_ERR_DATA_TYPE	: Invalid argument data type
EZNC_DATA_READ_ADDR	: Invalid system specification
EZNC_DATA_READ_READ	: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Returns the sequence number of the program for which search has been completed or currently in automatic operation.
This function is not supported with C80.

□ **Reference** **GetProgramNumber2(), GetBlockNumber(), GetSubProLevel()**

□ **Specifica-
tion** System

IEZNCProgram2::GetBlockNumber**Get block number**

□ Custom call procedure

```

HRESULT      GetBlockNumber(
                LONG IProgramType,           // (I) Program type
                LONG* pIBlockNo,            // (O) Block number
                LONG* pIRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Program_GetBlockNumber(
    IProgramType As LONG,           // (I) Program type
    pIBlockNo As LONG*             // (O) Block number
) As LONG                          // (O) Error code

```

□ *IProgramType*: Sets the program type.

Argument	Value	Meaning
	EZNC_MAINPRG	Main program
	EZNC_SUBPRG	Subprogram

pIBlockNo: Returns the block number of the program for which search has been completed or currently in automatic operation.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Returns the block number of the program for which search has been completed or currently in automatic operation.

□ **Reference** **GetProgramNumber2(), GetSequenceNumber(), GetSubProLevel()**

□ **Specifica-
tion** System

IEZNCProgram2::GetSubProLevel**Get subprogram call level**□ **Custom call procedure**

```

HRESULT      GetSubProLevel(
                LONG* pLevel,           // (O) Level
                LONG* pRet              // (O) Error code
            )

```

□ **Automation call procedure**

```

Program_GetSubProLevel(
    pLevel As LONG*           // (O) Level
) As LONG                    // (O) Error code

```

□ **Argument** *pLevel* : Returns the subprogram call level.
 Value: **0 to 8 (M700 series, C70 and C80)**
 0 to 10 (M800 series)

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK : Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Gets the sub-program call level of the set system.

□ **Function**

□ **Reference** **GetProgramNumber2(), GetSequenceNumber()**

□ **Specifica-
tion** System

IEZNCProgram2::GetInformation**Get program information**

□ Custom call procedure

```

HRESULT      GetInformation(
                LONG InfoType,           // (I) Information type
                LONG* pInfoData,         // (O) User machining program information
                LONG* pRet                // (O) Error code
            )

```

□ Automation call procedure

```

Program_GetInformation(
    InfoType As LONG,           // (I) Information type
    pInfoData As LONG*         // (O) User machining program information
) As LONG                      // (O) Error code

```

□ *InfoType*: Sets the type of information to get.

Argument	Value	Meaning
	EZNC_PRG_MAXNUM	Maximum number of registrable programs
	EZNC_PRG_CURNUM	Number of programs currently registered
	EZNC_PRG_RESTNUM	Remaining number of registrable programs
	EZNC_PRG_CHARNUM	Number of registered characters
	EZNC_PRG_RESTCHARNUM	Remaining number of registrable characters (250 character units)

pInfoData: Returns the program information specified by *InfoType*.

If **EZNC_PRG_MAXNUM** set, *pInfoData* means 1: 200 [programs]. When the return value is N, *pInfoData* means N x 200 [programs]. The data range depends on the specifications of the NC control unit.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_READ: Data is not readable

	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Gets program information. This function is not supported with C80.	

□ Reference

□ Specification

IEZNCProgram2::GetCurrentBlockByByte

Get number of bytes from
start of program

□ Custom call procedure

```

HRESULT      GetCurrentBlockByByte(
                LONG* pSize,                // (O) Number of bytes
                LONG* pRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Program_GetCurrentBlockByByte(
    pSize As LONG*                // (O) Number of bytes
) As LONG                        // (O) Error code

```

□ **Argument** *pSize* : Returns the number of bytes from start of program of the set part system.
Value: From 0

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the number of bytes from the start of the program in the searched block or the block currently stopped with single block stop. This function is not supported with C70/C80.	
Function		

□
Reference

□
**Specifica-
tion**

C70

M700

M800

IEZNcTime::GetClockData

Get date and time

□ **Custom call procedure**

HRESULT **GetClockData**(
 LONG* *pDate*, // (O) Year, month, day
 LONG* *pTime*, // (O) Hour, minute, second
 LONG* *pRet* // (O) Error code
)

□ **Automation call procedure**

Time_GetClockData(
 pDate **As LONG***, // (O) Year, month, day
 pTime **As LONG*** // (O) Hour, minute, second
) **As LONG** // (O) Error code

□ **Argument** *pDate*: Returns the date (year, month, day).
 Output example: 1998/12/05 = 19981205

pTime: Returns the time (hour, minute, second) of the clock in the NC.
 Value: 0 to 235959
 Output example: 23:59:59 = 235959

pRet: Returns an error code. (Upon automation, the return value is used.)
 S_OK: Normal termination
 EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the date and time from the clock in the NC.
 This function is not supported with C80.

□ **Reference** **SetClockData()**

□ **Specifica-
tion**

IEZNCtime::SetClockData**Set date and time**□ **Custom call procedure**

```

HRESULT      SetClockData(
                    LONG IDate,           // (I) Year, month, day
                    LONG ITime,         // (I) Hour, minute, second
                    LONG* pIRet         // (O) Error code
                )

```

□ **Automation call procedure**

```

Time_SetClockData(
    IDate As LONG,           // (I) Year, month, day
    ITime As LONG,         // (I) Hour, minute, second
    ) As LONG                // (O) Error code

```

□ **Argument** *IDate*: Sets the date (year, month, day).
 Setting example: 1998/12/05 = 19981205

ITime: Sets the time (hour, minute, second) for the clock in the NC.
 Value: 0 to 235959
 Setting example: 23:59:59 = 235959

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Sets the date and time for the clock in the NC.	
Function	This function is not supported with C80.	

□ **Reference** **GetClockData()**

□ **Specifica-
tion**

IEZNCtime::GetAliveTime**Get power-on time**□ **Custom call procedure**

```

HRESULT      GetAliveTime(
                LONG* pTime,           // (O) Power-on time
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Time_GetAliveTime(
    pTime As LONG*           // (O) Power-on time
) As LONG                   // (O) Error code

```

□ **Argument** *pTime*: Gets total power-on time (hour, minute, second) from the controller power ON to OFF.
 Value: 0 to 99995959
 Output example: 9999:59:59 = 99995959

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets total power-on time (hour, minute, second) from the controller power ON to OFF.
 Stops integration when the value reaches the maximum value, and retains the maximum value.
 This function is not supported with C80.

□ **Reference** **SetAliveTime()**

□ **Specifica-
tion**

IEZNCtime::SetAliveTime**Set power-on time**

□ Custom call procedure

```

HRESULT      SetAliveTime(
                LONG lTime,                // (I) Power-on time
                LONG* plRet                // (O) Error code
            )

```

□ Automation call procedure

```

Time_SetAliveTime(
    lTime As LONG                // (I) Power-on time
) As LONG                      // (O) Error code

```

- **Argument** *lTime*: Sets total power-on time (hour, minute, second) from the controller power ON to OFF.
 Value: 0 to 599995959 (M700/M800 series)
 Setting example: 59999:59:59 = 599995959
 Value: 0 to 99995959 (C70)
 Setting example: 9999:59:59 = 99995959

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Forcibly sets total power-on time (hour, minute, second) from the controller power ON to OFF.
 This function is not supported with C80.

□ **Reference** **GetAliveTime()**□ **Specifica-
tion**

IEZNCtime::GetRunTime**Get automatic operation time**□ **Custom call procedure**

```

HRESULT      GetRunTime(
                LONG* pTime,           // (O) Automatic operation time
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Time_GetRunTime(
    pTime As LONG*           // (O) Automatic operation time
) As LONG                   // (O) Error code

```

□ **Argument** *pTime*: Returns total processing time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by M02/M30 or the reset operation.
 Value: 0 to 99995959
 Output example: 9999:59:59 = 99995959

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets total processing time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by M02/M30 or the reset operation.
 Stops integration when the value reaches the maximum value, and retains the maximum value.
 This function is not supported with C80.

□ **SetRunTime()**□ **Reference**□ **Specifica-
tion**

IEZNCtime::SetRunTime**Set automatic operation time**□ **Custom call procedure**

```

HRESULT      SetRunTime(
                LONG ITime,                // (I) Automatic operation time
                LONG* pIRet                // (O) Error code
            )

```

□ **Automation call procedure**

```

Time_SetRunTime(
    ITime As LONG                // (I) Automatic operation time
) As LONG                       // (O) Error code

```

- **Argument** *ITime*: Sets total processing time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by M02/M30 or the reset operation.
 Value: 0 to 599995959 (M700/M800 series)
 Setting example: 59999:59:59 = 599995959
 Value: 0 to 99995959 (C70)
 Setting example: 9999:59:59 = 99995959

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Forcibly sets total processing time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by M02/M30 or the reset operation. This function is not supported with C80.	

□ **GetRunTime()**□ **Reference**□ **Specifica-
tion**

IEZNCtime::GetStartTime**Get automatic start time**□ **Custom call procedure**

```

HRESULT      GetStartTime(
                LONG* pTime,           // (O) Automatic start time
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Time_GetStartTime(
    pTime As LONG*           // (O) Automatic start time
) As LONG                   // (O) Error code

```

□ **Argument** *pTime*: Returns total automatic operation time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by feed hold, block stop, or reset.
 Value: 0 to 99995959
 Output example: 9999:59:59 = 99995959

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets total automatic operation time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by feed hold, block stop, or reset.
 This function is not supported with C80.

□ **Reference** **SetStartTime()**

□ **Specifica-
tion**

IEZNCtime::SetStartTime**Set automatic start time**□ **Custom call procedure**

```

HRESULT      SetStartTime(
                LONG ITime,                // (I) Automatic start time
                LONG* pIRet                // (O) Error code
            )

```

□ **Automation call procedure**

```

Time_SetStartTime(
    ITime As LONG                // (I) Automatic start time
) As LONG                        // (O) Error code

```

- **Argument** *ITime*: Sets total automatic operation time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by feed hold, block stop, or reset.
 Value: 0 to 599995959 (M700/M800 series)
 Setting example: 59999:59:59 = 599995959
 Value: 0 to 99995959 (C70)
 Setting example: 9999:59:59 = 99995959

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Sets total automatic operation time (hour, minute, second) from the automatic operation start using memory (tape) or in MDI mode to the termination by feed hold, block stop, or reset. This function is not supported with C80.	

□ **GetStartTime()**□ **Reference**□ **Specifica-
tion**

IEZNCtime::GetEstimateTime**Get external integration time**

□ Custom call procedure

```

HRESULT      GetEstimateTime(
                LONG IKind,                // (I) External integration time type
                LONG* pTime,              // (O) External integration time
                LONG* pRet                 // (O) Error code
            )

```

□ Automation call procedure

```

Time_GetEstimateTime(
    IKind As LONG,                // (I) External integration time type
    pTime As LONG*               // (O) External integration time
) As LONG                        // (O) Error code

```

□ *IKind*: Sets the external integration time type.

Argument	Value	Meaning
	0	External integration time 1 (programmable controller device C70: Y314 M700/M800 series: Y704): When counting with the device turned ON
	1	External integration time 2 (programmable controller device C70: Y315 M700/M800 series: Y705): When counting with the device turned ON

pTime: Returns the time (hour, minute, second) controlled by the programmable controller. Stops integration when the integration time display reaches the maximum value, and retains the display with the maximum value.

Value: 0 to 99995959

Output example: 9999:59:59 = 99995959

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Returns the time (hour, minute, second) controlled by the programmable controller. Starts counting when the user programmable controller device is turned ON. Refer to the "PLC Interface Manual" for each model because the device number differs depending on models. This function is not supported with C80.

□ **Reference** **SetEstimateTime()**□ **Specification**

IEZNCtime::SetEstimateTime**Set external integration time**

□ Custom call procedure

```

HRESULT      SetEstimateTime(
                LONG IKind,           // (I) External integration time type
                LONG ITime,          // (I) External integration time
                LONG* pIRet           // (O) Error code
            )

```

□ Automation call procedure

```

Time_SetEstimateTime(
    IKind As LONG,           // (I) External integration time type
    ITime As LONG           // (I) External integration time
) As LONG                  // (O) Error code

```

□ *IKind*: Sets the external integration time type.

Argument	Value	Meaning
	0	External integration time 1 (programmable controller device C70: Y314 M700/M800 series: Y704): When counting with the device turned ON
	1	External integration time 2 (programmable controller device C70: Y315 M700/M800 series: Y705): When counting with the device turned ON

ITime: Sets the time (hour, minute, second) controlled by the PLC.

Stops integration when the integration time display reaches the maximum value, and retains the display with the maximum value.

Value: 0 to 599995959 (M700/M800 series)

Setting example: 59999:59:59 = 599995959

Value: 0 to 99995959 (C70)

Setting example: 9999:59:59 = 99995959

pIRet: Returns an error code. (Upon automation, the return value is used.)**S_OK**: Normal termination**EZNC_DATA_WRITE_WRITE**: Data is not writable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Sets the time (hour, minute, second) controlled by the programmable controller. Starts counting when the user programmable controller device is turned ON. Refer to the "PLC Interface Manual" for each model because the device number differs depending on models. This function is not supported with C80.	

□ **GetEstimateTime()**

Reference

□ Specification

2.10 IEZNCAxisMonitor Interface

C70

C80

M700

M800

IEZNCAxisMonitor::GetServoMonitor

Get servo monitor

□ Custom call procedure

```
HRESULT      GetServoMonitor(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Data number
                LONG* pIData,          // (O) Monitor data
                LPOLESTR* lppwszBuffer, // (O) Monitor data character string
                LONG* plRet             // (O) Error code
            )
```

□ Automation call procedure

```
Monitor_GetServoMonitor(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Data number
    pIData As LONG*,          // (O) Monitor data
    lppwszBuffer As STRING*    // (O) Monitor data character string
) As LONG                    // (O) Error code
```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from **1**)
Sets in common to all part systems.
Example: When the 1st part system has 3 axes, the 1st axis No. of the 2nd part system is 4.

IIndex: Sets the desired data number. Refer to the table below.

pIData: Returns the monitor data.

lppwszBuffer: Outputs monitor data (return value) as a **UNICODE** character string when any of 100 to 104 is set for *IIndex*.

For the **M700/M800/C80 series**, outputs monitor data (return value) as a **UNICODE** character string when any of 11 to 15, 18 to 20, or 100 to 104 is set for *IIndex*.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_DATASIZE: Application does not fit into prepared buffer

EZNC_DATA_READ_DATATYPE: Invalid data type (parameter number)

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis specification

EZ_ERR_DATA_TYPE: Invalid argument data type

<i>IIndex</i>	Description	Data range	Remarks
0	GAIN. Position loop gain status display.	Unit: 1/s	
1	DROOP. (tracking delay)	Unit: i	
2	SPEED. Actual motor speed.	From 0 [rpm]	
3	CURRENT. Load current. Motor current (displayed by converting to continuous current when stalled).	From 0 [%]	
4	MAXCUR1. Maximum current I.	Unit: %	
5	MAXCUR2. Maximum current II.	Unit: %	
6	OVER LOAD. Overload.	Unit: %	
7	REGEN LOAD. Regenerative load.	Unit: %	
10	CYC CNT. Cycle counter.	Unit: Pulse	

□ Argument	<i>Index</i>	Description	Data range	Remarks
	11	GRIDSP. Grid interval.	Unit: Command unit*	
	12	GRID. Grid amount.	Unit: Command unit*	
	13	MACPOS. Machine position.	Unit: Command unit*	
	14	MOT POS. Motor end FB.	Unit: Command unit*	
	15	SCA POS. Machine end FB.	Unit: Command unit*	
	16	FB ERROR. FB error.	Unit: i	
	17	DFB COMP. DFB compensation amount.		
	18	Remain command	Unit: Command unit*	
	19	Currnt posn.	Unit: Command unit*	
	20	Manual interrupt amount.	Unit: Command unit*	
	100	AMP DISP. Amplifier display. 7-segment LED display on a drive unite.	Outputs a 3-digit character string from "00\0" to "FF\0".	
	101	Alarm 1.	Outputs a 3-digit character string.	
	102	Alarm 2.	Same as above	
	103	Alarm 3.	Same as above	
	104	Alarm 4.	Same as above	

* The M700/M800/C80 series allows acquisition of value (actual value) converted according to the command unit as a character string.

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets servo monitor information of the set axis No. in the set part system.
When the data range in the *Index* table is [Unit: Command unit], the value got needs to be converted according to the command unit sets for the Mitsubishi CNC. For the setting unit, refer to the Mitsubishi CNC specifications.

<For Linear axis>

	Metric system	Conversion for 1 of LONG type
For 10-μm specifications	-999999.99 to 999999.99	1 = 1/200 (mm)
For 1-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (mm)
For 0.1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (mm)
For 10-nm specifications	-999.99999 to 999.99999	1 = 1/200000 (mm)
For 1-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (mm)
	Inch system	Conversion for 1 of LONG type
For 10-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (inch)
For 1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (inch)
For 0.1-μm specifications	-999.99999 to 999.99999	1 = 1/200000 (inch)
For 10-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (inch)
For 1-nm specifications	-9.9999999 to 9.9999999	1 = 1/20000000 (inch)

<For Rotary axis>

	Metric/inch system	Conversion for 1 of LONG type
For 10-μm specifications	-999999.99 to 999999.99	1 = 1/200 (deg)
For 1-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (deg)
For 0.1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (deg)
For 10-nm specifications	-999.99999 to 999.99999	1 = 1/200000 (deg)
For 1-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (deg)

Conversion example)

For the linear axis with the 1-μm specifications in the Metric system, when the LONG value got is 710001,

$$710001 \div 2000 = 355.0005.$$

However, 355.0005 is rounded to minus infinity. Therefore the result is 355.000.

Data in units of 0.5 μm (1/2000 mm) is rounded to minus infinity when displayed.

This means that +0.5 μm is displayed as 0 and -0.5 μm is displayed as -1 μm.

As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with **CoTaskMemFree()**.

□ **GetServoVersion(), GetServoDiagnosis()**

Reference

□ **Specifica-** System, PLC axis, Axis number

tion

IEZNCAxisMonitor::GetServoMonitorEx**Get servo monitor**

□ Custom call procedure

```

HRESULT      GetServoMonitorEx(
                LONG lAxisNo,           // (I) Axis No.
                LONG lIndex,          // (I) Data number
                LONG* pIData,          // (O) Monitor data
                LPOLESTR* lppwszBuffer, // (O) Monitor data character string
                LONG* pIRet           // (O) Error code
                )

```

□ Automation call procedure

```

Monitor_GetServoMonitorEx(
    lAxisNo As LONG,           // (I) Axis No.
    lIndex As LONG,           // (I) Data number
    pIData As LONG*,          // (O) Monitor data
    lppwszBuffer As STRING* // (O) Monitor data character string
) As LONG                     // (O) Error code

```

□ *lAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

lIndex: Sets the desired data number. Refer to the table below.

pIData: Returns the monitor data. Refer to the table below.

lppwszBuffer: Returns the monitor data as a **UNICODE** character string.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_DATASIZE: Application does not fit into prepared buffer

EZNC_DATA_READ_DATATYPE: Invalid data type (parameter number)

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis specification

EZ_ERR_NOT_SUPPORT: Not supported

<i>lIndex</i>	Description	Data range	Return value		Remarks
			<i>pIData</i>	<i>lppwszBuffer</i>	
0	GAIN. Position loop gain status display.	Unit: 1/s	○	○	
1	DROOP. An error between the command position and actual machine position is called "droop". The error is proportional to the command speed value.	Unit: i	○	○	
2	SPEED. Actual motor speed.	From 0 [rpm]	○	○	
3	CURRENT. Load current. Motor current (displayed by converting to continuous current when stalled).	From 0 [%]	○	○	
4	MAXCUR1. Maximum current I.	Unit: %	○	○	
5	MAXCUR2. Maximum current II.	Unit: %	○	○	
6	OVER LOAD. Overload.	Unit: %	○	○	
7	OVER REG. Regenerative load.	Unit: %	○	○	
10	CYC CNT. Cycle counter.	Unit: Pulse	○	○	
11	GRIDSP. Grid interval.	Unit: Command unit		○	(*1)
12	GRID. Grid amount.	Unit: Command unit		○	(*1)
13	MACPOS. Machine position.	Unit: Command unit		○	(*1)

14	MOT POS. Motor end FB.	Unit: Command unit		○	(*1)
15	SCA POS. Machine end FB.	Unit: Command unit		○	(*1)
16	FB ERROR. FB error.	Unit: i	○	○	
17	DFB COMP. DFB compensation amount.		○	○	
18	Remain command	Unit: Command unit		○	(*1)
19	Currnt posn.	Unit: Command unit		○	(*1)
20	Manual interrupt amount.	Unit: Command unit		○	(*1)
100	AMP DISP. Amplifier display. 7-segment LED display on a driver.	Outputs a 3-digit character string from "00\0" to "FF\0".		○	
101	Alarm 1.	Outputs a 3-digit character string.		○	
102	Alarm 2.	Same as above		○	
103	Alarm 3.	Same as above		○	
104	Alarm 4.	Same as above		○	

When *//Index* is **11 to 15, 18 to 20, or 100 to 104**, "0" is returned to *p/Data*.

(*1) The value (actual value) converted according to the command unit can be acquired as a character string.

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure
	Gets servo monitor information of the set axis No.	

When the data range in the *//Index* table is [Unit: Command unit], the value got needs to be converted according to the command unit sets for the Mitsubishi CNC. For the setting unit, refer to the Mitsubishi CNC specifications.

<For Linear axis>

	Metric system	Conversion for 1 of LONG type
For 10-μm specifications	-999999.99 to 999999.99	1 = 1/200 (mm)
For 1-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (mm)
For 0.1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (mm)
For 10-nm specifications	-999.99999 to 999.99999	1 = 1/200000 (mm)
For 1-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (mm)
	Inch system	Conversion for 1 of LONG type
For 10-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (inch)
For 1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (inch)
For 0.1-μm specifications	-999.99999 to 999.99999	1 = 1/200000 (inch)
For 10-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (inch)
For 1-nm specifications	-9.9999999 to 9.9999999	1 = 1/20000000 (inch)

<For Rotary axis>

	Metric/inch system	Conversion for 1 of LONG type
For 10-μm specifications	-999999.99 to 999999.99	1 = 1/200 (deg)
For 1-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (deg)
For 0.1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (deg)
For 10-nm specifications	-999.99999 to 999.99999	1 = 1/200000 (deg)
For 1-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (deg)

Conversion example)

For the linear axis with the 1- μm specifications in the Metric system, when the LONG value got is 710001,

$$710001 \div 2000 = 355.0005.$$

However, 355.0005 is rounded to minus infinity. Therefore the result is 355.000.

Data in units of 0.5 μm (1/2000 mm) is rounded to minus infinity when displayed.

This means that +0.5 μm is displayed as 0 and -0.5 μm is displayed as -1 μm .

As the string area memory is allocated in EZSocket, the client using VC++ needs to release the string area memory explicitly with **CoTaskMemFree()**.

This function is not supported with **C70**.

☐ Reference	GetServoVersion(), GetServoDiagnosis()
--	---

☐ Specifica- tion	<table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>PLC axis</td></tr></table> , <table border="1" style="display: inline-table; vertical-align: middle;"><tr><td>Axis number</td></tr></table>	PLC axis	Axis number
PLC axis			
Axis number			

IEZNCAxisMonitor::GetServoVersion**Get servo axis unit information**

□ Custom call procedure

```

HRESULT      GetServoVersion(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Servo information
                LPOLESTR* IppwszBuffer, // (O) Servo information
                LONG* pIRet            // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetServoVersion(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Servo information
    IppwszBuffer As STRING* // (O) Servo information
) As LONG                   // (O) Error code

```

□ **Argument** IAxisNo: Sets the axis No. (From Axis 1 = from 1)

IIndex: Set the servo information. Refer to the table below.

IppwszBuffer: Sets servo information as a **UNICODE** character string.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis No. setting

IIndex	Description	Data range
0	Unit type	Up to 17 alphanumeric characters.
1	Unit serial No.	Up to 9 alphanumeric characters.
2	Software version	Up to 17 alphanumeric characters.
3	Control method.	Up to 7 alphanumeric characters.
4	Motor end detector	Up to 9 alphanumeric characters.
5	Machine end detector	Up to 9 alphanumeric characters.
6	Motor	Up to 9 alphanumeric characters.

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets servo version information.
 As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with **CoTaskMemFree()**.
 This function is not supported with C80.

□ **Reference** **GetServoMonitor(), GetServoDiagnosis()**

□ **Specifica-
tion** [System], [PLC axis], [Axis number]

IEZNCAxisMonitor::GetServoDiagnosis**Get servo diagnostics information**

□ Custom call procedure

```

HRESULT      GetServoDiagnosis(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Diagnostics information
                LONG* pData,           // (O) Diagnostics information value
                LPOLESTR* lppwszBuffer, // (O) Diagnostics information character string
                LONG* pRet              // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetServoDiagnosis(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Diagnostics information
    pData As LONG*            // (O) Diagnostics information value
    lppwszBuffer As STRING*    // (O) Diagnostics information character string
) As LONG                    // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from **1**)

IIndex: Sets servo diagnostics information. Refer to the table below.

pData: Returns the servo diagnostics information value.

lppwszBuffer: Gets diagnostics information as a **UNICODE** character string. Outputs a character string when *IIndex* is 21 to 30.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis specification

<i>IIndex</i>	Description	Data range
0	Work time	
1	Alarm history 1 (Alarm No).	Previous servo alarm number
2	" 2 (Alarm No).	Same as above
3	" 3 (Alarm No).	Same as above
4	" 4 (Alarm No).	Same as above
5	" 5 (Alarm No).	Same as above
6	" 6 (Alarm No).	Same as above
7	" 7 (Alarm No).	Same as above
8	" 8 (Alarm No).	Same as above
11	Alarm history 1 (time).	Previous servo alarm occurrence time
12	" 2 (time).	Same as above
13	" 3 (time).	Same as above
14	" 4 (time).	Same as above
15	" 5 (time).	Same as above
16	" 6 (time).	Same as above
17	" 7 (time).	Same as above
18	" 8 (time).	Same as above
21	MNT (1).	Up to 3 alphanumeric characters
22	MNT (2).	Same as above
23	MNT (3).	Same as above
24	MNT (4).	Same as above
30	SYS.	Up to 2 alphanumeric characters

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□	Gets servo diagnostics information.
Function	As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with CoTaskMemFree() . This function is not supported with C80.
□	GetServoMonitor(), GetServoVersion()
Reference	
□	
Specifica- tion	[System], [PLC axis], [Axis number]

IEZNCAxisMonitor::GetServoMotorTemperature**Get servo motor temperature**□ **Custom call procedure**

```

HRESULT      GetServoMotorTemperature(
                LONG IAxisNo,           // (I) Axis number
                LONG* pITemperature,    // (O) Servo motor temperature
                LONG* pIRet              // (O) Error code
            )

```

□ **Automation call procedure**

```

Monitor_GetServoMotorTemperature(
    IAxisNo As LONG           // (I) Axis number
    pITemperature As LONG*    // (O) Servo motor temperature
) As LONG                    // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

pITemperature: Returns the servo motor temperature.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis specification

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Gets servo motor temperature.

Function This function is not supported with **M700** series and **C70**.

□ **Reference**

□ **Specifica-
tion** PLC axis, Axis number

IEZNCAxisMonitor::GetPowerVersion**Get power supply version information**

□ Custom call procedure

```

HRESULT      GetPowerVersion(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Version information
                LPOLESTR* lppwszBuffer, // (O) Version information character string
                LONG* pIRet             // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetPowerVersion(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Version information
    lppwszBuffer As STRING*   // (O) Version information
) As LONG                    // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from **1**)

IIndex: Sets version information. Refer to the table below.

lppwszBuffer: Gets version information as a **UNICODE** character string.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

<i>IIndex</i>	Description	Data range
0	Unit type	Up to 17 alphanumeric characters.
1	Unit serial No.	Up to 9 alphanumeric characters.
2	Software version	Up to 17 alphanumeric characters.
3	Connected drive.	1 alphanumeric character.

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets power supply version information.
 As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with **CoTaskMemFree()**.
 This function is not supported with C80.

□ **Reference** **GetPowerDiagnosis()**

□ **Specifica-
tion** Axis number

IEZNCAxisMonitor::GetPowerDiagnosis

Get power supply diagnostics information

□ Custom call procedure

```

HRESULT      GetPowerDiagnosis(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Diagnostics information
                LONG* pIData,          // (O) Diagnostics information value
                LPOLESTR* lppwszBuffer, // (O) Diagnostics information character string
                LONG* pIRet            // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetPowerDiagnosis(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Diagnostics information
    pIData As LONG*,          // (O) Diagnostics information value
    lppwszBuffer As STRING*    // (O) Diagnostics information character string
) As LONG                    // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

IIndex: Sets diagnostics information. Refer to the table below.

pIData: Returns the diagnostics information value.

lppwszBuffer: Gets diagnostics information as a **UNICODE** character string.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

<i>IIndex</i>	Description	Data range
0	Work time	
1	Alarm history 1 (Alarm No).	Previous servo alarm number
2	" 2 (Alarm No).	Same as above
3	" 3 (Alarm No).	Same as above
4	" 4 (Alarm No).	Same as above
5	" 5 (Alarm No).	Same as above
6	" 6 (Alarm No).	Same as above
7	" 7 (Alarm No).	Same as above
8	" 8 (Alarm No).	Same as above
11	Alarm history 1 (time).	Previous servo alarm occurrence time
12	" 2 (time).	Same as above
13	" 3 (time).	Same as above
14	" 4 (time).	Same as above
15	" 5 (time).	Same as above
16	" 6 (time).	Same as above
17	" 7 (time).	Same as above
18	" 8 (time).	Same as above
21	MNT (1).	Up to 3 alphanumeric characters
22	MNT (2).	Same as above
23	MNT (3).	Same as above
24	MNT (4).	Same as above
30	SYS	Up to 2 alphanumeric characters

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets power supply diagnostics information.	
Function	As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with CoTaskMemFree() .	
	This function is not supported with C80.	
□	GetPowerVersion()	
Reference		
□	Axis number	
Specification		

IEZNCAxisMonitor::GetSpindleMonitor**Monitor spindle**

□ Custom call procedure

```

HRESULT      GetSpindleMonitor(
                LONG lIndex,                // (I) Monitor data
                LONG lSpindle,              // (I) Spindle number
                LONG* pData,                // (O) Monitor data value
                LPOLESTR* lppwszBuffer,     // (O) Monitor data character string
                LONG* pRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetSpindleMonitor(
    lIndex As LONG,                // (I) Monitor data
    lSpindle As LONG,              // (I) Spindle number
    pData As LONG*,               // (O) Monitor data value
    lppwszBuffer As STRING*,      // (O) Monitor data character string
) As LONG                        // (O) Error code

```

□ *lIndex*: Sets the parameter number for the set spindle.

Argument

lSpindle: Set the spindle number.

pData: Returns the spindle status.

lppwszBuffer: Gets spindle information as a **UNICODE** character string.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZ_ERR_DATA_TYPE: Invalid argument data type

<i>lIndex</i>	Description	Data range	Remarks
0	Gain. Spindle position loop gain.	Unit: 1/s	
1	Droop. Position deviation amount.	Unit: I	
2	Spindle (SR, SF) rotation speed. Actual spindle motor speed. Including override.	From 0 [rpm]	
3	Load. Spindle motor load.	From 0 [%]	
4	LED display. 7-segment LED display on a driver.	Outputs a 3-digit character string from "00\0" to "FF\0".	
5	Alarm 1.	Up to 3 alphanumeric characters.	
6	Alarm 2.	Same as above	
7	Alarm 3.	Same as above	
8	Alarm 4.	Same as above	M700/M800/C80 series only
10	Cycle counter.		
11	Control input 1.		
12	" 2.		
13	" 3.		
14	" 4.		
15	Control output 1.		
16	" 2.		
17	" 3.		
18	" 4.		

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ Gets the set spindle status.

Function

☐ **GetSpindleVersion(), GetSpindleDiagnosis()**

Reference

☐
**Specifica-
tion**

IEZNCAxisMonitor::GetSpindleVersion**Get spindle unit version information**

□ Custom call procedure

```

HRESULT      GetSpindleVersion(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Version information
                LPOLESTR* lppwszBuffer, // (O) Version information character string
                LONG* pIRet             // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetSpindleVersion(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Version information
    lppwszBuffer As STRING*   // (O) Version information
) As LONG                     // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from **1**)

IIndex: Sets version information. Refer to the table below.

lppwszBuffer: Gets version information as a **UNICODE** character string.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis No. setting

<i>IIndex</i>	Description	Data range
0	Unit type	Up to 17 alphanumeric characters.
1	Unit serial No.	Up to 9 alphanumeric characters.
2	Software version	Up to 17 alphanumeric characters.

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets spindle version information.
 As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with **CoTaskMemFree()**.
 This function is not supported with C80.

□ **Reference** **GetSpindleMonitor(), GetSpindleDiagnosis()**

□ **Specifica-
tion** Axis number

IEZNCAxisMonitor::GetSpindleDiagnosis**Get spindle diagnostics information**

□ Custom call procedure

```

HRESULT      GetSpindleDiagnosis(
                LONG lAxisNo,           // (I) Axis No.
                LONG lIndex,           // (I) Diagnostics information
                LONG* pData,           // (O) Diagnostics information value
                LPOLESTR* lppwszBuffer, // (O) Diagnostics information character string
                LONG* pRet              // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetSpindleDiagnosis(
    lAxisNo As LONG,           // (I) Axis No.
    lIndex As LONG,           // (I) Diagnostics information
    pData As LONG*,           // (O) Diagnostics information value
    lppwszBuffer As STRING*   // (O) Diagnostics information character string
) As LONG                     // (O) Error code

```

□ *lAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

lIndex: Sets diagnostics information.

pData: Returns the diagnostics information value.

lppwszBuffer: Gets diagnostics information as a **UNICODE** character string.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axins No. setting

EZNC_DATA_READ_AXIS: Invalid axis No. setting

EZ_ERR_DATA_TYPE: Invalid argument data type

<i>lIndex</i>	Description	Data range
0	Work time	
1	Alarm history 1 (Alarm No).	Previous spindle alarm number
2	" 2 (Alarm No).	Same as above
3	" 3 (Alarm No).	Same as above
4	" 4 (Alarm No).	Same as above
5	" 5 (Alarm No).	Same as above
6	" 6 (Alarm No).	Same as above
7	" 7 (Alarm No).	Same as above
8	" 8 (Alarm No).	Same as above
11	Alarm history 1 (time).	Previous spindle alarm time
12	" 2 (time).	Same as above
13	" 3 (time).	Same as above
14	" 4 (time).	Same as above
15	" 5 (time).	Same as above
16	" 6 (time).	Same as above
17	" 7 (time).	Same as above
18	" 8 (time).	Same as above
21	MNT (1).	Up to 3 alphanumeric characters.
22	MNT (2).	Same as above
23	MNT (3).	Same as above
24	MNT (4).	Same as above
30	SYS.	Up to 2 alphanumeric characters.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets power supply diagnostics information.	
Function	As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with CoTaskMemFree() .	
	This function is not supported with C80.	
□	GetSpindleMonitor(), GetSpindleVersion()	
Reference		
□	Axis number	
Specification		

IEZNCAxisMonitor::GetSpindleMotorTemperature

Get spindle motor temperature

□ Custom call procedure

```

HRESULT      GetSpindleMotorTemperature(
                LONG iSpindleNo,           // (I) Spindle number
                LONG* pTemperature,        // (O) Spindle motor temperature
                LONG* pRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetSpindleMotorTemperature(
    iSpindleNo As LONG           // (I) Spindle number
    pTemperature As LONG*        // (O) Spindle motor temperature
) As LONG                       // (O) Error code

```

□ **Argument** *iSpindleNo*: Sets the spindle No. (From Axis 1 = from 1)

pTemperature: Returns the spindle motor temperature.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis No. setting

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets spindle motor temperature.
This function is not supported with **M700** series and **C70**.

□ **Reference**

□ **Specifica-
tion** Axis number

IEZNCAxisMonitor::GetAbsPositionMonitor

Get absolute position
monitor information

□ Custom call procedure

```

HRESULT      GetAbsPositionMonitor(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Axis monitor information
                LONG* pData,           // (O) Monitor information value
                LPOLESTR* lppwszBuffer, // (O) Absolute position monitor information
                                                character string
                LONG* pRet              // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetAbsPositionMonitor(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Monitor information
    pData As LONG*,           // (O) Monitor information value
    lppwszBuffer As STRING*    // (O) Absolute position monitor information
                                character string
) As LONG                     // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

IIndex: Sets monitor information.

pData: Returns monitor information value.

lppwszBuffer: Gets information about the absolute position monitor as a **UNICODE** character string. Outputs a character string when *IIndex* is 0.

For the M700/M800 series, outputs the result as a **UNICODE** character string when *IIndex* is 0 to 3.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis No. setting

EZ_ERR_DATA_TYPE: Invalid argument data type

<i>IIndex</i>	Description	Data range	System	Axis	PLC axis
0	ABS SYS. Detection system.	0: Semi closed encoder (ES) 1: Semi closed high-speed serial encoder (ESS) 2: Incremental (INC)	○	○	○
1	POF POS. Power off position.	Unit: Command unit*	○	○	○
2	PON POS. Power on position.	Unit: Command unit*	○	○	○
3	MAC POS. Current position.	Unit: Command unit*	○	○	○
4	R0.		○	○	○
5	P0.		○	○	○
6	E0.		○	○	○
7	Rn.		○	○	○
8	Pn.		○	○	○
9	En.		○	○	○
10	ABSn.		○	○	○
11	ABS0.		○	○	○

* The M700/M800 series allows acquisition of value (actual value) converted according to the command unit as a character string. For conversion according to the command unit, refer to □ Function in IEZNCAxisMonitor::GetServoMonitor().

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets information about the absolute position monitor.	
Function	ABS0 is valid only for the M700/M800 series.	
	As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with CoTaskMemFree() .	
	This function is not supported with C80.	
□	Reference	
□	Specification	
	System, PLC axis, Axis number	

IEZNCAxisMonitor::GetAuxAxisMonitor**Get auxiliary axis monitor information**

□ Custom call procedure

```

HRESULT      GetAuxAxisMonitor (
                LONG lAxisNo,           // (I) Axis No.
                LONG lIndex,           // (I) Auxiliary axis information type
                LONG* plData,          // (O) Auxiliary axis information value
                LPOLESTR* lppwszBuffer, // (O) Auxiliary axis monitor information
                                                character string
                LONG* plRet           // (O) Error code
                )

```

□ Automation call procedure

```

Monitor_GetAuxAxisMonitor (
    lAxisNo As LONG,           // (I) Axis No.
    lIndex As LONG,           // (I) Auxiliary axis information type
    plData As LONG*,         // (O) Auxiliary axis information value
    lppwszBuffer As STRING* // (O) Auxiliary axis monitor information string
) As LONG                    // (O) Error code

```

□ *lAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

lIndex: Sets the auxiliary axis information type. Refer to the table below.

plData: Returns the auxiliary axis information.

lppwszBuffer: Gets monitor information as a **UNICODE** string.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZ_ERR_NOT_SUPPORT: Not supported

<i>lIndex</i>	Description	Data range
0	Droop.	-999 to 999
1	Rotation speed.	0 to 9999 [rpm]
2	Load current.	-999 to 999 [%]
3	Maximum current 1.	-999 to 999 [%]
4	Maximum current 2.	-999 to 999 [%]
5	Motor load	-999 to 999 [%]
6	Regen load.	-999 to 999 [%]
7	Current station No.	1 to 360
8	Current position.	Auxiliary axis information: Parameter value: Unit: Command unit Auxiliary axis monitor information character string: -99999.999 to 99999.999
9	Inst station No.	1 to 360
10	Inst posn:	Auxiliary axis information: Parameter value: Unit: Command unit Auxiliary axis monitor information character string: -99999.999 to 99999.999
11	Position loop gain 1.	0 to 999
12	Speed loop gain 1.	0 to 999
13	Position loop gain 2.	0 to 999
14	Speed loop gain 2.	0 to 999
15	Speed integral compensation.	0 to 999
16	Load inertia.	0 to 999.9

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Gets information about the auxiliary axis monitor.
When the data range in the *Index* table is [Unit: Command unit], the value got needs to be converted according to the command unit set for the Mitsubishi CNC. For the set unit, refer to the Mitsubishi CNC specifications.

<For Linear axis>

	Metric system	Conversion for 1 of LONG type
For 10-μm specifications	-999999.99 to 999999.99	1 = 1/200 (mm)
For 1-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (mm)
For 0.1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (mm)
For 10-nm specifications	-999.99999 to 999.99999	1 = 1/200000 (mm)
For 1-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (mm)
	Inch system	Conversion for 1 of LONG type
For 10-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (inch)
For 1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (inch)
For 0.1-μm specifications	-999.99999 to 999.99999	1 = 1/200000 (inch)
For 10-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (inch)
For 1-nm specifications	-9.9999999 to 9.9999999	1 = 1/20000000 (inch)

<For Rotary axis>

	Metric/inch system	Conversion for 1 of LONG type
For 10-μm specifications	-999999.99 to 999999.99	1 = 1/200 (deg)
For 1-μm specifications	-99999.999 to 99999.999	1 = 1/2000 (deg)
For 0.1-μm specifications	-9999.9999 to 9999.9999	1 = 1/20000 (deg)
For 10-nm specifications	-999.99999 to 999.99999	1 = 1/200000 (deg)
For 1-nm specifications	-99.999999 to 99.999999	1 = 1/2000000 (deg)

Conversion example)

For the linear axis with the 1-μm specifications in the Metric system, when the LONG value got is 710001,

$$710001 \div 2000 = 355.0005.$$

However, 355.0005 is rounded to minus infinity. Therefore the result is 355.000.

Data in units of 0.5 μm (1/2000 mm) is rounded to minus infinity when displayed.

This means that +0.5 μm is displayed as 0 and -0.5 μm is displayed as -1 μm.

As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with **CoTaskMemFree()**.

This function is not supported with C70/C80.

□ Reference

- **Specifica-
tion** Axis number

IEZNCAxisMonitor::GetAuxAxisVersion**Get auxiliary axis version information**□ **Custom call procedure**

```

HRESULT      GetAuxAxisVersion (
                LONG lAxisNo,           // (I) Axis No.
                LONG lIndex,           // (I) Auxiliary axis version information type
                LPOLESTR* lppwszBuffer, // (O) Auxiliary axis version information
                                                character string
                LONG* plRet           // (O) Error code
            )

```

□ **Automation call procedure**

```

Monitor_GetAuxAxisVersion (
    lAxisNo As LONG,           // (I) Axis No.
    lIndex As LONG,           // (I) Auxiliary axis version information type
    lppwszBuffer As STRING* // (O) Auxiliary axis version information
                                                character string
) As LONG                    // (O) Error code

```

□ **Argument** *lAxisNo*: Sets the axis No. (From Axis 1 = from **1**)

lIndex: Sets the auxiliary axis version information type.

lppwszBuffer: Sets the auxiliary axis version information as a **UNICODE** character string.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZ_ERR_NOT_SUPPORT: Not supported

<i>lIndex</i>	Description	Data range
0	Unit type	9 alphanumeric characters
1	Unit serial No.	16 alphanumeric characters
2	Motor	9 alphanumeric characters

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets auxiliary axis version information.
 As the string area memory is allocated in this product, the client using VC++ needs to release the string area memory explicitly with **CoTaskMemFree()**.
 This function is not supported with C70/C80.

□ **Reference**

□ **Specifica-
tion** Axis number

IEZNCAxisMonitor::GetDwellTime**Get remaining dwell time**

□ Custom call procedure

```

HRESULT      GetDwellTime(
                DOUBLE* pdTime,           // (O) Remaining dwell time
                LONG* plRet                // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetDwellTime(
    pdTime As DOUBLE*           // (O) Remaining dwell time
) As LONG                      // (O) Error code

```

□ **Argument** *pdTime*: Returns the remaining dwell (G04) time.
 Unit: Second
 Value: 0.000 to 99999.999 (s)

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Returns the remaining dwell (G04) time. Unit: Second, output up to second, 1/1000 (second).

Function□
Reference

□ **Specifica-
tion** System

IEZNCAxisMonitor::GetPowerConsumption**Get current power consumption**

□ Custom call procedure

```

HRESULT      GetPowerConsumption (
                LONG IAxisNo,                // (I) Axis No.
                DOUBLE** ppdPower,          // (O) Power consumption
                LONG* pIRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetPowerConsumption (
    IAxisNo As LONG,                // (I) Axis No.
    pvPower As VARIANT*            // (O) Power consumption
) As LONG                          // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)**Argument**

ppdPower: Returns the current total power consumption as an array. The data array is secured on the EZSocket side, so explicitly release it on the client side using CoTaskMemFree(). Refer to the index table for the got power consumption.

Index table

Array Index	Type of power consumption
0	Present consumption of entire drive system
1	Present power consumption of servo axis in drive system (fluctuating part)
2	Present power consumption of spindle in drive system (fluctuating part)

Automation argument:

IAxisNo: See the explanation of *IAxisNo*.

pvPower: Returns the power consumption array as a VARIANT.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure
□ Function	Gets the present consumption of entire drive system as an array. This function is not supported with the M700 series and C70/C80.	

□ **Reference**□ **Specifica-
tion**

IEZNCAxisMonitor::GetIntegralPower**Get integral power**

□ Custom call procedure

```

HRESULT      GetIntegralPower (
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) "Enable consumption accumulation
                                         n" signal
                DOUBLE** ppdPower,     // (O) Power consumption
                LONG* plRet            // (O) Error code
            )

```

□ Automation call procedure

```

Monitor_GetIntegralPower (
    IAxisNo As LONG,           // (I) Axis NO.
    IIndex As LONG,           // (I) "Enable consumption accumulation
                               n" signal
    pvPower As VARIANT*      // (O) Power consumption
) As LONG                    // (O) Error code

```

□ **Argument** IAxisNo: Sets the axis No. (From Axis 1 = from 1)

IIndex: Sets "n" for the "Enable consumption accumulation n" signal (PLC to NC: IPCE1 to 4, Y724 to 727). (1 to 4)

ppdPower: Returns the total power consumption as an array. The data array is secured on the EZSocket side, so explicitly release it on the client side using CoTaskMemFree().

Refer to the index table for the got power consumption.

Index table

Array Index	Type of power consumption
0	Accumulated consumption of entire drive system
1	Drive system's fixed consumption correction
2	Accumulated consumption of servo axis in drive system (fluctuating part)
3	Accumulated regeneration of servo axis in drive system (fluctuating part)
4	Accumulated consumption of spindle in drive system (fluctuating part)
5	Accumulated regeneration of spindle in drive system (fluctuating part)

Automation argument:

IAxisNo: See the explanation of IAxisNo.

IIndex: See the explanation of IIndex.

pvPower: Returns the power consumption array as a VARIANT.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the current total power consumption as an array.
This function is not supported with the M700 series and C70/C80.

□ **Reference**□ **Specifica-
tion**

IEZNCAxisMonitor::GetAccumulatedTravelDistance

Get accumulated travel distance

□ Custom call procedure

```

HRESULT      GetAccumulatedTravelDistance(
                                LONG IAxisNo,           // (I) Axis number
                                DWORD* pdwDistance,      // (O) Accumulated travel distance
                                LONG* pIRet              // (O) Error code
                                )

```

□ Automation call procedure

```

Monitor_GetAccumulatedTravelDistance(
    IAxisNo As LONG,           // (I) Axis number
    pIDistance As LONG*      // (O) Accumulated travel distance
    ) As LONG                 // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 to Axis 32)

□ Argument

pdwDistance: Returns the accumulated travel distance.

Automation argument:

pIDistance: See the explanation of *pdwDistance*. LONG type is used for automation call, while DWORD type is used for custom call. This is because unsigned 32-bit data type cannot be got in VBA. The returned data needs to be treated as unsigned 32-bit data.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets accumulated travel distance of servo axis.	
Function	This function is not supported with M700 series and C70 .	
□	Reference	
□	Specifica- tion	

C70

M700

M800

IEZNcRunStatus::GetInvalidStatus

Get disabled status

□ Custom call procedure

```
HRESULT      GetInvalidStatus(  
                LONG* pStatus,           // (O) Disabled status flag  
                LONG* pRet               // (O) Error code  
            )
```

□ Automation call procedure

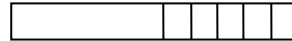
```
Status_GetInvalidStatus(  
    pStatus As LONG*           // (O) Disabled status flag  
    ) As LONG                  // (O) Error code
```

□ *pStatus* : Returns the disabled bit flag.

Argument

Value	Meaning
0	OFF
1	ON

31 4 3 2 1 0 (bit)



Disables single block stop
Waits for the MST command completion signal
Disables feed hold
Disables cutting override
Disables G09 block deceleration check

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid system specification

EZNC_DATA_READ_READ: Data is not readable

□ **Return value**

Value	Meaning
S_OK	Normal termination
S_FALSE	Communication failure

□ **Function**

Returns the disabled flag.
Disabled status: Disables single block stop
Waits for the MST command completion signal
Disables feed hold
Disables cutting override
Disables G09 block deceleration check
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** System

IEZNCRunStatus::GetCommandStatus**Get operation command status**

□ Custom call procedure

```

HRESULT      GetCommandStatus(
                LONG* pIStatus,           // (O) Operation command status
                LONG* pIRet              // (O) Error code
            )

```

□ Automation call procedure

```

Status_GetCommandStatus(
    pIStatus As LONG           // (O) Operation command status
) As LONG                     // (O) Error code

```

□ *pIStatus* : Returns the operation command status with any of the following numbers.

Argument	Value	Meaning	Value	Meaning
	0	Positioning (independent axis)	15	3rd reference position verification
	1	Positioning (linear)	16	4th reference position verification
	2	Linear interpolation	17	Automatic reference position return
	3	Circular interpolation (CW)	18	Return from the automatic reference position
	4	Circular interpolation (CCW)	19	2nd reference position return
	5	Helical interpolation (CW)	20	3rd reference position return
	6	Helical interpolation (CCW)	21	4th reference position return
	7	Reserved	22	Skip function
	8	Reserved	23	Multi-skip function 1
	9	Reserved	24	Multi-skip function 2
	10	Reserved	25	Multi-skip function 3
	11	Time command dwell	26	Threading
	12	Reserved	27	Reserved
	13	1st reference position verification	28	Reserved
	14	2nd reference position verification	29	Coordinate system setting

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the operation command status.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** System

IEZNCRunStatus::GetCuttingMode**Get cutting mode status**□ **Custom call procedure**

```

HRESULT      GetCuttingMode(
                LONG* pIMode,           // (O) Cutting mode
                LONG* pIRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Status_GetCuttingMode(
    pIMode As LONG           // (O) Cutting mode
) As LONG                   // (O) Error code

```

□ **Argument** *pIMode*: Returns the cutting mode.

Value	Meaning
1	In G01, G02, G03, G31, G33, G34, or G35 mode
0	In any of other modes

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the cutting mode.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** System

IEZNCRunStatus::GetAxisStatus**Get servo axis status**

□ Custom call procedure

```

HRESULT      GetAxisStatus(
                LONG IAxisNo,           // (I) Axis No.
                LONG IType,             // (I) Status type
                LONG* pIStatus,         // (O) Servo axis status
                LONG* pIRet              // (O) Error code
            )

```

□ Automation call procedure

```

Status_GetAxisStatus(
    IAxisNo As LONG,           // (I) Axis No.
    IType As LONG,             // (I) Status type
    pIStatus As LONG*         // (O) Servo axis status
) As LONG                     // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)
Valid when *IType* = 4. All axes information in the set part system can be got when *Type* = number other than 4.

IType: Sets the status type.

pIStatus: Returns the servo axis status.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_READ: Data is not readable

<i>IType</i>	Description	Data range
0	1st reference position return completion.	A bit corresponding to the returned axis becomes 1. Example) 00000101 = The 1st and 3rd axes returns have been completed.
1	2nd reference position return completion.	A bit corresponding to the returned axis becomes 1. Example) 00000010 = The 2nd axis return has been completed.
2	3rd reference position return completion.	A bit corresponding to the returned axis becomes 1. Example) 00001010 = The 2nd and 3th axes returns have been completed.
3	Fourth reference position return completion.	A bit corresponding to the returned axis becomes 1. Example) 00001000 = The 4th axis return has been completed.
4	Axis status (axis being removed). The axis specification is required.	0: Axis not being removed 1: Axis being removed
5	Axis status (servo off).	Axis with servo turned off A bit corresponding to the axis becomes 1.
6	Axis status (mirror image).	Mirror image axis. A bit corresponding to the axis set for mirror image becomes 1.

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the servo axis status.
This function is not supported with C80.

□

Reference

□

**Specifica-
tion**

System (Part system setting is required when the M700/M800 series is used or IType is 4 or 6.),

Axis number

IEZNCRunStatus::GetRunStatus**Get operation status**

□ Custom call procedure

```

HRESULT      GetRunStatus(
                LONG lIndex,                // (I) Operation type
                LONG* pIStatus,             // (O) Operation status
                LONG* pIRet                  // (O) Error code
            )

```

□ Automation call procedure

```

Status_GetRunStatus(
    lIndex As LONG,                // (I) Operation type
    pIStatus As LONG*             // (O) Operation status
) As LONG                         // (O) Error code

```

□ *lIndex*: Sets the status number.

Argument

pIStatus: Returns the set operation status.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

<i>lIndex</i>	Description	Data range
0	Tool length measurement.	0: Tool length not being measured 1: Tool length being measured
1	In automatic operation "run". Gets the status indicating that the system is operating automatically.	0: Not operating in automatically 1: Operating automatically
2	Automatic operation "start". Gets the status indicating that the system is operating automatically and that a movement command or M, S, T, B process is being executed.	0: Not starting automatic operation 1: Starting automatic operation
3	Automatic operation "pause". Gets the status indicating that automatic operation is paused while executing a movement command or miscellaneous command with automatic operation.	0: Automatic operation not paused 1: Automatic operation paused

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

-
- **Function** Gets the operation status.
Automatic operation paused is enabled only for the M700/M800/C80 series.
Signals that indicate the status of automatic operation with the PLC interface include 'in automatic operation', 'automatic operation start', and 'automatic operation pause'. The ON/OFF statuses of these three signals in each state are shown below.

	In automatic operation "RUN" (OP)	In automatic operation "START" (STL)	In automatic operation "PAUSE" (SPL)
In "reset"	0	0	0
Automatic operation stop condition	1	0	0
Automatic operation pause condition	1	0	1
Automatic operation start condition	1	1	0

Each status represents the following type of state.

* Reset state

Automatic operation is stopped because of the reset conditions.

(All states in which the system is not operating automatically correspond to this.)

* Automatic operation stopped state

Automatic operation is stopped after executing one block.

(Single block stop corresponds to this.)

* Automatic operation pause state

Automatic operation is stopped during the execution of one block.

(The automatic operation pause (*SP) signal OFF state corresponds to this.)

* Automatic operation started state

Automatic operation is actually being executed.

Refer to the "PLC Interface Manual" for details.

-
- **Reference**

- **Specifica-
tion** System
-

IEZNCRunStatus::GetCycleTime**Get cycle time**□ **Custom call procedure**

```

HRESULT      GetCycleTime(
                LONG* pTime,           // (O) Cycle time
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Status_GetCycleTime(
    pTime As LONG*           // (O) Cycle time
) As LONG                   // (O) Error code

```

□ **Argument** *pTime*: Returns the total hours, minutes and seconds of cycle time.
 Value: 0 to 995959
 Output example: 99:59:59 = 995959

pRet : Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets total hours, minutes and seconds of cycle time.
 This function is not supported with **M700** series and **C70**.

□ **Reference**

□ **Specifica-
tion** System

IEZNCFile6::FindDir2**Search directory**□ **Custom call procedure**

```

HRESULT FindDir2(
    LPCOLESTR lpcwszDirectory, // (I) Directory name
    LONG IFileType,           // (I) Read type and format
    LPOLESTR* lppwszFileInfo, // (O) File information character string
    LONG* plRet               // (O) Error code
)

```

□ **Automation call procedure**

```

File_FindDir2(
    lpcwszDirectory As STRING, // (I) Directory name
    IFileType As LONG,         // (I) Read type and format
    lppwszFileInfo As STRING* // (O) File information character string
) As LONG                     // (O) Error code

```

□ *lpcwszDirectory*: Sets the directory name as a **UNICODE** character string.

Argument Specify directory with an absolute path as follows:

Drive name + ":\Directory name\File name ...Gets the set file name information. (Note 1)
 Drive name + ":\Directory name" ...Gets the set directory name information. (Note 1)
 Drive name + ":\Directory name\" ...Gets the set directory information.

(Note 1) This setting is for the M700/M800 series and C80.

IFileType: Sets the type and format of data to be read.

The following can also be set with pipe (|). When **NULL** is set, file information is read.

Value	Meaning
EZNC_DISK_DIRTYPE	Directory information read
EZNC_DISK_COMMENT	Comment information read (on the NC control unit side only)
EZNC_DISK_DATE	Date information read (on the personal computer side only)
EZNC_DISK_SIZE	Size information read

lppwszFileInfo: Gets file information as a **UNICODE** character string.

The format of file information becomes as follows:

File name\tSize\tDate\tComment\0

A **TAB** code is inserted between file name, size, date, and comments.

The end of the data becomes a **NULL** code.

plRet: Returns whether or not there is file information read or returns an error code. (Upon automation, the return value is used.)

0: When there is no file information

1 or more: When there is file information

EZNC_FILE_DIR_DATASIZE: Exceeded maximum data size

EZNC_FILE_DIR_NOTOPEN: Not open

EZNC_FILE_DIR_READ: File information read error

EZNC_FILE_DIR_ALREADYOPENED: A different directory is already opened

EZNC_FILE_DIR_NODRIVE: Drive does not exist

EZNC_FILE_DIR_NODIR: Directory does not exist

(Note 2) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes

EZNC_PCFILE_....

□ Return value	Value	Meaning
	S_OK S_FALSE	Normal termination Communication failure
□ Function	Searches directory. In FindDir2(), information of one file can be read by reading once. To continuously get directory information, a list of file names in the set directory can be got by calling FindNextDir2() repeatedly. The format of file information to be stored in the area indicated by <i>lpszFileInfo</i> is as follows: File name\tSize\tDate\tComment\t0 A TAB code is inserted between file name, size, date, and comments. The end of the data becomes a NULL code. Only information that follows file name set in the read type is stored. For example, if "EZNC_DISK_COMMENT EZNC_DISK_DATE" is set, information is as follows: Filename\tDate\tComment\t0 If "EZNC_DISK_SIZE EZNC_DISK_COMMENT" is set for a file to which comments cannot be added, the comment information will not be output and the comment will be as follows: File name\tSize\t\t0 For a file from which date cannot get, setting of "EZNC_DISK_SIZE EZNC_DISK_DATE EZNC_DISK_COMMENT" becomes as follows with no date information output. File name\tSize\t\t\tComment\t0 * The file from which a date cannot get refers to the file on the NC control unit side. As the character string area memory is allocated in this product, the client using VC++ needs to release the character string area memory explicitly with CoTaskMemFree(). (Note 1) Reading the directory size information on the NC-side compact flash (M700 series) or SD card (M800 series) is not supported. The directory size information read is invalid. (Note 2) For the C70, when the file on the personal computer is specified and 0 is specified for <i>FileType</i>, \t is added to the end of file information got (filename\t\t0). To use file information got, remove \t before using.	
□ Restrictions	When FindDir2() is used, FindDir2(), OpenFile4(), OpenNcFile3() cannot be executed until ResetDir() is executed. When executed, an error "EZNC_FILE_DIR_ALREADYOPENED (0x80030101) A different directory is already opened" occurs. When using it, immediately execute ResetDir() after executing FindDir2().	
□ Reference	FindNextDir2(), ResetDir()	
□ Specification		

IEZNCFile6::FindNextDir2**Search next directory**□ **Custom call procedure**

```

HRESULT FindNextDir2(
    LPOLESTR* lppwszFileInfo, // (O) File information character string
    LONG* plRet // (O) Error code
)

```

□ **Automation call procedure**

```

File_FindNextDir2(
    lppwszFileInfo As STRING* // (O) File information character string
    ) As LONG // (O) Error code

```

□ *lppwszFileInfo*: Gets file information as a **UNICODE** character string.

Argument The format of file information becomes as follows:

File name\tSize\tDate\tComment\0

A **TAB** code is inserted between file name, size, date, and comments.

The end of the data becomes a **NULL** code.

plRet: Returns whether or not there is file information read or returns an error code. (Upon automation, the return value is used.)

0: When there is no file information

1 or more: When there is file information

EZNC_FILE_DIR_DATASIZE: Exceeded maximum data size

EZNC_FILE_DIR_NOTOPEN: Not open

EZNC_FILE_DIR_READ: File information read error

EZNC_FILE_DIR_NODRIVE: Drive does not exist

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes **EZNC_PCFILE_....**

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Continuously searches for a directory.
 To continuously get directory information after executing **FindDir2()**, a list of file names in the set directory can get by calling **FindNextDir2()** repeatedly. The format of file information that is stored in the area indicated by *lpszFileInfo* is the same as that for **FindDir2()**.
 As the character string area memory is allocated in this product, the client using VC++ needs to release the character string area memory explicitly with **CoTaskMemFree()**.

□ **Reference** **FindDir2(), ResetDir()**

□ **Specifica-
tion**

IEZNCFile6::ResetDir**Terminate directory search**□ **Custom call procedure**

```
HRESULT      ResetDir(
                LONG* pIRet                // (O) Error code
            )
```

□ **Automation call procedure**

```
File_ResetDir( ) As LONG                // (O) Error code
```

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)

EZNC_FILE_DIR_DATASIZE: Exceeded maximum data size

EZNC_FILE_DIR_NOTOPEN: Not open

EZNC_FILE_DIR_READ: File information read error

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes **EZNC_PCFILE_...**

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Terminates directory search.
To search for a directory again, execute **FindDir2()**.

□ **Reference** **FindDir2()**

□ **Specifica-
tion**

IEZNCFile6::CreateDir**Create directory**

□ Custom call procedure

```

HRESULT      CreateDir(
                    LPCOLESTR lpcwszDirectryName,      // (I) Directory name
                    LONG* plRet                        // (O) Error code
                )

```

□ Automation call procedure

```

File_CreateDir(
    lpcwszDirectoryName As STRING      // (I) Directory name
) As LONG                             // (O) Error code

```

- **Argument** *lpcwszDirectryName*: Sets the directory name to be created.
 Directory is set with an absolute path as follows:
 Drive name + ":" + \Directory name\

plRet: Returns an error code. (Upon automation, the return value is used.)

EZNC_FILE_CREATEDIR_FILEEXIST: Directory already exists.

EZNC_FILE_CREATEDIR_FILESYSTEM: File system error.

EZNC_FILE_CREATEDIR_ILLEGALNAME: Invalid file name format.

EZNC_FILE_CREATEDIR_NODIR: Directory does not exist.

EZNC_FILE_CREATEDIR_NOTSUPPORTED: Not supported (unsupported directoy is set)

EZNC_FILE_CREATEDIR_NAMELENGTH: Path is long.

EZNC_FILE_CREATEDIR_MEMORYOVER: Memory capacity exceeded.

EZNC_FILE_CREATEDIR_ALREADYOPENED: The other directory is already open.

EZNC_FILE_CREATEDIR_ROOTDIRFULL: Maximum number of root directories exceeded.

EZNC_FILE_CREATEDIR_WRITEERR: Cannot write.

EZNC_FILE_CREATEDIR_WRITE_PROTECT: Write protect error.

EZNC_FS_OPEN_FILE_FILEFULL: Maximum number of open files exceeded

EZNC_FS_OPEN_FILE_ALREADYOPEN: Already open

EZNC_FS_OPEN_FILE_BUSY: File cannot be opened (during operation)

EZNC_FS_OPEN_FILE_OPEN: File cannot be opened

EZNC_FS_OPEN_FILE_MALLOC: Work area cannot be allocated

EZNC_FS_OPEN_FILE_NOTSUPPORTED: Not supported (CF not supported)

EZNC_FS_OPEN_FILE_NAMELENGTH: File path is too long

EZNC_FS_OPEN_FILE_SORT: Cannot open while sorting tool data

EZNC_FS_OPEN_FILE_SAFE_NOPASSWD: Safety password not authenticated

EZNC_FS_OPEN_FILE_PROTECT: Protect data setting error.

EZNC_FS_OPEN_FILE_WRITE_PROTECT: Write protect error.

EZNC_FILE_DIR_NODRIVE: Drive does not exist

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes **EZNC_PCFILE_...**

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Creates the specified directory.	
Function	NC control unit or personal computer can be set as the drive name.	
	When an NC control unit is set as the drive name, directories cannot be created except the following one(s) for each model.	
	M700 series: CF card	
	M800 series: SD card, USB memory	
	This is valid only for the M700/M800 series. This function is not supported with C70/C80.	
□	DeleteDir()	
Reference		
□		
Specifica- tion		

IEZNCFile6::DeleteDir**Delete directory**

□ Custom call procedure

```

HRESULT      DeleteDir(
                LPCOLESTR lpcwszDirectory, // (I) Directory name
                LONG* plRet                // (O) Error code
            )

```

□ Automation call procedure

```

File_DeleteDir(
    lpcwszDirectory As STRING // (I) Directory name
) As LONG                // (O) Error code

```

- **Argument** *lpcwszDirectory*: Sets the directory name to be deleted as a UNICODE character string.
 Directory is set with an absolute path as follows:
 Drive name + ":" + \Directory name\

plRet: Returns an error code. (Upon automation, the return value is used.)

EZNC_FILE_DELETEDIR_FILESYSTEM: File system error
EZNC_FILE_DELETEDIR_ILLEGALNAME: Invalid file name format
EZNC_FILE_DELETEDIR_NODIR: Directory does not exist
EZNC_FILE_DELETEDIR_NOTSUPPORTED: Not supported (inapplicable directory path is specified.)
EZNC_FILE_DELETEDIR_NAMELENGTH: Path is too long.
EZNC_FILE_DELETEDIR_NOTEMPTY: Directory is not empty.
EZNC_FILE_DELETEDIR_NOTDELETE: Cannot delete.
EZNC_FILE_DELETEDIR_ALREADYOPENED: The other directory is already open.
EZNC_FILE_DELETEDIR_WRITE_PROTECT: Write protect error
EZNC_FS_REMOVE_FILE_ALREADYOPEN: File is already open
EZNC_FS_REMOVE_FILE_BUSY: File cannot be deleted (during operation)
EZNC_FS_REMOVE_FILE_NOFILE: File does not exist
EZNC_FS_REMOVE_FILE_REMOVEERR: File deletion error
EZNC_FS_REMOVE_FILE_NOTSUPPORTED: Not supported (CF not supported)
EZNC_FS_REMOVE_FILE_NAMELENGTH: File path is too long
EZNC_FS_REMOVE_FILE_PROTECT: Protect data setting error
EZNC_FS_REMOVE_FILE_WRITE_PROTECT: Write protect error
EZNC_FILE_DIR_NODRIVE: Drive does not exist
EZNC_FILE_DIR_NODIR: Directory does not exist

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes **EZNC_PCFILE_...**

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Deletes the specified directory.
 NC control unit or personal computer can be set as the drive name.
 When an NC control unit is set as the drive name, directories cannot be deleted except the following one(s) for each model.
 M700 series: CF card
 M800 series: SD card, USB memory

The directory to be deleted must be empty.
 The length of each path is as follows:
 NC card-side drive: 180 characters or less
 PC-side drive: 250 characters or less
 SD card and USB memory: 128 characters or less

This is valid only for the M700/M800 series. This function is not supported with C70/C80.

☐ **CreateDir()**

Reference

☐
**Specifica-
tion**

IEZNCFile6::Copy2**Copy file**

□ Custom call procedure

```

HRESULT      Copy2(
                LPCOLESTR lpcwszSrcFileName,    // (I) Transfer source file name
                LPCOLESTR lpcwszDstFileName,    // (I) Transfer destination file name
                LONG* pRet                       // (O) Error code
            )

```

□ Automation call procedure

```

File_Copy2(
    lpcwszSrcFileName As STRING,    // (I) Transfer source file name
    lpcwszDstFileName As STRING    // (I) Transfer destination file name
) As LONG                          // (O) Error code

```

□ *lpcwszSrcFileName*: Sets transfer source file name using **UNICODE** character strings.

Argument

lpcwszDstFileName: Sets transfer destination file name as a **UNICODE** character strings.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_COPY_BUSY: Copy is disabled (during operation)

EZNC_FILE_COPY_ENTRYOVER: Registration limit exceeded

EZNC_FILE_COPY_FILEEXIST: Copy destination file already exists

EZNC_FILE_COPY_FILESYSTEM: File system error

EZNC_FILE_COPY_ILLEGALNAME: Invalid file name format

EZNC_FILE_COPY_MEMORYOVER: Memory capacity exceeded

EZNC_FILE_COPY_NODIR: Directory does not exist

EZNC_FILE_COPY_NODRIVE: Drive does not exist

EZNC_FILE_COPY_NOFILE: File does not exist

EZNC_FILE_COPY_PLCRUN: Copy is disabled (programmable controller in operation)

EZNC_FILE_COPY_READ: Transfer source file is not readable

EZNC_FILE_COPY_WRITE: Transfer destination file is not writable

EZNC_FILE_COPY_PROTECT: Copying is disabled (protected)

EZNC_PCFILE_COPY_CREATE: File cannot be created (PC only)

EZNC_PCFILE_COPY_OPEN: File cannot be opened (personal computer only)

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes

EZNC_PCFILE_...

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Function	Copies the file set by <i>lpcwszSrcFileName</i> to <i>lpcwszDstFileName</i> . Set a file name with an absolute path as follows: Drive name + ":" + \Directory name\File name
	<i>lpcwszDstFileName</i> must not be a file name that already exists. The transfer destination directory must already exist.
	In the C70, multiple program files under \PRG\USER can be collectively copied to a file on a personal computer. In this case, specify "*" as a file name for <i>lpcwszSrcFileName</i> (example: M01:\PRG\USER*.*). Set any file name on a personal computer for <i>lpcwszDstFileName</i> (example: C:\PLURAL.PRG). To expand files combined into one in the C70, set any file name on a personal computer for <i>lpcwszSrcFileName</i> (example: C:\PLURAL.PRG) and set "ALL.PRG" as a file name under \PRG\USER of <i>lpcwszDstFileName</i> (example: M01:\PRG\USER\ALL.PRG).
	This method does not check whether the set directory and file name are appropriate or not. It is recommended to check the appropriateness of file name and directory for irregular operations such as transfer between files with different types and applications (example: overwriting the user program (\PRG\USER\ to.PRG) with parameter file (PARAMET.BIN)) or copying a file to a directory with different purposes.
	(Note) Do not perform write or overwrite operation during automatic operation of the NC control unit.

□ Reference	Delete2(), Rename2()
□ Specifica- tion	

IEZNCFile6::Delete2**Delete file**

□ Custom call procedure

```

HRESULT      Delete2(
                LPCOLESTR lpcwszFileName,    // (I) File name
                LONG* pRet                    // (O) Error code
            )

```

□ Automation call procedure

```

File_Delete2(
    lpcwszFileName As STRING    // (I) File name
) As LONG                      // (O) Error code

```

□ *lpcwszFileName*: Sets the file name as a **UNICODE** character string.

□ Argument

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK : Normal termination

EZNC_FILE_DEL_BUSY: Deletion is disabled (during operation)

EZNC_FILE_DEL_FILESYSTEM: File system error

EZNC_FILE_DEL_ILLEGALNAME: Invalid file name format

EZNC_FILE_DEL_PROTECT: Deletion is disabled (protected)

EZNC_FILE_DEL_NODIR: Directory does not exist

EZNC_FILE_DEL_NODRIVE: Drive does not exist

EZNC_FILE_DEL_NOFILE: File does not exist

EZNC_PCFIL_DEL_NOTDELETE: File cannot be deleted

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes **EZNC_PCFIL_...**

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Deletes the file set in <i>lpcwszFileName</i> .	
Function	Set a file name with an absolute path as follows:	
	Drive name + ":" + \Directory name\File name	

(Note) Do not perform this operation during automatic operation of the NC control unit.

□ **Copy2(), Rename2()**

□ Reference

□ Specification

IEZNCFile6::Rename2**Change file name**□ **Custom call procedure**

```

HRESULT      Rename2(
                LPCOLESTR lpcwszSrcFileName,    // (I) Old file name
                LPCOLESTR lpcwszDstFileName,    // (I) New file name
                LONG* plRet                     // (O) Error code
            )

```

□ **Automation call procedure**

```

File_Rename2(
    lpcwszSrcFileName As STRING    // (I) Old file name
    lpcwszDstFileName As STRING    // (I) New file name
) As LONG                        // (O) Error code

```

□ *lpcwszSrcFileName*: Sets an old file name.

Argument

lpcwszDstFileName: Sets a new file name.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK : Normal termination

EZNC_FILE_REN_BUSY: Rename is disabled (during operation)

EZNC_FILE_REN_FILEEXIST: New file name already exists

EZNC_FILE_REN_FILESYSTEM: File system error

EZNC_FILE_REN_ILLEGALNAME: Invalid file name format

EZNC_FILE_REN_PROTECT: Rename is disabled (protected)

EZNC_FILE_REN_NODIR: Directory does not exist

EZNC_FILE_REN_NODRIVE: Drive does not exist

EZNC_FILE_REN_NOFILE: File does not exist

EZNC_PCFIL_REN_NOTRENAME: Rename is disabled

EZNC_PCFIL_REN_SAMENAME: New and old file names are identical

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes

EZNC_PCFIL_....

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Changes the file name set for *lpcwszSrcFileName* to that set for *lpcwszDstFileName*.
For *lpszSrcFileName*, set with an absolute path.
Drive name + ":" + \Directory name\File name

For *lpcwszDstFileName*, set only file name without drive name and directory. For *lpcwszDstFileName*, a file name that already exists must not be set.

A directory name can also be changed with M700/M800 series. To change the directory name, "lpszSrcFileName" is set with an absolute path as follows:

Drive name + ":" + \Directory name\

Only the directory name which is to be changed can be set with "lpcwszDstFileName".

The directory name, which is set with "lpcwszDstFileName", must be non-existent directory.

(Note) Do not perform this operation during automatic operation of the NC control unit.

□ **Copy2(), Delete2()**

Reference

□
**Specifica-
tion**

IEZNCFile6::GetDriveInformation**Get drive information**□ **Custom call procedure**

```

HRESULT      GetDriveInformation(
                    LPOLESTR* lppwszDriveInfo,    // (O) Drive information character string
                    LONG* pRet                    // (O) Error code
                )

```

□ **Automation call procedure**

```

File_GetDriveInformation(
    lppwszDriveInfo As STRING*    // (O) Drive information character string
) As LONG                        // (O) Error code

```

- **Argument** *lppwszDriveInfo*: Gets drive information as a **UNICODE** character string.
The format of drive information is as follows:
Drive name:**CRLF**Drive name:**CRLF**...Drive name:**CRLF**\0
To separate drive names, CR, LF codes are inserted between them. The end of the data becomes **CR**, **LF** codes and a **NULL** code. The end of the data becomes a **NULL** code.
- pRet*: Returns the size of drive information got or an error code. (Upon automation, the return value is used.)
0: When a drive does not exist
1 or more: Number of bytes
EZNC_FILE_DRVLIST_READ: Drive information read error
EZNC_FILE_DIR_NODRIVE: Drive does not exist
(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes **EZNC_PCFILE_...**

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Reads drive information of the NC control unit that is currently connected.	
Function	The format of drive information is as follows:	
	Drive name: CRLF Drive name: CRLF ...Drive name: CRLF \0	
	To separate drive names, CR, LF codes are inserted between them. The end of the data becomes CR , LF codes and a NULL code. The end of the data becomes a NULL code.	
	Drive information on a personal computer cannot be read.	
	As the character string area memory is allocated in this product, the client using VC++ needs to release the character string area memory explicitly with CoTaskMemFree() .	

□ **GetDriveSize()**□ **Reference**□ **Specifica-
tion**

IEZNCFile6::GetDriveSize**Get free drive space**□ **Custom call procedure**

```

HRESULT      GetDriveSize(
                    LPCOLESTR lpcwszDirectryName,      // (I) Directory name
                    LONG* pIDriveSize,                // (O) Free space
                    LONG* pIRet                      // (O) Error code
                )

```

□ **Automation call procedure**

```

File_GetDriveSize(
    lpcwszDirectryName As STRING,      // (I) Directory name
    pIDriveSize As LONG*             // (O) Free space
) As LONG                             // (O) Error code

```

□ **Argument** *lpcwszDirectryName*: Sets the directory name as a **UNICODE** character string.
 Directory is set with an absolute path as follows:
 Drive name + ":" + \Directory name\

pIDriveSize: Gets free space of the set directory. (unit: byte)

pIRet: Returns an error code. (Upon automation, the return value is used.)

EZNC_FILE_REN_FILESYSTEM: File system error

EZNC_FILE_DIR_NODRIVE: Drive does not exist

EZNC_FILE_DISKFREE_NODIR: Directory does not exist

EZNC_FILE_DISKFREE_NODRIVE: Drive does not exist

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes **EZNC_PCFILE_....**

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Returns free space of the directory specified for *lpcwszDirectryName*.
 The unit of free space is a byte.
 Directory is specified with an absolute path as follows:
 Drive name + ":" + \Directory name\

When a drive on a personal computer is set for drive name, setting of directory is ignored, and free space of the drive is returned.

When the NC control unit is set for a drive name, free space of the set directory is returned. When a subdirectory exists in the set directory, usage in the subdirectory is excluded from calculation of free space.

(Note) When a directory name (M01:\IC1\) that corresponds to the compact flash (M700 series) or SD card (M800 series) on the NC side is set, up to 2GB can get.

This function is not supported with C80.

□ **GetDriveInformation()**□ **Reference**□ **Specifica-
tion**

IEZNCFile6::GetDriveSize2**Get free drive space**□ **Custom call procedure**

```

HRESULT      GetDriveSize2(
                LPCOLESTR lpcwszDirectryName, // (I) Directory name
                LONG* plDriveSize,           // (O) Free space
                LONG* plRet                  // (O) Error code
                )

```

□ **Automation call procedure**

```

File_GetDriveSize2(
    lpcwszDirectryName As STRING, // (I) Directory name
    plDriveSize As LONG*         // (O) Free space
    ) As LONG                     // (O) Error code

```

□ *lpcwszDirectryName*: Sets the directory name as a **UNICODE** character string.

Argument Directory is set with an absolute path as follows:

Drive name + ":" + \Directory name\

plDriveSize: Gets free space of the set directory. (unit: byte)

Automation argument:

lpcwszDirectryName: See the explanation of *lpcwszDirectryName*.

pvDriveSize: Gets the free space of the set directory as a character string.

plRet: Returns an error code. (Upon automation, the return value is used.)

EZNC_FILE_REN_FILESYSTEM: File system error

EZNC_FILE_DIR_NODRIVE: Drive does not exist

EZNC_FILE_DISKFREE_NODIR: Directory does not exist

EZNC_FILE_DISKFREE_NODRIVE: Drive does not exist

(Note) If an error occurs on the personal computer, the error code **EZNC_FILE_...** becomes

EZNC_PCFILE_...

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Returns free space of the directory set for *lpcwszDirectryName*.

Function The unit of free space is a byte.

Directory is set with an absolute path as follows:

Drive name + ":" + \Directory name\

When a drive on a personal computer is set for drive name, setting of directory is ignored, and free space of the drive is returned.

When the NC control unit is set for a drive name, free space of the set directory is returned.

When a subdirectory exists in the set directory, usage in the subdirectory is excluded from calculation of free space.

This is valid only for the M800 series.

This function is not supported with the M700 series and C70/C80. (EZ_ERR_NOT_SUPPORT is returned to *plRet*.)

□ **GetDriveSize()**

Reference

□ **Specifica-
tion**

IEZNCFile6::OpenFile3**Open file**□ **Custom call procedure**

```

HRESULT      OpenFile3(
                LPCOLESTR lpcwszFileName,    // (I) File name containing a path
                LONG IMode,                  // (I) Open mode
                LONG* pIRet                  // (O) Error code
            )

```

□ **Automation call procedure**

```

File_OpenFile3(
    bstrFileName As STRING,                // (I) File name containing a path
    IMode As LONG                          // (I) Open mode
) As LONG                                   // (O) Error code

```

- **Argument** *lpcwszFileName*: Sets the file name containing a path as a **UNICODE** character string.
 A file is set with an absolute path as follows:
 Drive name + ":" + \Directory name\File name

For a list of files in the NC control unit that is accessible, refer to Table 2-1, Table 2-2, Table 2-3, and Table 2-4. All files except for machining programs in the NC control unit can be backed up, but cannot be edited.

bstrFileName: Refer to the explanation of *lpcwszFileName*.

IMode: Sets open mode.

Value	Meaning
EZNC_FILE_READ	Read mode
EZNC_FILE_WRITE	Write mode
EZNC_FILE_OVERWRITE	Overwrite mode (writes even if the specified file exists.)

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_OPEN_OPEN: File cannot be opened

EZNC_FILE_OPEN_ALREADYOPENED: File is already open

EZNC_FILE_OPEN_FILEEXIST: File already exists (in write mode)

EZNC_FILE_OPEN_FILENOEXIST: File does not exist (in read mode)

EZNC_FILE_OPEN_MODE: Invalid open mode

EZNC_FILE_OPEN_NOTOPEN: File cannot be opened

EZNC_FILE_OPEN_CREATE: Temporary file cannot be created (in write mode)

EZNC_FILE_READFILE_CREATE: Temporary file cannot be created (in read mode)

EZNC_FILE_DIR_NODRIVE: Drive does not exist

EZNC_FILE_DIR_ALREADYOPENED: A different directory is already opened

EZNC_FILE_OPEN_ILLEGALPATH: Invalid file path

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ Function	Opens a file in the specified mode. The directory that creates a temporary file is created in the following order of priority: • Directory set with environment variable TMP • Directory to which the product is installed
----------------------	--

The temporary file name is MELDASn. A number is placed in n.

(Note 1) Make sure to close the open file with CloseFile2() (or AbortFile2()). The temporary file will remain if CloseFile2() is not used.

(Note 2) Do not perform write or overwrite operation during automatic operation of the NC control unit.

(Note 3) It takes C70/C80 approximately 20 s to read SRAM.BIN (SRAM data (binary format)). Other methods cannot be used during that time.

□ Reference	CloseFile2(), AbortFile2(), ReadFile2(), WriteFile()
-----------------------	---

□ Specifica- tion

Table 2-1 M700 series (version A0 or later) List of accessible files

File description	Directory	File name	Remarks
Machining program	M01:\PRG\USER\	Program No.	
MTB macro	M01:\PRG\MMACRO\	Program No.	Program No. is 0100001000 to 0199999999.
Fixed cycle program	M01:\PRG\FIX\	Program No.	Program No. is 0100000010 to 100009999.
MDI program	M01:\PRG\MDI\	MDI.PRG	
Parameters [User, machine]	M01:\PRM\	ALL.PRM	
Auxiliary axis parameter	M01:\PRM\	AUXAXIS.PRM	M700/M700VW only
DeviceNet parameter file	M01:\PRM\	DEVICENT.PRM	
Rotary axis geometric deviation parameter file	M01:\PRM\	GEOMETRY.PRM	
PLC program	M01:\LAD\	USERPLC.LAD	
Workpiece offset data	M01:\DAT\	WORK.OFS	
Tool compensation amount data	M01:\DAT\	TOOL.OFS	
Common variable data	M01:\DAT\	COMMON.VAR	
Custom variable data	M01:\DAT\	CUSTOM.VAR	
SRAM data (binary format)	M01:\DAT\	SRAM.BIN	
Sampling data file (binary format)	M01:\DAT\	SAMPLE.BIN	
Tool life management data file	M01:\DAT\	TLIFE.TLF	
Tool management data file	M01:\DAT\	TOOLMNG.DAT	
SRAM open data file	M01:\DAT\	SRAMOPEN.DAT	
Device open data file	M01:\DAT\	DEVOPEN.DAT	
Machining surface data	M01:\DAT\	RNAV.DAT	
Extended SRAM data (binary format)	M01:\DAT\	EXTSRAM.BIN	
All history	M01:\LOG\	ALLLOGLOG	Key history, alarm history, programmable controller I/O signal history, AC input power supply faults history
Key history	M01:\LOG\	KEYLOGLOG	
Sampling data file	M01:\LOG\	NCSAMP.CSV	
NC-side compact flash	M01:\IC1\	Any	The NC-side compact flash (hereinafter referred to as the "NC-side CF card") is recognized as DS (data server) from the NC control unit. Used for data backup, storing large-capacity programs, etc.

Table 2-2 M800 series List of accessible files

File description	Directory	File name	Remarks
Machining program	M01:\PRG\USER\	Program No.	
MTB macro	M01:\PRG\MMACRO\	Program No.	Program No. is 100010000 to 199999999.
Fixed cycle program	M01:\PRG\FIX\	Program No.	Program No. is 100000010 to 100009999.
MDI program	M01:\PRG\MDI\	MDI.PRG	
Parameters [User, machine]	M01:\PRM\	ALL.PRM	The header (1st line) is different from the M700 series. An M700 file can be used with M800, but an M800 file cannot be used with the M800 series.
Auxiliary axis parameter	M01:\PRM\	AUXAXIS.PRM	
DeviceNet parameter file	M01:\PRM\	DEVICENT.PRM	
Rotary axis geometric deviation parameter file	M01:\PRM\	GEOMETRY.PRM	
Safety parameter file	M01:\PRM\	SAFEPARA.BIN	
System files for maintenance and service	M01:\PRM\	SYSTEM.PRM	This cannot be accessed by the user.
PLC program file	M01:\LAD\	USERPLC.LAD	Not compatible with M700 series.
PLC program file for each project	M01:\LAD\	PROJECTxx.LAD	xx01 to usable project numbers
Own station safety PLC program file	M01:\LAD\	SAFEPLC1.LAD	
Other station safety PLC program file	M01:\LAD\	SAFEPLC2.LAD	
Workpiece offset data file	M01:\DAT\	WORK.OFS	
Tool compensation amount data	M01:\DAT\	TOOL.OFS	
Tool life management data file	M01:\DAT\	TLIFE.TLF	
Common variable data file	M01:\DAT\	COMMON.VAR	The format is different from M700 series
SRAM data	M01:\DAT\	SRAM.BIN	Not compatible with M700 series.
Tool management data file	M01:\DAT\	TOOLMNG.DAT	
SRAM open data file	M01:\DAT\	SRAMOPEN.DAT	
Device open data file	M01:\DAT\	DEVOPEN.DAT	
Machining surface data	M01:\DAT\	RNAV.DAT	
Tool safety data file	M01:\DAT\	TOOLALL.DAT	
Machine manufacturer macro variable data file	M01:\DAT\	MMACRO.VAR	
All history	M01:\LOG\	ALLLOGLOG	The format is different from M700 series.
Key history data file	M01:\LOG\	KEYLOGLOG	The format is different from M700 series.
Touchscreen history	M01:\LOG\	TOUCHLOG.LOG	
MELDAS-NET data file	M01:\LOG\	TRACK.MNT	
Sampling data file	M01:\LOG\	NCSAMP.CSV	The format may be different from M700 series depending on the parameter settings.
Sampling data file	M01:\LOG\	NCSAMP.BIN	
PLC message data file (English)	M01:\PLCMMSG\	PLCMMSG_ENG.TXT	
PLC message data file (Japanese)	M01:\PLCMMSG\	PLCMMSG_JPN.TXT, etc.	
NC-side SD card	M01:\IC1\	Any	For panel-in type display, the system accesses to an SD card (a memory card) mounted on the card slot on the front side of the display unit. For separate type display, the system accesses to an SD card (DS) in the NC control unit.

Table 2-3 C70 List of accessible files

File description	Directory	File name	Remarks
Machining program	M01:\PRG\USER\	Program No. PRG	
Fixed cycle program	M01:\PRG\FIX\	Program No. PRG	
MDI program	M01:\PRG\MDI\	MDI.PRG	
Parameters [User, machine]	M01:\PRM\	ALL.PRM	
PLC program file	M01:\LAD\	USERPLC.LAD	
Workpiece offset	M01:\DAT\	WORK.OFS	
Tool offset data	M01:\DAT\	TOOL.OFS	
Common variable data	M01:\DAT\	COMMON.VAR	
SRAM data (binary format)	M01:\DAT\	SRAM.BIN	For maintenance
Sampling data	M01:\LOG\	NCSAMP.CSV	For maintenance
Operation history data	M01:\LOG\	TRACE.TRC	For maintenance

Table 2-4 C80 List of accessible files

File description	Directory	File name	Remarks
Machining program (Basic area (memory))	M01:\PRG\USER\	Program No.	Program No. is 1 to 99999999. The same name as Extended area (memory2) can be set. Unlike C70, ALL.PRG can not be set.
Machining program (Extended area (memory2))	M01:\BMEM\	Program No.	Program No. is 1 to 99999999. The same name as Basic area (memory) can be set. This can only be used when the applicable C80 supports 2560m and 5120m program memory capacity options.
MTB macro	M01:\PRG\MMACRO\	Program No.	Program No. is 100010000 to 199999999.
Fixed cycle program	M01:\PRG\FIX\	Program No.	Program No. is 100000010 to 100009999.
MDI program	M01:\PRG\MDI\	MDI.PRG	
Parameters [User, machine]	M01:\PRM\	ALL.PRM	
Safety parameters	M01:\PRM\	SAFE PARA.BIN	
Own station safety PLC program file	M01:\LAD\	SAFEPLC1.LAD	
Other station safety PLC program file	M01:\LAD\	SAFEPLC2.LAD	
Workpiece offset data file	M01:\DAT\	WORK.OFS	
Tool offset data file	M01:\DAT\	TOOL.OFS	
Common variable data file	M01:\DAT\	COMMON.VAR	
SRAM data file	M01:\DAT\	SRAM.BIN	
All tool data file	M01:\DAT\	TOOLALL.DAT	
All history	M01:\LOG\	ALLLOGLOG	
Key history data file	M01:\LOG\	KEYLOGLOG	
Touchscreen history	M01:\LOG\	TOUCHLOG.LOG	
Sampling data file	M01:\LOG\	NCSAMP.CSV	
Sampling data file	M01:\LOG\	NCSAMP.BIN	

* All files are not compatible with C70 directly.



Cautions for writing to a file

(Note 1) Carefully check a file before writing to the file for the NC control unit. Writing to an incorrect file may cause unexpected operation, resulting in a serious accident.

(Note 2) Depending on the CNC version and status, you may not be able to access the intended file. When you can not access the intended file, check the CNC specifications.

IEZNCFile6::OpenFile4**Open file**□ **Custom call procedure**

```

HRESULT      OpenFile4(
                LPCOLESTR lpcwszFileName,    // (I) File name containing a path
                LONG IMode,                  // (I) Open mode
                LONG* pIRet                  // (O) Error code
                )

```

□ **Automation call procedure**

```

File_OpenFile4(
    bstrFileName As STRING,                // (I) File name containing a path
    IMode As LONG                          // (I) Open mode
    ) As LONG                               // (O) Error code

```

- **Argument** *lpcwszFileName*: Sets the file name containing a path as a **UNICODE** character string.
 A file is set with an absolute path as follows:
 Drive name + ":" + \Directory name\File name

For a list of files in the NC control unit that is accessible, refer to Table 2-1, Table 2-2 and Table 2-3. All files except for machining programs in the NC control unit can be backed up, but cannot be edited.

bstrFileName: Refer to the explanation of *lpcwszFileName*.

IMode: Sets open mode.

Value	Meaning
EZNC_FILE_READ	Read mode
EZNC_FILE_WRITE	Write mode
EZNC_FILE_OVERWRITE	Overwrite mode (writes even if the specified file exists.)

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_OPEN_OPEN: File cannot be opened

EZNC_FILE_OPEN_ALREADYOPENED: File is already open

EZNC_FILE_OPEN_FILEEXIST: File already exists (in write mode)

EZNC_FILE_OPEN_FILENOEXIST: File does not exist (in read mode)

EZNC_FILE_OPEN_MODE: Invalid open mode

EZNC_FILE_OPEN_NOTOPEN: File cannot be opened

EZNC_FILE_OPEN_CREATE: Temporary file cannot be created (in write mode)

EZNC_FILE_READFILE_CREATE: Temporary file cannot be created (in read mode)

EZNC_FILE_DIR_NODRIVE: Drive does not exist

EZNC_FILE_DIR_ALREADYOPENED: A different directory is already opened

EZNC_FILE_OPEN_ILLEGALPATH: Invalid file path

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□	Function	Opens a file in the specified mode. The directory that creates a temporary file is created in the following order of priority: • Directory set with environment variable TMP • Directory to which the product is installed The temporary file name is MELDASn. A number is placed in n. (Note 1) Make sure to close the open file with CloseFile3() (or AbortFile3()). The temporary file will remain if CloseFile3() is not used. (Note 2) Do not perform write or overwrite operation during automatic operation of the NC control unit. (Note 3) It takes C70/C80 approximately 20 s to read SRAM.BIN (SRAM data (binary format)). Other methods cannot be used during that time. (Note 4) This method cannot be used combined with CloseFile2(), AbortFile2(), or ReadFile2(). Use CloseFile3(), AbortFile3(), or ReadFile3() to operate files opened with this method. WriteFile() can be used in the same manner consecutively.
□	Reference	CloseFile3(), AbortFile3(), ReadFile3(), WriteFile()
□	Specifica- tion	

IEZNCFile6::CloseFile2**Close file**□ **Custom call procedure**

```

HRESULT      CloseFile2(
                LONG* pIRet
            )
// (O) Error code

```

□ **Automation call procedure**

```

File_CloseFile2(
    ) As LONG
// (O) Error code

```

□ *pIRet*: Returns an error code. (Upon automation, the return value is used.)

□ **Argument** **S_OK**: Normal termination

EZNC_FILE_WRITEFILE_WRITE: File is not writable

□	Value	Meaning
---	-------	---------

□ Return value	S_OK	Normal termination
-----------------------	-------------	--------------------

	S_FALSE	Communication failure
--	----------------	-----------------------

□ **Function** Closes a file. When the file was opened with **OpenFile3()**, make sure to close it with **CloseFile2()** (or **AbortFile2()**).

(Note) Do not perform this operation during automatic operation of the NC control unit when a writing or overwriting operation has been performed.

□ **Reference** **OpenFile3(), AbortFile2(), ReadFile2(), WriteFile()**

□ **Specifica-
tion**

IEZNCFile6::CloseFile3**Close file**□ **Custom call procedure**

```

HRESULT      CloseFile3(
                LONG* pIRet
            )
// (O) Error code

```

□ **Automation call procedure**

```

File_CloseFile3(
    ) As LONG
// (O) Error code

```

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_WRITEFILE_WRITE: File is not writable

Return value	Value	Meaning
S_OK		Normal termination
S_FALSE		Communication failure

□ **Function** Closes a file. When the file was opened with **OpenFile4()**, make sure to close it with **CloseFile3()** (or **AbortFile3()**).

(Note 1) Do not perform this operation during automatic operation of the NC control unit when a writing or overwriting operation has been performed.

(Note 2) This method cannot be used combined with **OpenFile3()**.

□ **Reference** **OpenFile4()**, **AbortFile3()**, **ReadFile3()**, **WriteFile()**

□ **Specifica-
tion**

IEZNCFile6::AbortFile2**Force close file**□ **Custom call procedure**

```

HRESULT      AbortFile2(
                LONG* pIRet
            )

```

// (O) Error code

□ **Automation call procedure**

```

File_AbortFile2(
    ) As LONG

```

// (O) Error code

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Forcibly closes an open file. Use this to stop writing. After writing is stopped, the file which was being written to will be deleted.
 The difference from **CloseFile2()** is that an error is not output.
 This function is not supported with C80.

□ **Reference** **OpenFile3(), CloseFile2(), ReadFile2(), WriteFile()**

□ **Specifica-
tion**

IEZNCFile6::AbortFile3**Force close file**

□ Custom call procedure

```

HRESULT      AbortFile3(
                LONG* pIRet
            )
// (O) Error code

```

□ Automation call procedure

```

File_AbortFile3(
    ) As LONG
// (O) Error code

```

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Forcibly closes an open file. Use this to stop writing. After writing is stopped, the file which was being written to will be deleted.
The difference from **CloseFile3()** is that an error is not output.

(Note 1) This method cannot be used combined with **OpenFile3()**.
This function is not supported with C80.

□ **Reference** **OpenFile4(), CloseFile3(), ReadFile3(), WriteFile()**

□ **Specifica-
tion**

IEZNCFile6::ReadFile2**Read file**

□ Custom call procedure

```

HRESULT      ReadFile2(
                DWORD dwLength,           // (I) Size of data to be read
                BYTE** ppbData,          // (O) Read data
                DWORD* pdwNumRead,       // (O) Read data size
                LONG* plRet               // (O) Error code
            )

```

□ Automation call procedure

```

File_ReadFile2(
    ILength As LONG,           // (I) Size of data to be read
    pvData As VARIANT*        // (O) Read data
) As LONG                     // (O) Error code

```

□ **Argument** *dwLength*: Sets the size of data to be read at a time in the number of bytes.

ppbData: Returns the pointer for the read byte data array. As the read data area is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

pdwNumRead: Returns the number of bytes that were actually read. In automation call, the **VARIANT** data includes the number of bytes.

Automation argument:

ILength: Refer to the explanation of *dwLength*.

pvData: Returns the read byte data array in VARIANT.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_READFILE_NOTOPEN: No file is open in the read mode

EZNC_FILE_READFILE_READ: File is not readable

EZNC_FILE_READFILE_CREATE: Temporary file cannot be created

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Reads data from the file opened with OpenFile3() . Data to be read returns a byte data array and its number of bytes. Determines as the end of file when <i>pdwNumRead</i> is smaller than <i>dwLength</i> . Sets the size of data to be read at a time. When reading a large file, it can be read in multiple parts. The file can be read in sequence until CloseFile2() is executed.	
Function		

□ **Reference** **OpenFile3(), CloseFile2(), AbortFile2(), WriteFile()**

□ **Specifica-
tion**

IEZNCFile6::ReadFile3**Read file**

□ Custom call procedure

```

HRESULT      ReadFile3(
                DWORD dwLength,           // (I) Size of data to be read
                BYTE** ppbData,          // (O) Read data
                DWORD* pdwNumRead,       // (O) Read data size
                LONG* plRet               // (O) Error code
            )

```

□ Automation call procedure

```

File_ReadFile3(
    ILength As LONG,           // (I) Size of data to be read
    pvData As VARIANT*        // (O) Read data
) As LONG                     // (O) Error code

```

□ **Argument** *dwLength*: Sets the size of data to be read at a time in the number of bytes.

ppbData: Returns the pointer for the read byte data array. As the read data area is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

pdwNumRead: Returns the number of bytes that were actually read. In automation call, the **VARIANT** data includes the number of bytes.

Automation argument:

ILength: Refer to the explanation of *dwLength*.

pvData: Returns the read byte data array in VARIANT.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_READFILE_NOTOPEN: No file is open in the read mode

EZNC_FILE_READFILE_READ: File is not readable

EZNC_FILE_READFILE_CREATE: Temporary file cannot be created

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Reads data from the file opened with **OpenFile4()**. Data to be read returns a byte data array and its number of bytes. Determines as the end of file when *pdwNumRead* is smaller than *dwLength*. Setting the size of data to be read at a time. When reading a large file, it can be read in multiple parts. The file can be read in sequence until **CloseFile4()** is executed.

(Note1) This method cannot be used combined with **OpenFile3()**.

□ **Reference** **OpenFile4(), CloseFile3(), AbortFile3(), WriteFile()**

□ **Specifica-
tion**

IEZNCFile6::WriteFile**Write file**□ **Custom call procedure**

```

HRESULT      WriteFile(
                DWORD dwLength,           // (I) Size of data to be written
                BYTE* pbData,           // (I) Data to be written
                LONG* pRet              // (O) Error code
            )

```

□ **Automation call procedure**

```

File_WriteFile(
    vData As VARIANT           // (I) Data to be written
) As LONG                     // (O) Error code

```

□ **Argument** *dwLength*: Sets the size of data that is written at a time in the number of bytes.

pbData: Sets data to be written as byte array.

Automation argument:

vData: Creates data to be written as a byte array and sets it by substituting it in *vData* (VARIANT type) as shown in the example below.

```

Example) Dim vWriteFile As Variant
         Dim byteWrite() As Byte
         vWriteFile = byteWrite

```

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_WRITEFILE_NOTOPEN: No file is open in write mode

EZNC_FILE_WRITEFILE_WRITE: Cannot be written to a file

EZNC_FILE_LENGTH: Invalid write data size

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Writes data to a file opened with OpenFile4() . Data to be written is data in a byte array. Sets the size of data to be written at a time. When writing a large amount of data, it can be written in multiple parts. Data can be written in sequence until CloseFile3() is executed. (Note 1) When a file in the NC control unit except for a machining program is changed, extra care should be taken because it may cause the NC control unit malfunction. Make sure to back up in advance to restore to its original state.	
Function		

□ **Reference** **OpenFile4(), CloseFile3(), AbortFile3(), ReadFile3()**□ **Specifica-
tion**

Cautions for writing to a file

Carefully check a file before writing to the file for the NC control unit. Writing to an incorrect file may cause unexpected operation, resulting in a serious accident.

IEZNCFile6::ForceReadFile**Force read file**□ **Custom call procedure**

```

HRESULT      ForceReadFile(
                LPCOLESTR lpcwszFileName,           // (I) File name containing a path
                DWORD dwLength,                     // (I) Size of data to be read
                BYTE** ppbData,                     // (O) Read data
                DWORD* pdwNumRead,                 // (O) Read data size
                LONG* plRet                         // (O) Error code
            )

```

□ *lpcwszFileName*: Sets the file name containing a path as UNICODE string.

Argument The file name is set with an absolute path as follows:
 Drive name + ":" + \Directory name\file name\

dwLength: Sets the size of data to be read at a time in the number of bytes.

ppbData: Returns the pointer for the read byte data array. As the read data area is allocated in this product, the client needs to release it explicitly with CoTaskMemFree().

pdwNumRead: Returns the number of bytes that were actually read. In automation call, the VARIANT data includes the number of bytes.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_OPEN_FILENOEXIST: File does not exist

EZNC_FILE_READFILE_CREATE: Temporary file cannot be created

EZNC_FILE_DIR_NODRIVE: Drive does not exist

EZNC_FILE_OPEN_ILLEGALPATH: Invalid file path

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Sets the size of data to be read, and reads it from the file forcibly.	
Function	Data to be read returns a byte data array and its number of bytes. Determines as the end of file when <i>pdwNumRead</i> is smaller than <i>dwLength</i> .	

This is valid only for the C70. This function is not supported with the other models.

□ **Reference**

□ **Specifica-
tion**

IEZNCFile6::SpecialReadFile**Special read file**□ **Custom call procedure**

```

HRESULT      ForceReadFile(
                LPCOLESTR lpcwszFileName,           // (I) File name containing a path
                BYTE** ppbData,                     // (O) Read data
                DWORD* pdwNumRead,                 // (O) Read data size
                LONG* plRet                       // (O) Error code
                )

```

□ **Automation call procedure**

```

File_SpecialReadFile(
    bstrFileName As STRING,           // (I) File name containing a path
    pvData As VARIANT*              // (O) Read data
    ) As LONG                        // (O) Error code

```

□ **Argument** *lpcwszFileName*: Sets the file name containing a path as a **UNICODE** character string.
 A file is set with an absolute path as follows:
 Drive name + ":" + \Directory name\File name

dwLength: Sets the size of data to be read at a time in the number of bytes.

ppbData: Returns the pointer for the read byte data array. As the read data area is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

pdwNumRead: Returns the number of bytes that were actually read. In automation call, the **VARIANT** data includes the number of bytes.

Automation argument:

bstrFileName: See the explanation of *lpcwszFileName*.

pvData: Returns the read byte data array in **VARIANT**.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_DIR_NODRIVE: Drive does not exist

EZNC_FS_OPEN_FILE_OPEN: File cannot be opened

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Data set by <i>lpcwszFileName</i> can be read from the NC system. Special files, which cannot be opened with OpenFile4() , can be read. Data to be read returns a byte data array and its number of bytes.	
Function		

This function is valid only for the M700/M800 series and C80. This function is not supported with the other models.

□ **Reference**

□ **Specifica-
tion**

IEZNCFile6::OpenNCFile2**Open machining program dedicated file**□ **Custom call procedure**

```

HRESULT      OpenNCFile2 (
                LPCOLESTR lpcwszFileName,           // (I) File name containing a path
                LONG IMode,                         // (I) Open mode
                LONG* plRet                         // (O) Error code
            )

```

□ **Automation call procedure**

```

File_OpenNCFile2 (
    bstrFileName As STRING,           // (I) File name containing a path
    IMode As LONG                    // (I) Open mode
) As LONG                           // (O) Error code

```

□ *lpcwszFileName*: Sets the file name containing a path as a **UNICODE** character string.

Argument A file is set with an absolute path as follows:
Drive name + ":" + \Directory name\File name

Paths other than those shown below cannot be used.

Model	Machining program
M700 series	M01:\PRG\USER\Machining Program No. M01:\PRG\UMACRO\Machining Program No. M01:\PRG\MMACRO\Machining Program No. M01:\PRG\FIX\Machining Program No. M01:\PRG\MDI\Machining Program No.
M800 series	M01:\PRG\USER\Machining Program Name (32 or less alphanumeric characters including extension) M01:\PRG\MMACRO\Machining Program No. (100010000 –199999999) M01:\PRG\FIX\Machining Program No. (100000010 –100009999) M01:\PRG\MDI\MDI.PRG

bstrFileName: Refer to the explanation of *lpcwszFileName*.

IMode: Sets open mode.

Value	Meaning
EZNC_FILE_READ	Read mode
EZNC_FILE_WRITE	Write mode
EZNC_FILE_OVERWRITE	Overwrite mode (writes even if the specified file exists.)

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_OPEN_ALREADYOPENED: File is already open

EZNC_FILE_OPEN_FILEEXIST: File already exists (in write mode)

EZNC_FILE_OPEN_MODE: Invalid open mode

EZNC_FILE_OPEN_NOTOPEN: File cannot be opened

EZNC_FILE_OPEN_CREATE: File cannot be created (in write mode)

EZNC_FILE_OPEN_ILLEGALPATH: Invalid path

EZNC_FILE_OPEN_FILENOTEXIST: File does not exist

EZNC_FILE_OPEN_OPEN: File cannot be opened

EZNC_FILE_DIR_ALREADYOPENED: A different directory is already opened

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Function Opens the machining program file in the set mode. The directory that creates a temporary file is created in the following order of priority: • Directory specified with environment variable TMP • Directory to which the product is installed The temporary file name is "MELDASn". A number is placed in n. OpenFile3() cannot be used concurrently. This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to plRet.) (Note 1) Make sure to close the open file with CloseNCFile2() (or AbortNCFile2()). If CloseNCFile2() is not used, a temporary file will remain. (Note 2) Do not perform a writing or overwriting operation during automatic operation of the NC control unit.	
□	CloseNCFile2(), AbortNCFile2(), ReadNCFile2(), WriteNCFile()	
Reference		
□		
Specifica- tion		

IEZNCFile6::OpenNCFile3**Open machining program dedicated file**

□ Custom call procedure

```

HRESULT      OpenNCFile3 (
                LPCOLESTR lpcwszFileName,           // (I) File name containing a path
                LONG IMode,                         // (I) Open mode
                LONG* plRet                         // (O) Error code
            )

```

□ Automation call procedure

```

File_OpenNCFile3 (
    bstrFileName As STRING,           // (I) File name containing a path
    IMode As LONG                    // (I) Open mode
) As LONG                           // (O) Error code

```

□ *lpcwszFileName*: Sets the file name containing a path as a **UNICODE** character string.

Argument A file is set with an absolute path as follows:
Drive name + ":" + \Directory name\File name

Paths other than those shown below cannot be used.

Model	Machining program
M700 series	M01:\PRG\USER\Machining Program No. M01:\PRG\UMACRO\Machining Program No. M01:\PRG\MMACRO\Machining Program No. M01:\PRG\FIX\Machining Program No. M01:\PRG\MDI\Machining Program No.
M800 series	M01:\PRG\USER\Machining Program Name (32 or less alphanumeric characters including extension) M01:\PRG\MMACRO\Machining Program No. (100010000 –199999999) M01:\PRG\FIX\Machining Program No. (100000010 –100009999) M01:\PRG\MDI\MDI.PRG

bstrFileName: Refer to the explanation of *lpcwszFileName*.

IMode: Sets open mode.

Value	Meaning
EZNC_FILE_READ	Read mode
EZNC_FILE_WRITE	Write mode
EZNC_FILE_OVERWRITE	Overwrite mode (writes even if the specified file exists.)

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_OPEN_ALREADYOPENED: File is already open

EZNC_FILE_OPEN_FILEEXIST: File already exists (in write mode)

EZNC_FILE_OPEN_MODE: Invalid open mode

EZNC_FILE_OPEN_NOTOPEN: File cannot be opened

EZNC_FILE_OPEN_CREATE: File cannot be created (in write mode)

EZNC_FILE_OPEN_ILLEGALPATH: Invalid path

EZNC_FILE_OPEN_FILENOTEXIST: File does not exist

EZNC_FILE_OPEN_OPEN: File cannot be opened

EZNC_FILE_DIR_ALREADYOPENED: A different directory is already opened

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Function Opens the machining program file in the set mode. The directory that creates a temporary file is created in the following order of priority: • Directory specified with environment variable TMP • Directory to which the product is installed The temporary file name is "MELDASn". A number is placed in n. OpenFile4() cannot be used concurrently. This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to plRet.) (Note 1) Make sure to close the open file with CloseNCFile3() (or AbortNCFile3()). If CloseNCFile3() is not used, a temporary file will remain. (Note 2) Do not perform a writing or overwriting operation during automatic operation of the NC control unit. (Note 3) This method cannot be used combined with CloseNCFile2(), CloseNCFile2(), or ReadNCFile2(). Use CloseNCFile3(), AbortNCFile3(), or ReadNCFile3() to operate files opened with this method. WriteNCFile() can be used in the same manner consecutively.	
□	CloseNCFile3(), AbortNCFile3(), ReadNCFile3(), WriteNCFile()	
Reference		
□		
Specifica- tion		

IEZNCFile6::CloseNCFile2**Close machining program dedicated file**□ **Custom call procedure**

```
HRESULT      CloseNCFile2(
                LONG* pRet                                // (O) Error code
            )
```

□ **Automation call procedure**

```
File_CloseNCFile2(
    ) As LONG                                            // (O) Error code
```

□ **Argument** *pRet*: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_WRITEFILE_WRITE: File is not writable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Closes the machining program file.

This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pRet.)

(Note 1) Do not perform this operation during automatic operation of the NC control unit when a writing or overwriting operation has been performed.

(Note 2) When the file was opened with **OpenNCFile2()**, make sure to close it with **CloseNCFile2()** (or **AborNCtFile2()**).

□ **Reference** **OpenNCFile2()**, **AbortNCFile2()**, **ReadNCFile2()**, **WriteNCFile()**

□ **Specifica-
tion**

IEZNCFile6::CloseNCFile3**Close machining program dedicated file**□ **Custom call procedure**

```

HRESULT      CloseNCFile3(
                LONG* pIRet                                // (O) Error code
            )

```

□ **Automation call procedure**

```

File_CloseNCFile3(
    ) As LONG                                            // (O) Error code

```

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_WRITEFILE_WRITE: File is not writable

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Closes the machining program file.
This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pIRet.)

(Note 1) Do not perform this operation during automatic operation of the NC control unit when a writing or overwriting operation has been performed.

(Note 2) When the file was opened with **OpenNCFile3()**, make sure to close it with **CloseNCFile3()** (or **AbortNCFile3()**).

(Note 3) This method cannot be used combined with **OpenNCFile2()**.

□ **Reference** **OpenNCFile3()**, **AbortNCFile3()**, **ReadNCFile3()**, **WriteNCFile()**

□ **Specifica-
tion**

IEZNCFile6::AbortNCFile2**Force close machining program dedicated file**□ **Custom call procedure**

```
HRESULT      AbortNCFile2(
                LONG* pIRet                                // (O) Error code
            )
```

□ **Automation call procedure**

```
File_AbortNCFile2(
    ) As LONG                                            // (O) Error code
```

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination
EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Force closes the open machining program file. Use this to stop writing. After writing is stopped, the file which was being written to will be deleted.
 The difference from **CloseNCFile2()** is that an error is not output.
 This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pIRet.)

□ **Reference** OpenNCFile2(), CloseNCFile2(), ReadNCFile2(), WriteNCFile()

□ **Specifica-
tion**

IEZNCFile6::AbortNCFile3**Force close machining program dedicated file**□ **Custom call procedure**

```
HRESULT      AbortNCFile3(
                LONG* pRet                                     // (O) Error code
            )
```

□ **Automation call procedure**

```
File_AbortNCFile3(
    ) As LONG                                           // (O) Error code
```

□ **Argument** *pRet*: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination
EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Force closes the open machining program file. Use this to stop writing. After writing is stopped, the file which was being written to will be deleted.
The difference from **CloseNCFile3()** is that an error is not output.
This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pRet.)

(Note 1) This method cannot be used combined with **OpenNCFile2()**.

□ **Reference** OpenNCFile3(), CloseNCFile3(), ReadNCFile3(), WriteNCFile()

□ **Specifica-
tion**

IEZNCFile6::ReadNCFile2**Read machining program dedicated file**

□ Custom call procedure

```

HRESULT      ReadNCFile2(
                DWORD dwLength,           // (I) Size of data to be read
                BYTE** ppbData,          // (O) Read data
                DWORD* pdwNumRead,        // (O) Read data size
                LONG* plRet               // (O) Error code
            )

```

□ Automation call procedure

```

File_ReadNCFile2(
    ILength As LONG,           // (I) Size of data to be read
    pvData As VARIANT*        // (O) Read data
) As LONG                     // (O) Error code

```

□ **Argument** *dwLength*: Sets the size of data to be read at a time in the number of bytes.

ppbData: Returns the pointer for the read byte data array. As the read data area is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

pdwNumRead: Returns the number of bytes that were actually read. In automation call, the **VARIANT** data includes the number of bytes.

Automation argument:

ILength: Refer to the explanation of *dwLength*.

pvData: Returns the read byte data array in VARIANT.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_READFILE_NOTOPEN: No file is open in the read mode

EZNC_FILE_READFILE_READ: File is not readable

EZNC_FILE_READFILE_CREATE: Temporary file cannot be created

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Data is read from the machining program file opened with OpenNCFile2() . Data to be read returns a byte data array and its number of bytes. Determines as the end of file when <i>pdwNumRead</i> is smaller than <i>dwLength</i> .	
	Sets the size of data to be read at a time. When reading a large file, it can be read in multiple parts. The file can be read in sequence until CloseNCFile2() is executed.	
	This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to <i>plRet</i> .)	

□ **Reference** **OpenNCFile2(), CloseNCFile2(), AbortNCFile2(), WriteNCFile()**

□ **Specifica-
tion**

IEZNCFile6::ReadNCFile3**Read machining program dedicated file**□ **Custom call procedure**

```

HRESULT      ReadNCFile3(
                DWORD dwLength,           // (I) Size of data to be read
                BYTE** ppbData,          // (O) Read data
                DWORD* pdwNumRead,       // (O) Read data size
                LONG* plRet              // (O) Error code
            )

```

□ **Automation call procedure**

```

File_ReadNCFile3(
    lLength As LONG,           // (I) Size of data to be read
    pvData As VARIANT*       // (O) Read data
    ) As LONG                 // (O) Error code

```

□ **Argument** *dwLength*: Sets the size of data to be read at a time in the number of bytes.

ppbData: Returns the pointer for the read byte data array. As the read data area is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

pdwNumRead: Returns the number of bytes that were actually read. In automation call, the **VARIANT** data includes the number of bytes.

Automation argument:

lLength: Refer to the explanation of *dwLength*.

pvData: Returns the read byte data array in **VARIANT**.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_READFILE_NOTOPEN: No file is open in the read mode

EZNC_FILE_READFILE_READ: File is not readable

EZNC_FILE_READFILE_CREATE: Temporary file cannot be created

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Data is read from the machining program file opened with **OpenNCFile3()**. Data to be read returns a byte data array and its number of bytes. Determines as the end of file when *pdwNumRead* is smaller than *dwLength*.
Sets the size of data to be read at a time. When reading a large file, it can be read in multiple parts. The file can be read in sequence until **CloseNCFile3()** is executed.
This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to *plRet*.)

(Note 1) This method cannot be used combined with **OpenNCFile2()**.

□ **Reference** **OpenNCFile3(), CloseNCFile3(), AbortNCFile3(), WriteNCFile()**

□ **Specifica-
tion**

IEZNCFile6::WriteNCFile**Write machining program dedicated file**□ **Custom call procedure**

```

HRESULT      WriteNCFile(
                DWORD dwLength,           // (I) Size of data to be written
                BYTE* pbData,             // (I) Data to be written
                LONG* pRet                 // (O) Error code
            )

```

□ **Automation call procedure**

```

File_WriteNCFile(
    vData As VARIANT           // (I) Data to be written
) As LONG                     // (O) Error code

```

□ **Argument** *dwLength*: Sets the size of data that is written at a time in the number of bytes.

pbData: Sets data to be written as byte array. In automation call, *vData* includes the number of bytes.

Automation argument:

vData: Creates data to be written as a byte array and sets it by substituting it in *vData* (VARIANT type) as shown in the example below.

```

Example) Dim vWriteFile As Variant
         Dim byteWrite() As Byte
         vWriteFile = byteWrite

```

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_WRITEFILE_NOTOPEN: No file is open in write mode

EZNC_FILE_WRITEFILE_WRITE: Cannot be written to a file

EZNC_FILE_LENGTH: Invalid write data size

EZ_ERR_NOT_SUPPORT: Not supported

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Data is written to the machining program file opened with **OpenNCFile3()**. Data to be written is data in a byte array.
 Sets the size of data to be written at a time. When writing a large amount of data, it can be written in multiple parts. Data can be written in sequence until **CloseNCFile3()** is executed.
 This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to *pRet*.)
 (Note 1) Do not perform a writing or overwriting operation during automatic operation of the NC control unit.
 (Note 2) For the M700/M800 series, when edit lock B (#8105) parameter is 1, programs 8000 to 9999 cannot be written. When edit lock C (#1121) parameter is 1, programs 9000 to 9999 cannot be written.

□ **Reference** **OpenNCFile3(), CloseNCFile3(), AbortNCFile3(), ReadNCFile3()**

□ **Specifica-
tion**



DANGER

Cautions for writing to a file

Carefully check a file before writing to the file for the NC control unit. Writing to an incorrect file may cause unexpected operation, resulting in a serious accident.

2.13 IEZNCCommonVariable2 Interface

C70

C80

M700

M800

IEZNCCommonVariable2::CommonVRead

Read common variables

□ Custom call procedure

```
HRESULT CommonVRead(
    LONG lIndex,           // (I) Variable number
    DOUBLE* pdData,        // (O) Variable value
    LONG* plType            // (O) Type
    LONG* plRet            // (O) Error code
)
```

□ Automation call procedure

```
CommonVariable_Read2(
    lIndex As LONG,        // (I) Variable number
    pdData As DOUBLE*,    // (O) Variable value
    plType As LONG*       // (O) Type
) As LONG                 // (O) Error code
```

□ *lIndex*: Sets the common variable number to be read.

Argument

	M700/M800 series	C70	C80
Number	100 to 199 400 to 999 100100 to 100199 200100 to 200199 300100 to 300199 400100 to 400199 500100 to 500199 600100 to 600199 700100 to 700199 800100 to 800199 900000 to 907399	100 to 199 500 to 999	100 to 199 400 to 999 100100 to 100199 200100 to 200199 300100 to 300199 400100 to 400199 500100 to 500199 600100 to 600199 700100 to 700199 800100 to 800199

pdData: Returns the common variable value of the set common variable number.

plType: Returns the variable value type. (For the **M700/M800 series**, enabled.)

Value	Meaning
1	Numerical value
0	Not set

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination
EZNC_DATA_READ_ADDR: Invalid system specification
EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Reads common variables. There is limit to the common variable number that can be handled, depending on specifications for the number of sets for common variables. For common variables #100 to #199, system specification is necessary. Set the existing arbitrary part system when using in common to all part systems.

□ **CommonVWrite(), GetSize()**

Reference

□ **Specifica-
tion** ([System] common variables #100 to #199 only)

IEZNCCommonVariable2::CommonVWrite**Write common variables**

□ Custom call procedure

```

HRESULT CommonVWrite(
    LONG lIndex,           // (I) Variable number
    DOUBLE dData,          // (I) Variable value
    LONG lType,            // (I) Type
    LONG* plRet            // (O) Error code
)

```

□ Automation call procedure

```

CommonVariable_Write2(
    lIndex As LONG,        // (I) Variable number
    dData As DOUBLE,       // (I) Variable value
    lType As LONG          // (I) Type
) As LONG                // (O) Error code

```

□ *lIndex*: Set the common variable number to be written.

Argument

	M700/M800 series	C70
Number	100 to 199 400 to 999 100100 to 100199 200100 to 200199 300100 to 300199 400100 to 400199 500100 to 500199 600100 to 600199 700100 to 700199 800100 to 800199 900000 to 907399	100 to 199 500 to 999

dData: Sets the common variable value to be written to the set common variable number.*lType*: Specifies the type. (For the **M700/M800**, enabled.)

Value	Meaning
1	Numerical value
0	Not set

plRet: Returns an error code. (Upon automation, the return value is used.)**S_OK**: Normal termination**EZNC_DATA_WRITE_ADDR**: Invalid part system setting**EZNC_DATA_WRITE_WRITE**: Data is not writable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Writes common variables. There is limit to the common variable number that can be handled, depending on specifications for the number of sets for common variables. For common variables #100 to #199, part system setting is necessary. Set the existing arbitrary part system when using in common to all part systems.
This function is not supported with C80.

□ **Reference** **CommonVRead()**, **GetSize()**

□ **Specifica-
tion** ([System] common variables #100 to #199 only)

IEZNCCommonVariable2::GetSize**Get number of sets for common variables**□ **Custom call procedure**

```

HRESULT      GetSize(
                LONG IType,           // (I) Common variable type
                LONG* pData,          // (O) Number of sets
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

CommonVariable_GetSize(
    IType As LONG,           // (I) Common variable type
    pData As LONG*          // (O) Number of sets
) As LONG                   // (O) Error code

```

□ Argument	<i>IType</i> : Specifies the common variable type to be read.	
	Value	Meaning
	0	When the number of sets of common variables #100 and greater is got.
	1	When the number of sets of common variables #500 and greater is got.

pData: Returns the number of sets of common variable type.

Value meaning: 40 = 40 [sets]

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ Function	Reads the number of sets of common variables. For common variables #100 to #199, part system setting is necessary. This function is not supported with C80.
----------------------	--

□ Reference	CommonVRead(), CommonVWrite()
-----------------------	--------------------------------------

□ Specifica- tion	(System common variables #100 to #199 only)
---------------------------------	--

IEZNCCommonVariable2::GetName**Get names of common variables**□ **Custom call procedure**

```

HRESULT      GetName(
                LONG lIndex,           // (I) Common variable number
                LPOLESTR* lppwszName, // (O) Common variable name character string
                LONG* plRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

CommonVariable_GetName(
    lIndex As LONG,           // (I) Common variable number
    lppwszName As STRING*    // (O) Common variable name character string
) As LONG                   // (O) Error code

```

□ **Argument** *lIndex*: Sets the common variable number to be read.

Value:
 (For M700 series and C70)
 500 to 519
 (For M800 series)
 500 to 599

lppwszName: Returns the common variable name as a **UNICODE** character string.
 Names are seven alphanumeric characters starting with an alphabet letter. The character string ends with a **NULL** code.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_DATASIZE: Exceeded maximum data size

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Reads common variable names. Names are seven alphanumeric characters starting with an alphabet letter. The character string ends with a **NULL** code.
 As the character string area memory is allocated in this product, the client using VC++ needs to release the character string area memory explicitly with **CoTaskMemFree()**.
 This function is not supported with C80.

□ **Reference** **CommonVRead(), CommonVWrite()**

□ **Specifica-
tion**

IEZNCCommonVariable2::SetName**Set names of common variables**□ **Custom call procedure**

```

HRESULT      SetName(
                LONG lIndex,           // (I) Common variable number
                LPCOLESTR lpcwszName,  // (I) Common variable name character string
                LONG* pRet              // (O) Error code
            )

```

□ **Automation call procedure**

```

CommonVariable_SetName(
    lIndex As LONG,           // (I) Common variable number
    lpcwszName As STRING     // (I) Common variable name character string
) As LONG                    // (O) Error code

```

□ **Argument** *lIndex*: Sets the common variable number to be written.

Value:
 (For M700 series and C70)
 500 to 519
 (For M800 series)
 500 to 599

lpcwszName: Set the common variable name as a **UNICODE** character string.
 Names are seven alphanumeric characters starting with an alphabet letter. The character string ends with a **NULL** code.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Writes common variable names. Names are seven alphanumeric characters starting with an alphabet letter. The character string ends with a NULL code. This function is not supported with C80.	
Function		

□ **GetName()**

Reference

□ **Specification**

IEZNCCommonVariable2::GetCVNullData**Get value when no numerical
value is set**□ **Custom call procedure**

```

HRESULT      GetCVNullData(
                DOUBLE* pdData,      // (O) Value when no numerical value is set
                LONG* plRet           // (O) Error code
            )

```

□ **Automation call procedure**

```

CommonVariable_GetNullData(
    pdData As DOUBLE*      // (O) Value when no numerical value is set
) As LONG                 // (O) Error code

```

□ **Argument** *pdData*: Returns the value when no numerical value is set.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the value when no numerical value of variable (#100 to 199, #500 to 519) is set.	

Function□ **Reference**□ **Specifica-
tion**

C70

M700

M800

IEZNcLocalVariable2::LocalVRead**Read local variable**

□ Custom call procedure

```

HRESULT      LocalVRead(
                LONG lIndex,           // (I) Variable number
                LONG lLevel,          // (I) Level
                DOUBLE* pdData,       // (O) Variable value
                LONG* plType          // (O) Type
                LONG* plRet           // (O) Error code
                )

```

□ Automation call procedure

```

LocalVariable_Read2(
    lIndex As LONG,           // (I) Variable number
    lLevel As LONG,          // (I) Level
    pdData As DOUBLE*,       // (O) Variable value
    plType As LONG*,        // (O) Type
    ) As LONG                // (O) Error code

```

□ **Argument** *lIndex*: Sets the local variable number to be read.
Value: 1 to 33

lLevel: Sets the macro subprogram execution level.
Value: 0 to 4

pdData: Returns the local variable value of the set local variable number of the set system.

plType: Returns the type. (Unused)

Value	Meaning
1	Numerical value
0	Not set

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Reads the local variable value of the specified system.
This function is not supported with C80.

□ **Reference** **GetMacroLevel()**

□ **Specifica-
tion** System

IEZNCLocalVariable2::GetMacroLevel**Get macro subprogram call level**□ **Custom call procedure**

```

HRESULT      GetMacroLevel(
                LONG* pData,           // (O) Level
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

LocalVariable_GetMacroLevel(
    pData As LONG           // (O) Level
) As LONG                  // (O) Error code

```

□ **Argument** *pData*: Returns the macro subprogram call level.
Value: 0 to 8

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the macro subprogram call level.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** System

IEZNCLocalVariable2::GetLVNullData**Get value when no numerical value is set**□ **Custom call procedure**

```

HRESULT      GetLVNullData(
                  DOUBLE* pdData,      // (O) Value when no numerical value is set
                  LONG* plRet         // (O) Error code
                )

```

□ **Automation call procedure**

```

LocalVariable_GetNullData(
  pdData As DOUBLE*      // (O) Value when no numerical value is set
) As LONG                // (O) Error code

```

□ **Argument** *pdData*: Returns the value when no numerical value is set.*plRet*: Returns an error code. (Upon automation, the return value is used.)**S_OK**: Normal termination**EZNC_DATA_READ_READ**: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the value when no numerical value of variable (#1 to 33) is set.
 This function is not supported with C80.

□ **Reference**□ **Specifica-
tion**

C70

M700

M800

IEZNcTool3::GetToolSetSize

Get number of sets for tool offset

Custom call procedure

```
HRESULT      GetToolSetSize(
                LONG* pSize,           // (O) Number of sets
                LONG* pRet             // (O) Error code
            )
```

Automation call procedure

```
Tool_GetToolSetSize(
    pSize As LONG*           // (O) Number of sets
) As LONG                   // (O) Error code
```

- Argument**

pSize: Returns the number of sets for tool offset of the set part system.
 The number of sets is determined by NC specifications.
 Value meaning: 200 = 200 [sets]

pRet: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination
EZNC_DATA_READ_READ: Data is not readable

	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Gets the number of sets for tool offset of the set part system. The number of sets is determined by NC specifications. This function is not supported with C80.	
Reference	GetType()	
Specification	System	

IEZNCtool3::GetType**Get tool offset type**

□ Custom call procedure

```

HRESULT      GetType(
                LONG* pType,           // (O) Type
                LONG* pRet             // (O) Error code
            )

```

□ Automation call procedure

```

Tool_GetType(
    pType As LONG*           // (O) Type
) As LONG                  // (O) Error code

```

□ *pType*: Returns the tool offset type of the set part system.

Argument	Value	Meaning
	1	M system type I: 1 axis compensation amount
	4	M system type II: 1 axis compensation amount with wear compensation amount
	6	L system type: 2 axes compensation amount

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ Gets the tool offset type of the set part system.

Function This function is not supported with C80.

□ **GetToolSetSize()**

Reference

□ **Specifica-
tion** System

IEZNCtool3::GetOffset**Get tool offset amount**□ **Custom call procedure**

```

HRESULT      GetOffset(
                LONG IType,           // (I) Tool offset type
                LONG IKind,           // (I) Offset amount type
                LONG IToolSetNo,       // (I) Tool set number
                DOUBLE* pdOffset,      // (O) Offset amount
                LONG* pINo,            // (O) Hypothetical tool nose pointnumber
                LONG* plRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_GetOffset(
    IType As LONG,           // (I) Tool offset type
    IKind As LONG,           // (I) Offset amount type
    IToolSetNo As LONG,      // (I) Tool set number
    pdOffset As DOUBLE*,     // (O) Offset amount
    pINo As LONG*,           // (O) Hypothetical tool nose point number
    ) As LONG                // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use **GetOffset3()** when newly starting use of EZSocket.

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the tool offset amount of the set part system/axis No. The range shown in the parameter table varies depending on the command unit such as inch system and metric system of the Mitsubishi CNC. For details, refer to the instruction manual for each Mitsubishi CNC. This function is not supported with C80.

□ **Reference** **GetType(), SetOffset(), GetToolSetSize()**

□ **Specifica-
tion** System

IEZNCtool3::GetOffset2**Get tool offset amount**□ **Custom call procedure**

```

HRESULT      GetOffset2(
                LONG IType,           // (I) Tool offset type
                LONG IKind,           // (I) Offset amount type
                LONG IToolSetNo,       // (I) Tool set number
                DOUBLE* pdOffset,      // (O) Offset amount
                LONG* pINo,            // (O) Hypothetical tool nose point number
                LONG* plRet            // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_GetOffset2(
    IType As LONG,           // (I) Tool offset type
    IKind As LONG,           // (I) Offset amount type
    IToolSetNo As LONG,       // (I) Tool set number
    pdOffset As DOUBLE*,     // (O) Offset amount
    pINo As LONG*,           // (O) Hypothetical tool nose point number
    ) As LONG                // (O) Error code

```

- **Argument** This method can be used as a backward compatible model. Use **GetOffset3()** when newly starting use of EZSocket.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Gets the tool offset amount of the set part system/axis No. The range shown in the parameter table varies depending on the command unit such as inch system and metrik system of the Mitsubishi CNC. For details, refer to the instruction manual for each Mitsubishi CNC. This function is not supported with C80.

- **Reference** **GetType(), SetOffset(), GetToolSetSize()**

- **Specifica-
tion** System

IEZNCTool3::GetOffset3**Get tool offset amount**□ **Custom call procedure**

```

HRESULT      GetOffset3(
                LONG IType,           // (I) Tool offset type
                LONG IKind,           // (I) Offset amount type
                LONG IToolSetNo,       // (I) Tool set number
                DOUBLE* pdOffset,      // (O) Offset amount
                LONG* pINo,            // (O) Hypothetical tool nose point number
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_GetOffset3(
    IType As LONG,           // (I) Tool offset type
    IKind As LONG,           // (I) Offset amount type
    IToolSetNo As LONG,      // (I) Tool set number
    pdOffset As DOUBLE*,     // (O) Offset amount
    pINo As LONG*,           // (O) Hypothetical tool nose point number
) As LONG                   // (O) Error code

```

□ *IType*: Sets the tool offset type. Refer to the parameter table.

Argument

IKind: Sets the type of tool offset amount. Refer to the parameter table.

IToolSetNo: Sets the tool offset set number.

The number of sets can get with **GetToolSetSize()**.

pdOffset: Returns the tool offset amount. Refer to the parameter table.

pINo: Returns the Hypothetical tool nose point number. Refer to the parameter table.
L system type only. Returns none except for the L system type.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_DATATYPE: Invalid data type

EZNC_DATA_READ_SUBJECT: Invalid subsection number

EZ_ERR_NOT_SUPPORT: Not supported

Parameter table

Value: Type	Value: Type of tool offset amount	Data range
1: M system type I	0: Tool offset amount	-9999.999 to 9999.999 [mm]
4: M system type II	0: Tool length compensation amount (dimensions) 1: (Wear) 2: Tool radius compensation amount (dimensions) 3: (Wear)	-9999.999 to 9999.999 [mm]
6: L system type	0: Tool wear amount 1st axis 1: 2nd axis 2: Additional axis 3: Tool length 1st axis 4: 2nd axis 5: Additional axis 6: Tool radius (nose R) R 7: Tool radius (nose R) wear amount r 8: Hypothetical tool nose point number P 9: Tool wear amount 4th axis 10: Tool length 4th axis	C70: -99.999 to 99.999 [mm] M700/M800 series : -9999.999 to 9999.999 [mm] C70: -999.999 to 999.999 [mm] M700/M800 series: -9999.999 to 9999.999 [mm] C70: -999.999 to 999.999 [mm] M700/M800 series: -9999.999 to 9999.999 [mm] 0 to 8 (Refer to Figure 1) M800 series: -9999.999 to 9999.999 [mm] M800 series: -9999.999 to 9999.999 [mm]

* For the M700/M800 series

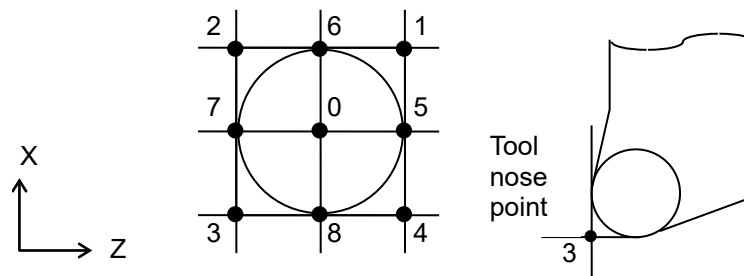


Figure 1 Hypothetical tool nose point

□	Value	Meaning
Return value	S_OK S_FALSE	Normal termination Communication failure
□	Gets the tool offset amount of the set part system/axis No. The range shown in the parameter table varies depending on the command unit such as inch system and metric system of the Mitsubishi Electric CNC. For details, refer to the instruction manual for each Mitsubishi Electric CNC.	
Function	This function is not supported with C80. The existing interface IEZNCtool3: : GetOffset() and IEZNCtool3: : GetOffset2() can be used as a backward compatible model. Use this method when newly starting use of EZSocket.	
□	GetType(), SetOffset2(), GetToolSetSize()	
Reference		
□	System	
Specifica- tion		

IEZNCtool3::SetOffset**Set tool offset amount**

□ Custom call procedure

```

HRESULT      SetOffset(
                LONG IType,           // (I) Tool offset type
                LONG IKind,           // (I) Offset amount type
                LONG IToolSetNo,       // (I) Tool set number
                DOUBLE dOffset,        // (I) Offset amount
                LONG INo,              // (I) Hypothetical tool nose point number
                LONG* pIRet            // (O) Error code
            )

```

□ Automation call procedure

```

Tool_SetOffset(
    IType As LONG,           // (I) Tool offset type
    IKind As LONG,           // (I) Offset amount type
    IToolSetNo As LONG,       // (I) Tool set number
    dOffset As DOUBLE,        // (I) Offset amount
    INo As LONG              // (I) Hypothetical tool nose point point number
) As LONG                   // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use **SetOffset2()** when newly starting use of EZSocket.

	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Configures the tool offset amount of the set part system/axis No. The range shown in the parameter table varies depending on the command unit such as inch system and metrik system of the Mitsubishi CNC. For details, refer to the instruction manual for each Mitsubishi CNC. This function is not supported with C80.

□ **Reference** **GetType(), GetOffset(), GetToolSetSize()**

□ **Specifica-
tion** System

IEZNCTool3::SetOffset2**Set tool offset amount**□ **Custom call procedure**

```

HRESULT      SetOffset2(
                LONG IType,           // (I) Tool offset type
                LONG IKind,           // (I) Offset amount type
                LONG IToolSetNo,       // (I) Tool set number
                DOUBLE dOffset,        // (I) Offset amount
                LONG INo,              // (I) Hypothetical tool nose point number
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_SetOffset2(
    IType As LONG,           // (I) Tool offset type
    IKind As LONG,           // (I) Offset amount type
    IToolSetNo As LONG,       // (I) Tool set number
    dOffset As DOUBLE,        // (I) Offset amount
    INo As LONG              // (I) Hypothetical tool nose point point number
) As LONG                   // (O) Error code

```

□ *IType*: Sets the tool offset type. Refer to the parameter table.

Argument

IKind: Sets the type of tool offset amount. Refer to the parameter table.

IToolSetNo: Sets the tool offset set number.

The number of sets can get with **GetToolSetSize()**.

dOffset: Sets the tool offset amount in the absolute values. Refer to the parameter table.

INo: Sets the hypothetical tool nose point number. Refer to the parameter table.

L system type only. Disabled for the M system type.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_WRITE_DATATYPE: Invalid data type

EZNC_DATA_WRITE_SUBSECT: Invalid subsection number

EZ_ERR_NOT_SUPPORT: Not supported

Parameter table

Value: Type	Value: Type of tool offset amount	Data range
1: M system type I	0: Tool offset amount	-9999.999 to 9999.999 [mm]
4: M system type II	0: Tool length compensation amount (dimensions) 1: (Wear) 2: Tool radius compensation amount (dimensions) 3: (Wear)	-9999.999 to 9999.999 [mm]
6: L system type	0: Tool wear amount 1st axis	C70: -99.999 to 99.999 [mm]
	1: 2nd axis	M700/M800 series:
	2: Additional axis	-9999.999 to 9999.999 [mm]
	3: Tool length 1st axis	C70: -999.999 to 999.999 [mm]
	4: 2nd axis	M700/M800 series:
	5: Additional axis	-9999.999 to 9999.999 [mm]
	6: Tool radius (nose R) R	C70: -999.999 to 999.999 [mm] M700/M800 series: -9999.999 to 9999.999 [mm]
	7: Tool radius (nose R) wear amount r	C70: -99.999 to 99.999 [mm] M700 /M800 series: -9999.999 to 9999.999 [mm]
	8: Hypothetical tool nose point number P	0 to 8 (Refer to Figure 1)
	9: Tool wear amount 4th axis	M800 series: -9999.999 to 9999.999 [mm]
	10: Tool length 4th axis	M800 series: -9999.999 to 9999.999 [mm]

* For the M700/M800 series

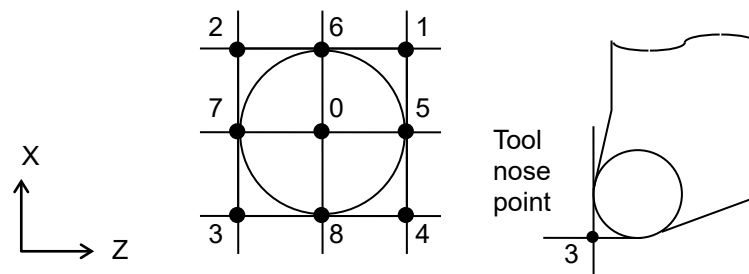


Figure 1 Hypothetical tool nose point number

Return value	Value	Meaning
	S_OK S_FALSE	Normal termination Communication failure
Function	Configures the tool offset amount of the set part system/axis No. The range shown in the parameter table varies depending on the command unit such as inch system and metric system of the Mitsubishi Electric CNC. For details, refer to the instruction manual for each Mitsubishi Electric CNC. This function is not supported with C80. The existing interface IEZNCtool3::SetOffset() can be used as a backward compatible model. Use this method when newly starting use of EZSocket.	
Reference	GetType(), GetOffset3(), GetToolSetSize()	
Specification	System	

IEZNCtool3::GetToolWorkOffset**Get workpiece coordinate system offset**□ **Custom call procedure**

```

HRESULT      GetToolWorkOffset(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Workpiece coordinate system number
                DOUBLE* pdOffset,      // (O) Offset value
                LONG* plRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_GetToolWorkOffset(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Workpiece coordinate system number
    pdOffset As DOUBLE*      // (O) Offset value
) As LONG                     // (O) Error code

```

□ **Argument** IAxisNo: Sets the axis. (From Axis 1 = from 1)

IIndex: Sets the workpiece coordinate system number to be read.

Value	Meaning
54	G54 offset
55	G55 offset
56	G56 offset
57	G57 offset
58	G58 offset
59	G59 offset
60	EXT offset

pdOffset: Returns the offset value of the workpiece coordinate of the set part system/axis No.
Value: -99999.999 to 99999.999 [mm]

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_AXIS: Invalid axis No. setting

EZ_ERR_NOT_SUPPORT: Not supported

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the offset value of the workpiece coordinate of the set part system/axis No. For details, refer to the instruction manual for each Mitsubishi CNC.

□ **Reference**

□ **Specifica-
tion** [System], [Axis number]

IEZNCtool3::GetToolWorkOffset2

Get workpiece coordinate system
offset

□ Custom call procedure

```

HRESULT      GetToolWorkOffset2(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Workpiece coordinate system number
                DOUBLE* pdOffset,       // (O) Offset value
                LONG* pIRet             // (O) Error code
            )

```

□ Automation call procedure

```

Tool_GetToolWorkOffset2(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Workpiece coordinate system number
    pdOffset As DOUBLE*       // (O) Offset value
) As LONG                    // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

IIndex: Sets the workpiece coordinate system number to be read.

Value	Meaning
54	G54 offset
55	G55 offset
56	G56 offset
57	G57 offset
58	G58 offset
59	G59 offset
60	EXT offset

pdOffset: Returns the offset value of the workpiece coordinate of the set part system/axis No.
Value: -99999.999 to 99999.999 [mm]

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid parft system setting

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_AXIS: Invalid axis No. setting

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the offset value of the workpiece coordinate of the set part system/axis No. For details, refer to the instruction manual for each Mitsubishi CNC.

Setting mode

(1) For setting absolute value,

Sets the set offset value as the current offset value.

(2) For setting additional value,

Sets the offset value got by adding the set offset value to the current offset value.

This function is not supported with C80.

□

Reference

□ Specifica-
tion

System, Axis number

IEZNCtool3::SetToolWorkOffset**Set workpiece coordinate system offset**

□ Custom call procedure

```

HRESULT      SetToolWorkOffset(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Workpiece coordinate system number
                DOUBLE dOffset,        // (I) Offset value
                LONG IMode,            // (I) Mode
                LONG* pIRet             // (O) Error code
            )

```

□ Automation call procedure

```

Tool_SetToolWorkOffset(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Workpiece coordinate system number
    dOffset As DOUBLE,        // (I) Offset value
    IMode As LONG             // (I) Mode
) As LONG                     // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

IIndex: Sets the workpiece coordinate system number to be read.

Value	Meaning
54	G54 offset
55	G55 offset
56	G56 offset
57	G57 offset
58	G58 offset
59	G59 offset
60	EXT offset

dOffset: Sets the offset value of the workpiece coordinate of the set part system/axis No.
Value: -99999.999 to 99999.999 [mm]

IMode: Sets the setting mode (absolute value setting/additional value setting).

Value	Meaning
0	Sets absolute value
1	Sets additional value

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_ADDR: Invalid part system setting

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_WRITE_AXIS: Invalid axis No. setting

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Sets the offset value of the workpiece coordinate of the set part system/axis No.
For details, refer to the instruction manual for each Mitsubishi CNC.

Setting mode

(1) For setting absolute value,
Sets the set offset value as the current offset value.

(2) For setting additional value,
Sets the offset value got by adding the set offset value to the current offset value.

This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** System, Axis number

IEZNCtool3::SetToolWorkOffset2**Set workpiece coordinate system offset**

□ Custom call procedure

```

HRESULT      SetToolWorkOffset2(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Workpiece coordinate system number
                DOUBLE dOffset,        // (I) Offset value
                LONG IMode,            // (I) Mode
                LONG* pIRet            // (O) Error code
            )

```

□ Automation call procedure

```

Tool_SetToolWorkOffset2(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Workpiece coordinate system number
    dOffset As DOUBLE,        // (I) Offset value
    IMode As LONG             // (I) Mode
) As LONG                    // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

IIndex: Sets the workpiece coordinate system number to be read.

Value	Meaning
54	G54 offset
55	G55 offset
56	G56 offset
57	G57 offset
58	G58 offset
59	G59 offset
60	EXT offset

dOffset: Sets the offset value of the workpiece coordinate of the set part system/axis No.
Value: -99999.999 to 99999.999 [mm]

IMode: Sets the setting mode (absolute value setting/additional value setting).

Value	Meaning
0	Sets absolute value
1	Sets additional value

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_ADDR: Invalid part system setting

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_WRITE_AXIS: Invalid axis No. setting

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Configures the offset value of the workpiece coordinate of the set part system/axis No.
For details, refer to the instruction manual for each Mitsubishi CNC.

Setting mode

(1) For setting absolute value,
Sets the set offset value as the current offset value.

(2) For setting additional value,
Sets the offset value got by adding the set offset value to the current offset value.

This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion** [System], [Axis number]

IEZNCtool3::GetToolWorkOffsetEX**Get extended workpiece coordinate system offset**□ **Custom call procedure**

```

HRESULT      GetToolWorkOffsetEX(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Extended workpiece coordinate system No.
                DOUBLE* pdOffset,      // (O) Offset value
                LONG* pIRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_GetToolWorkOffsetEX(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Extended workpiece coordinate system No.
    pdOffset As DOUBLE*       // (O) Offset value
) As LONG                    // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis. (From Axis 1 = from 1)

IIndex: Sets the extended workpiece coordinate system No. to be read.

Value: 1 to 96 (M700 series)

Value: 1 to 300 (M800 series)

Value	Meaning
1	G54.1P1 offset
2	G54.1P2 offset
:	
300	G54.1P300 offset

pdOffset: Returns the offset value of the extended workpiece coordinate system No. of the set part system/axis No.

Value: -99999.999 to 99999.999 [mm]

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_AXIS: Invalid axis No. setting

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the offset value of the extended workpiece coordinate system No. of the set part system/axis No. For details, refer to the instruction manual for each Mitsubishi CNC.

This is valid only for the M700/M800 series.

This function is not supported with C70/C80.

□ **Reference**

□ **Specifica-
tion** [System], [Axis number]

IEZNCtool3::SetToolWorkOffsetEX**Set extended workpiece coordinate system
offset**

□ Custom call procedure

```

HRESULT      SetToolWorkOffsetEX(
                LONG IAxisNo,           // (I) Axis No.
                LONG IIndex,           // (I) Extended workpiece coordinate system No.
                DOUBLE dOffset,        // (I) Offset value
                LONG IMode,            // (I) Mode
                LONG* pIRet            // (O) Error code
            )

```

□ Automation call procedure

```

Tool_SetToolWorkOffsetEX(
    IAxisNo As LONG,           // (I) Axis No.
    IIndex As LONG,           // (I) Extended workpiece coordinate system No.
    dOffset As DOUBLE,        // (I) Offset value
    IMode As LONG             // (I) Mode
) As LONG                    // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from **1**)

IIndex: Sets the extended workpiece coordinate system No. to be written.

Value: 1 to 96 (M700 series)

Value: 1 to 300 (M800 series)

Value	Meaning
1	G54.1P1 offset
2	G54.1P2 offset
:	
300	G54.1P300 offset

dOffset: Sets the offset value of the extended workpiece coordinate system No. of the set part system/axis No.

Value: -99999.999 to 99999.999 [mm]

IMode: Sets the setting mode (absolute value setting/additional value setting).

Value	Meaning
0	Sets absolute value
1	Sets additional value

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_ADDR: Invalid part system setting

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_WRITE_AXIS: Invalid axis No. setting

EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Sets the offset value of the extended workpiece coordinate system No. of the set part system/axis No. For details, refer to the instruction manual for each Mitsubishi CNC.

Setting mode

(1) For setting absolute value,

Configures the set offset value to the current offset value.

(2) For setting additional value,

Adds the set offset value to the current offset value.

This is valid only for the M700/M800 series.

This function is not supported with C70/C80.

□
Reference

□
**Specifica-
tion**

System **Axis number**

IEZNCTool3::GetSurface**Get reference surface height**□ **Custom call procedure**

```

HRESULT      GetSurface(
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdHight,       // (O) Reference surface height
                LONG* pIRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_GetSurface(
    IAxisNo As LONG,           // (I) Axis No.
    pdHight As DOUBLE*       // (O) Reference surface height
) As LONG                    // (O) Error code

```

□ **Argument** *IAxisNo*: Sets the axis No. (From Axis 1 = from **1**)

pdHight: Returns the reference surface coordinate position of the tool length measurement II of the set part system/axis No.

Value: -99999.999 to 99999.999 [mm]

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis No. setting

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Gets the reference surface coordinate position of the tool length measurement II of the set part system/axis No. This function is not supported with C80.	

□ **Reference**

□ **Specifica-
tion** [System], [Axis number]

IEZNCTool3::GetSurface2**Get reference surface height**□ **Custom call procedure**

```

HRESULT      GetSurface2(
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE* pdHight,       // (O) Reference surface height
                LONG* pIRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_GetSurface2(
    IAxisNo As LONG,           // (I) Axis No.
    pdHight As DOUBLE*        // (O) Reference surface height
) As LONG                     // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from 1)

Argument

pdHight: Returns the reference surface coordinate position of the tool length measurement II of the set part system/axis No.

Value: -99999.999 to 99999.999 [mm]

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_DATA_READ_AXIS: Invalid axis No. setting

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Gets the reference surface coordinate position of the tool length measurement II of the set axis No. of the set part system. This function is not supported with C80.	

□ **Reference**

□ **Specifica-
tion** System, Axis number

IEZNCtool3::SetSurface**Set reference surface height**

□ Custom call procedure

```

HRESULT      SetSurface(
                LONG IAxisNo,           // (I) Axis No.
                DOUBLE dHight,         // (I) Reference surface height
                LONG* pIRet             // (O) Error code
            )

```

□ Automation call procedure

```

Tool_SetSurface(
    IAxisNo As LONG,           // (I) Axis No.
    dHight As DOUBLE*         // (I) Reference surface height
) As LONG                     // (O) Error code

```

□ *IAxisNo*: Sets the axis No. (From Axis 1 = from **1**)

Argument

dHight: Sets the reference surface coordinate position of the tool length measurement II of the set part system/axis No.

Value: -99999.999 to 99999.999 [mm]

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_WRITE_ADDR: Invalid part system, axis No. setting

EZNC_DATA_WRITE_AXIS: Invalid axis No. setting

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
Function	Sets the reference surface coordinate position of the tool length measurement II of the set part system/axis No. This function is not supported with C80.	

□
Reference

□
**Specifica-
tion**

System, Axis number

IEZNCTool3::GetToolLifeType2**Get tool life management method**□ **Custom call procedure**

```

HRESULT      GetToolLifeType2(
                LONG* pIType,           // (O) Tool life management method
                LONG* pIRet           // (O) Error code
                )

```

□ **Automation call procedure**

```

Tool_GetToolLifeType2(
    pIType As LONG*           // (O) Tool life management method
) As LONG                   // (O) Error code

```

□	<i>pIType</i> : Returns the tool life control type.	
Argument	Value	Meaning
	0	Disabled
	1	Type I
	2	Type II
	3	Type III (Machining center system of M700/M800 series only)

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_DATA_RANGE: Invalid argument data range

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets tool life management method.	
Function	This function is not supported with C80.	

□	SetToolLifeType2()
Reference	

□	
Specifica- tion	

IEZNCtool3::SetToolLifeType2**Select tool life management method**□ **Custom call procedure**

```

HRESULT      SetToolLifeType2(
                LONG IType,           // (I) Tool life management method
                LONG* pIRet           // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_SetToolLifeType2(
    IType As LONG           // (I) Tool life management method
) As LONG                  // (O) Error code

```

□ **Argument** *IType*: Sets the tool life management method.

Value	Meaning
0	Disabled (cannot be specified)
1	Tool life management I
2	Tool life management II
3	Tool life management III (Machining center system of M700/M800 series only)

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZ_ERR_DATA_RANGE: Invalid argument data range

EZ_ERR_DATA_TYPE: Invalid argument data type

EZ_ERR_NOT_SUPPORT: Not supported

Value	Meaning
S_OK	Normal termination
S_FALSE	Communication failure

□ **Function** Selects tool life management. Write is inhibited by password mode.
(EZNC_DATA_WRITE_WRITE is returned.)
 This function is not supported with C80.

□ **Reference** **GetToolLifeType2()**

□ **Specifica-
tion**

IEZNCtool3::GetToolLifeGroupList**Get tool life management group No. list**

□ Custom call procedure

```

HRESULT      GetToolLifeGroupList (
                LPDWORD lpdwLength,           // (O) Number of groups
                LPDWORD* lppdwGroup,          // (O) Array of group numbers
                LONG* pRet                      // (O) Error code
            )

```

□ Automation call procedure

```

Tool_GetToolLifeGroupList(
    pvGroup As VARIANT*           // (O) Group number
) As LONG                        // (O) Error code

```

□ *lpdwLength*: Returns the number of sets of groups.

Argument

lppdwGroup: Returns the list of group numbers as an array. As the group number array is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

Automation argument:

pvGroup: Returns the group number array as **VARIANT**.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_MEMORY_ALLOC: Memory cannot be allocated.

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets tool life management group No.	
Function	This function is not supported with C80.	
□	ChangeToolLifeGroup(), DeleteToolLifeGroup()	
Reference		
□	System (M700/M800 series only)	
Specifica- tion		

IEZNCtool3::ChangeToolLifeGroup**Change tool life management group No.**□ **Custom call procedure**

```

HRESULT      ChangeToolLifeGroup (
                DWORD dwSrcGroup,    // (I) Old group number
                DWORD dwDstGroup,    // (I) New group number
                LONG* pIRet          // (O) Error code
                )

```

□ **Automation call procedure**

```

Tool_ChangeToolLifeGroup(
    ISrcGroup As LONG,    // (I) Old group number
    IDstGroup As LONG    // (I) New group number
    ) As LONG             // (O) Error code

```

□ **Argument** *dwSrcGroup*: Sets the old group number.

dwDstGroup: Sets the new group number.

Automation argument:

ISrcGroup: Refer to the explanation of *dwSrcGroup*.

IDstGroup: Refer to the explanation of *dwDstGroup*.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable.

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_EXIST: Group number already exists

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Changes the set group number to the new group number.
 The set old group tool is changed to the new group.
 This function is not supported with C80.
 This function is not supported with M800 tool life management III for M system.
 (**EZNC_DATA_TLFGROUP_OUTOFSPEC** is returned to *pIRet*.)

□ **Reference** **GetToolLifeGroupList(), DeleteToolLifeGroup()**

□ **Specifica-
tion** System (M700/M800 series only)

IEZNCtool3::DeleteToolLifeGroup**Delete tool life management group No.**□ **Custom call procedure**

```

HRESULT      DeleteToolLifeGroup (
                    DWORD dwGroup,      // (I) Group number
                    LONG* pRet          // (O) Error code
                )

```

□ **Automation call procedure**

```

Tool_DeleteToolLifeGroup(
    IGroup As LONG          // (I) Group number
) As LONG                 // (O) Error code

```

□ **Argument** *dwGroup*: Sets the group number to be deleted. Specify "1" for tool life management III for M800 M system.

Automation argument:

*I*Group: Refer to the explanation of *dwGroup*.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable.

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_READ_ADDR: Invalid part system setting

EZNC_DATA_TLFGROUP_ADDR: Invalid address (part system setting)

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Deletes the group number.
This function is not supported with C80.

□ **Reference** **GetToolLifeGroupList(), ChangeToolLifeGroup()**

□ **Specifica-
tion** System

IEZNCtool3::GetToolLifeToolNoList**Get tool No. list within tool life management group**□ **Custom call procedure**

```

HRESULT      GetToolLifeToolNoList (
                DWORD dwGroup,           // (I) Group number
                LPDWORD lpdwLength,      // (O) Number of registered tools
                LPDWORD *lppdwToolNo,    // (O) Array of tool numbers
                LONG* plRet              // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_ GetToolLifeToolNoList (
    IGroup As LONG,           // (I) Group number
    pvToolNo As VARIANT      // (O) Array of tool numbers
) As LONG                   // (O) Error code

```

□ **Argument** *dwGroup*: Sets the group number for which the list of tool numbers is got. Specify "1" for tool life management III for M800 M system.

lpdwLength: Returns the number of registered tools included in the group (array length of the list of tool numbers).

lppdwToolNo: Returns the list of tool numbers included in the group as an array. As the tool number list array is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

Automation argument:

IGroup: Refer to the explanation of *dwGroup*.

pvToolNo: Returns the list of tool numbers included in the group as VARIANT.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_MEMORY_ALLOC: Memory cannot be allocated

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the list of tool numbers of the set group.
This function is not supported with C80.

□ **Reference** **AddToolLifeToolNo(), ChangeToolLifeToolNo(), DeleteToolLifeToolNo()**

□ **Specifica-
tion** System

IEZNCtool3::AddToolLifeToolNo**Add tool No. within tool life management group**□ **Custom call procedure**

```

HRESULT      AddToolLifeToolNo (
                    DWORD dwGroup,           // (I) Group number
                    DWORD dwToolNo,         // (I) Tool number
                    LONG* pRet              // (O) Error code
                )

```

□ **Automation call procedure**

```

Tool_ AddToolLifeToolNo (
    IGroup As LONG,           // (I) Group number
    IToolNo As LONG         // (I) Tool number
) As LONG                   // (O) Error code

```

- **Argument** *dwGroup*: Sets the group numbers to which tool numbers are added. Specify "1" for tool life management III for M800 M system.

dwToolNo: Sets the tool numbers to be added.

Automation argument:

IGroup: Refer to the explanation of *dwGroup*.

IToolNo: Refer to the explanation of *dwToolNo*.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_DUPLICATE: Duplicated numbers

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZNC_DATA_TLFTOOL_EXIST: Tool number already exists

EZNC_DATA_TLFTOOL_OUTOFSPEC: Set tool number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Adds the tool numbers to the specified group.
This function is not supported with C80.

□ **Reference** **GetToolLifeToolNoList(), ChangeToolLifeToolNo(), DeleteToolLifeToolNo()**

- **Specifica-
tion** System

IEZNCtool3::ChangeToolLifeToolNo**Change tool life management tool No.**□ **Custom call procedure**

```

HRESULT      ChangeToolLifeToolNo(
                DWORD dwGroup,           // (I) Group number
                DWORD dwSrcToolNo,       // (I) Old tool number
                DWORD dwDstToolNo,       // (I) New tool number
                LONG* pRet               // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_ ChangeToolLifeToolNo (
    IGroup As LONG,           // (I) Group number
    ISrcToolNo As LONG,       // (I) Old tool number
    IDstToolNo As LONG       // (I) New tool number
) As LONG                    // (O) Error code

```

□ **Argument** *dwGroup*: Sets the group number in which the tool number is changed. Specify "1" for tool life management III for M800 M system.

dwSrcToolNo: Sets the old tool number.

dwDstToolNo: Sets the new tool number.

Automation argument:

IGroup: Refer to the explanation of *dwGroup*.

ISrcToolNo: Refer to the explanation of *dwSrcToolNo*.

IDstToolNo: Refer to the explanation of *dwDstToolNo*.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZNC_DATA_TLFTOOL_EXIST: Tool number already exists

EZNC_DATA_TLFTOOL_NONEXIST: Tool number does not exist

EZNC_DATA_TLFTOOL_OUTOFSPEC: Set tool number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

□ Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Changes the set tool number to the new tool number.
This function is not supported with C80.

□ **Reference** **GetToolLifeToolNoList(), AddToolLifeToolNo(), DeleteToolLifeToolNo()**

□ **Specifica-
tion** System

IEZNCtool3::DeleteToolLifeToolNo**Delete tool life management tool No.**□ **Custom call procedure**

```

HRESULT      DeleteToolLifeToolNo (
                    DWORD dwGroup,           // (I) Group number
                    DWORD dwToolNo,         // (I) Tool number
                    LONG* pIRet             // (O) Error code
                )

```

□ **Automation call procedure**

```

Tool_ DeleteToolLifeToolNo (
    IGroup As LONG,           // (I) Group number
    IToolNo As LONG          // (I) Tool number
) As LONG                   // (O) Error code

```

□ **Argument** *dwGroup*: Sets the group number from which tool numbers are deleted. Specify "1" for tool life management III for M800 M system.

dwToolNo: Sets the tool numbers to be deleted.

Automation argument:

IGroup: Refer to the explanation of *dwGroup*.

IToolNo: Refer to the explanation of *dwToolNo*.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZNC_DATA_TLFTOOL_NONEXIST: Tool number does not exist

EZNC_DATA_TLFTOOL_OUTOFSPEC: Set tool number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Deletes the set tool numbers.
This function is not supported with C80.

□ **Reference** **GetToolLifeToolNoList(), AddToolLifeToolNo(), ChangeToolLifeToolNo()**

□ **Specifica-
tion** System

IEZNCTool3::GetToolLifeValue**Get tool life management data**□ **Custom call procedure**

```

HRESULT      GetToolLifeValue (
                DWORD dwGroup,           // (I) Group number
                DWORD dwToolNo,         // (I) Tool number
                LPOLESTR** lpppwszData, // (O) Tool life management data value
                                                character string array
                LONG* pRet               // (O) Error code
                )

```

□ **Automation call procedure**

```

Tool_GetToolLifeValue (
    IGroup As LONG,           // (I) Group number
    IToolNo As LONG,          // (I) Tool number
    pvData As VARIANT*       // (O) Tool life management data value
                                character string
    ) As LONG                 // (O) Error code

```

□ **Argument** *dwGroup*: Sets the group number for which tool life is got. Specify "1" for tool life management III for M800 M system.

dwToolNo: Sets the tool number for which tool life is got.

lpppwszData: Returns the life management data value as a **UNICODE** character string array. As the data value array is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree** (). Refer to the index table.

Automation argument:

IGroup: Refer to the explanation of *dwGroup*.

IToolNo: Refer to the explanation of *dwToolNo*.

pvData: Returns the life control data value (UNICODE character string) array as VARIANT. For life control data values, refer to the index.

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_MEMORY_ALLOC: Memory cannot be allocated

EZ_ERR_DATA_RANGE: Invalid argument data range

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZNC_DATA_TLFTOOL_NONEXIST: Tool number does not exist

EZNC_DATA_TLFTOOL_OUTOFSPEC: Set tool number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

Index table

Array Index	Tool life type (data range)	
	C70 M system	M700/M800 series M system
0	Tool number (1 to 99999999)	Tool number (1 to 99999999)
1	Status (depends on MTB specifications)	Status (0x00 to 0xFF)
2	Method (000 to 222)*	Method (000 to 222)*
3	Length dimension (± 1 to 99999.999)	Length dimension (± 99999.999)
4	Radius dimension (± 1 to 99999.999)	Radius dimension (± 99999.999)
5	Life (Time: 0 to 4000, Count: 0 to 65000)	Life (Time: 0 to 4000, Count: 0 to 65000)
6	Used (Time: 0 to 4000, Count: 0 to 65000)	Used (Time: 0 to 4000, Count: 0 to 65000)
7	Auxiliary (0 to 65535, depends on MTB specifications)	Auxiliary (0 to 65535, depends on MTB specifications)
8	Length wear (Reserved: 0)	Length wear (Reserved: 0)
9	Radius wear (Reserved: 0)	Radius wear (Reserved: 0)
10	Group (1 to 99999999)	Group (1 to 99999999)

Array Index	Tool life type (data range)	
	M700/M800 series L system (TYPE I), C70 L system (TYPE I)	M700/M800 series L system (TYPE II)
0	Application of time management (0 to 995959)	Tool number (1 to 999999)
1	Application of number of times management (0 to 9999)	Compensation number (0 to 80)
2	Status A (0 to 2)	Usage (Time: 0 to 99999999, Count: 0 to 999999)
3	Life of time management (0 to 995959)	ST (0 to 3)
4	Life of count management (0 to 9999)	Method (Time: 0, Count: 1)
5	Status B (depends on MTB specifications)	Life (0 to 999999)
6 to 10	-	-

Array Index	Tool life type (data range)	
	C70 L system (TYPE II)	
0	Tool number (1 to 999999)	
1	Group (1 to 9999)	
2	Method (0: Time, 1: Count)	
3	Compensation number (1 to 80)	
4	Status (0 to 3)	
5	Life (Time: 0 to 999999, Count: 0 to 999999)	
6	Used (Time: 0 to 999999, Count: 0 to 999999)	
7 to 10	-	

□ Return value	Value	Meaning
	S_OK S_FALSE	Normal termination Communication failure
□ Function	Gets life control data for the set tool number. Note that the number of elements of the character string array that returns life control data varies depending on models. * For the "method" for tool life management data, refer to the instruction manual for each Mitsubishi CNC. This function is not supported with C80.	
□ Reference	SetToolLifeValue()	
□ Specification	System	

IEZNCTool3::SetToolLifeValue**Set individual tool life management data**□ **Custom call procedure**

```

HRESULT      SetToolLifeValue (
                DWORD dwGroup,           // (I) Group number
                DWORD dwToolNo,        // (I) Tool number
                DWORD dwKind,          // (I) Type of tool life management data
                LPCOLESTR lpcwszData,  // (I) Tool life management
                LONG* plRet            // (O) Error code
            )

```

□ **Automation call procedure**

```

Tool_ SetToolLifeValue (
    lGroup As LONG,           // (I) Group number
    lToolNo As LONG,         // (I) Tool number
    lKind As LONG,          // (I) Type of tool life management I data
    lpcwszData As STRING    // (I) Tool life management data
) As LONG                   // (O) Error code

```

□ **Argument** *dwGroup*: Sets the group number for which tool life is set. Specify "1" for tool life management III for M800 M system.

dwToolNo: Sets the tool number for which tool life is set.

dwKind: Sets the type of tool life. Refer to the parameter table.

lpcwszData: Sets the specified type of life data.

Automation argument:

lGroup: Refer to the explanation of *dwGroup*.

lToolNo: Refer to the explanation of *dwToolNo*.

lKind: Refer to the explanation of *dwKind*.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_NOT_EXIST: Data does not exist

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_DATA_RANGE: Invalid argument data range (*dwKind*)

EZ_ERR_DATA_TYPE: Invalid argument data type

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZNC_DATA_TLFTOOL_NONEXIST: Tool number does not exist

EZNC_DATA_TLFTOOL_PARAMERR: Invalid type specified for life control data

EZNC_DATA_TLFTOOL_MAXMINERR: Setting data is out of range

EZNC_DATA_TLFTOOL_OUTOFSPEC: Set tool number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

Parameter table

Value	Tool life type (data range)	
	C70 M system	M700/M800 series M system
1	Tool number (1 to 99999999)	Tool number (1 to 99999999)
2	Status (depends on machine manufacturer specifications)	Status (0x00 to 0xFF)
3	Method (000 to 222)*	Method (000 to 222)*
4	Length dimension (± 99999.999)	Length dimension (± 9999.999)*
5	Radius dimension (± 99999.999)	Radius dimension (± 9999.999)*
6	Life (Time: 0 to 4000, Count: 0 to 9999)	Life (Time: 0 to 4000, Count: 0 to 9999/65000)
7	Used (Time: 0 to 4000, Count: 0 to 9999)	Used (Time: 0 to 4000, Count: 0 to 9999/65000)
8	Auxiliary (0 to 65535, depends on MTB specifications)	Auxiliary (0 to 65535, depends on MTB specifications)
9	Length wear (Reserved: 0)	Length wear (Reserved: 0)
10	Radius wear (Reserved: 0)	Radius wear (Reserved: 0)
11	Group (1 to 99999999)	Group (1 to 99999999)

Value	Tool life type (data range)	
	M700/M800 series L system (TYPE I), C70 L system (TYPE I)	M700/M800 series L system (TYPE II)
1	Application of time management (0 to 995959)	Tool number (1 to 999999)
2	Application of count management (0 to 9999)	Compensation number (0 to 80)
3	Status A (0 to 2)	Used (Time: 0 to 99999999, Count: 0 to 999999)
4	Life of time management (0 to 995959)	ST (0 to 3)
5	Life of count management (0 to 9999)	Method (Time: 0, Count: 1)
6	Status B (depends on MTB specifications)	Life (0 to 999999)

Value	Tool life type (data range)	
	C70 L system (TYPE II)	
1	Tool number (1 to 999999)	
2	Group (1 to 9999)	
3	Method (0: Time, 1: Count)	
4	Compensation number (1 to 80)	
5	Status (0 to 3)	
6	Life (Time: 0 to 999999, Count: 0 to 999999)	
7	Used (Time: 0 to 999999, Count: 0 to 999999)	

	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Function	Individually sets life control data for the set tool number. This method is used for updating. To newly add a tool, use the following procedure. 1) AddToolLifeToolNo() 2) SetToolLifeValue()
-------------------	---

* For the "method" for tool life management data, refer to the instruction manual for each Mitsubishi CNC.

This function is not supported with C80.

□ Reference	GetToolLifeValue(), AddToolLifeToolNo()
□ Specification	System

IEZNCtool3::SetToolLifeValue2**Set tool life management data**

□ Custom call procedure

```

HRESULT      SetToolLifeValue2 (
                DWORD dwGroup,           // (I) Group number
                DWORD dwToolNo,         // (I) Tool number
                LPCOLESTR* lppcwszData, // (I) Tool life management data character
                                                string array
                LONG* pRet              // (O) Error code
            )

```

□ Automation call procedure

```

Tool_ SetToolLifeValue2 (
    IGroup As LONG,           // (I) Group number
    IToolNo As LONG,         // (I) Tool number
    vData As VARIANT         // (I) Tool life management data character
                                string array
    ) As LONG                 // (O) Error code

```

□ **Argument** *DwGroup*: Sets the group number for which tool life is set. Specify "1" for tool life management III for M800 M system.

dwToolNo: Sets the tool number for which tool life is set.

lppcwszData: Sets a **UNICODE** character string array for the set type of life data.

Automation argument:

IGroup: Refer to the explanation of *dwGroup*.

IToolNo: Refer to the explanation of *dwToolNo*.

vData: Creates a **UNICODE** character string array for the specified type of life data and sets by substituting it in *vData* (VARIANT type). For examples of substitution, refer to "WriteFile".

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_DATA_RANGE: Invalid argument data range (*dwGroup*, *dwToolNo*)

EZ_ERR_NULLPTR: Argument is NULL pointer

EZNC_DATA_TLFGROUP_ADDR: Invalid address (system specification)

EZNC_DATA_TLFGROUP_NONEXIST: Group number does not exist

EZNC_DATA_TLFGROUP_OUTOFSPEC: The set group number is out of specifications

EZNC_DATA_TLFTOOL_NONEXIST: Tool number does not exist

EZNC_DATA_TLFTOOL_MAXMINERR: Setting data is out of range

EZNC_DATA_TLFTOOL_OUTOFSPEC: Set tool number is out of specifications

EZ_ERR_NOT_SUPPORT: Not supported

Parameter table

Array Index	Tool life type (data range)	
	C70 M system	M700/M800 series M system
0	Tool number (1 to 99999999)	Tool number (Reserved: 0)
1	Status (depends on MTB specifications)	Status (0x00 to 0xFF)
2	Method (000 to 222)*	Method (000 to 222)*
3	Length dimension (± 99999.999)	Length dimension (± 9999.999)*
4	Radius dimension (± 99999.999)	Radius dimension (± 9999.999)*
5	Life (Time: 0 to 4000, Count: 0 to 9999)	Life (Time: 0 to 4000, Count: 0 to 65000)
6	Used (Time: 0 to 4000, Count: 0 to 9999)	Usage (Time: 0 to 4000, Count: 0 to 65000)
7	Auxiliary (0 to 65535, depends on MTB specifications)	Auxiliary (0 to 65535, depends on MTB specifications)
8	Length wear (Reserved: 0)	Length wear (Reserved: 0)
9	Radius wear (Reserved: 0)	Radius wear (Reserved: 0)
10	Group (1 to 99999999)	Group (Reserved: 0)

Array Index	Tool life type (data range)	
	M700/M800 series L system (TYPE I), C70 L system (TYPE I)	M700/M800 series L system (TYPE II)
0	Application of time management (0 to 995959)	Tool number (Reserved: 0)
1	Application of count management (0 to 9999)	Compensation number (0 to 80)
2	Status A (0 to 2)	Usage (Time: 0 to 99999999, Number of times: 0 to 999999)
3	Life of time management (0 to 995959)	ST (0 to 3)
4	Life of count management (0 to 9999)	Method (Time: 0, Count: 1)
5	Status B (depends on MTB specifications)	Life (0 to 999999)
6 to 10	-	-

Array Index	Tool life type (data range)	
	C70 L system (TYPE II)	
0	Tool number (1 to 999999)	
1	Group (1 to 9999)	
2	Method (0: Time, 1: Count)	
3	Compensation number (1 to 80)	
4	Status (0 to 3)	
5	Life (Time: 0 to 999999, Count: 0 to 999999)	
6	Usage (Time: 0 to 999999, Count: 0 to 999999)	
7 to 10	-	

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Sets tool life management data for the set tool number. This method is used for updating. To newly add a tool, use the following procedure.	
Function	1) AddToolLifeGroup() 2) AddToolLifeToolNo() 3) SetToolLifeValue2()	
	For the "method" for tool life management data, refer to the instruction manual for each Mitsubishi CNC.	
	[Example] LPOLESTR* lppwszData; lppwszData = new LPOLESTR[11]; lppwszData[0] =L"0"; lppwszData[1] =L"1"; lppwszData[2] =L"220"; lppwszData[3] =L"10.000"; lppwszData[4] =L"20.000"; lppwszData[5] =L"40.000"; lppwszData[6] =L"18.000"; lppwszData[7] =L"0"; lppwszData[8] =L"0.000"; lppwszData[9] =L"0.000"; lppwszData[10] =L"0"; hr = pIEZNCtool->SetToolLifeValue2(1, 100, (LPCOLESTR*)lppwszData, &lRet); if(S_OK != hr){ wprintf(L"HRESULT Code = 0x%x, lRet Code = 0x%x\n", hr, lRet); } delete[] lppwszData; This function is not supported with C80.	
□	GetToolLifeValue(), AddToolLifeToolNo()	
Reference		
□	System	
Specifica- tion		

IEZNcATC3::GetMGNControl

Get ATC tool registration control parameter

□ **Custom call procedure**

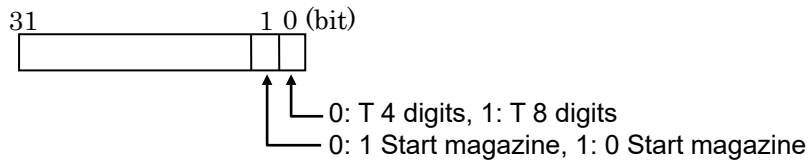
```
HRESULT      GetMGNControl(  
                LONG* pData,           // (O) Parameter value  
                LONG* pRet             // (O) Error code  
            )
```

□ **Automation call procedure**

```
ATC_GetMGNControl(  
    pData As LONG*           // (O) Parameter value  
    ) As LONG                // (O) Error code
```

□ *pData*: Returns a parameter that controls start magazine.

□ **Argument**



pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the control parameter value for ATC tool registration.	
Function	This function is not supported with C80.	

□ **Reference**

□ **Specifica-
tion**

IEZNCATC3::GetMGNSize

Get total number of sets of magazine pots for ATC tool registration

□ **Custom call procedure**

```

HRESULT      GetMGNSize(
                  LONG* pSize,           // (O) Total number of sets of magazine pots
                  LONG* pRet            // (O) Error code
                )

```

□ **Automation call procedure**

```

ATC_GetMGNSize(
  pSize As LONG*           // (O) Total number of sets of magazine pots
) As LONG                  // (O) Error code

```

□ **Argument** *pSize*: Returns the total number of sets of magazine pots.
 Value: 0 to 360 (maximum)

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the total number of sets of magazine pots.
 This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion**

IEZNCATC3::GetMGNSize2

Get number of sets of pots for each magazine for ATC tool registration

□ **Custom call procedure**

```

HRESULT      GetMGNSize2(
                    LONG IMagazineNo,      // (I) Magazine number
                    LONG* pSize,          // (O) Number of sets of magazine pots
                    LONG* pRet           // (O) Error code
                )

```

□ **Automation call procedure**

```

ATC_GetMGNSize2(
    IMagazineNo As LONG, // (I) Magazine number
    pSize As LONG*      // (O) Number of sets of magazine pots
) As LONG               // (O) Error code

```

□ **Argument** *IMagazineNo*: Set the magazine number.
 Value: 1 to 5 (maximum) M700/M800 series
 Value: 1 to 3 (maximum) C70

pSize: Returns the number of sets of magazine pots.
 Value: 0 to 360 (maximum) M700/M800 series
 Value: 0 to 80 (maximum) C70

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_DATA_RANGE: Invalid argument data range (*IMagazineNo*)

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the number of sets of the set magazine pots.
 This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion**

IEZNCATC3::GetMGNReady2**Get tool number for ATC tool registration**□ **Custom call procedure**

```

HRESULT      GetMGNReady2(
                LONG IMagazineNo,           // (I) Magazine number
                LONG IReady,                // (I) On standby
                LONG* pToolNo,              // (O) Tool number
                LONG* pRet                   // (O) Error code
            )

```

□ **Automation call procedure**

```

ATC_GetMGNReady2(
    IMagazineNo As LONG,           // (I) Magazine number
    IReady As LONG,                // (I) On standby
    pToolNo As LONG*               // (O) Tool number
) As LONG                          // (O) Error code

```

□ **Argument** *IMagazineNo*: Sets the magazine number.
Value: 1 to 2 (Even if value is set for M700/M800 series, it is invalid.)

IReady: Sets standby state.

Value	Meaning
0	Tool number to be installed
1	Tool number on standby 1
2	Tool number on standby 2
3	Tool number on standby 3
4	Tool number on standby 4

pToolNo: Returns the tool number.
Value: 1 to 99999999 (maximum)

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_DATA_RANGE: Invalid argument data range (*IMagazineNo*, *IReady*)

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the tool number for ATC tool registration.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion**

IEZNCATC3::GetMGNPot

Get tool number for magazine pot for ATC
tool registration

□ Custom call procedure

```

HRESULT      GetMGNPot(
                LONG lIndex,           // (I) Magazine pot number
                LONG* pToolNo,         // (O) Tool number
                LONG* pRet              // (O) Error code
            )

```

□ Automation call procedure

```

ATC_GetMGNPot(
    lIndex As LONG,           // (I) Magazine pot number
    pToolNo As LONG*         // (O) Tool number
) As LONG                    // (O) Error code

```

□ **Argument** *lIndex*: Sets the magazine pot number.
 Value: 1 to 360 (maximum) M700/M800 series
 Value: 1 to 80 (maximum) C70

pToolNo: Returns the tool number.
 Value: 0 to 99999999 (maximum)

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

EZ_ERR_DATA_RANGE: Invalid argument data range (*lIndex*)

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the tool number which is stored in the set pot of the magazine.
 This function is not supported with C80.

□ **Reference** **SetMGNPot()**, **GetMGNPotEx()**

□ **Specifica-
tion**

IEZNCATC3::GetMGNPot3

Get tool number for each magazine pot for ATC
tool registration

□ Custom call procedure

```

HRESULT      GetMGNPot3(
                LONG IMagazineNo,           // (I) Magazine number
                LONG IIndex,                // (I) Pot number
                LONG* pToolNo,              // (O) Tool number
                LONG* pRet                   // (O) Error code
            )

```

□ Automation call procedure

```

ATC_GetMGNPot3(
    IMagazineNo As LONG,           // (I) Magazine number
    IIndex As LONG,                // (I) Pot number
    pToolNo As LONG*              // (O) Tool number
) As LONG                          // (O) Error code

```

□ **Argument** *IMagazineNo*: Sets the magazine number.
 Value: 1 to 5 (maximum) M700/M800 series
 Value: 1 to 3 (maximum) C70

IIndex: Sets the pot number.
 Value: 1 to 360 (maximum) M700/M800 series
 Value: 1 to 80 (maximum) C70

pToolNo: Returns the tool number.
 Value: 0 to 99999999 (maximum)

pRet: Returns an error code. (Upon automation, the return value is used.)
S_OK: Normal termination
EZNC_DATA_READ_READ: Data is not readable
EZ_ERR_DATA_RANGE: Invalid argument data range (*IMagazineNo*, *IIndex*)

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the tool number which is stored in the pot of the set magazine number.	
Function	This function is not supported with C80.	

□ **Reference** **SetMGNPot3()**

□ **Specifica-
tion**

IEZNCATC3::SetMGNPot

Set tool number for magazine pots for ATC
tool registration

□ Custom call procedure

```

HRESULT      SetMGNPot(
                LONG lIndex,           // (I) Pot number
                LONG lToolNo,         // (I) Tool number
                LONG* pRet             // (O) Error code
            )

```

□ Automation call procedure

```

ATC_SetMGNPot(
    lIndex As LONG,           // (I) Pot number
    lToolNo As LONG          // (I) Tool number
) As LONG                    // (O) Error code

```

□ **Argument** *lIndex*: Sets the pot number.
 Value: 1 to 360 (maximum) M700/M800 series
 Value: 1 to 80 (maximum) C70

lToolNo: Sets the tool number.
 Value: 1 to 99999999 (maximum)

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZ_ERR_DATA_RANGE: Invalid argument data range (*lIndex*, *lToolNo*)

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Sets the tool number to be stored in the magazine pot. Care should be taken to avoid overlaps in tool numbers because double registration of a tool number is not checked. An alarm is raised when double registration is made. Alarm information can be got with GetAlarm().
 For the **M700/M800 series**, if the tool number of the Mitsubishi CNC is specified for 4 digits, the lower 4 digits are registered when the number with 5 digits or more is specified, and the 5th digit and higher will be discarded.
 This function is not supported with C80.

□ **Reference** GetMGNPot(), SetMGNPotEx(), GetAlarm()

□ **Specifica-
tion**

IEZNCATC3::SetMGNPot3**Set tool number for each magazine pot for ATC tool registration**

□ Custom call procedure

```

HRESULT      SetMGNPot3(
                LONG IMagazineNo,           // (I) Magazine number
                LONG IIndex,                // (I) Pot number
                LONG IToolNo,               // (I) Tool number
                LONG* pIRet                  // (O) Error code
            )

```

□ Automation call procedure

```

ATC_SetMGNPot3(
    IMagazineNo As LONG,           // (I) Magazine number
    IIndex As LONG,                // (I) Pot number
    IToolNo As LONG                // (I) Tool number
) As LONG                          // (O) Error code

```

□ **Argument** *IMagazineNo*: Sets the magazine number.
Value: 1 to 5 (maximum) M700/M800 series
Value: 1 to 3 (maximum) C70

IIndex: Sets the pot number.
Value: 1 to 360 (maximum) M700/M800 series
Value: 1 to 80 (maximum) C70

IToolNo: Sets the tool number.
Value: 0 to 99999999 (maximum) M700/M800 series
Value: 1 to 99999999 (maximum) C70

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_WRITE: Data is not writable

EZ_ERR_DATA_RANGE: Invalid argument data range (*IMagazineNo*, *IIndex*, *IToolNo*)

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Sets the tool number to be stored in the pot of the set magazine number. Care should be taken to avoid overlaps in tool numbers because double registration of a tool number is not checked. An alarm is raised when double registration is made. Alarm information can be got with GetAlarm(). If the tool number of the Mitsubishi CNC is specified for 4 digits, the lower 4 digits are registered when the number with 5 digits or more is specified, and the 5th digit and higher will be discarded. This function is not supported with C80.

□ **Reference** **GetMGNPot3(), GetAlarm()**

□ **Specifica-
tion**

IEZNCATC3::GetMGNAux**Get user PLC interface for ATC tool registration**□ **Custom call procedure**

```

HRESULT      GetMGNAux(
                LONG* pData,           // (O) Data
                LONG* pRet             // (O) Error code
            )

```

□ **Automation call procedure**

```

ATC_GetMGNAux(
    pData As LONG*           // (O) Data
) As LONG                   // (O) Error code

```

□ **Argument** *pIData*: Returns sequence processing data for user programmable controller.
Value: 0 to 65535

pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Returns sequence processing data for user PLC.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion**

IEZNCATC3::SetMGNAux**Set user PLC interface for ATC tool registration**□ **Custom call procedure**

```

HRESULT      SetMGNAux(
                LONG IData,           // (I) Data
                LONG* pIRet           // (O) Error code
            )

```

□ **Automation call procedure**

```

ATC_SetMGNAux(
    IData As LONG           // (I) Data
) As LONG                  // (O) Error code

```

□ **Argument** *IData*: Sets sequence processing data for userPLC.
Value: 0 to 65535

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_READ: Data is not readable

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Sets sequence processing data for user PLC.
This function is not supported with C80.

□ **Reference**

□ **Specifica-
tion**

IEZNCParameter3::GetParameterData2**Get parameters**

□ Custom call procedure

```

HRESULT      GetParameterData2(
                LONG IGroup,           // (I) Group number
                LONG IItem,           // (I) First item number
                LONG ISize,           // (I) Number of items
                LONG IAxis,           // (I) Axis No.
                LPOLESTR* lppwszValue, // (O) Parameter value character string array
                LONG* pIRet           // (O) Error code
            )

```

□ Automation call procedure

```

Parameter_GetData2(
    IGroup As LONG,           // (I) Group number
    IItem As LONG,           // (I) First item number
    ISize As LONG,           // (I) Number of items
    IAxis As LONG,           // (I) Axis No.
    pvValue As VARIANT*     // (O) Parameter value character string array
) As LONG                   // (O) Error code

```

□ *IGroup*: Sets the group number of parameter.

Argument	Model	Setting
	M700 series	Disabled
	C70	Disabled

IItem: Sets the first item number of parameter. This must be set. The parameter number described in the setup manual for each model will be this item number.

ISize: Sets the number of items of parameter. The range starts from 1.

IAxis: Sets the axis No. whose parameter is to be got. This argument needs not be set unless the parameter to be got is axis-dependent.

lppwszValue: Gets the parameter value as a **UNICODE** character string array. Though the character string area will internally be allocated in this product, the pointer of the character string (for *ISize*) needs to be reserved by the client. As the character string area is internally allocated in this product, the client using VC++ needs to release the memory area explicitly with **CoTaskMemFree()**.

The parameter value will be got as signed value regardless of its item number. In the case of unsigned **SHORT** type data "65535", "-1" will be got.

pvValue: See the explanation of *lppwszValue*.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_PARAM_FILENOTEXIST: No parameter information file

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Gets the parameter.
This function is not supported with the M800 Series and C80. (EZ_ERR_NOT_SUPPORT is returned to *pIRet*.)

□ **Reference** **SetParameterData2()**

□ **Specifica-
tion**

IEZNCParameter3::GetParameterData3**Get parameters**

□ Custom call procedure

```

HRESULT      GetParameterData3(
                LONG IGroup,           // (I) Group number
                LONG IItem,           // (I) First item number
                LONG ISize,           // (I) Number of items
                LONG IAxis,           // (I) Axis No.
                LPOLESTR* lppwszValue, // (O) Parameter value character string array
                LONG* plRet            // (O) Error code
            )

```

□ Automation call procedure

```

Parameter_GetData3(
    IGroup As LONG,           // (I) Group number
    IItem As LONG,           // (I) First item number
    ISize As LONG,           // (I) Number of items
    IAxis As LONG,           // (I) Axis No.
    pvValue As VARIANT*      // (O) Parameter value character string array
) As LONG                    // (O) Error code

```

□ *IGroup*: This is the reserved value. Set "0".

□ **Argument**

IItem: Sets the first item number of parameter. This must be set. The parameter number described in the setup manual for each model will be this item number.

ISize: Sets the number of items of parameter. The range starts from 1.

IAxis: Sets the axis whose parameter is to be got. This argument needs not be set unless the parameter to be got is axis-dependent.

lppwszValue: Gets the parameter value as a **UNICODE** character string array. Though the character string area will internally be allocated in this product, the pointer of the character string (for *ISize*) needs to be reserved by the client. As the character string area is internally allocated in this product, the client using VC++ needs to release the memory area explicitly with **CoTaskMemFree()**.

The parameter value will be got as signed value regardless of its item number. In the case of unsigned SHORT type data "65535", "-1" will be got.

pvValue: See the explanation of *lppwszValue*.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_PARAM_FILENOTEXIST: No parameter information file

EZNC_DATA_NOT_EXIST: Data does not exist

□	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Gets the parameter.	
□ Function	This function is not supported with C80.	
□	SetParameterData3()	
□ Reference		
□		
□ Specifica- tion		

IEZNCParameter3::SetParameterData2

Set parameters

□ Custom call procedure

```

HRESULT      SetParameterData2(
                LONG IGroup,           // (I) Group number
                LONG IItem,           // (I) First item number
                LONG ISize,           // (I) Number of items
                LONG IAxis,           // (I) Axis No.
                LPCOLESTR* lppcwszValue, // (I) Parameter value character string array
                LONG* pIRet            // (O) Error code
            )

```

□ Automation call procedure

```

Parameter_SetData2(
    IGroup As LONG,           // (I) Group number
    IItem As LONG,           // (I) First item number
    IAxis As LONG,           // (I) Axis No.
    vValue As VARIANT         // (I) Parameter value character string array
) As LONG                    // (O) Error code

```

□ *IGroup*: Sets the group number of parameter.

Argument	Model	Setting
	C70	Disabled
	M700 series	Disabled

IItem: Sets the first item number of parameter. This must be set. The parameter number described in the setup manual for each model will be this item number.

ISize: Sets the number of items of parameter. The range starts from 1.

IAxis: Sets the axis whose parameter is to be got. This argument needs not be set unless the parameter to be got is axis-dependent.

lppcwszValue: Sets the parameter value as a **UNICODE** character string array.

Automation argument:

vValue: See the explanation of *lppcwszValue*.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_ADDR: Invalid part system, axis No. setting

EZNC_PARAM_FILENOTEXIST: No parameter information file

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ *Function* Sets the parameter.

When setting the machine parameters, the NC must be set to the machine parameter setting mode state. Refer to the setup manual for each Mitsubishi CNC for details on setting the machine parameter setting mode.

This function is not supported with the M800 Series and C80. (EZ_ERR_NOT_SUPPORT is returned to *pIRet*.)

□ **GetParameterData2()**

Reference

□ Specification

IEZNCParameter3::SetParameterData3**Set parameters**

□ Custom call procedure

```

HRESULT      SetParameterData3(
                LONG IGroup,           // (I) Group number
                LONG IItem,           // (I) First item number
                LONG ISize,           // (I) Number of items
                LONG IAxis,           // (I) Axis No.
                LPCOLESTR* lppcwszValue, // (I) Parameter value character string array
                LONG* pIRet            // (O) Error code
            )

```

□ Automation call procedure

```

Parameter_SetData3(
    IGroup As LONG,           // (I) Group number
    IItem As LONG,           // (I) First item number
    IAxis As LONG,           // (I) Axis No.
    vValue As VARIANT         // (I) Parameter value character string array
) As LONG                    // (O) Error code

```

□ *IGroup*: Sets the group number of parameter.

Argument	Model	Setting
	C70	Disabled
	M700/M800 series	Disabled

IItem: Sets the first item number of parameter. This must be set. The parameter number described in the setup manual for each model will be this item number.

ISize: Sets the number of items of parameter. The range starts from 1.

IAxis: Sets the axis whose parameter is to be got. This argument needs not be set unless the parameter to be got is axis-dependent.

lppcwszValue: Sets the parameter value as a **UNICODE** character string array.

vValue: See the explanation of *lppcwszValue*.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_ADDR: Invalid part system, axis No.setting

EZNC_PARAM_FILENOTEXIST: No parameter information file

EZNC_DATA_NOT_EXIST: Data does not exist

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Sets the parameter.

To set a machine parameter, the NC needs to be in the machine parameter setting mode. Refer to the setup manual for each Mitsubishi CNC for details on setting the machine parameter setting mode.

This function is not supported with C80.

□ **Reference** **GetParameterData3()**□ **Specifica-
tion**

C70

M700

M800

IEZNcOperation::Search**Operation search**□ **Custom call procedure**

```

HRESULT      Search(
                LPCOLESTR lpcwszSelectProgram,    // (I) Program file name
                LONG ISequenceNo,                 // (I) Sequence number
                LONG IBlockNo,                     // (I) Block number
                LONG* plRet                         // (O) Error code
            )

```

□ **Automation call procedure**

```

Operation_Search(
    lpcwszSelectProgram As STRING,    // (I) Program file name
    ISequenceNo As LONG,             // (I) Sequence number
    IBlockNo As LONG                 // (I) Block number
) As LONG                            // (O) Error code

```

□ **Argument** *lpcwszSelectProgram*: Sets the program file name for operation search as a **UNICODE** character string.

ISequenceNo: Sets the sequence number to be searched.

IBlockNo: Sets the block number to be searched.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK : Normal termination

EZNC_OPE_SELECTPRG_ADDR: Invalid system specification

EZNC_OPE_SELECTPRG_FILESYSTEM: File system error

EZNC_OPE_SELECTPRG_NOPRG: No program file

EZNC_OPE_SELECTPRG_PRGFORMAT: Invalid program file name format

EZNC_OPE_SELECTPRG_RUNNING: Program operation in progress

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Executes operation search.

Function Use *lpcwszSelectProgram* to set the program file name whose operation is to be started.

The name of the program files in the \PRG\USER\ directory of NC control unit or in the M□:\IC1\ directory of NC's CF card (M700 Series) or NC's SD card (M800S/M80: at the front panel SD card slot, M800W: at the rear panel SD card slot #2) can be set.

Note that the file name of the program file in the \PRG\USER\ directory of NC control unit does not need to contain the drive name and directory path.

Use a character string as below to specify the program file name.

C70: "<program number>.PRG" Example) "1000.PRG"

M700/M800 series: "<program file name>" Example) "1000"

M700/M800 series: "M□:\IC1\<program file name>" Example) "M01:\IC1\1000"

ISequenceNo or *IBlockNo* can be used to set the sequence number or block number whose operation is to be started. Set 0 in *ISequenceNo* and *IBlockNo* when operation is to be started from the top of the program.

This function is not supported with C80.

□ **IEZNcProgram::CurrentBlockRead()**

Reference

□ **Specifica-
tion** System

IEZNCOperation::Run**Start PLC program**□ **Custom call procedure**

```
HRESULT      Run(
                LONG* pIRet
            )
// (O) Error code
```

□ **Automation call procedure**

```
Operation_Run( ) As LONG
// (O) Error code
```

□ **Argument** *pIRet*: Returns an error code. (Upon automation, the return value is used.)

S_OK : Normal termination

EZNC_OPE_ACTPLC_ADDR: Invalid NC control unit

	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Starts the PLC program.

This function is not supported with C80.

□ **Reference** **Stop()**

□ **Specifica-
tion**

IEZNCOperation::Stop**Stop PLC program**

□ Custom call procedure

```

HRESULT      Stop(
                LONG* pIRet
            )

```

// (O) Error code

□ Automation call procedure

```

Operation_Stop( ) As LONG

```

// (O) Error code

□ *pIRet*: Returns an error code. (Upon automation, the return value is used.)

Argument **S_OK** : Normal termination

EZNC_OPE_ACTPLC_ADDR: Invalid NC control unit

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ Stops the PLC program.

Function This function is not supported with C80.

□ **Run()**

Reference

□
**Specifica-
tion**

IEZNcDevice::SetDevice**Set device**

□ Custom call procedure

```

HRESULT      SetDevice(
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR* lppcwszDevice, // (I) Device character string
                LPDWORD lpdwDataType,    // (I) Data type
                LPDWORD lpdwValue,       // (I) Device value array
                LONG* plRet               // (O) Error code
            )

```

□ Automation call procedure

```

Device_SetDevice(
    vDevice As VARIANT,           // (I) Device character string
    vDataType As VARIANT,        // (I) Data type
    vValue As VARIANT,           // (I) Device value array
    ) As LONG                     // (O) Error code

```

□ **Argument** *dwLength*: Sets the number of device points to be set. The maximum value is 1K points.

lppcwszDevice: Sets the array of the device character string to be set. The device character string needs to be specified as UNICODE string. However if word format (or double word format) is set in the data type, the device character string needs to be set in multiples of 16 (or 32).

lpdwDataType: Sets each data type of the device to be set as an array.

Value	Meaning
EZNC_PLC_BIT	Bit
EZNC_PLC_WORD	Word
EZNC_PLC_DWORD	Double word

lpdwValue: Sets the array used to set the device value. When reading, set a dummy value that has the same number of array elements as that of the device character string.

Automation argument:

vDevice: Sets the array of the device character string to be set as **VARIANT**. It needs to be sets as UNICODE character string. However if word format (or double word format) is set in the data type, the device character string needs to be set in multiples of 16 (or 32).

vDataType: Sets the array of the data type of the device value to be set as **VARIANT**.

Value	Meaning
EZNC_PLC_BIT	Bit
EZNC_PLC_WORD	Word
EZNC_PLC_DWORD	Double word

vValue: Sets the array of the device value to be set as **VARIANT**. When reading, set a dummy value that has the same number of array elements as that of the device character string.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_DATATYPE: Invalid data type

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Function	
	Defines the device to be used in the user PLC. The device settings are all based on one-shot operation. The settings for one-shot operation may return to the original state after one cycle of the PLC operation. The settings cannot be retrieved when making the settings next time. This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to plRet.)	
□	Reference	
	ReadDevice2(), WriteDevice2(), DeleteDeviceAll()	
□	Specification	

Refer to the following manuals for the devices that can be set.

Manual list

Model	Manual name	Manual number
Mitsubishi Electric CNC M700/M70 series	700/70 Series PLC Programming Manual	IB-1500036(ENG)
Mitsubishi Electric CNC M700V/M70V series	M700V/M70V/E70 Series PLC Programming Manual	IB-1500918(ENG)
Mitsubishi Electric CNC M800/M80 series	M800/M80/E80/C80 Series PLC Interface Manual	IB-1501272(ENG)
Mitsubishi Electric CNC M800V/M80V series	M800V/M80V Series PLC Interface Manual	IB-1501616(ENG)

IEZNCDevice::DeleteDeviceAll**Delete all devices**□ **Custom call procedure**

```

HRESULT      DeleteDeviceAll(
                LONG* pRet
            )
// (O) Error code

```

□ **Automation call procedure**

```

Device_DeleteAll( ) As LONG
// (O) Error code

```

□ **Argument** *pRet*: Returns an error code. (Upon automation, the return value is used.)
 S_OK: Normal termination
 EZ_ERR_NOT_SUPPORT: Not supported

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Deletes all the data set in **SetDevice()**.
 This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pRet.)

□ **Reference** **SetDevice()**

□ **Specifica-
tion**

IEZNCDevice::ReadDevice**Read device**□ **Custom call procedure**

```

HRESULT      ReadDevice(
                LPDWORD lpdwLength,           // (O) Number of read device points
                LPDWORD* lppdwValue,         // (O) Array of read device value
                LONG* pRet                     // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_Read(
    pvValue As VARIANT*           // (O) Device value array
) As LONG                        // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use ReadDevice2() when newly starting use of EZSocket.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Reads all the devices **set in SetDevice()**.
This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pRet.)

□ **Reference** **SetDevice(), WriteDevice()**

□ **Specifica-
tion**

IEZNCDevice::WriteDevice**Write device**□ **Custom call procedure**

```

HRESULT      WriteDevice(
                LONG* pIRet                                // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_Write( ) As LONG                                // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use WriteDevice2() when newly starting use of EZSocket.

	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Writes all the devices **set in SetDevice()**.
This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to pIRet.)

□ **Reference** **SetDevice(), ReadDevice()**

□ **Specifica-
tion**

IEZNCDevice::ReadBlockDevice**Batch read devices**□ **Custom call procedure**

```

HRESULT      ReadBlockDevice(
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR lpcwszDevice,    // (I) Head device character string
                DWORD dwDataType,         // (I) Data type
                LPDWORD* lppdwValues,     // (O) Read device value array
                LONG* plRet               // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_ReadBlock(
    lLength As LONG,           // (I) Number of device points
    bstrDevice As STRING,      // (I) Head device character string
    lDataType As LONG,        // (I) Data type
    pvValues As VARIANT*     // (O) Read device value array
) As LONG                    // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use ReadBlockDevice2() when newly starting use of EZSocket.

	Value	Meaning
□ Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** A device with continuous device points from the head device character string is read out. This is valid only for the M700V/M70V/M800 series. This function is not supported with the M700/M70 series and C70/C80. (EZ_ERR_NOT_SUPPORT is returned to plRet.)

□ **Reference** **WriteBlockDevice()**

□ **Specifica-
tion**

IEZNCDevice::WriteBlockDevice**Batch write devices**□ **Custom call procedure**

```

HRESULT      WriteBlockDevice(
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR lpcwszDevice,   // (I) Head device character string
                DWORD dwDataType,         // (I) Data type
                LPDWORD lppdwValues,     // (I) Write device value array
                LONG* plRet               // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_WriteBlock(
    lLength As LONG,           // (I) Number of device points
    bstrDevice As STRING,     // (I) Head device character string
    lDataType As LONG,        // (I) Data type
    vValues As VARIANT       // (I) Write device value array
) As LONG                     // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use WriteBlockDevice2() when newly starting use of EZSocket.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** A device with continuous device points from the head device character string is read out.

This is valid only for the M700V/M70V/M800 series.
 This function is not supported with the M700/M70 series and C70/C80.
 (EZ_ERR_NOT_SUPPORT is returned to plRet.)

□ **Reference** **ReadBlockDevice()**

□ **Specifica-
tion**

IEZNcDevice::ReadRandomDevice**Random read device**□ **Custom call procedure**

```

HRESULT      ReadRandomDevice(
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR lpcwszDevice,    // (I) Device character string array
                LPDWORD* lppdwValues,     // (O) Read device value array
                LONG* plRet               // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_ReadRandom (
    lLength As LONG,           // (I) Number of device points
    vDevice As VARIANT,       // (I) Device name character string
    pvValues As VARIANT*     // (O) Read device value array
) As LONG                   // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use ReadRandomDevice2() when newly starting use of EZSocket.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Writes all the values of the set device arrays.

This is valid only for the M700/M800 series.

This function is not supported with C70/C80. (**EZ_ERR_NOT_SUPPORT** is returned to **plRet**.)

□ **WriteRandomDevice ()**

□ **Reference**□ **Specifica-
tion**

IEZNCDevice::WriteRandomDevice**Random write device**□ **Custom call procedure**

```

HRESULT      WriteRandomDevice (
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR lpcwszDevice,   // (I) Device character string array
                LPDWORD lpdwValues,       // (I) Write device value array
                LONG* plRet                // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_WriteRandom(
    lLength As LONG,           // (I) Number of device points
    vDevice As VARIANT,       // (I) Device name character string
    vValues As VARIANT        // (I) Write device value array
) As LONG                    // (O) Error code

```

□ **Argument** This method can be used as a backward compatible model. Use WriteRandomDevice2() when newly starting use of EZSocket.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Reads all the values of the set device arrays.
 This is valid only for the M700/M800 series.
 This function is not supported with C70/C80. (**EZ_ERR_NOT_SUPPORT** is returned to **plRet**.)

□ **Reference** **ReadRandomDevice()**

□ **Specifica-
tion**

IEZNCDevice::ReadDevice2**Read device**□ **Custom call procedure**

```

HRESULT      ReadDevice2(
                LONG IProjectNo,           // (I) Project number designation
                LPDWORD IpdwLength,       // (O) Number of read device points
                LPDWORD* IppdwValue,      // (O) Array of read device value
                LONG* pIRet                // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_Read2(
    IProjectNo AS LONG           // (I) Project number designation
    pvValue As VARIANT*        // (O) Device value array
    ) As LONG                   // (O) Error code

```

- **Argument** *IProjectNo*: Sets the project number of the multiple projects (0 to 6).
 The range of the value and operation when "0" is set differs depending on the NC specifications.
 Accesses to the set project device when the multi-project function is enabled.
 Set "0" when the multi-project function is disabled.
 This is ignored **when using the models which do not support the multi-project function**.
 When the project number is outside the setting range, **EZNC_DATA_PROJECT_RANGE** is returned to *pIRet*.

IpdwLength: Returns the number of read devices.

IppdwValue: Returns the array that contains the device value. As the device value array is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.

Automation argument:

IProjectNo: See the explanation of *IProjectNo*.

pvValue: Returns the device value array in **VARIANT**.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_READ_DATATYPE: Invalid data type

EZNC_DATA_READ_READ: Data is not readable.

EZNC_DATA_READ_WRITEONLY: Write-only data

EZNC_DATA_PROJECT_RANGE: Project number out of range

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Reads all the devices set in **SetDevice()**.
 This function supports the multi-project function and reads the devices of the project number set in the argument *IProjectNo*.
 The project number is ignored when accessing the common devices (such as X, Y, and F device).
 For details of the common devices, refer to "M800/M80/E80/C80 Series PLC Interface Manual (IB-1501272(ENG))".
 This function is not supported with C70/C80. (**EZ_ERR_NOT_SUPPORT** is returned to *pIRet*.)

The existing interface **IEZNCDevice::ReadDevice()** can be used as a backward compatible model. Use this method when newly starting use of **EZSocket**.

□ **Reference** **SetDevice()**, **WriteDevice2()**□ **Specifica-
tion**

IEZNCDevice::WriteDevice2**Write device**□ **Custom call procedure**

```

HRESULT      WriteDevice2(
                LONG IProjectNo,      // (I) Project number designation
                LONG* pIRet           // (O) Error code
            )

```

□ **Automation call procedure**

```

Device_Write2(
    IProjectNo AS LONG      // (I) Project number designation
) AS LONG                  // (O) Error code

```

- **Argument** *IProjectNo*: Sets the project number of the multiple projects (0 to 6).
 The range of the value and operation when "0" is set differs depending on the NC specifications.
 Accesses to the set project device when the multi-project function is enabled.
 Set "0" when the multi-project function is disabled.
 This is ignored **when using the models which do not support the multi-project function**.
 When the project number is outside the setting range, **EZNC_DATA_PROJECT_RANGE** is returned to *pIRet*.

pIRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_DATA_WRITE_DATATYPE: Invalid data type

EZNC_DATA_WRITE_WRITE: Data is not writable.

EZNC_DATA_WRITE_READONLY: Read-only data

EZNC_DATA_PROJECT_RANGE: Project number out of range

EZ_ERR_NOT_SUPPORT: Not supported

Automation argument:

IProjectNo: See the explanation of *IProjectNo*.

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

- **Function** Writes all the devices set in **SetDevice()**.

This function supports the multi-project function and writes the devices of the project number set in the argument *IProjectNo*.

The project number is ignored when accessing the common devices (such as X, Y, and F device).

For details of the common devices, refer to "M800/M80/E80/C80 Series PLC Interface Manual (IB-1501272(ENG))".

This function is not supported with C70/C80.

(**EZ_ERR_NOT_SUPPORT** is returned to *pIRet*.)

The existing interface **IEZNCDevice::WriteDevice()** can be used as a backward compatible model. Use this method when newly starting use of **EZSocket**.

- **Reference** **SetDevice()**, **ReadDevice2()**

- **Specifica-
tion**

IEZNCDevice::ReadBlockDevice2**Batch read devices**

□ Custom call procedure

```

HRESULT      ReadBlockDevice2 (
                LONG IProjectNo,           // (I) Project number designation
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR IpcwszDevice,    // (I) Head device character string
                DWORD dwDataType,         // (I) Data type
                LPDWORD* IppdwValues,     // (O) Read device value array
                LONG* pIRet                // (O) Error code
            )

```

□ Automation call procedure

```

Device_ReadBlock2 (
    IProjectNo As LONG           // (I) Project number designation
    ILength As LONG,            // (I) Number of device points
    bstrDevice As STRING,       // (I) Head device character string
    IDataType As LONG,          // (I) Data type
    pvValues As VARIANT*        // (O) Read device value array
) As LONG                      // (O) Error code

```

□ **Argument** *IProjectNo*: Sets the project number of the multiple projects (0 to 6).
The range of the value and operation when "0" is set differs depending on the NC specifications.
Accesses to the set project device when the multi-project function is enabled.
Set "0" when the multi-project function is disabled.
This is ignored **when using the models which do not support the multi-project function**.
When the project number is outside the setting range, **EZNC_DATA_PROJECT_RANGE** is returned to *pIRet*.

dwLength: Sets the number of device points to be set. (2 or more)
The maximum number of points that can be got is determined by the data type of the device being got.

Device data type	Maximum number of getting points
------------------	----------------------------------

EZNC_PLC_BIT	1280 points
EZNC_PLC_BYTE	1280 points
EZNC_PLC_WORD	640 points
EZNC_PLC_DWORD	320 points

EZ_ERR_DATA_RANGE is returned to *pIRet* when two or less is set.

IpcwszDevice: Sets the array of the device character string to be set. The device character string needs to be specified as **UNICODE** string. However if word format (or double word format) is set in the data type, the device character string needs to be set in multiples of 16 (or 32).
EZNC_DATA_READ_DATATYPE is returned to *IRet* when the incorrect device is set for the device character string.

dwDataType: Sets each data type of the device to be set.

Value	Meaning
EZNC_PLC_BIT	Bit
EZNC_PLC_BYTE	Byte
EZNC_PLC_WORD	Word
EZNC_PLC_DWORD	Double word

IppdwValue: Sets the array used to set the device value. The data array is secured on the EZSocket side, so explicitly release it on the client side using *CoTaskMemFree()*.
The values are got in the device corresponds to the size for the data type at the lower end of the array.

Example) Data type: **EZNC_PLC_BYTE**, Set device value: 0x82
Read device value: 0x00000082

Automation argument:

IProjectNo : See the explanation of *IProjectNo*
ILength : See the explanation of *dwLength*
bstrDevice : See the explanation of *IpcwszDevice*
IDataType : See the explanation of *IppdwDataType*

pvValue: Returns the device value array in **VARIANT**.
pRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination
EZNC_DATA_READ_DATATYPE: Invalid data type
EZNC_DATA_READ_READ: Data is not readable.
EZNC_DATA_READ_WRITEONLY: Write-only data
EZNC_DATA_PROJECT_RANGE: Project number out of range
EZ_ERR_NOT_SUPPORT: Not supported

☐	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

☐ **Function** Reads all the devices of the consecutive device points from the head device character string. This function supports the multi-project function and reads the devices of the project number set in the argument **IPProjectNo** in block units.

The project number is ignored when accessing the common devices (such as X, Y, and F device).
For details of the common devices, refer to "M800/M80/E80/C80 Series PLC Interface Manual (IB-1501272(ENG))".

This function is not supported with C70/C80.
(**EZ_ERR_NOT_SUPPORT** is returned to *pRet*.)

The existing interface IEZNcDevice::WriteBlockDevice() can be used as a backward compatible model. Use this method when newly starting use of EZSocket.

☐	WriteBlockDevice2()
Reference	
☐	
Specifica- tion	

IEZNCDevice::WriteBlockDevice2**Batch write devices**

□ Custom call procedure

```

HRESULT      WriteBlockDevice2 (
                LONG lProjectNo,           // (I) Project number designation
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR lpcwszDevice,    // (I) Head device character string
                DWORD dwDataType,         // (I) Data type
                LPDWORD lpdwValues,       // (I) Write device value array
                LONG* plRet               // (O) Error code
            )

```

□ Automation call procedure

```

Device_WriteBlock2 (
    lProjectNo AS LONG           // (I) Project number designation
    lLength AS LONG,             // (I) Number of device points
    bstrDevice AS STRING,        // (I) Head device character string
    lDataType AS LONG,          // (I) Data type
    vValues AS VARIANT         // (I) Write device value array
    ) AS LONG                   // (O) Error code

```

□ **Argument** *lProjectNo*: Sets the project number of the multiple projects (0 to 6).
 The range of the value and operation when "0" is set differs depending on the NC specifications.
 Accesses to the set project device when the multi-project function is enabled.
 Set "0" when the multi-project function is disabled.
 This is ignored **when using the models which do not support the multi-project function**.
 When the project number is outside the setting range, **EZNC_DATA_PROJECT_RANGE** is returned to *plRet*.

dwLength: Sets the number of device points to be set. (2 or more)
 The maximum number of points that can be got is determined by the data type of the device being got.

Device data type	Maximum number of gettable points
EZNC_PLC_BIT	1280 points
EZNC_PLC_BYTE	1280 points
EZNC_PLC_WORD	640 points
EZNC_PLC_DWORD	320 points

lpcwszDevice: Sets the array of the head device character string to be set. The device character string needs to be set as **UNICODE** string. However if word format (or double word format) is set in the data type, the device character string needs to be set in multiples of 16 (or 32).

dwDataType: Sets each data type of the device to be set.

Value	Meaning
EZNC_PLC_BIT	Bit
EZNC_PLC_BYTE	Byte
EZNC_PLC_WORD	Word
EZNC_PLC_DWORD	Double word

lpdwValue: Sets the array used to set the device value.
 The value set in the device corresponds to the size for the data type at the lower end of the array.
 Example) Data type: **EZNC_PLC_BYTE**, Written device value: 0x12345678
 Set device value: 0x78

Automation argument:

lProjectNo : See the explanation of *lProjectNo*
lLength : See the explanation of *dwLength*
bstrDevice : See the explanation of *lpcwszDevice*
lDataType : See the explanation of *lpdwDataType*

vValue: Returns the device value array in **VARIANT**.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination
EZNC_DATA_READ_DATATYPE: Invalid data type
EZNC_DATA_WRITE_WRITE: Data is not writable.
EZNC_DATA_WRITE_READONLY: Read-only data
EZNC_DATA_PROJECT_RANGE: Project number out of range
EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
□	Writes all the devices of the consecutive device points from the head device character string.	
Function	This function supports the multi-project function and writes the devices of the project number set in the argument <i>lProjectNo</i> in block units.	
	The project number is ignored when accessing the common devices (such as X, Y, and F device).	
	For details of the common devices, refer to "M800/M80/E80/C80 Series PLC Interface Manual (IB-1501272(ENG))".	
	This is valid only for the M700V/M70V/M800 series.	
	This function is not supported with the M700/M70 series and C70/C80.	
	(EZ_ERR_NOT_SUPPORT is returned to <i>pRet</i> .)	
	The existing interface IEZNcDevice::ReadBlockDevice() can be used as a backward compatible model. Use this method when newly starting use of EZSocket.	
□	ReadBlockDevice2()	
Reference		
□		
Specification		

IEZNCDevice::ReadRandomDevice2**Random read device**

□ Custom call procedure

```

HRESULT      ReadRandomDevice2 (
                LONG IProjectNo,                // (I) Project number designation
                DWORD dwLength,                // (I) Number of device points
                LPCOLESTR lpcwszDevice,        // (I) Device character string array
                LPDWORD* lppdwValues,          // (O) Read device value array
                LONG* plRet                    // (O) Error code
            )

```

□ Automation call procedure

```

Device_ReadRandom2(
    IProjectNo AS LONG                // (I) Project number designation
    ILength As LONG,                 // (I) Number of device points
    vDevice As VARIANT,              // (I) Device name character string
    pvValues As VARIANT*             // (O) Read device value array
) As LONG                           // (O) Error code

```

□ **Argument** *IProjectNo*: Sets the project number of the multiple projects (0 to 6).
 The range of the value and operation when "0" is set differs depending on the NC specifications.
 Accesses to the set project device when the multi-project function is enabled.
 Set "0" when the multi-project function is disabled.
 This is ignored **when using the models which do not support the multi-project function**.
 When the project number is outside the setting range, **EZNC_DATA_PROJECT_RANGE** is returned to *plRet*.

dwLength: Sets the number of device points to be set.

lpcwszDevice: Sets the array of the device character string to be get. The device character string needs to be specified as UNICODE string. For details of the range of the corresponding device arrays, refer to "Manual list" of **SetDevice()**.

* Set "CC" to specify the C device and "TC" to specify the T device.

lppdwValue: Returns the array that contains the device value. As the device value array is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree()**.
 The acquired device data (regardless of the data type; bit device or word device) is stored as DWORD(32bit).

Automation argument:
 See the explanation of *IProjectNo*: *IProjectNo*.

ILength: See the explanation of *dwLength*.

vDevice: Sets the device character string array to be set as **VARIANT**. The device character string needs to be specified as **UNICODE** string.

pvValue: Returns the device value array in **VARIANT**.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination
EZNC_COMM_NOTMODULE: No submodule
EZNC_COMM_CREATEPC: EZSocketPc objects cannot be created
EZNC_DATA_READ_READ: Data is not readable
EZNC_DATA_READ_WRITEONLY: Write-only data
EZNC_DATA_PROJECT_RANGE: Project number out of range
EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure

□	Reads the device values from the set device name character string.
Function	
	This function supports the multi-project function and reads the devices of the project number set in the argument <i>lProjectNo</i> .
	The project number is ignored when accessing the common devices (such as X, Y, and F device).
	For details of the common devices, refer to "M800/M80/E80/C80 Series PLC Interface Manual (IB-1501272(ENG))".
	This is valid only for the M700/M70/M700V/M70V/M800 series.
	This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to <i>pRet</i> .)
	The existing interface IEZNcDevice::WriteRandomDevice() can be used as a backward compatible model. Use this method when newly starting use of EZSocket.

□	WriteRandomDevice2()
Reference	
□	
Reference	

IEZNCDevice::WriteRandomDevice2**Random write device**

□ Custom call procedure

```

HRESULT      WriteRandomDevice2 (
                LONG IProjectNo,           // (I) Project number designation
                DWORD dwLength,           // (I) Number of device points
                LPCOLESTR lpcwszDevice,    // (I) Device character string array
                LPDWORD lpdwValues,       // (I) Write device value array
                LONG* plRet               // (O) Error code
            )

```

□ Automation call procedure

```

Device_WriteRandom2(
    IProjectNo AS LONG           // (I) Project number designation
    ILength As LONG,            // (I) Number of device points
    vDevice As VARIANT,         // (I) Device name character string
    vValues As VARIANT         // (I) Write device value array
) As LONG                       // (O) Error code

```

□ **Argument** *IProjectNo*: Sets the project number of the multiple projects (0 to 6).
The range of the value and operation when "0" is set differs depending on the NC specifications.
Accesses to the set project device when the multi-project function is enabled.
Set "0" when the multi-project function is disabled.
This is ignored **when using the models which do not support the multi-project function**.
When the project number is outside the setting range, **EZNC_DATA_PROJECT_RANGE** is returned to *plRet*.

dwLength: Sets the number of device points to be set.

lpcwszDevice: Sets the array of the device character string to be set. The device character string needs to be specified as **UNICODE** string. For details of the range of the corresponding device arrays, refer to "Manual list" of **SetDevice()**.

* Set "CC" to specify the C device and "TC" to specify the T device.

lpdwValue: Sets the array that the device value is set.
The value set to the device (regardless of the data type; bit device or word device) is stored as **DWORD**(32bit).

Automation argument:

IProjectNo: See the explanation of *IProjectNo*.

ILength: See the explanation of *dwLength*.

vDevice: Sets the device character string array to be set as **VARIANT**. The device character string needs to be specified as **UNICODE** string.

vValue: Returns the device value array in **VARIANT**.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_COMM_NOTMODULE: No submodule

EZNC_COMM_CREATEPC: EZSocketPc objects cannot be created

EZNC_DATA_WRITE_WRITE: Data is not writable.

EZNC_DATA_WRITE_READONLY: Read-only data

EZNC_DATA_PROJECT_RANGE: Project number out of range

EZ_ERR_NOT_SUPPORT: Not supported

□	Value	Meaning
Return	S_OK	Normal termination
value	S_FALSE	Communication failure

□ Function	Writes the device values from the set device name character string. This function supports the multi-project function and writes randomly the devices of the project number set in the argument IProjectNo. The project number is ignored when accessing the common devices (such as X, Y, and F device). For details of the common devices, refer to "M800/M80/E80/C80 Series PLC Interface Manual (IB-1501272(ENG))". This is valid only for M700/M70/M700V/M70V/M800. This function is not supported with C70/C80. (EZ_ERR_NOT_SUPPORT is returned to <i>pRet</i>.) The existing interface IEZNcDevice::ReadRandomDevice() can be used as a backward compatible model. Use this method when newly starting use of EZSocket.
------------------------------	---

□ Reference	ReadRandomDevice2()
-------------------------------	----------------------------

□ Specifica- tion	
---	--

C70

M700

M800

IEZNCSubFunction3::Changelnit2**Initialize subfunction**

□ Custom call procedure

```

HRESULT      Changelnit2 (
                LONG ISystemType,           // (I) Mitsubishi CNC type
                LONG IReserve1,           // (I) Reservation 1
                LONG IReserve2,           // (I) Reservation 2
                LONG* pIRet                // (O) Error code
            )

```

□ Automation call procedure

```

                Changelnit2 (
                    ISystemType As LONG,           // (I) Mitsubishi CNC type
                    IReserve1 As LONG,           // (I) Reservation 1
                    IReserve2 As LONG           // (I) Reservation 2
                ) As LONG                          // (O) Error code

```

□ *ISystemType*: Sets the Mitsubishi CNC type.

Argument	Value	Meaning
	EZNC_SYS_MELDASC70	Perform initialization on C70.
	EZNC_SYS_MELDAS700M	Perform initialization on M700 M series.
	EZNC_SYS_MELDAS700L	Perform initialization on M700 L series.
	EZNC_SYS_MELDAS800M	Perform initialization on M800 M series.
	EZNC_SYS_MELDAS800L	Perform initialization on M800 L series.

IReserve1: Not used. (Always set 0.)*IReserve2*: Not used. (Always set 0.)*pIRet*: Returns an error code. (Upon automation, the return value is used.)**S_OK**: Normal termination**EZ_ERR_DATA_RANGE**: Invalid data range

Return value	Value	Meaning
	S_OK	Normal termination
	S_FALSE	Communication failure

□ **Function** Initializes **IEZNCSubFunction**.
 This function is not supported with C80.

□ **Reference**□ **Specifica-
tion**

IEZNCSubFunction3::GetToolWorkOffsetOfFile

Get data from workpiece
offset file

□ Custom call procedure

```

HRESULT      GetToolWorkOffsetOfFile(
                LPCOLESTR lpcwszFileName,    // (I) File name containing a path
                LONG lHead,                  // (I) Part system
                LONG lIndex,                  // (I) Workpiece coordinate system number
                LPCOLESTR* lppcwszAxis,        // (I) Axis name character string array
                LPOLESTR** lpppwszData,        // (O) Workpiece coordinate data value
                                                    character string array
                LONG* plRet                    // (O) Error code
            )

```

□ Automation call procedure

```

GetToolWorkOffsetOfFile(
    bstrFileName As STRING,    // (I) File name containing a path
    lHead As LONG,              // (I) Part system
    lIndex As LONG,             // (I) Workpiece coordinate system number
    vAxis As VARIANT,           // (I) Axis name character string array
    pvData As VARIANT*         // (O) Workpiece coordinate data value
                                character string array
) As LONG                      // (O) Error code

```

□ **Argument** *lpcwszFileName*: Sets the file name including path of the workpiece offset file as a **UNICODE** character string.

Set the file with absolute path as below.

Drive name + ":" + \directory name\file name

lHead: Sets the system. Up to 2 part systems can be set for M700/M700V/M70/M70V/M800.

lIndex: Sets the workpiece coordinate system number to be read.

Value	Meaning
54	G54 offset
55	G55 offset
56	G56 offset
57	G57 offset
58	G58 offset
59	G59 offset
60	EXT offset
61	P1 offset
62	P2 offset
:	:
108	P48 offset

lppcwszAxis: Sets the axis name as a **UNICODE** character string array. (Example: "X") The number of array elements is 8 (0 to 7). Set a **NULL** character string to the axes not applicable. (A NULL pointer cannot be set.) This is used only in C70. Set a NULL character string to all elements in any other model.

lpppwszData: Returns the workpiece offset data value as a UNICODE character string array. As the data value array is allocated in this product, the client needs to release it explicitly with **CoTaskMemFree ()**.

<i>lpppwszData</i>	Tool change data type	Remarks
0	1st axis	
1	2nd axis	
2	3rd axis	
3	4th axis	
4	5th axis	
5	6th axis	
6	7th axis	
7	8th axis	

The unit is [inch] or [mm] depending on the parameter setting of NC.

Automation argument:

bstrFileName: See the explanation of *lpcwszFileName*.

vAxis: See the explanation of *lppcwszAxis*.

pvData: Returns the workpiece offset data value as VARIANT.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_OPEN_FILENOTEXIST: File does not exist

EZNC_FILE_OPEN_OPEN: File cannot be opened

EZNC_FILE_READFILE_READ: Data is not readable

EZNC_DATA_NOT_EXIST: Data does not exist

EZ_ERR_MEMORY_ALLOC: Memory cannot be allocated.

☐	Value	Meaning
Return value	S_OK	Normal termination
	S_FALSE	Communication failure
☐	Gets the offset value of the workpiece coordinate system of the set part system and axis No. Only 2 part systems/3 axes can be set for M700/M700V/M70/M70V/M800. This function is not supported with C80.	
Function		
☐	OpenFile3(), CloseFile2(), ReadFile2(), WriteFile(), SetToolWorkOffsetFile()	
Reference		
☐		
Specifica- tion		

IEZNCSubFunction3::SetToolWorkOffsetOfFile**Set data to workpiece offset file**□ **Custom call procedure**

```

HRESULT      SetToolWorkOffsetOfFile(
                LPCOLESTR lpcwszFileName,    // (I) File name containing a path
                LONG lMode,                  // (I) Setting mode
                LONG lHead,                  // (I) Part System
                LONG lIndex,                 // (I) Workpiece coordinate system number
                LPCOLESTR* lppcwszAxis,       // (I) Axis name string character array
                LPCOLESTR* lppcwszData,       // (I) Workpiece coordinate data value
                                                // character string array
                LONG* plRet                  // (O) Error code
            )

```

□ **Automation call procedure**

```

SetToolWorkOffsetOfFile(
    bstrFileName As STRING,    // (I) File name containing a path
    lMode As LONG,             // (I) Setting mode
    lHead As LONG,             // (I) Part system
    lIndex As LONG,            // (I) Workpiece coordinate system number
    vAxis As VARIANT,          // (I) Axis name character string array
    vData As VARIANT           // (I) Workpiece coordinate data value
                                // character string array
) As LONG                     // (O) Error code

```

□ **Argument** *lpcwszFileName*: Sets the file name including path of the workpiece offset file as a **UNICODE** character string.

Set the file with absolute path as below.

Drive name + ":" + \directory name\file name

lMode: Sets the setting mode of the tool life management file.

Value	Meaning
-------	---------

EZNC_FILE_CREATE	Creates a new tool life management file.
-------------------------	--

EZNC_FILE_OPEN	Modifies an existing tool life management file
-----------------------	--

lHead: Sets the system. Up to 2 part systems can be set for M700/M700V/M70/M70V/M800.

lIndex: Sets the workpiece coordinate system number to be written to.

Value	Meaning
54	G54 offset
55	G55 offset
56	G56 offset
57	G57 offset
58	G58 offset
59	G59 offset
60	EXT offset
61	P1 offset
62	P2 offset
:	:
108	P48 offset

lppcwszAxis: Sets the axis name as a **UNICODE** character string. Set a **NULL** character string to the axes that do not exist. This is used only in C70. Set a NULL character string to all elements in any other model.

lppcwszData: Returns the workpiece offset data as a UNICODE character string. Set a **NULL** character string to the axes that do not exist.
The unit is [inch] or [mm] depending on the parameter setting of CNC.

Index table

Array Index	Tool change data type	Remarks
0	1st axis	
1	2nd axis	
2	3rd axis	
3	4th axis	
4	5th axis	
5	6th axis	
6	7th axis	
7	8th axis	

Automation argument:

bstrFileName: See the explanation of *lpcwszFileName*.

vAxis: See the explanation of *lppcwszAxis*.

vData: Creates the workpiece offset data as a UNICODE character string and sets it by substituting it in *vData(VARIANT)*. For details of the workpiece offset data, see the explanation of *lppcwszData* and the index above.

plRet: Returns an error code. (Upon automation, the return value is used.)

S_OK: Normal termination

EZNC_FILE_OPEN_FILENOTEXIST: File does not exist

EZNC_FILE_OPEN_OPEN: File cannot be opened

EZNC_FILE_WRITEFILE_WRITE: Data is not writable

EZ_ERR_NULLPTR: Argument is NULL pointer

```

[Example]
LPOLESTR* lppwszAxis;
lppwszAxis = new LPOLESTR[8];
lppwszAxis [0] =L"";
lppwszAxis [1] =L"";
lppwszAxis [2] =L"";
lppwszAxis [3] =L"";
lppwszAxis [4] =L"";
lppwszAxis [5] =L"";
lppwszAxis [6] =L"";
lppwszAxis [7] =L"";
LPOLESTR* lppwszData;
lppwszData = new LPOLESTR[11];
lppwszData[0] =L"-1.000";
lppwszData[1] =L"1.000";
lppwszData[2] =L"3.000";
lppwszData[3] =L"";
lppwszData[4] =L"";
lppwszData[5] =L"";
lppwszData[6] =L"";
lppwszData[7] =L"";
hr = pIEZNCTool->SetToolLifeValueOfFile(L"C:\\TEMP\\OFFSET.WRK",EZNC_FILE_OPEN, 1, 54,
(LPCOLESTR*)lppwszAxis,(LPCOLESTR*)lppwszData, &IRet);

if( S_OK != hr ){
    wprintf(L"HRESULT Code = 0x%x, IRet Code = 0x%x\\n", hr, IRet );
}
delete[ ] lppwszData;

```

□ Return value	Value	Meaning
	S_OK S_FALSE	Normal termination Communication failure
□ Function	Sets the offset value of the workpiece coordinate system of the set part system and axis No. Only 2 part systems/3 axes can be set for M700/M700V/M70/M70V/M800. This function is not supported with C80.	
□ Reference	OpenFile3(), CloseFile2(), ReadFile2(), WriteFile(), GetToolWorkOffsetFile()	
□ Specifica- tion		

3. ERROR CODE LIST

This section provides a list of error codes.

No.	Error code	Number	Description
1.	EZ_ERR_NOT_OPEN	0x80A00101	Communication lines are not open.
2.	EZ_ERR_DOUBLE_OPEN	0x80A00104	Double open error
3.	EZ_ERR_DATA_TYPE	0x80A00105	Invalid argument data type
4.	EZ_ERR_DATA_RANGE	0x80A00106	Invalid argument data range
5.	EZ_ERR_NOT_SUPPORT	0x80A00107	Not supported
6.	EZ_ERR_CANNOT_OPEN	0x80A00109	Communication line cannot be opened.
7.	EZ_ERR_NULLPTR	0x80A0010A	Argument is NULL pointer.
8.	EZ_ERR_DATA_LENGTH	0x80A0010B	Invalid argument data
9.	EZ_ERR_OPEN_COMM	0x80A0010C	COMM port handle error
10.	EZ_ERR_MEMORY_ALLOC	0x80B00101	Memory cannot be allocated.
11.	EZNC_ERR_CANNOT_GETPCERR	0x80B00102	EZSocketPc error cannot be obtained.
12.	EZNC_FILE_OPEN_MODE	0x80B00201	Invalid mode specification
13.	EZNC_FILE_OPEN_NOTOPEN	0x80B00202	File is not open.
14.	EZNC_FILE_OPEN_FILEEXIST	0x80B00203	File already exists.
15.	EZNC_FILE_OPEN_ALREADYOPENED	0x80B00204	File is already open.
16.	EZNC_FILE_OPEN_CREATE	0x80B00205	Temporary file cannot be created.
17.	EZNC_FILE_WRITEFILE_NOTOPEN	0x80B00206	File is open without write mode specification.
18.	EZNC_FILE_WRITEFILE_LENGTH	0x80B00207	Invalid write data size
19.	EZNC_FILE_WRITEFILE_WRITE	0x80B00208	Not writable
20.	EZNC_FILE_READFILE_NOTOPEN	0x80B00209	File is open without read mode specification.
21.	EZNC_FILE_READFILE_READ	0x80B0020A	Not readable
22.	EZNC_FILE_READFILE_CREATE	0x80B0020B	Temporary file cannot be created.
23.	EZNC_FILE_OPEN_FILENOTEXIST	0x80B0020C	File does not exist (READ mode).
24.	EZNC_FILE_OPEN_OPEN	0x80B0020D	File cannot be opened.
25.	EZNC_FILE_OPEN_ILLEGALPATH	0x80B0020E	Invalid file path
26.	EZNC_FILE_READFILE_ILLEGALFILE	0x80B0020F	Invalid read file
27.	EZNC_FILE_WRITEFILE_ILLEGALFILE	0x80B00210	Invalid write file
28.	EZNC_COMM_CANNOT_OPEN	0x80B00301	Host name for local connection used for automation call is invalid.
29.	EZNC_COMM_NOTSETUP_PROTOCOL	0x80B00302	TCP/IP communication is not configured.
30.	EZNC_COMM_ALREADYOPENED	0x80B00303	Cannot be set because communication is already in progress.
31.	EZNC_COMM_NOTMODULE	0x80B00304	No submodule
32.	EZNC_COMM_CREATEPC	0x80B00305	EZSocketPc objects cannot be created.
33.	EZNC_DATA_NOT_EXIST	0x80B00401	Data does not exist.
34.	EZNC_DATA_DUPLICATE	0x80B00402	Duplicate data
35.	EZNC_DATA_PROJECT_RANGE	0x80B00403	Project number out of range
36.	EZNC_PARAM_FILENOTEXIST	0x80B00501	No parameter information file
37.	EZNC_SYSFUNC_IOCTL_ADDR	0x80020190	Invalid NC control unit number
38.	EZNC_SYSFUNC_IOCTL_NOTOPEN	0x80020102	Device is not open.
39.	EZNC_SYSFUNC_IOCTL_FUNCTION	0x80020132	Invalid command
40.	EZNC_SYSFUNC_IOCTL_DATA	0x80020133	Invalid communication parameter data range
41.	EZNC_FILE_DIR_FILESYSTEM	0x80030143	File system error
42.	EZNC_FILE_DIR_NODIR	0x80030191	Directory does not exist.
43.	EZNC_FILE_DIR_NODRIVE	0x8003019B	Drive does not exist.
44.	EZNC_PCFILE_DIR_NODIR	0x800301A2	Directory does not exist.
45.	EZNC_PCFILE_DIR_NODRIVE	0x800301A8	Drive does not exist.
46.	EZNC_OPE_CURRALM_ADDR	0x80050D90	Invalid part system, axis No. setting
47.	EZNC_OPE_CURRALM_ALMTYPE	0x80050D02	Invalid alarm type
48.	EZNC_OPE_CURRALM_DATAERR	0x80050D03	Error in communication data between NC and personal computer
49.	EZNC_OPE_CURRALM_DATASIZE	0x80050D93	Application does not fit into prepared buffer.
50.	EZNC_OPE_CURRALM_DATATYPE	0x80050D94	Invalid data type

No.	Error code	Number	Description
51.	EZNC_OPE_CURRALM_NOS	0x80050D01	The number of obtained messages is invalid.
52.	EZNC_OPE_CURRALMEX_NOS	0x80052201	The number of obtained messages is invalid.
53.	EZNC_OPE_CURRALMEX_ALMTYPE	0x80052202	Invalid alarm type
54.	EZNC_OPE_CURRALMEX_DATAERR	0x80052203	Error in communication data between NC and personal computer
55.	EZNC_OPE_CURRALMEX_ADDR	0x80052290	Invalid address
56.	EZNC_OPE_CURRALMEX_DATASIZE	0x80052293	Application does not fit into prepared buffer.
57.	EZNC_OPE_CURRALMEX_DATATYPE	0x80052294	Invalid data type
58.	EZNC_OPE_GETPRGBLK_NOSEARCH	0x80050C02	Operation search is not completed.
59.	EZNC_OPE_GETPRGBLK_DATAERR	0x80050C03	Error in communication data between NC and personal computer
60.	EZNC_OPE_GETPRGBLK_ADDR	0x80050C90	Invalid part system specification
61.	EZNC_OPE_GETPRGBLK_DATASIZE	0x80050C93	Application does not fit into prepared buffer.
62.	EZNC_OPE_GETPRGBLK_DATATYPE	0x80050C94	Invalid data type
63.	EZNC_OPE_GETPRGBLK_NOS	0x80050C01	The number of specified blocks is invalid.
64.	EZNC_OPE_SELECTPRG_RESET	0x80051004	Operation search cannot be performed (in reset state).
65.	EZNC_OPE_SELECTPRG_LONGPATH	0x80051005	File path name is long.
66.	EZNC_OPE_SELECTPRG_TIMEOUT	0x80051007	Time-out
67.	EZNC_OPE_SELECTPRG_ADDR	0x80051090	Invalid part system specification
68.	EZNC_OPE_SELECTPRG_FILEREAD	0x80051094	File reading error
69.	EZNC_OPE_SELECTPRG_FILEWRITE	0x80051095	File writing error
70.	EZNC_OPE_SELECTPRG_FILESYSTEM	0x80051043	File system error
71.	EZNC_OPE_SELECTPRG_NOPRG	0x80051002	No program file
72.	EZNC_OPE_SELECTPRG_PRGFORMAT	0x80051001	Format error for program file name
73.	EZNC_OPE_SELECTPRG_RUNNING	0x80051003	Program operation is in progress.
74.	EZNC_OPE_SELECTPRG_NOTPRG	0x80051002	No program file
75.	EZNC_OPE_SELECTPRG_NCPCCOM	0x80051006	No "ncpccom.exe"
76.	EZNC_OPE_SELECTPRG_SEARCHING	0x8005100A	Operation search cannot be performed (searching is in progress).
77.	EZNC_OPE_SELECTPRG_CHECKING	0x8005100D	Search cannot be performed (program check is in progress).
78.	EZNC_OPE_SELECTPRG_MODE	0x80051091	Mode error
79.	EZNC_OPE_SELECTPRG_NOTSUPPORT ED	0x80051049	Cannot search (no option)
80.	EZNC_OPE_SELECTPRG_PGLKC	0x8005104B	Cannot search (Program display lock C)
81.	EZNC_OPE_SELECTPRG_ILLEGALPOS	0x8005100E	Invalid position specification
82.	EZNC_OPE_SELECTPRG_REVERSE	0x8005100F	In arbitrary reverse run
83.	EZNC_OPE_SELECTPRG_PROTECT	0x8005104A	Cannot search (operating or editing program is inhibited.)
84.	EZNC_OPE_ACTPLC_ADDR	0x80050990	NC card error
85.	EZNC_OPE_ACTPLC_MODE	0x80050991	Mode error
86.	EZNC_DATA_TLFGROUP_ADDR	0x80041090	Invalid address (part system specification)
87.	EZNC_DATA_TLFGROUP_EXIST	0x80041091	Group No. already exists.
88.	EZNC_DATA_TLFGROUP_NONEXIST	0x80041092	Group No. does not exist.
89.	EZNC_DATA_TLFGROUP_OVER	0x80041093	Number of registered groups exceeds limit.
90.	EZNC_DATA_TLFGROUP_NONFORMAT	0x80041094	Formatting is not completed.
91.	EZNC_DATA_TLFGROUP_UNMACH	0x80041096	Specified group No. is not matched.
92.	EZNC_DATA_TLFGROUP_OUTOFSPEC	0x80041097	Specified group No. is not applicable.
93.	EZNC_DATA_TLFTOOL_ADDR	0x80041190	Invalid address (part system specification)
94.	EZNC_DATA_TLFTOOL_EXIST	0x80041191	Tool No. already exists.
95.	EZNC_DATA_TLFTOOL_NONEXIST	0x80041192	Tool No. does not exist.
96.	EZNC_DATA_TLFTOOL_OVER	0x80041193	Number of registered tools exceeds limit.
97.	EZNC_DATA_TLFTOOL_PARAMERR	0x80041194	Invalid type specified for life control data
98.	EZNC_DATA_TLFTOOL_MAXMINERR	0x80041195	Setting data is out of range.
99.	EZNC_DATA_TLFTOOL_UNMACH	0x80041196	Specified tool number mismatch
100.	EZNC_DATA_TLFTOOL_OUTOFSPEC	0x80041197	Specified tool number is out of specifications.
101.	EZNC_DATA_READ_ADDR	0x80040190	Invalid part system, axis No. setting

No.	Error code	Number	Description
102.	EZNC_DATA_READ_SECT	0x80040191	Invalid section number
103.	EZNC_DATA_READ_SUBSECT	0x80040192	Invalid subsection number
104.	EZNC_DATA_READ_DATASIZE	0x80040196	Application does not fit into prepared buffer.
105.	EZNC_DATA_READ_DATATYPE	0x80040197	Invalid data type
106.	EZNC_DATA_READ_READ	0x8004019D	Data is not readable.
107.	EZNC_DATA_READ_WRITEONLY	0x8004019F	Write-only data
108.	EZNC_DATA_READ_AXIS	0x800401A0	Invalid axis specification
109.	EZNC_DATA_READ_DATANUM	0x800401A1	Invalid data number
110.	EZNC_DATA_READ_NODATA	0x800401A3	Read data not found
111.	EZNC_DATA_READ_VALUE	0x8004019A	Invalid read data range
112.	EZNC_DATA_WRITE_ADDR	0x80040290	Invalid part system, axis No. setting
113.	EZNC_DATA_WRITE_SECT	0x80040291	Invalid section number.
114.	EZNC_DATA_WRITE_SUBSECT	0x80040292	Invalid subsection number
115.	EZNC_DATA_WRITE_DATASIZE	0x80040296	Application does not fit into prepared buffer.
116.	EZNC_DATA_WRITE_DATATYPE	0x80040297	Invalid data type.
117.	EZNC_DATA_WRITE_READONLY	0x8004029B	Read-only data
118.	EZNC_DATA_WRITE_WRITE	0x8004029E	Data is not writable.
119.	EZNC_DATA_WRITE_AXIS	0x800402A0	Invalid axis specification
120.	EZNCDATA_WRITE_SAFETYLOCK	0x8004024D	Safety password locked
121.	EZNCDATA_WRITE_UOPEN_FORMAT	0x800402A2	Formatting canceled because of invalid SRAM open parameter
122.	EZNCDATA_WRITE_EDTFILE_REGIST	0x800402A4	Cannot register edit file (already being edited).
123.	EZNCDATA_WRITE_EDTFILE_RELEASE	0x800402A5	Cannot release edit file.
124.	EZNCDATA_WRITE_NODATA	0x800402A3	No data at write destination
125.	EZNCDATA_WRITE_VALUE	0x8004029A	Invalid write data range
126.	EZNCDATA_WRITE_SAFE_NOPASSWD	0x800402A6	Safety password not set
127.	EZNCDATA_WRITE_SAFE_CHECKERR	0x800402A7	Safety data consistency check error
128.	EZNCDATA_WRITE_SAFE_DATATYPE	0x800402A9	Safety data type invalid
129.	EZNCDATA_WRITE_SORT	0x800402A8	Cannot write while sorting tool data
130.	EZNC_DATA_MDLCANCEL_NOTREGIST	0x80040501	Not registered for fast read
131.	EZNC_DATA_MDLREGIST_PRIORITY	0x80040402	Invalid priority specification
132.	EZNC_DATA_MDLREGIST_REGIST	0x80040401	Exceeded the limit of registrations
133.	EZNC_DATA_MDLREGIST_ADDR	0x80040490	Invalid address
134.	EZNC_DATA_MDLREGIST_SECT	0x80040491	Invalid section number
135.	EZNC_DATA_MDLREGIST_SUBSECT	0x80040492	Invalid subsection number
136.	EZNC_DATA_MDLREGIST_DATATYPE	0x80040497	Invalid data type
137.	EZNC_DATA_MDLREGIST_READONLY	0x8004049B	Read-only data
138.	EZNC_DATA_MDLREGIST_READ	0x8004049D	Data is not readable.
139.	EZNC_DATA_MDLREGIST_WRITEONLY	0x8004049F	Write-only data
140.	EZNC_DATA_MDLREGIST_AXIS	0x800404A0	Invalid axis specification
141.	EZNC_DATA_RETHREADWRITE_NODATA	0x80040BA3	Rethread cut position not set
142.	EZNC_FILE_DIR_ALREADYOPENED	0x80030101	A different directory is already opened.
143.	EZNC_FILE_DIR_DATASIZE	0x80030103	Exceeded maximum data size
144.	EZNC_FILE_DIR_NAMELENGTH	0x80030148	File name is too long.
145.	EZNC_FILE_DIR_ILLEGALNAME	0x80030198	Invalid file name format
146.	EZNC_FILE_DIR_NOTOPEN	0x80030190	Not open
147.	EZNC_FILE_DIR_READ	0x80030194	File information read error
148.	EZNC_PCFILE_DIR_ALREADYOPENED	0x80030102	A different directory is already opened (personal computer only).
149.	EZNC_PCFILE_DIR_NOTOPEN	0x800301A0	Not open
150.	EZNC_PCFILE_DIR_NOFILE	0x80030142	File does not exist.
151.	EZNC_PCFILE_DIR_READ	0x800301A5	File information read error.
152.	EZNC_FILE_COPY_BUSY	0x80030447	Copying is disabled (during operation).
153.	EZNC_FILE_COPY_ENTRYOVER	0x80030403	Exceeded the limit of registrations
154.	EZNC_FILE_COPY_FILEEXIST	0x80030401	Copy destination file already exists.
155.	EZNC_FILE_COPY_FILESYSTEM	0x80030443	File system error

No.	Error code	Number	Description
156.	EZNC_FILE_COPY_NAMELENGTH	0x80030448	File name is too long.
157.	EZNC_FILE_COPY_ILLEGALNAME	0x80030498	Invalid file name format
158.	EZNC_FILE_COPY_MEMORYOVER	0x80030404	Memory capacity exceeded.
159.	EZNC_FILE_COPY_NODIR	0x80030491	Directory does not exist.
160.	EZNC_FILE_COPY_NODRIVE	0x8003049B	Drive does not exist.
161.	EZNC_FILE_COPY_NOFILE	0x80030442	File does not exist.
162.	EZNC_FILE_COPY_PLCRUN	0x80030446	Copying is disabled (PLC in operation).
163.	EZNC_FILE_COPY_READ	0x80030494	Transfer source file is not readable.
164.	EZNC_FILE_COPY_WRITE	0x80030495	Transfer destination file is not writable.
165.	EZNC_FILE_COPY_PROTECT	0x8003044A	Copying is disabled (protected).
166.	EZNC_FILE_COPY_DIFFER	0x80030405	Verification error
167.	EZNC_FILE_COPY_NOTSUPPORTED	0x80030449	Verification function is not supported.
168.	EZNC_FILE_COPY_EXECUTING	0x8003044C	Copying file
169.	EZNC_FILE_COPY_NOTOPEN	0x80030490	File is not open.
170.	EZNC_FILE_COPY_WRITE_WARNING	0x80030495	Transfer destination file is not writable.
171.	EZNC_FILE_COPY_SAFETYPWLOCK	0x8003044D	Safety password locked
172.	EZNC_FILE_COPY_ILLEGALFORMAT	0x8003049D	Invalid file format
173.	EZNC_FILE_COPY_WRONGPASSWORD	0x8003049E	Password is wrong.
174.	EZNC_PCFILE_COPY_CREATE	0x800304A4	File cannot be created (personal computer only).
175.	EZNC_PCFILE_COPY_OPEN	0x800304A3	File cannot be opened (personal computer only).
176.	EZNC_PCFILE_COPY_FILEEXIST	0x80030402	Copy destination file already exists.
177.	EZNC_PCFILE_COPY_ILLEGALNAME	0x800304A7	Invalid file name format
178.	EZNC_PCFILE_COPY_NODIR	0x800304A2	Directory does not exist.
179.	EZNC_PCFILE_COPY_NODRIVE	0x800304A8	Drive does not exist.
180.	EZNC_PCFILE_COPY_NOFILE	0x800304A1	File does not exist.
181.	EZNC_PCFILE_COPY_READ	0x800304A5	Transfer source file is not readable.
182.	EZNC_PCFILE_COPY_WRITE	0x800304A6	Transfer destination file is not writable.
183.	EZNC_PCFILE_COPY_MEMORYOVER	0x80030406	Disk space exceeded.
184.	EZNC_PCFILE_COPY_NOTOPEN	0x800304A0	File is not open.
185.	EZNC_FILE_DEL_NOTDELETE	0x80030201	File cannot be deleted.
186.	EZNC_FILE_DEL_NOFILE	0x80030242	File does not exist.
187.	EZNC_FILE_DEL_FILESYSTEM	0x80030243	File system error
188.	EZNC_FILE_DEL_BUSY	0x80030247	Deletion is disabled (during operation).
189.	EZNC_FILE_DEL_NAMELENGTH	0x80030248	File name is too long.
190.	EZNC_FILE_DEL_PROTECT	0x8003024A	Deletion is disabled (protected).
191.	EZNC_FILE_DEL_NODIR	0x80030291	Directory does not exist.
192.	EZNC_FILE_DEL_ILLEGALNAME	0x80030298	Invalid file name format
193.	EZNC_FILE_DEL_NODRIVE	0x8003029B	Drive does not exist.
194.	EZNC_PCFILE_DEL_NOTDELETE	0x80030202	File cannot be deleted.
195.	EZNC_PCFILE_DEL_ILLEGALNAME	0x800302A7	Invalid file name format
196.	EZNC_PCFILE_DEL_NODIR	0x800302A2	Directory does not exist.
197.	EZNC_PCFILE_DEL_NODRIVE	0x800302A8	Drive does not exist.
198.	EZNC_PCFILE_DEL_NOFILE	0x800302A1	File does not exist.
199.	EZNC_FILE_REN_FILEEXIST	0x80030301	New file name already exists.
200.	EZNC_FILE_REN_NOFILE	0x80030342	File does not exist.
201.	EZNC_FILE_REN_FILESYSTEM	0x80030343	File system error.
202.	EZNC_FILE_REN_BUSY	0x80030347	Renaming is disabled (during operation).
203.	EZNC_FILE_REN_NAMELENGTH	0x80030348	File name is too long.
204.	EZNC_FILE_REN_PROTECT	0x8003034A	Renaming is disabled (protected).
205.	EZNC_FILE_REN_NODIR	0x80030391	Directory does not exist.
206.	EZNC_FILE_REN_ILLEGALNAME	0x80030398	Invalid file name format
207.	EZNC_FILE_REN_NODRIVE	0x8003039B	Drive does not exist.
208.	EZNC_PCFILE_REN_NOTRENAME	0x80030304	Renaming is disabled.
209.	EZNC_PCFILE_REN_SAMENAME	0x80030306	New and old file names are identical.
210.	EZNC_PCFILE_REN_FILEEXIST	0x80030302	New file name already exists.

No.	Error code	Number	Description
211.	EZNC_PCFILE_REN_ILLEGALNAME	0x800303A7	Invalid file name format
212.	EZNC_PCFILE_REN_NODIR	0x800303A2	Directory does not exist.
213.	EZNC_PCFILE_REN_NODRIVE	0x800303A8	Drive does not exist.
214.	EZNC_PCFILE_REN_NOFILE	0x800303A1	File does not exist.
215.	EZNC_FILE_DISKFREE_NODIR	0x80030691	Directory does not exist.
216.	EZNC_FILE_DISKFREE_NODRIVE	0x8003069B	Drive does not exist.
217.	EZNC_FILE_DISKFREE_FILESYSTEM	0x80030643	File system error
218.	EZNC_FILE_DISKFREE_NAMELENGTH	0x80030648	File name is too long.
219.	EZNC_FILE_DISKFREE_ILLEGALNAME	0x80030648	Invalid file name format
220.	EZNC_PCFILE_DISKFREE_NODIR	0x800306A2	Directory does not exist (personal computer only).
221.	EZNC_PCFILE_DISKFREE_NODRIVE	0x800306A8	Drive does not exist (personal computer only).
222.	EZNC_FILE_DRVLIST_DATASIZE	0x80030701	Application does not fit into prepared buffer.
223.	EZNC_FILE_DRVLIST_READ	0x80030794	Drive information read error
224.	EZNC_FILE_CREATEDIR_FILEEXIST	0x80030A03	Directory already exists.
225.	EZNC_FILE_CREATEDIR_FILESYSTEM	0x80030A43	File system error
226.	EZNC_FILE_CREATEDIR_ILLEGALNAME	0x80030A98	Invalid file name format.
227.	EZNC_FILE_CREATEDIR_NODIR	0x80030A91	Directory does not exist.
228.	EZNC_FILE_CREATEDIR_NOTSUPPORTED	0x80030A49	Not supported (inapplicable directory path is specified.)
229.	EZNC_FILE_CREATEDIR_NAMELENGTH	0x80030A48	Path is long.
230.	EZNC_FILE_CREATEDIR_MEMORYOVER	0x80030A05	Exceeded memory capacity
231.	EZNC_FILE_CREATEDIR_ALREADYOPENED	0x80030A01	The other directory is already open.
232.	EZNC_FILE_CREATEDIR_ROOTDIRFULL	0x80030A07	Maximum number of root directories exceeded
233.	EZNC_FILE_CREATEDIR_WRITEERR	0x80030A95	Cannot write
234.	EZNC_FILE_CREATEDIR_WRITE_PROTECTED	0x80030AB2	Write protected
235.	EZNC_PCFILE_CREATEDIR_FILEEXIST	0x80030A04	Directory already exists (personal computer only).
236.	EZNC_PCFILE_CREATEDIR_ILLEGALNAME	0x80030AA7	Invalid file name format (personal computer only)
237.	EZNC_PCFILE_CREATEDIR_NODIR	0x80030AA2	Directory does not exist (personal computer only).
238.	EZNC_PCFILE_CREATEDIR_NAMELENGTH	0x80030AA9	Path is long (personal computer only).
239.	EZNC_PCFILE_CREATEDIR_MEMORYOVER	0x80030A06	Exceeded memory capacity (personal computer only)
240.	EZNC_PCFILE_CREATEDIR_ALREADYOPENED	0x80030A02	The other directory is already open (personal computer only).
241.	EZNC_PCFILE_CREATEDIR_ROOTDIRFULL	0x80030A08	The other directory is already open (personal computer only).
242.	EZNC_PCFILE_CREATEDIR_WRITEERR	0x80030AA6	Cannot write (personal computer only)
243.	EZNC_FILE_DELETEDIR_FILESYSTEM	0x80030B43	File system error
244.	EZNC_FILE_DELETEDIR_ILLEGALNAME	0x80030B98	Invalid file name format
245.	EZNC_FILE_DELETEDIR_NODIR	0x80030B91	Directory does not exist.
246.	EZNC_FILE_DELETEDIR_NOTSUPPORTED	0x80030B49	Not supported (inapplicable directory path is specified.)
247.	EZNC_FILE_DELETEDIR_NAMELENGTH	0x80030B48	Path is long.
248.	EZNC_FILE_DELETEDIR_NOTEMPTY	0x80030B03	Directory is not empty.
249.	EZNC_FILE_DELETEDIR_ALREADYOPENED	0x80030B01	The other directory is already open.
250.	EZNC_FILE_DELETEDIR_NOTDELETE	0x80030B05	Cannot delete
251.	EZNC_PCFILE_DELETEDIR_ILLEGALNAME	0x80030BA7	Invalid file name format (personal computer only)
252.	EZNC_PCFILE_DELETEDIR_NODIR	0x80030BA2	Directory does not exist (personal computer only).

No.	Error code	Number	Description
253.	EZNC_PCFILE_DELETEDIR_NAMELENGT H	0x80030BA9	Path is long (personal computer only).
254.	EZNC_PCFILE_DELETEDIR_NOTEMPTY	0x80030B04	Directory is not empty (personal computer only).
255.	EZNC_PCFILE_DELETEDIR_ALREADYOP ENED	0x80030B02	The other directory is already open (personal computer only).
256.	EZNC_PCFILE_DELETEDIR_NOTDELETE	0x80030B06	Cannot delete (personal computer only)
257.	EZNC_ENET_ALREADYOPEN	0x82020001	Already open.
258.	EZNC_ENET_NOTOPEN	0x82020002	Not open
259.	EZNC_ENET_CARDNOTEXIST	0x82020004	Card does not exist.
260.	EZNC_ENET_BADCHANNEL	0x82020006	Invalid channel number
261.	EZNC_ENET_BADFD	0x82020007	Invalid file descriptor
262.	EZNC_ENET_NOTCONNECT	0x8202000A	Not connected
263.	EZNC_ENET_NOTCLOSE	0x8202000B	Not closed
264.	EZNC_ENET_TIMEOUT	0x82020014	Time-out
265.	EZNC_ENET_DATAERR	0x82020015	Invalid data
266.	EZNC_ENET_CANCELED	0x82020016	Terminated by cancel request
267.	EZNC_ENET_ILLEGALSIZ	0x82020017	Invalid packet size
268.	EZNC_ENET_TASKQUIT	0x82020018	Terminated due to by end of task
269.	EZNC_ENET_UNKNOWNFUNC	0x82020032	Invalid command
270.	EZNC_ENET_SETDATAERR	0x82020033	Invalid setting data
271.	EZNC_READ_CACHE_REGIST	0x80060001	Invalid data read cache
272.	EZNC_READ_CACHE_ADDR	0x80060090	Invalid address
273.	EZNC_READ_CACHE_SECT	0x80060091	Invalid section number
274.	EZNC_READ_CACHE_SUBSECT	0x80060092	Invalid subsection number
275.	EZNC_READ_CACHE_DATATYPE	0x80060097	Invalid data type
276.	EZNC_READ_CACHE_DATA	0x8006009A	Invalid data range
277.	EZNC_READ_CACHE_READ	0x8006009D	Data is not readable
278.	EZNC_READ_CACHE_WRITEONLY	0x8006009F	Invalid data type
279.	EZNC_READ_CACHE_AXIS	0x800600A0	Invalid axis specification
280.	EZNC_FS_OPEN_FILE_MALLOC	0x80070140	Work area cannot be allocated.
281.	EZNC_FS_OPEN_FILE_OPEN	0x80070142	File cannot be opened.
282.	EZNC_FS_OPEN_FILE_BUSY	0x80070147	File cannot be opened (during operation).
283.	EZNC_FS_OPEN_FILE_NAMELENGTH	0x80070148	File path is too long.
284.	EZNC_FS_OPEN_FILE_NOTSUPPORTED	0x80070149	Not supported (CF not supported)
285.	EZNC_FS_OPEN_FILE_PROTECT	0x8007014A	Protect data setting error
286.	EZNC_FS_OPEN_FILE_ALREADYOPEN	0x80070192	Already open
287.	EZNC_FS_OPEN_FILE_FILEFULL	0x80070199	Maximum number of open files exceeded.
288.	EZNC_FS_OPEN_FILE_SORT	0x8007019F	Cannot open while sorting tool data
289.	EZNC_FS_OPEN_FILE_SAFE_NOPASSW D	0x800701B0	Safety password not authenticated
290.	EZNC_FS_OPEN_FILE_WRITE_PROTECT	0x800701B2	Write protect error
291.	EZNC_FS_CLOSE_FILE_NOTOPEN	0x80070290	File is not open.
292.	EZNC_FS_CREATE_FILE_MALLOC	0x80070340	Work area cannot be allocated.
293.	EZNC_FS_CREATE_FILE_BUSY	0x80070347	File cannot be created (during operation).
294.	EZNC_FS_CREATE_FILE_NAMELENGTH	0x80070348	File path is too long.
295.	EZNC_FS_CREATE_FILE_NOTSUPPORT ED	0x80070349	Not supported (CF not supported)
296.	EZNC_FS_CREATE_FILE_ALREADYOPE N	0x80070392	Already created
297.	EZNC_FS_CREATE_FILE_CREATE	0x80070393	File cannot be created.
298.	EZNC_FS_CREATE_FILE_FILEFULL	0x80070399	Maximum number of open files exceeded.
299.	EZNC_FS_CREATE_FILE_NODRIVE	0x8007039B	Drive does not exist.
300.	EZNC_FS_READ_FILE_NOTOPEN	0x80070490	File is not open.
301.	EZNC_FS_READ_FILE_READ	0x80070494	File information read error
302.	EZNC_FS_WRITE_FILE_NOTSUPPORTE D	0x80070549	Write is not available.
303.	EZNC_FS_WRITE_FILE_NOTOPEN	0x80070590	File is not open.

No.	Error code	Number	Description
304.	EZNC_FS_WRITE_FILE_WRITE	0x80070595	File write error
305.	EZNC_FS_REMOVE_FILE_REMOVEERR	0x80070740	File deletion error
306.	EZNC_FS_REMOVE_FILE_NOFILE	0x80070742	File does not exist.
307.	EZNC_FS_REMOVE_FILE_BUSY	0x80070747	File cannot be deleted (during operation).
308.	EZNC_FS_REMOVE_FILE_NAMELENGTH	0x80070748	File path is too long.
309.	EZNC_FS_REMOVE_FILE_NOTSUPPORTED	0x80070749	Not supported (CF not supported)
310.	EZNC_FS_REMOVE_FILE_PROTECT	0x8007074A	Protect data setting error
311.	EZNC_FS_REMOVE_FILE_ALREADYOPEN	0x80070792	File is already open.
312.	EZNC_FS_REMOVE_FILE_NODRIVE	0x8007079B	Drive does not exist.
313.	EZNC_FS_REMOVE_FILE_WRITE_PROTECT	0x800707B2	Write protect error
314.	EZNC_FS_RENAME_FILE_NOFILE	0x80070842	File does not exist.
315.	EZNC_FS_RENAME_FILE_NOTRENAME	0x80070843	File cannot be renamed.
316.	EZNC_FS_RENAME_FILE_NAMELENGTH	0x80070848	File path is too long.
317.	EZNC_FS_RENAME_FILE_NOTSUPPORTED	0x80070849	Not supported (CF not supported)
318.	EZNC_FS_RENAME_FILE_ALREADYOPEN	0x80070892	File is already open.
319.	EZNC_FS_RENAME_FILE_FILEFULL	0x80070899	Maximum number of open files exceeded.
320.	EZNC_FS_RENAME_FILE_NODRIVE	0x8007089B	Drive does not exist.
321.	EZNC_FS_IOCTL_FILE_FUNCTION	0x80070944	Invalid command (not supported)
322.	EZNC_FS_IOCTL_FILE_NOTOPEN	0x80070990	Not open
323.	EZNC_FS_IOCTL_FILE_READ	0x80070994	Read error
324.	EZNC_FS_IOCTL_FILE_WRITE	0x80070995	Write error
325.	EZNC_FS_IOCTL_FILE_DATASIZE	0x80070996	Application does not fit into prepared buffer.
326.	EZNC_FS_IOCTL_FILE_DATATYPE	0x80070997	Invalid data type
327.	EZNC_FS_IOCTL_FILE_NOTSUPPORTED	0x80070949	Not supported (CF not supported)
328.	EZNC_FS_OPEN_DIR_MALLOC	0x80070A40	Work area cannot be allocated.
329.	EZNC_FS_OPEN_DIR_BUSY	0x80070A47	Directory cannot be opened (during operation).
330.	EZNC_FS_OPEN_DIR_NAMELENGTH	0x80070A48	File path is too long.
331.	EZNC_FS_OPEN_DIR_NOTSUPPORTED	0x80070A49	Not supported (CF not supported)
332.	EZNC_FS_OPEN_DIR_NODIR	0x80070A91	Directory does not exist.
333.	EZNC_FS_OPEN_DIR_NOTOPEN	0x80070A92	Already open.
334.	EZNC_FS_OPEN_DIR_FILEFULL	0x80070A99	Maximum number of open directories exceeded.
335.	EZNC_FS_OPEN_DIR_NODRIVE	0x80070A9B	Drive does not exist.
336.	EZNC_FS_READ_DIR_NOTOPEN	0x80070B90	Directory is not open.
337.	EZNC_FS_READ_DIR_NODIR	0x80070B91	Directory does not exist.
338.	EZNC_FS_READ_DIR_DATASIZE	0x80070B96	Application does not fit into prepared buffer.
339.	EZNC_FS_CLOSE_DIR_NOTOPEN	0x80070D90	Directory is not open.
340.	EZNC_FS_STAT_FILE_NAMELENGTH	0x80070E48	File path is too long.
341.	EZNC_FS_STAT_FILE_NOTSUPPORTED	0x80070E49	Supported (CF not supported)
342.	EZNC_FS_STAT_FILE_STATERR	0x80070E94	File information read error
343.	EZNC_FS_STAT_FILE_FILEFULL	0x80070E99	Maximum number of open files exceeded.
344.	EZNC_FS_STAT_FILE_NODRIVE	0x80070E9B	Drive does not exist.
345.	EZNC_FS_FSTAT_FILE_NAMELENGTH	0x80070F48	File path is too long.
346.	EZNC_FS_FSTAT_FILE_NOTSUPPORTED	0x80070F49	Not supported (CF not supported)
347.	EZNC_FS_FSTAT_FILE_STATERR	0x80070F94	File information read error
348.	EZNC_FS_FSTAT_FILE_NOTOPEN	0x80070F90	File is not open.
349.	EZNC_FS_FSTAT_FILE_NODRIVE	0x80070F9B	Drive does not exist.
350.	EZNC_FS_IOCTL_UOPEN_FORMAT	0x8007099C	Formatting canceled because of invalid SRAM open parameter.
351.	Error codes output by NC control unit.	0xF00000FF	Invalid argument
352.	Error codes output by NC control unit.	0xFFFFFFFF	Data is not readable/writable.

No.	Error code	Number	Description
353.	MELSEC communication error codes	0x0101XXXX 0x0180XXXX	For details, refer to each error code indicated as follows. However, refer to the manual of each CPU unit for the following error codes. 0x01010000 to 0x0101FFFF: Motion Controller CPU (some of the error codes are also described in this table.) 0x01070000 to 0x0107FFFF: CC-Link IE Controller Network Board, MELSECNET/H Board, CC-Link Ver.2 Board 0x01090000 to 0x0109FFFF: FXCPU 0x010A0000 to 0x010AFFFF: QCPU (Q Mode), QSCPU, RCP, FX5CPU 0x010B0000 to 0x010BFFFF: Q Series compatible C24 0x010C0000 to 0x010CFFFF: Q Series compatible E71 0x010D0000 to 0x010DFFFF: Personal Computer CPU Unit 0x010F0000 to 0x010FFFFF: GOT
354.	-	0x01010002	Timeout error
355.	-	0x01010005	Message error
356.	-	0x01010010	Programmable controller No. error
357.	-	0x01010011	Mode error/Command not supported.
358.	-	0x01010012	Special unit specification error
359.	-	0x01010013	Other data error/Communication cannot be made for some cause.
360.	-	0x01010018	Remote request error
361.	-	0x01010020	Link error
362.	-	0x01010021	Special unit bus error
363.	-	0x01800001	No command error The method does not support.
364.	-	0x01800002	Memory lock error
365.	-	0x01800003	Memory securing error
366.	-	0x01800004	DLL load error
367.	-	0x01800005	Resource securing error
368.	-	0x01801001	Resource timeout error
369.	-	0x01801002	Multi-line open error
370.	-	0x01801003	Open not yet executed
371.	-	0x01801004	Open type error
372.	-	0x01801005	Specified port error
373.	-	0x01801006	Specified module error
374.	-	0x01801007	Specified CPU error
375.	-	0x01801008	Target station access error
376.	-	0x01801009	Registry open failure
377.	-	0x0180100A	Packet type error
378.	-	0x0180100B	Protocol type error
379.	-	0x0180100C	Registry search failure
380.	-	0x0180100D	GetProcAddress failure
381.	-	0x0180100E	DLL non-load error
382.	-	0x0180100F	Another object in execution
383.	-	0x01802001	Device error
384.	-	0x01802002	Device number error
385.	-	0x01802003	Program type error
386.	-	0x01802004	Sumcheck error
387.	-	0x01802005	Size error
388.	-	0x01802006	Block number error
389.	-	0x01802007	Receive data error
390.	-	0x01802008	Write protect error

No.	Error code	Number	Description
391.	-	0x01802009	Reading parameters error
392.	-	0x0180200A	Writing parameters error
393.	-	0x0180200B	Programmable controller type mismatch
394.	-	0x0180200C	Request cancel error
395.	-	0x0180200D	Drive name error
396.	-	0x0180200E	Beginning step error
397.	-	0x0180200F	Parameter type error
398.	-	0x01802010	File name error
399.	-	0x01802011	Status error
400.	-	0x01802012	Detailed condition field error
401.	-	0x01802013	Step condition error
402.	-	0x01802014	Bit device condition error
403.	-	0x01802015	Parameter settings error
404.	-	0x01802016	Error in specifying telephone exchange number
405.	-	0x01802017	Keyword error
406.	-	0x01802018	Read/Write flag error
407.	-	0x01802019	Refresh method error
408.	-	0x0180201A	Buffer access method error
409.	-	0x0180201B	Start mode/Stop mode error
410.	-	0x0180201C	Written clock data error
411.	-	0x0180201D	Online clock data write error
412.	-	0x0180201E	ROM drive error
413.	-	0x0180201F	While tracing error
414.	-	0x01802020	First I/O number error
415.	-	0x01802021	First address error
416.	-	0x01802022	Pattern error
417.	-	0x01802023	SFC block No. error
418.	-	0x01802024	SFC step No. error
419.	-	0x01802025	Step No. error
420.	-	0x01802026	Data error
421.	-	0x01802027	System data error
422.	-	0x01802028	Error in number of TC settings value
423.	-	0x01802029	Clear mode error
424.	-	0x0180202A	Signal flow error
425.	-	0x0180202B	Version control error
426.	-	0x0180202C	Monitor not registered error
427.	-	0x0180202D	PI type error
428.	-	0x0180202E	PI No. error
429.	-	0x0180202F	Error in number of PIs
430.	-	0x01802030	Shift error
431.	-	0x01802031	File type error
432.	-	0x01802032	Specified unit error
433.	-	0x01802033	Error check flag error
434.	-	0x01802034	Step RUN operation error
435.	-	0x01802035	Step RUN data error
436.	-	0x01802036	During step RUN error
437.	-	0x01802037	Write error while running program corresponding to E2PROM
438.	-	0x01802038	Clock data read/write error
439.	-	0x01802039	Trace not completed error
440.	-	0x0180203A	Registration clear flag error
441.	-	0x0180203B	Operation error
442.	-	0x0180203C	Error in the number of exchanges
443.	-	0x0180203D	Error in number of loops specified
444.	-	0x0180203E	Retrieve data selection
445.	-	0x0180203F	Error in number of SFC cycles

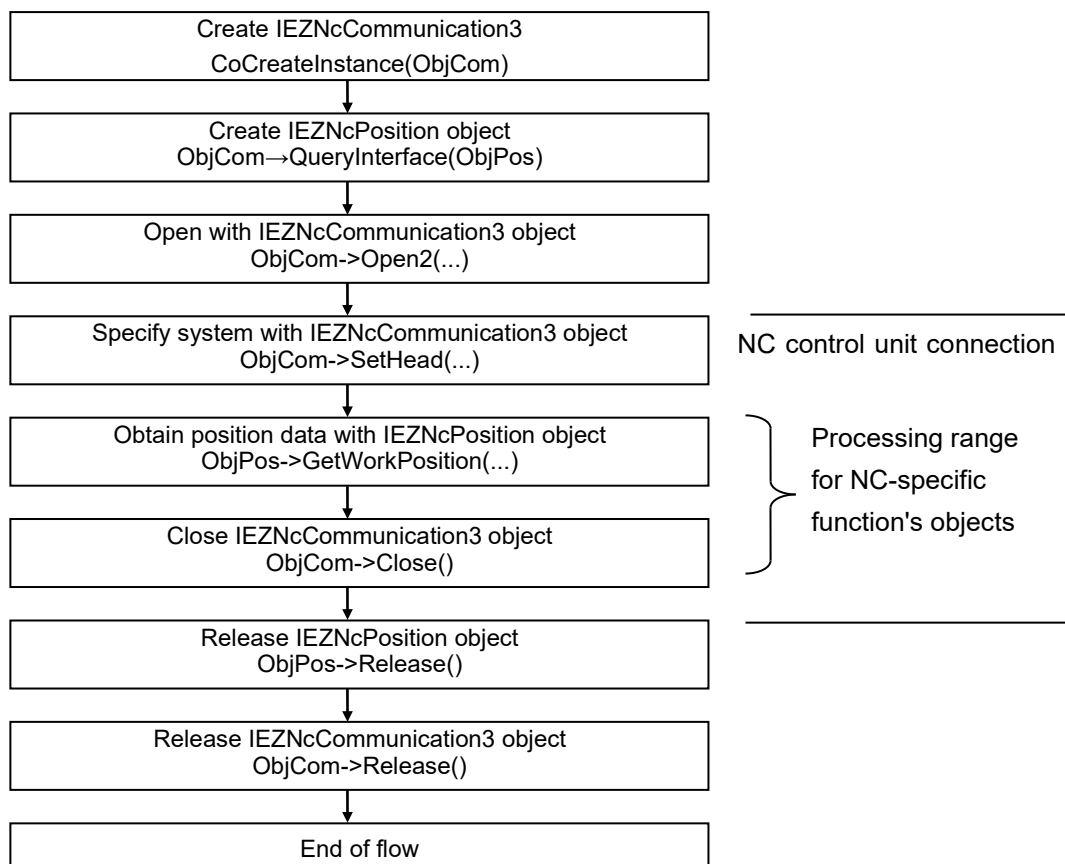
No.	Error code	Number	Description
446.	-	0x01802040	Motion programmable controller error
447.	-	0x01802041	Motion programmable controller communication error
448.	-	0x01802042	Fixed execution time setting error
449.	-	0x01802043	Error in number of functions
450.	-	0x01802044	System information specification error
451.	-	0x01802045	Registration Condition Not Formed
452.	-	0x01802046	Function No. error
453.	-	0x01802047	RAM operation error
454.	-	0x01802048	ROM drive error at the booting side
455.	-	0x01802049	Transfer mode specification error at the booting side
456.	-	0x0180204A	Insufficient memory error
457.	-	0x0180204B	Back up drive ROM error
458.	-	0x0180204C	Block size error
459.	-	0x0180204D	Detached during RUN state error
460.	-	0x0180204E	Unit already registered error
461.	-	0x0180204F	Password registration data full error
462.	-	0x01802050	Password not registered error
463.	-	0x01802051	Remote password error
464.	-	0x01802052	IP address error
465.	-	0x01802053	Timeout value out of range error
466.	-	0x01802054	Command not detected error
467.	-	0x01802055	Trace execution type error
468.	-	0x01802056	Version error
469.	-	0x01802057	Tracking cable error
470.	-	0x0180205C	Keyword protection error
471.	-	0x0180205D	Keyword disable error
472.	-	0x0180205E	Keyword protecting error
473.	-	0x0180205F	Keyword entry error Ether direct
474.	-	0x01802060	Keyword deletion failure
475.	-	0x01802062	Received packet CRC check error
476.	-	0x01802063	Received packet CRC check error (whole data)
477.	-	0x01802064	FX series programmable controller connection error
478.	-	0x01802070	Online change program error
479.	-	0x01802071	Ether direct communication multiple response receive error
480.	-	0x01802072	Ether direct communication error
481.	-	0x01802073	Programmable controller CPU search response error
482.	-	0x01802074	Redundant system other system connection diagnostics error
483.	-	0x01808001	Multiple open error
484.	-	0x01808002	Channel number specifying error
485.	-	0x01808003	Driver not yet started
486.	-	0x01808004	Error in in overlap event generation
487.	-	0x01808005	MUTEX generation error
488.	-	0x01808006	Error in socket object generation
489.	-	0x01808007	Socket object generation error
490.	-	0x01808008	Port connection error
491.	-	0x01808009	COM port handle error
492.	-	0x0180800A	Buffer size setting error
493.	-	0x0180800B	DCB value acquisition error
494.	-	0x0180800C	DCB setting error
495.	-	0x0180800D	Time-out value setting error
496.	-	0x0180800E	Shared memory open error

No.	Error code	Number	Description
497.	-	0x01808101	Duplex close error
498.	-	0x01808102	Handle close error
499.	-	0x01808103	Driver close error
500.	-	0x01808104	Overlap event close error
501.	-	0x01808105	Mutex handle close error
502.	-	0x01808106	COM port handle close error
503.	-	0x01808201	Send error
504.	-	0x01808202	Send data size error
505.	-	0x01808203	Queue clear error
506.	-	0x01808301	Receive error
507.	-	0x01808302	Not sent error
508.	-	0x01808303	Error in retrieving overlap event
509.	-	0x01808304	Receive buffer size shortage
510.	-	0x01808401	Control error
511.	-	0x01808402	Signal line control error
512.	-	0x01808403	Signal line specifying error
513.	-	0x01808404	Open not yet executed
514.	-	0x01808405	Communication parameter error
515.	-	0x01808406	Transmission speed value specifying error
516.	-	0x01808407	Data length setting error
517.	-	0x01808408	Parity specifying error
518.	-	0x01808409	Stop bit specifying error
519.	-	0x0180840A	Communication control setting error
520.	-	0x0180840B	Time-out error
521.	-	0x0180840C	Connect error
522.	-	0x0180840D	Duplex connect error
523.	-	0x0180840E	Attach failure
524.	-	0x0180840F	Signal line status acquisition failure
525.	-	0x01808410	CD signal line OFF
526.	-	0x01808411	Password mismatch error
527.	-	0x01808412	TEL communication error
528.	-	0x01808501	USB driver load error
529.	-	0x01808502	USB driver connect error
530.	-	0x01808503	USB driver send error
531.	-	0x01808504	USB driver receive error
532.	-	0x01808505	USB driver timeout error
533.	-	0x01808506	USB driver initialization error
534.	-	0x01808507	Other USB error

4. OPERATING PROCEDURE

This section provides instructions and procedures for using the product.

Use the product by performing the following steps:



4.1 Initialization for Enabling OLE/COM Interface

The product uses the OLE/COM interface. Thus, the VC++ project must support OLE/COM. For the project that was created without OLE/COM being enabled, it is possible to enable OLE/COM by modifying appropriate parts of the two files created by VC++, as shown below.

Note that a project name is Project in the explanation below.

Project.cpp

```
BOOL CProjectApp::InitInstance()
```

```
{  
    // Initialize OLE library.  
    if (!AfxOleInit())  
    {  
        // Show error message.  
        return FALSE;  
    }  
  
    // The rest is omitted.  
}
```

Stdafx.h

```
// stdafx.h: Describes standard system include files  
//           or project specific include files that are used frequently  
//           but changed infrequently.  
//  
#define VC_EXTRALEAN           // Excludes rarely-used stuff from Windows headers.  
//  
#include <afxwin.h>             // MFC core and standard components  
#include <afxext.h>             // MFC extensions  
#include <afxdisp.h>           // MFC OLE/COM  
#ifndef _AFX_NO_AFXCMN_SUPPORT  
#include <afxcmn.h>             // MFC support for Windows Common Controls  
#endif // _AFX_NO_AFXCMN_SUPPORT
```

4.2 Object Creation

The product uses the OLE/COM interface; therefore, objects must be created/released in the thread where OLE/COM initialization was performed. This is not a matter of concern if your program is single-threaded.

4.3 VB Programming of Automation Interface

This section explains programming with Microsoft Visual Basic (hereinafter referred to as "VB"). The VC++ programs and VB programs can be written in a similar flow; therefore, VB can be used to create a prototype of your application and verify it at an early stage.

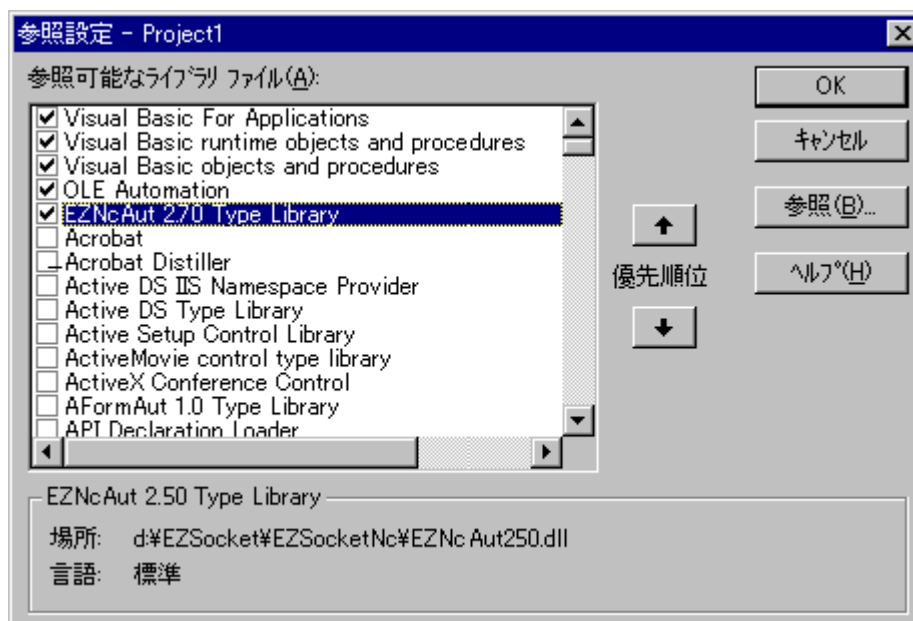
VB's functions that assist programming also help make programming efficient.

4.3.1 Using OLE automation interface with VB

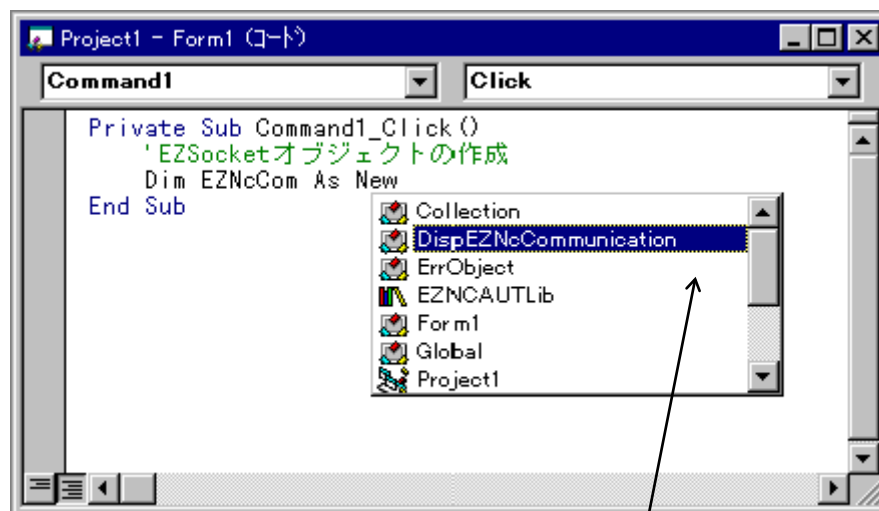
- (1) Setting references: Check (select) object libraries.

This section describes a way to enable early binding by setting references. Setting references will enable VB's object browsing function.

(Install the product prior to this procedure.)

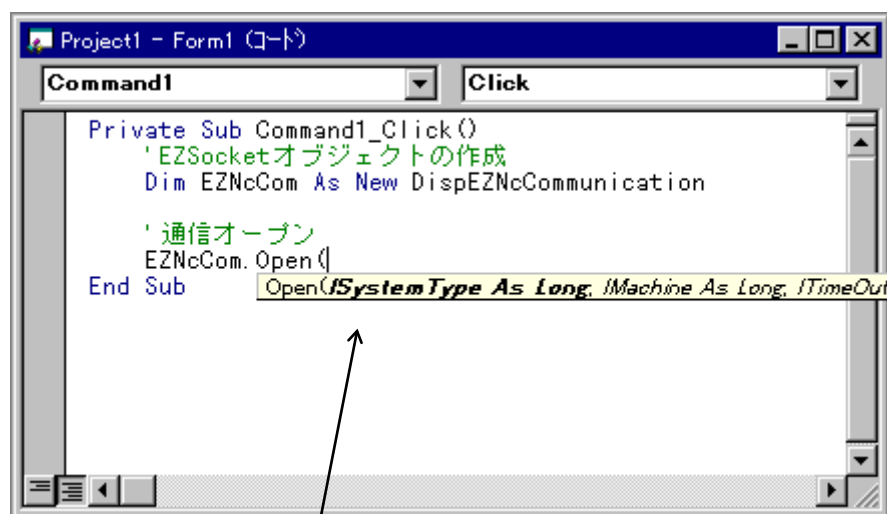


- (2) Object browser: In your programming window, select the EZNCCommunication object.



Object browser

(3) Method browser: In your programming window, select a method for the EZNcCom object and check its argument.

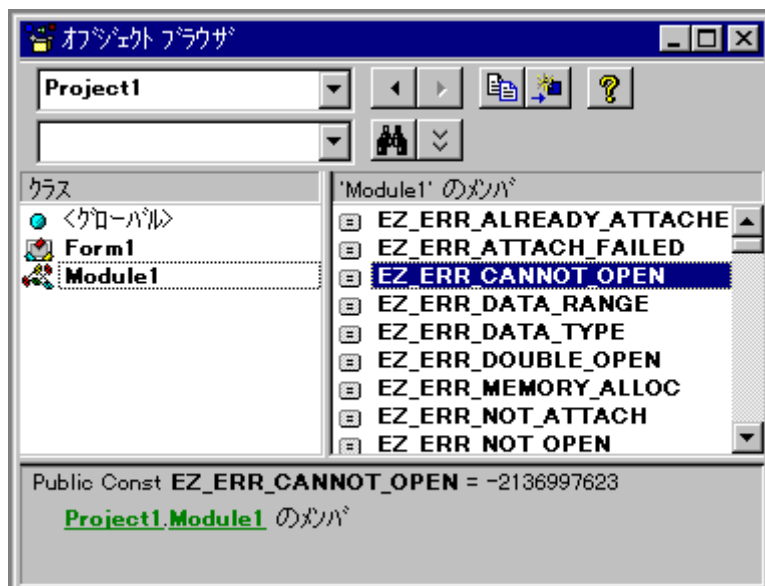


Method browser

(4) Module file: Sets definitions and error code references.

Add a module file so that the definitions of the product and error codes can be used in the VB. Select "Project", "Add a standard module", and then "Existing file" to add EZNcDef.bas, EZNcErr.bas, and EZComErr.bas module files to the project.

Definitions and error codes can now be referred to easily through VB's object browser function.



4.3.2 VB program flow (early binding)

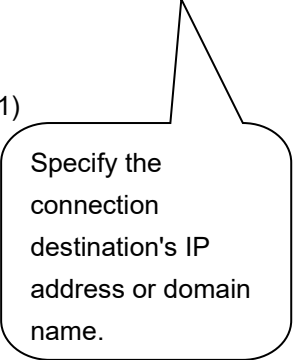
This section shows the flow of the program that uses early binding. Reference setting in the product is required.

```
Private Sub Command1_Click()  
    'Create object.  
    Dim EZNcCom As New DispEZNcCommunication  
  
    'Open communication.  
    Dim IRet As Long  
    IRet = EZNcCom.Open2(EZNC_SYS_MAGICBOARD64, 1, 1)  
    If IRet <> 0 Then GoTo Error_Proc  
  
    'Processing  
    IRet = EZNcCom.SetHead(1)  
    If IRet <> 0 Then GoTo Error_Proc  
  
    Dim CurPos(1 To 3) As Double  
    For Axis = 1 To 3  
        IRet = EZNcCom.Position_GetCurrentPosition(Axis, CurPos(Axis))  
        If IRet <> 0 Then GoTo Error_Proc  
    Next Axis  
  
    X.Text = CurPos(1)  
    Y.Text = CurPos(2)  
    Z.Text = CurPos(3)  
  
    'Close.  
    IRet = EZNcCom.Close  
    If IRet <> 0 Then GoTo Error_Proc  
  
    GoTo Last_Proc  
Error_Proc:  
    MsgBox ("Error! Code = " + "&H" + CStr(Hex(IRet)))  
Last_Proc:  
    'Release object.  
    Set EZNcCom = Nothing  
End Sub
```


4.3.3 VB program flow (late binding)

This section shows the flow of the program that uses late binding. Before you call each method, the declaration of parameter arrays to be specified for the arguments is required. Reference setting in the product is not required. This sample program acquires offset amounts of a tool and a workpiece.

```
Private Sub Command1_Click()  
    'Create object.  
    Dim EZNcCom As Object  
    Set EZNcCom = CreateObject("EZNCut.DispEZNCCommunication","192.168.1.1")  
  
    'Open communication.  
    Dim IRet As Long  
    IRet = EZNcCom.Open2(EZNC_SYS_MAGICBOARD64, 1, 1)  
    If IRet <> 0 Then GoTo Error_Proc  
  
    'Processing  
    IRet = EZNcCom.SetHead(1)  
    If IRet <> 0 Then GoTo Error_Proc  
  
    Dim CurPos(1 To 3) As Double  
    For Axis = 1 To 3  
        IRet = EZNcCom.Position_GetCurrentPosition(Axis, CurPos(Axis))  
        If IRet <> 0 Then GoTo Error_Proc  
    Next Axis  
  
    X.Text = CurPos(1)  
    Y.Text = CurPos(2)  
    Z.Text = CurPos(3)  
  
    'Close.  
    IRet = EZNcCom.Close  
    If IRet <> 0 Then GoTo Error_Proc  
  
    GoTo Last_Proc  
Error_Proc:  
    MsgBox ("Error! Code = " + "&H" + CStr(Hex(IRet)))  
Last_Proc:  
    'Release object.  
    Set EZNcCom = Nothing  
End Sub
```



(Note) For the first argument setting of CreateObject(), refer to *2 in "1.10.1 VC++ program flow (1)".

4.4 VC# Programming of EZSocket Automation Interface

This section explains programming with VC#. The VC++ programs and VC# programs can be written in a similar flow; therefore, VC# can be used to create a prototype of your application and verify it at an early stage.

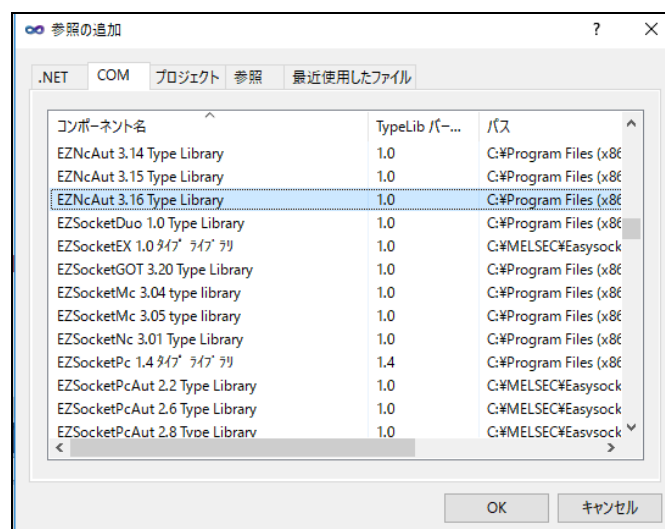
VC#'s functions that assist programming also help make programming efficient.

4.4.1 Using EZSocketNc OLE automation interface with VC#

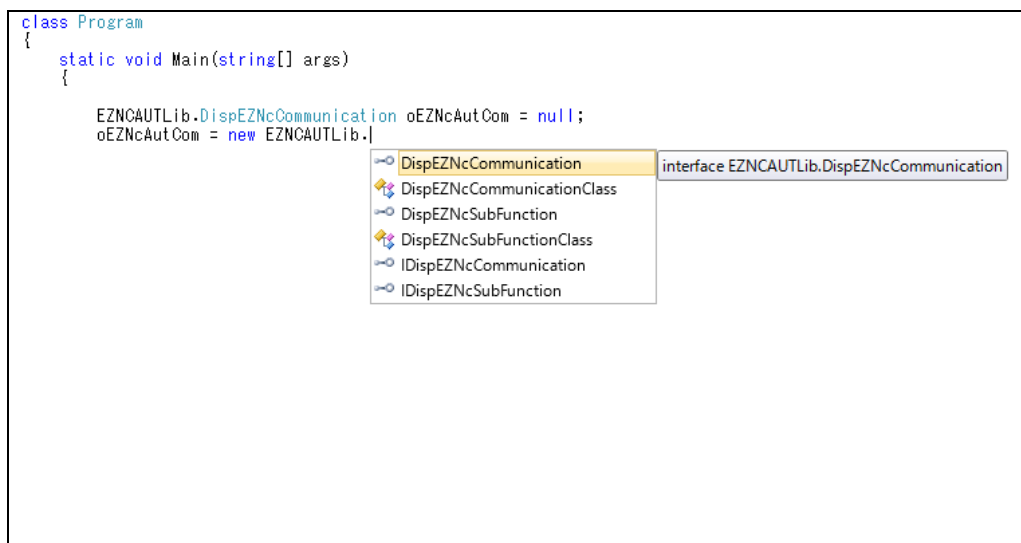
(1) Setting references: Check (select) object libraries.

This section describes a way to enable early binding by setting references. Setting references will enable VC#'s object browsing function.

(Install the product prior to this procedure.)



(2) Object browser: In your programming window, select the **EZNACommunication** object.



(3) Method browser: In your programming window, select a method for the oEZNCutCom object and check its argument.

```
class Program
{
    static void Main(string[] args)
    {
        EZNCUTLib.DispEZNCCommunication oEZNCutCom = null;
        oEZNCutCom = new EZNCUTLib.DispEZNCCommunication();

        int result = 0;

        result = oEZNCutCom.SetTCP/IPProtocol("192.168.1.1", 683);

        :
    }
}
```

4.4.2 VC# program flow (early binding)

This section shows the flow of the program that uses early binding. Reference setting in the product is required. This sample program acquires the workpiece coordinate system offset amount.

```
//Create object
EZNCut.DispEZNCCommunication oEZNCutCom = null;
oEZNCutCom = new EZNCut.DispEZNCCommunication();

// [Local variable declaration]
object BlockValue = "";
int Ret = 0;
double pdoffset = 0;

//Open communication
Ret = oEZNCutCom.SetTCP/IPProtocol("192.168.1.1", 683);
Ret = oEZNCutCom.Open3(9, 1, 10, "EZNC_LOCALHOST");

//Acquire the workpiece coordinate system offset amount
Ret = oEZNCutCom.Tool_GetToolWorkOffset2(1, 54, out pdoffset);

//Close.
Ret = oEZNCutCom.Close();

//Release object
oEZNCutCom = null;
```

* The program above follows the specification of VC# Ver. 7.0 or later.

4.4.3 VC# program flow (late binding)

This section shows the flow of the program that uses late binding. Reference setting in the product is not required. Before you call each method, the declaration of parameter arrays to be specified for the arguments is required. This sample program acquires the workpiece coordinate system offset amount.

```
//EZSocket object declaration
Type objClassType = Type.GetTypeFromProgID("EZNCAut.DispEZNCCommunication");
//EZSocket object creation
object objEZNC_Late = Activator.CreateInstance(objClassType);

//Local variable declaration
object Ret = 0;
double pdoffset = 0;

//Declaration of parameter arrays to be specified for the arguments
object[] tcpip_arg = {"192.168.1.1", 683};
object[] open_arg = {9, 1, 10, "EZNC_LOCALHOST"};
object[] getwork_arg = { 1, 54, out pdoffset};
object[] close_arg = { };

//Open communication
Ret = objEZNC_Late.GetType().InvokeMember("SetTCPIPProtocol",
                                           BindingFlags.InvokeMethod, null, objEZNC_Late, tcpip_arg);
Ret = objEZNC_Late.GetType().InvokeMember("Open3", BindingFlags.InvokeMethod, null, objEZNC_Late, open_arg);

//Acquire the workpiece coordinate system offset amount
Ret = objEZNC_Late.GetType().InvokeMember("Tool_GetToolWorkOffSet",
                                           BindingFlags.InvokeMethod, null, objEZNC_Late, getwork_arg);

//Close.
Ret = objEZNC_Late.GetType().InvokeMember("Close", BindingFlags.InvokeMethod, null, objEZNC_Late, close_arg);

//Release object
objEZNC_Late = null;
```

* The program above follows the specification of VC# Ver. 7.0 or later.

When the earlier version than VC#7.0 is used for the development environment, the ParameterModifier argument of InvokeMember() is used to output the data for reference.

5. SAMPLE APPLICATION

5.1 Overview

The sample application that uses this product is provided with compilable project files for Visual C++ Version 6.0 and Visual Basic Version 6.0.

The sample application includes the following:

- Position data display application: \samples\Vc\Position\Position.dsw
- Monitoring application: \samples\Vb\EZNcAutSample\EZNcAutSample.vbp
- Console program sample: (Refer to "5.4. CONSOLE PROGRAM SAMPLE")

5.2 Position Data Display Application

This section explains the sample application for Visual C++ Version 6.0 using this product.

5.2.1 Operating requirements

The sample application operates in the following system configuration:

Compiler	Microsoft Visual C++ Version 6.0
Controller	M700 series, C70, M800 series

5.2.2 Installation and uninstallation

This section explains installation and uninstallation of the sample application.

For installation of operating systems and VC++ other than the product as well as operations of hardware, refer to the respective instruction manuals.

(1) Installation

The sample application is created in the samples folder when this product is installed.

The sample application has the subfolders with respective project names and each contains its source code and execution file. The sample application includes the Visual C++ 6.0 project workspace files. Opening the corresponding project workspace file enables Visual C++ to open the project.

(2) Uninstallation

To uninstall the sample application, delete the subfolder with the project name or delete the samples folder.

5.2.3 Executing the sample application

This section explains execution of the sample application.

The execution file is stored under the Debug folder or the Release folder in the sample application folder. To open the position data display application, execute **Position.exe**.

For instructions for using the position data display application, refer to the following sections.

Note that this sample application is a monitor application for the Mitsubishi Electric CNC. Operations such as operation search and cycle start are required for the computerized numerical controller. For details on the operation methods, refer to the instruction manuals.

5.2.4 Function list

This section explains the functions of the sample application.

The position data display application monitors specified position data and displays values obtained as counters.

Table 5-1 Position data display application function list

[File]	[Exit application]	Ends the position data display application.
[Edit]	[Position data]	Edits the position data type to be displayed. • Current position • Workpiece coordinate position • Machine position • Command remaining distance
[Display]	[Refresh cycle]	Edits the refresh cycle for the display.
[Communication]	[Communication selection]	Selects a communication target. • CNC C70 • CNC M700M (*1) • CNC M700L (*2) • CNC M800M(*3) • CNC M800L(*4)
	[Execution]	Starts/Stops communication.
[Help]	[Version information]	Displays version information of the position data display application.

(*1) A communication target is machining center system M700 series.

(*2) A communication target is lathe system M700 series.

(*3) A communication target is machining center system M800 series.

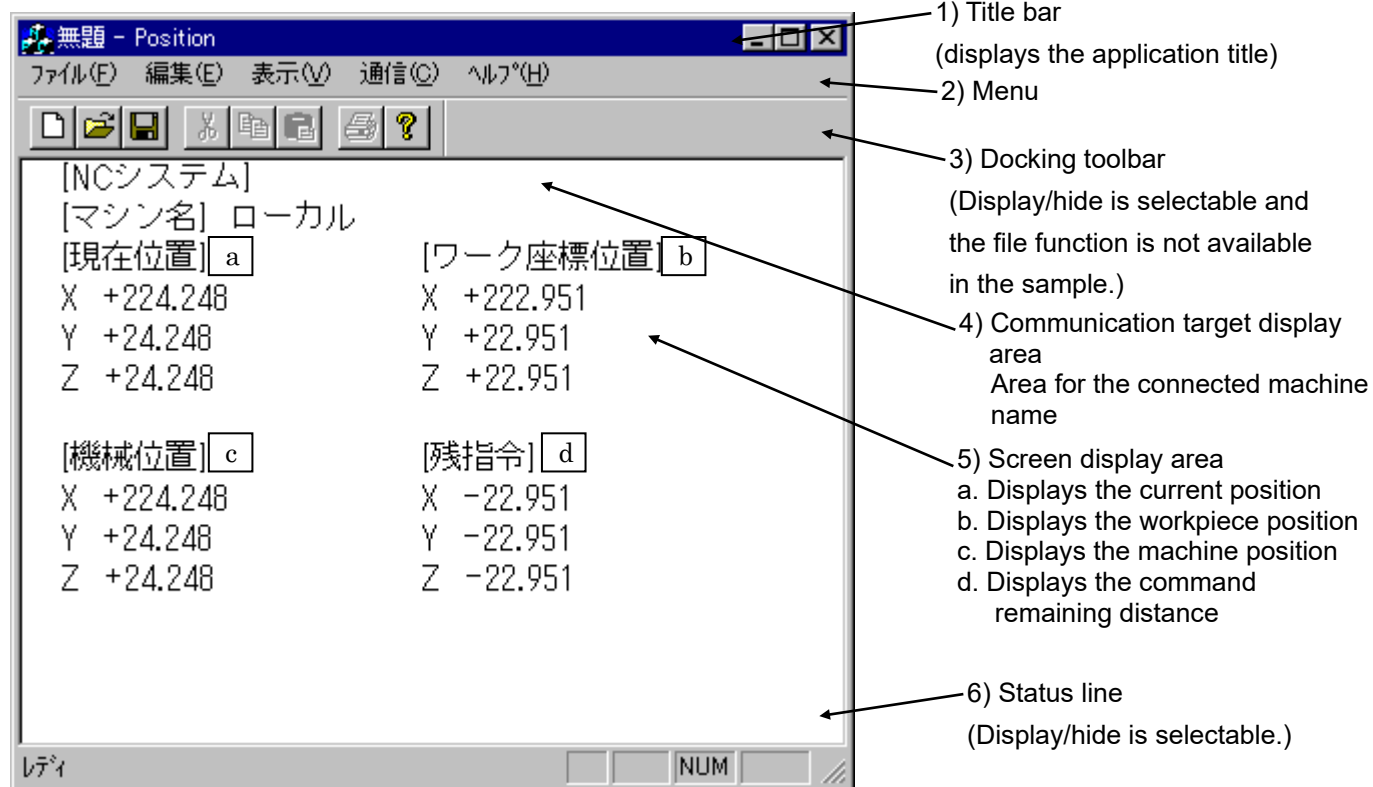
(*4) A communication target is lathe system M800 series.

5.2.5 Screen structure and functions

This section describes the screen structure for the position data display application and the functions for each menu item.

(1) Basic screen structure

The basic screen is shown below:



(2) File function

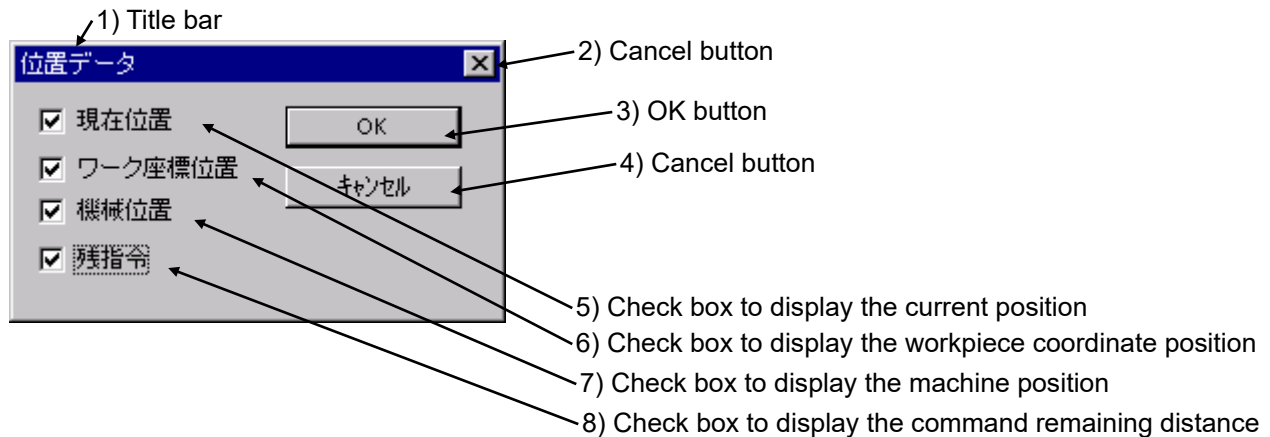
In the position data display application, the available file function is the exit of the application only.

There is no file selection function.

(3) Edit function

a. Position data function dialog

Selects the position data types to be displayed.



1) to 4): Explanation omitted. The following explanation is also omitted.

5) Check box to display the current position:

Selects whether to display or hide the relative position to the position at a completion of the dog type zero point return or to the preset position configured by G92/origin set/counter set.

6) Check box to display the workpiece coordinate position:

Selects whether to display or hide the coordinate position in the current workpiece coordinate system.

7) Check box to display the machine position:

Selects whether to display or hide the coordinate position for each axis in the basic machine coordinate system.

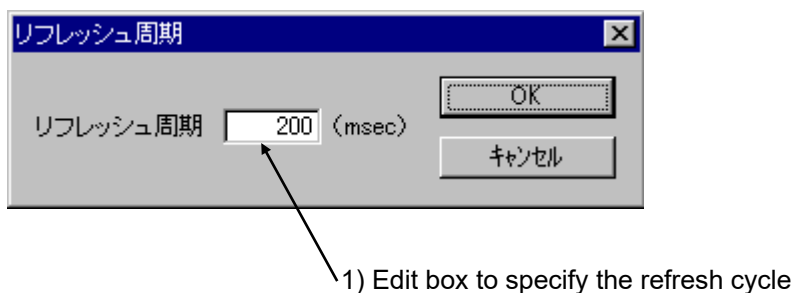
8) Check box to display the command remaining distance:

Selects whether to display or hide the remaining distance for the travel command being executed.

(4) Display function

Refresh cycle function dialog

Sets the refresh cycle for the screen display.



1) Edit box to specify the refresh cycle:

Specifies the refresh cycle for the position data displayed on the screen.

The range is from 200 to 10000 (ms).

(5) Communication function

Communication selection function dialog

Selects a communication target.



1) Text box for the remotely connected machine

2) Combo box for communication target selection (*1)

1) Text box for the remotely connected machine:

Sets the machine name of a personal computer equipped with the NC.

Allows the domain name and IP address to be specified.

2) Combo box for the communication target selection:

Sets an NC control unit communication target.

Selection range is as follows: MELDASMAGIC64, MELDAS6x5M, MELDAS6x5L, MELDASC6/C64, CNC C70, CNC M700M, CNC M700L, CNC M800M, CNC M800L.

(6) Execution function

Connects/disconnects the selected communication target.

If connection fails, an error message is output and the message box appears.

(7) Version display

"Help - Version display" displays the dialog box for version information of the position data display application.

5.2.6 Setting project workspaces

This section explains how to set the project workspace used for creating the position data display application. The application project configuration is as follows.

Table 5-2 Project configuration

Setting item	Setting value
Application type	SDI (Single Document Interface).
Database support	Not supported.
Automation support	Supported.
OLE compound document support	Not supported.
Functions to be embedded into the application	Docking toolbar. Initial status bar. 3D control.
MAPI support	Not supported.
Windows Sockets support	Not supported.
Number of the latest files to be displayed.	4 files. (The default value is applied to the advanced settings.)

5.2.7 IEZNCCommunication object

The IEZNCCommunication3 object is used for connection to the communication circuit.

This sample application uses the following methods:

Open3() Open line method
Close() Line disconnect method
SetHead()..... System specification method

5.2.8 IEZNcPosition object

The IEZNcPosition object executes position information acquisition for the opened NC control unit. This sample application uses the following methods:

GetWorkPosition()..... Workpiece coordinate position acquisition method
GetMachinePosition()... Machine position acquisition method
GetCurrentPosition()... Current position acquisition method
GetDistance()..... Command remaining distance acquisition method

5.3 Monitoring Application

This section explains the sample application for Visual Basic Version 6.0 using this product.

5.3.1 Operating requirements

The sample application operates in the following system configuration:

Compiler	Microsoft Visual Basic Version 6.0
Controller	M700 series, C70, M800 series

5.3.2 Installation and uninstallation

This section explains installation and uninstallation of the sample application.

For installation of operating systems and VB other than the product as well as operations of hardware, refer to the respective instruction manual.

(1) Installation

The sample application is created in the samples folder when this product is installed.

The sample application has the subfolders with respective project names and each contains its source code and execution file. The sample application includes the Visual Basic 6.0 project workspace files. Opening the corresponding project workspace file enables Visual Basic to open the project.

(2) Uninstallation

To uninstall the sample application, delete the subfolder with the project name or delete the samples folder.

5.3.3 Executing the sample application

This section explains execution of the sample application.

The execution file is stored under the sample application folder. To open the monitoring application, execute **EZNcAutSample.exe**.

For instruction for using the monitoring application, refer to the following sections.

Note that this sample application is a monitor application for the computerized numerical controller. Operations such as operation search and cycle start are required for the computerized numerical controller. For details on the operation methods, refer to the instruction manuals.

5.3.4 Function list

This section explains the functions of the sample application.

The monitoring application monitors the currently-running NC program and the current position, and displays the obtained values as counters.

Table 5-3 Monitoring application function list

Function item	Overview
NC control unit communication start/stop	Sets NC control unit communication parameters and starts/stops communication. • Sets the NC control unit number. • Sets communication time-out value. • Executes communication. • Displays the NC system version.
Position display	Reads the position data. • Displays the current coordinate position. • Displays the workpiece coordinate position. • Displays the machine coordinate position.
Alarm display	Displays the current alarm.
Program display	Displays the currently-running program and the ongoing line position. • Displays the currently-running program list • Displays the running block position and sequence number.
Error display	Displays the API error codes for this product.

5.3.5 Screen structure and functions

This section describes the screen structure for the position data display application and the functions for each menu item.

(1) Basic screen structure

The basic screen is shown below:

The screenshot shows the 'EZSocketNc Sample Application' window. It contains several sections:

- NCカード通信開始/終了**: Includes fields for '接続ノード名' (mpcat18), 'NCタイプ' (dropdown), '通信開始' (selected radio button), 'NCカードNo.' (1), '通信終了' (radio button), 'TimeOut値' (1), and 'システム番号/名称'.
- 位置表示**: Divided into '現在座標位置' (X: 287.909, Y: 87.909, Z: 87.909) and 'ワーク座標位置' (X: 286.612, Y: 86.612, Z: 86.612). It also has a '機械位置' section with X, Y, Z values.
- プログラム実行プログラム表示**: A list box showing 'S2150', 'G01X-100.Y-100.Z-100.F2000', 'M98P10', 'M99', and '%'. Below it are 'プログラムNo.' (11) and 'シーケンス番号' (0).
- アラーム表示**: A text area showing 'エラーなし'.
- EZSocket Error Code**: An empty text area.

 Numbered callouts point to:

- 1) Edit box for the connected machine name (接続ノード名)
- 2) Button for connection/disconnection (切断)
- 3) Edit box for the NC control unit number (NCカードNo.)
- 4) Edit box for the communication time-out value (TimeOut値)
- 5) Radio button to start/stop communication (通信開始)
- 6) Execution button to start/stop communication (実行)

1) Edit box for the connected machine name

Sets the machine name of a personal computer equipped with the NC.

Allows the domain name and IP address to be specified.

2) Button for connection/disconnection

Connects/disconnects a specified machine.

3) Edit box for the NC control unit number

Sets the NC control unit number. The NC control unit number is determined when setting up the NC control unit.

4) Edit box for the communication time-out value

Sets the communication time-out value for the NC control unit.

5) Radio button to start/stop communication

Sets to start or stop communication with the NC control unit.

6) Execution button to start/stop communication

Executes a communication start or stop after setting 1) to 3).

5.3.6 Setting project workspaces

This section explains how to set the project workspace used for creating the position data display application. The application project configuration is as follows.

This project uses late binding to call methods of this product.

Table 5-4 Project configuration

Setting item	Setting value
Application type	Standard EXE
Addition of the standard module	Select EZNcDef.bas or EZNcErr.bas from [Add standard module] in [Project].

5.4 CONSOLE PROGRAM SAMPLE

5.4.1 Console Program to Connect C70 (via Ethernet)

```
//
// Simple sample program for the console application
//
// Copyright(C) 2008 MITSUBISHI ELECTRIC CORPORATION
#include "stdafx.h"

#include "stdio.h"
#include <locale.h>

// EZSocket header file
#include "EZSocketNc.h"
#include "EZSocketNcStr.h"
#include "EZSocketNcDef.h"

int main(int argc, char* argv[])
{
    HRESULT                hr = S_OK;
    LONG                   lRet = 0;
    EZNCST_OPEN            stOpen;
    LPOLESTR                lpwszBuffer = NULL;
    HANDLE                  hSampleFile = NULL;
    BYTE*                   pbData = NULL;
    const DWORD             dwLength = 256;
    DWORD                   dwNumRead = 0;
    DWORD                   dwWrittenSize = 0;

    memset(&stOpen, 0x00, sizeof(stOpen));

    // COM initialization
    hr = CoInitialize(NULL);
    if( S_OK != hr ){
        wprintf(L"Failed in CoInitialize!\n");
        return 0;
    }

    setlocale( LC_ALL, "Japanese" );
    IEZNcCommunication3    *pIEZNcCom = NULL;    // Communication object
    IEZNcFile6              *pIEZNcFile = NULL;   // File object

    //
    // IEZNcCommunication object creation
    CLSID clsid;
    CLSIDFromProgID( L"EZSocketNc.EZNcCommunication", &clsid );
    hr = CoCreateInstance(clsid,
                          NULL,
                          CLSCTX_INPROC_SERVER,
                          IID_IEZNcCommunication3,
                          (void **)&pIEZNcCom);

    if( S_OK != hr )
    {
        wprintf(L"EZSocket is not installed!\n");
        goto END;
    }

    //
    // IEZNcFile object creation
    //
    if(pIEZNcCom->QueryInterface(IID_IEZNcFile6, (void**)&pIEZNcFile) != S_OK){
        wprintf(L"EZSocket is not installed!\n");
        goto END;
    }

    //
    // Open parameter setting
    //
    stOpen.lNetworkNumber    = 0x01;
    stOpen.lStationNumber    = 0x01;
    stOpen.lUnitNumber       = 0x00;
    stOpen.lConnectUnitNumber = 0x00;
    stOpen.lIONumber         = 0x3E1;
```

```

stOpen.ICpuType           = 0x0901;
stOpen.IUnitType          = 0x1a;
stOpen.IPacketType        = 0x01;
stOpen.IProtocolType      = 0x08;
stOpen.IPortNumber        = 0x00;
stOpen.IBaudRate          = 0x00;
stOpen.IDataBits          = 0x00;
stOpen.IParity             = 0x00;
stOpen.IStopBits          = 0x00;
stOpen.IControl           = 0x00;
stOpen.IpcwszHostAddress  = L"192.168.1.1";
stOpen.ICpuTimeOut        = 0x00;
stOpen.ITimeOut           = 1000;
stOpen.ISumCheck          = FALSE;
stOpen.ISourceNetworkNumber = 0x01;
stOpen.ISourceStationNumber = 0x04;
stOpen.IDestinationPortNumber = 5001;
stOpen.IDestinationIONumber = 0x00;
stOpen.IConnectChannelNumber = 0x00;
stOpen.IMultiDropChannelNumber = 0x00;
stOpen.IThroughNetworkType = 0x00;
stOpen.IIntelligentPreferenceBit = 0x00;
stOpen.IDidPropertyBit    = 0x01;
stOpen.IDsidPropertyBit   = 0x01;

hr = pIEZNcCom->SetMelsecProtocol(&stOpen, &IRet);
if( S_OK != hr ){
    wprintf(L"Can't SetMelsecProtocol! Error Code = 0x%x\n",IRet);
    goto END;
}

// IEZNcCommunication3 open
hr = pIEZNcCom->Open3(EZNC_SYS_MELDASC70, 1, 20, &IRet);
if( S_OK != hr ){
    wprintf(L"Can't Open3! Error Code = 0x%x\n",IRet);
    goto END;
}

//
// File search
//
hr = pIEZNcFile->FindDir2(L"M01:\\PRG\\USER\\", 0x00, &lpwszBuffer, &IRet);
if( S_OK != hr ){
    wprintf(L"Can't FindDir! Error Code = 0x%x\n",IRet);
    goto END;
}

while(IRet >= 1){
    if(wcsncmp(lpwszBuffer, L"10.PRG") == 0){
        break;
    }
    CoTaskMemFree(lpwszBuffer);
    lpwszBuffer = NULL;
    hr = pIEZNcFile->FindNextDir2(&lpwszBuffer, &IRet);
    if( S_OK != hr ){
        wprintf(L"Can't FindNextDir! Error Code = 0x%x\n",IRet);
        goto END;
    }
}
if(IRet == 0){
    wprintf(L"File is not found.\n");
    pIEZNcFile->ResetDir(&IRet);
    goto END;
}
hr = pIEZNcFile->ResetDir(&IRet);
if( S_OK != hr ){
    wprintf(L"Can't ResetDir! Error Code = 0x%x\n",IRet);
    goto END;
}

//
// File read
//
hr = pIEZNcFile->OpenFile3(L"M01:\\PRG\\USER\\10.PRG", EZNC_FILE_READ, &IRet);
if( S_OK != hr ){

```



```

        wprintf(L"Can't OpenFile3! Error Code = 0x%x\n",IRet);
        goto END;
    }

    CreateDirectory("C: \\temp", NULL);

    hSampleFile = ::CreateFile("C: \\temp\\SAMPLE.PRG",
                                GENERIC_WRITE,
                                FILE_SHARE_READ,
                                NULL,
                                OPEN_ALWAYS,
                                FILE_ATTRIBUTE_NORMAL,
                                NULL);
    if(hSampleFile == INVALID_HANDLE_VALUE ){
        wprintf(L"Can't create file.\n");
        goto END;
    }
    do{
        hr = pIEZNcFile->ReadFile2(dwLength, &pbData, &dwNumRead, &IRet);
        if( S_OK != hr ){
            wprintf(L"Can't ReadFile2! Error Code = 0x%x\n",IRet);
            pIEZNcFile->AbortFile2(&IRet);
            ::CloseHandle(hSampleFile);
            goto END;
        }
        if(dwNumRead != 0){
            ::WriteFile(hSampleFile,
                        (LPCVOID)pbData,
                        dwNumRead,
                        &dwWrittenSize,
                        NULL);
            CoTaskMemFree(pbData);
            pbData = NULL;
        }
    }while(dwLength == dwNumRead);
    ::CloseHandle(hSampleFile);

    hr = pIEZNcFile->CloseFile2(&IRet);
    if( S_OK != hr ){
        wprintf(L"Can't CloseFile2! Error Code = 0x%x\n",IRet);
        goto END;
    }
END:
    if(lpwszBuffer != NULL){
        CoTaskMemFree(lpwszBuffer);
        lpwszBuffer = NULL;
    }
    if(pbData != NULL){
        CoTaskMemFree(pbData);
        pbData = NULL;
    }

    // IEZNcCommunication3 close
    if(pIEZNcCom != NULL){
        pIEZNcCom->Close(&IRet);
    }

    // Object release
    if(pIEZNcFile != NULL){
        pIEZNcFile->Release();
        pIEZNcFile = NULL;
    }
    if(pIEZNcCom != NULL){
        pIEZNcCom->Release();
        pIEZNcCom = NULL;
    }

    // COM library release
    CoUninitialize();
    return 0;
}

```

Revision History

Date of revision	Manual No.	Revision details										
Dec.2013	IB(NA)1501209-A	First edition created.										
Jan. 2014	IB(NA)1501209-B	1. Changed the compatible NC models Changed the compatible NC models to be M700 and C70 Corrected the description so that only the compatible models are given in each item. 2. Added the descriptions and corrections on C70 Added the compatible methods Added the description and caution on I/F in accordance with the added methods Added a C70 sample program 3. Added the description on the compatibility with 64bit OS Added the description and caution on x64 platform 4. Corrected the method versions<table><tr><th>Before correction</th><th>After correction</th></tr><tr><td>IEZnCFile5 FindDir FindNextDir</td><td>IEZnCFile6 FindDir2 FindNextDir2</td></tr><tr><td>IEZnCAtc2 GetMNGReady</td><td>IEZnCAtc3 GetMNGReady2</td></tr><tr><td>IEZnCParameter2 GetParameter SetParameter</td><td>IEZnCParameter3 GetParameter2 SetParameter2</td></tr><tr><td>IEZnCSubFunciton ChangeInit</td><td>IEZnCSubFunciton2 ChangeInit2</td></tr></table> Applied these version changes also to the other parts of this manual. 5. Error codes Added the related error codes to the I/F description Revised the list of error codes 6. Others Corrected errors, modified the layout, etc.	Before correction	After correction	IEZnCFile5 FindDir FindNextDir	IEZnCFile6 FindDir2 FindNextDir2	IEZnCAtc2 GetMNGReady	IEZnCAtc3 GetMNGReady2	IEZnCParameter2 GetParameter SetParameter	IEZnCParameter3 GetParameter2 SetParameter2	IEZnCSubFunciton ChangeInit	IEZnCSubFunciton2 ChangeInit2
Before correction	After correction											
IEZnCFile5 FindDir FindNextDir	IEZnCFile6 FindDir2 FindNextDir2											
IEZnCAtc2 GetMNGReady	IEZnCAtc3 GetMNGReady2											
IEZnCParameter2 GetParameter SetParameter	IEZnCParameter3 GetParameter2 SetParameter2											
IEZnCSubFunciton ChangeInit	IEZnCSubFunciton2 ChangeInit2											
Jul. 2014	IB(NA)1501209-C	1. Corrected the method versions<table><tr><th>Before correction</th><th>After correction</th></tr><tr><td>IEZnCParameter3 GetParameter2 SetParameter2</td><td>IEZnCParameter3 GetParameter3 SetParameter3</td></tr></table> 2. Others Corrected errors.	Before correction	After correction	IEZnCParameter3 GetParameter2 SetParameter2	IEZnCParameter3 GetParameter3 SetParameter3						
Before correction	After correction											
IEZnCParameter3 GetParameter2 SetParameter2	IEZnCParameter3 GetParameter3 SetParameter3											
Sept. 2015	IB(NA)1501209-D	1. Added information related to M800 series 2. Added and corrected information for M700 series and C70 series 3. Others Corrected errors, modified the layout, etc.										

Date of revision	Manual No.	Revision details
Jun. 2019	IB(NA)1501209-E	Revised the contents corresponding to A3 Version of FCSB1224W000. - Added the operating environment for Windows10, Visual C#, and Visual Studio 2015. - Added E80 for applicable model. - Added and revised the methods for EZSocketNc Ver. 3.13 to 3.16 - Added the following chapters 1.7 Redistributable Installer for Run-time Library 1.10.3 VC# Program flow 2.3 Precautions for Using Methods 4.5 Overview of VC# Programming of EZSocket Automation Interface - Added error codes in "3. ERROR CODE LIST" - Deleted "5. APPLICATION INSTALLATION PROCEDURE" - Others - Corrected errors
Nov. 2019	IB(NA)1501209-F	Revised the contents corresponding to A5 Version of FCSB1224W000. - Added C80 for applicable model. - Revised "3. ERROR CODE LIST". - Others - Corrected errors, modified the layout, etc.
May. 2021	IB(NA)1501209-G	Revised the contents corresponding to A6 Version of FCSB1224W000. - Deleted description of Windows 7. - Added the operating environment (Visual C# 2015, 2017). - Deleted the operating environment (Visual C++ Ver.5.0, Visual Basic Ver.5.0, Visual Basic for Applications Ver.5.0). - Added and corrected information for M700 series, M800 series, and C80 series. - Revised the following chapters. 1.6 Installation 1.10.2 VB program flow 2.1 Common Items 2.2 Method List 3. ERROR CODE LIST 5. SAMPLE APPLICATION - Added the following methods. - GetOffset3 - SetOffset2 - ReadDevice2 - WriteDevice2 - ReadBlockDevice2 - WriteBlockDevice2 - ReadRandomDevice2 - WriteRandomDevice2 - Others - Corrected errors

Date of revision	Manual No.	Revision details
Feb. 2023	IB(NA)1501209-H	Revised the contents corresponding to A7 Version of FCSB1224W000. - Corrected description of Windows 10 in the supported OS. - Added M800V/M80V series to the applicable models. - Revised the following chapters. 2. I/F DETAILED SPECIFICATIONS 4. OPERATING PROCEDURE - Added the following methods. - GetAlarm3 - GetServoMonitorEx - GetServoMotorTemperature - GetSpindleMotorTemperature - GetAccumulatedTravelDistance - GetCycleTime - Deleted the following method. - GetAuxAxisDiagnosis - Others - Corrected errors.

Global Service Network

AMERICA

mitsubishi electric automation inc. (america fa center)

Central Region Service Center (Chicago)
500 CORPORATE WOODS PARKWAY, VERNON HILLS, ILLINOIS 60061, U.S.A.
TEL: +1-847-478-2500 / FAX: +1-847-478-2650
Minneapolis, MN Service Satellite
Detroit, MI Service Satellite
Grand Rapids, MI Service Satellite
Milwaukee, WI Service Satellite
Cleveland, OH Service Satellite
Indianapolis, IN Service Satellite
St. Louis, MO Service Satellite

South/East Region Service Center (Georgia)

1845 SATELLITE BOULEVARD STE. 450, DULUTH, GEORGIA 30097, U.S.A.
TEL: +1-678-258-4529 / FAX: +1-678-258-4519
Charleston, SC Service Satellite
Charlotte, NC Service Satellite
Raleigh, NC Service Satellite
Dallas, TX Service Satellite
Houston, TX Service Satellite
Hartford, CT Service Satellite
Knoxville, TN Service Satellite
Nashville, TN Service Satellite
Baltimore, MD Service Satellite
Pittsburg, PA Service Satellite
Tampa, FL Service Satellite
Syracuse, NY Service Satellite
Orlando, FL Service Satellite
Lafayette, LA Service Satellite
Philadelphia, PA Service Satellite

Western Region Service Center (California)

5900-B KATELLA AVE. - 5900-A KATELLA AVE. CYPRESS, CALIFORNIA 90630, U.S.A.
TEL: +1-714-699-2625 / FAX: +1-847-478-2650
San Jose, CA Service Satellite
Seattle, WA Service Satellite
Denver, CO Service Satellite

Canada Region Service Center (Toronto)

4299 14TH AVENUE MARKHAM, ONTARIO L3R 0J2, CANADA
TEL: +1-905-475-7728 / FAX: +1-905-475-7935
Edmonton, AB Service Satellite
Montreal, QC Service Satellite

Mexico Region Service Center (Queretaro)

Parque Tecnológico Innovación Querétaro, Lateral Carretera Estatal 431, Km 2+200, Lote 91 Modulos 1 y 2
Hacienda la Machorra, CP 76246, El Marqués, Querétaro, México
TEL: +52-442-153-6050
Monterrey, NL Service Satellite
Mexico City, DF Service Satellite

BRAZIL

mitsubishi electric do brasil comércio e serviços ltda.

Votorantim Office
AV. GISELE CONSTANTINO, 1578, PARQUE BELA VISTA, VOTORANTIM-SP, BRAZIL CEP:18.110-650
TEL: +55-15-3023-9000
Blumenau, Santa Catarina Office

EUROPE

mitsubishi electric europe b.v.

European Service Headquarters (Dusseldorf, GERMANY)
Mitsubishi-Electric-Platz 1 40882 RATINGEN, GERMANY
TEL: +49-2102-486-5000 / FAX: +49-2102-486-5910

South Germany Service Center (Stuttgart)

SCHELMENWASENSTRASSE 16-20, 70567 STUTTGART, GERMANY
TEL: + 49-711-770598-0 / FAX: +49-711-770598-141

France Service Center (Paris)

2 RUE DE L'UNION, 92565 RUEIL-MALMAISON CEDEX, FRANCE
TEL: +33-1-41-02-83-13 / FAX: +33-1-49-01-07-25

France Service Satellite (Lyon)

240, ALLEE JACQUES MONOD 69800 SAINT PRIEST FRANCE
TEL: +33-1-41-02-83-13 / FAX: +33-1-49-01-07-25

Italy Service Center (Milan)

VIA ENERGY PARK 14, VIMERCATE 20871 (MB) ITALY
TEL: +39-039-6053-342 / FAX: +39-039-6053-206

Italy Service Satellite (Padova)

VIA G. SAVELLI, 24 - 35129 PADOVA, ITALY
TEL: +39-039-6053-342 / FAX: +39-039-6053-206

U.K. Service Center

TRAVELLERS LANE, HATFIELD, HERTFORDSHIRE, AL10 8XB, U.K.
TEL: +44-1707-288-780 / FAX: +44-1707-278-695

Spain Service Center

CTRA. RUBI, 76-80 8174 SAINT CUGAT DEL VALLES, BARCELONA, SPAIN
TEL: +34-935-65-2236 / FAX: +34-935-89-1579

Poland Service Center

UL.KRAKOWSKA 50, 32-083 BALICE, POLAND
TEL: +48-12-347-6500 / FAX: +48-12-630-4701

Hungary Service Center

BUDAÖRS OFFICE PARK, SZABADSÁG ÚT 117., 2040 BUDAÖRS, HUNGARY
TEL: +48-12-347-6500 / FAX: +48-12-630-4701

Turkey Service Center

MITSUBISHI ELECTRIC TURKEY ELEKTRİK ÜRÜNLERİ A.Ş
SERIFALI MAHALLESİ KALE SOKAKI NO.41 34775
UMRANIYE, ISTANBUL, TURKEY
TEL: +90-216-969-2500 / FAX: +90-216-661-44-47

Czech Republic Service Center

AutoCont Control Systems s.r.o (Service Partner)
KAFKOVA 1853/3, 702 00 OSTRAVA 2, CZECH REPUBLIC
TEL: +420-59-5691-185 / FAX: +420-59-5691-199

Russia Service Center

MITSUBISHI ELECTRIC RUSSIA LLC
LETNIKOVSKAYA STREET 2, BLD.1, 5TH 115114 MOSCOW, RUSSIA
TEL: +7-495-721-2070 / FAX: +7-495-721-2071

Sweden Service Center

HAMMARBACKEN 14, P.O.BOX 750 SE-19127, SOLLENTUNA, SWEDEN
TEL: +46-8-6251200 / FAX: +46-8-6251014

Bulgaria Service Center

AKHNATON Ltd. (Service Partner)
4 ANDREJ LJAPCHEV BLVD. POB 21, BG-1756 SOFIA, BULGARIA
TEL: +359-2-8176009 / FAX: +359-2-9744061

Ukraine Service Center (Kiev)

CSC Automation Ltd. (Service Partner)
4 B. YEVHENA SVERSTYUKA STR., 02002 KIEV, UKRAINE
TEL: +380-44-494-3344 / FAX: +380-44-494-3366

Belarus Service Center

TECHNIKON Ltd. (Service Partner)
NEZAVISIMOSTI PR.177, 220125 MINSK, BELARUS
TEL: +375-17-393-1177 / FAX: +375-17-393-0081

South Africa Service Center

Adroit Technologies (Service Partner)
20 WATERFORD OFFICE PARK, WATERFORD DRIVE, CNR OF WITKOPPEN ROAD,
FOURWAYS JOHANNESBURG SOUTH AFRICA
TEL: +27-11-658-8100 / FAX: +27-11-658-8101

ASEAN

MITSUBISHI ELECTRIC ASIA PTE. LTD. (ASEAN FA CENTER)

Singapore Service Center
307 ALEXANDRA ROAD MITSUBISHI ELECTRIC BUILDING SINGAPORE 159943
TEL: +65-6473-2308 / FAX: +65-6476-7439

PHILIPPINES

MELCO FACTORY AUTOMATION PHILIPPINES INC.

Head Office
128 LOPEZ RIZAL STREET, BRGY., HIGHWAY HILLS, MANDALUYONG CITY , MM PHILIPPINES 1550
TEL: +63-2-8256-8042 / FAX: +632-8637-2294

Philippines Service Center
KM.23 WEST SERVICE ROAD SSH, CUPANG ,MUNTINLUPA CITY, PHILIPPINES
TEL: +63-2-8807-0420 / FAX: +63-2-8842-5202

VIETNAM

MITSUBISHI ELECTRIC VIETNAM CO., LTD.

Vietnam Ho Chi Minh Service Center
11TH & 12TH FLOOR, VIETTEL TOWER B, 285 CACH MANG THANG 8 STREET, WARD 12, DISTRICT 10,
HO CHI MINH CITY, VIETNAM
TEL: +84-28-3910-5945 / FAX: +84-28-3910-5947

Vietnam Hanoi Service Center
14TH FLOOR, CAPITAL TOWER, 109 TRAN HUNG DAO STREET, CUA NAM WARD,
HOAN KIEM DISTRICT, HA NOI CITY, VIETNAM
TEL: +84-24-3937-8075 / FAX: +84-24-3937-8076

INDONESIA

PT. MITSUBISHI ELECTRIC INDONESIA

Indonesia Service Center (Cikarang)
JL. KENARI RAYA BLOK G2-07A, DELTA SILICON 5, LIPPO CIKARANG - BEKASI 17550, INDONESIA
TEL: +62-21-2961-7797 / FAX: +62-21-2961-7794

MALAYSIA

MITSUBISHI ELECTRIC SALES MALAYSIA SDN. BHD.

Malaysia Service Center (Kuala Lumpur Service Center)
LOT 11, JALAN 219, P.O BOX 1036, 46860 PETALING JAYA, SELANGOR DARUL EHSAN, MALAYSIA
TEL: +60-3-7626-5032
Johor Bahru Service Satellite
Pulau Pinang Service Satellite

THAILAND

MITSUBISHI ELECTRIC FACTORY AUTOMATION (THAILAND) CO., LTD.

Thailand Service Center (Bangkok)
101, TRUE DIGITAL PARK OFFICE, 5TH FLOOR, SUKHUMVIT ROAD, BANGCHAK, PHRA KHANONG,
BANGKOK, 10260 THAILAND
TEL: +66-2-092-8600 / FAX: +66-2-043-1231-33

INDIA

MITSUBISHI ELECTRIC INDIA PVT., LTD.

CNC Technical Center (Bangalore)
PLOT NO. 56, 4TH MAIN ROAD, PEENYA PHASE 3,
PEENYA INDUSTRIAL AREA, BANGALORE 560058, KARNATAKA, INDIA
TEL : +91-80-4655-2121
Chennai Service Satellite
Coimbatore Service Satellite
Hyderabad Service Satellite

North India Service Center (Gurgaon)
PLOT 517, GROUND FLOOR, UDYOG VIHAR PHASE-III, GURUGRAM 122008, HARYANA, INDIA
TEL : +91-124-463-0300
Ludhiana Service Satellite
Panthnagar Service Satellite
Delhi Service Satellite
Jamshedpur Service Satellite
Manesar Service Satellite

West India Service Center (Pune)
ICC-Devi GAURAV TECHNOLOGY PARK, UNIT NO.402, FOURTH FLOOR, NORTH WING,
SURVEY NUMBER 191-192 (P), NEXT to INDIAN CARD CLOTHING COMPANY Ltd,
OPP. VALLABH NAGAR, PIMPRI, PUNE- 411 018, MAHARASHTRA, INDIA
TEL : +91-20-6819-2274
Kolhapur Service Satellite
Aurangabad Service Satellite
Mumbai Service Satellite

West India Service Center (Ahmedabad)
204-209, 2ND FLOOR, 31FIVE, CORPORATE ROAD PRAHLADNAGAR,
AHMEDABAD -390015, GUJARAT, INDIA
TEL : + 91-79-6777-7888
Rajkot Service Satellite

CHINA

MITSUBISHI ELECTRIC AUTOMATION (CHINA) LTD. (CHINA FA CENTER)

CNC Call Center
TEL: +86-400-921-5130

Shanghai Service Center
NO.1386 HONG QIAO ROAD, CHANG NING QU, SHANGHAI 200336, CHINA
TEL: +86-21-2322-3030 / FAX: +86-21-2322-3000*8422
Qingdao Service Center
Suzhou Service Center
Wuhan Service Center
Ningbo Service Center
Hefei Service Center
Beijing Service Center
Tianjin Service Center
Xian Service Center
Dalian Service Center
Chengdu Service Center

Shenzhen Service Center
LEVEL8, GALAXY WORLD TOWER B, 1 YABAO ROAD, LONGGANG DISTRICT,
SHENZHEN 518129, CHINA
TEL: +86-755-2399-8272 / FAX: +86-755-8229-3686
Dongguan Service Center
Xiamen Service Center

KOREA

MITSUBISHI ELECTRIC AUTOMATION KOREA CO., LTD. (KOREA FA CENTER)

Korea Service Center
8F GANGSEO HANGANG XI-TOWER A, 401 YANGCHEON-RO, GANGSEO-GU,
SEOUL 07528 KOREA
TEL: +82-2-3660-9631 / FAX: +82-2-3664-8668
Korea Daegu Service Satellite

TAIWAN

MITSUBISHI ELECTRIC TAIWAN CO., LTD. (TAIWAN FA CENTER)

Taiwan Taichung Service Center
NO. 8-1, GONGYEQU 16th RD., XITUN DIST., TAICHUNG CITY 40768 , TAIWAN
TEL: +886-4-2359-0688 / FAX: +886-4-2359-0689

Taiwan Taipei Service Center
11F, NO.88, SEC.6, ZHONGSHAN N. RD., SHILIN DIST., TAIPEI CITY 11155, TAIWAN
TEL: +886-2-2833-5430 / FAX: +886-2-2833-5433

Taiwan Tainan Service Center
11F.-1, NO.30, ZHONGZHENG S. RD., YONGKANG DIST., TAINAN CITY 71067, TAIWAN
TEL: +886-6-252-5030 / FAX: +886-6-252-5031

OCEANIA

MITSUBISHI ELECTRIC AUSTRALIA PTY. LTD.

Oceania Service Center
348 VICTORIA ROAD, RYDALMERE, N.S.W. 2116 AUSTRALIA
TEL: +61-2-9684-7269/ FAX: +61-2-9684-7245

Notice

Every effort has been made to keep up with software and hardware revisions in the contents described in this manual. However, please understand that in some unavoidable cases simultaneous revision is not possible.

Please contact your Mitsubishi Electric dealer with any questions or comments regarding the use of this product.

Duplication Prohibited

This manual may not be reproduced in any form, in part or in whole, without written permission from Mitsubishi Electric Corporation.

© 2013-2023 MITSUBISHI ELECTRIC CORPORATION
ALL RIGHTS RESERVED.

mitsubishi **MITSUBISHI ELECTRIC CORPORATION**

HEAD OFFICE : TOKYO BLDG.,2-7-3 MARUNOUCHI,CHIYODA-KU,TOKYO 100-8310,JAPAN

MODEL	FCSB1224W000
MODEL CODE	—
Manual No.	IB-1501209